Two Trees in One Ball

Norm Ladders, Peel Steps and Knot Boosts

Jeromie Beasley

Prepared with AI assistance (Claude, Anthropic). Every Lean statement is kernel-checked and every number is reproduced by the scripts at the end. Version 4, 30 October 2026: Section 8 answers the open questions of version 3.

Abstract

Garrett's tree is built from the six Lipschitz quaternions of norm 5, $1 \pm 2i$, $1 \pm 2j$ and $1 \pm 2k$. Every step of the tree multiplies the norm by 5. In the peel coordinate of VOTKP, a step that multiplies a ratio by z has Smith coordinate $\Gamma = (z - 1)/(z + 1)$ and hyperbolic length $\ln z$, and steps compose by Einstein addition. So a norm-5 step has Smith coordinate 2/3 and length $\ln 5$. Each generator is sent to the Lorentz boost of rapidity $\ln 5$ along its own imaginary axis, and its conjugate to the inverse boost. This places the whole tree in the Poincaré ball, and every distance in the picture is a true hyperbolic distance between the placed points. Over the reals the unit quaternion $(1 + 2u)/\sqrt{5}$ acts as a rotation, not a boost; the boost is the peel-coordinate reading of the norm step, and its length $\ln 5$ is the length of one edge of the 5-adic tree.

Because $\cosh \ln 5 = 13/5$ and $\sinh \ln 5 = 12/5$, every point of the tree has rational coordinates, and the main facts are exact:

- straight paths add their lengths;
- one right-angle turn gives $\cosh d = \cosh^2(\ln 5)$;
- the straight path is the farthest of all words of its length;
- two perpendicular steps leave a Thomas rotation with $\cos \theta = 65/97$ and $\tan(\theta/2) = \Gamma^2 = 4/9$.

We prove for every word length that $5^L \cosh d$ is never divisible by 5. So no non-backtracking word ever returns to the origin, and the three boosts generate a free group. The same residue argument works for every whole-number peel ratio $z \geq 2$ that is not a power of 2, and a ping-pong at the prime $1 + i$ of the Gaussian integers covers every even ratio, so the boost group is free for every whole-number ratio. Garrett's own drawing uses the step $\ln 7$, and its caption numbers ($3 \ln 7 = 5.838$, and 4.742 for the word i j i) come out exactly.

The line $\Gamma^2 = 1/2$ divides two behaviours sharply. Above it ($\ln 7$, $\Gamma = 3/4$) the boosts form a Schottky group and the tree sits in the ball with distances kept in proportion. Below it ($\ln 5$, $\Gamma = 2/3$) the commutator of two perpendicular steps is a rotation through an angle that is no rational part of a turn. So the norm-5 group is free but not discrete, and its placement does not keep distances in proportion: the word $(i\, j\, \bar{\imath}\, \bar{\jmath})^n$ has 4n letters yet never lands more than 3.5255 from the centre.

On the knot side, the Riley meridians of a two-bridge knot on its real peripheral curve are boosts of the Smith disk of length 2x. The longitude is a boost of length $2|y|$, and the knot relation closes the word walk into a loop. The trefoil and figure-eight relations factor through their Riley polynomials over any commutative ring.

The relator lifts to the identity even in the universal cover, while the area of the drawn relator polygon depends on where it is started. It vanishes on a whole geodesic, because a reflection swaps the two meridians and maps the polygon to itself with orientation reversed.

The commutator of the two meridians satisfies $\mathrm{tr}[a, b] = 2 - v((s - 1/s)^2 - v)$. So it is a rotation only when the meridian axes cross, and, below the same line $\Gamma^2 = 1/2$, always when they do. For the torus knots T(2, 3), T(2, 5) and T(2, 7) the axes never cross, and this is proved. In a census of 26 two-bridge knots, nineteen have a rotating commutator on some real branch and seven never do. The boost group at $\ln 5$ is not conjugate to the Lubotzky–Phillips–Sarnak group, although both act simply transitively on the same 5-adic tree. The distance constants at $\ln 7$ are proved and optimal, and the ping-pong at $\ln 7$ is checked in Lean.

So the paper sets two trees side by side in one disk: Garrett's tree, which never closes, and a knot's tree, which does. One hundred and fifty-nine statements are checked in core Lean 4 without Mathlib, in five files.

1. Introduction

Garrett posted a picture of the norm-5 quaternion tree drawn in the Poincaré ball, with a caption giving hyperbolic lengths: $3 \ln 7 = 5.838$ along a straight path and 4.742 along a path with turns [1]. The question this paper answers is whether that drawing is only a picture, or whether the tree really sits in hyperbolic space with those distances.

It does. The tool is the peel coordinate of VOTKP [2]: a ratio z between two levels becomes a point $\Gamma = (z - 1)/(z + 1)$ of the Smith (Poincaré) disk, at hyperbolic distance $\ln z$ from the centre, and successive ratios multiply while their Smith points add by Einstein addition. A quaternion step multiplies the norm by 5, so it is a peel step of ratio 5. The quaternion also carries a direction, its imaginary axis. Length from the norm ratio and direction from the axis together give one Lorentz boost per generator. The tree becomes a set of points on the hyperboloid, with exact rational coordinates.

The same coordinate already carries Jeromie's knot work. On the real peripheral curve of a two-bridge knot, the two meridians act on the Smith disk as boosts, and the knot relation is a closed loop of boosts. That gives a second tree in the same disk, and the contrast between the two trees is the point of the paper.

A plain statement of what is chosen

Garrett's group is free, so *any* three boosts define a map of his tree into hyperbolic space that respects the tree's rules. The placement here is not forced by the group alone. What fixes it is the peel coordinate:

- the length of a step is the logarithm of its norm ratio;
- its direction is the quaternion's own axis.

Three consequences of this choice are worth saying at the start. First, over the real numbers the unit quaternion $(1 + 2u)/\sqrt{5}$ acts on space as a rotation, not a boost; the boost is what the peel coordinate makes of the norm step, and its length $\ln 5$ agrees with one edge of the 5-adic (Bruhat–Tits) tree, whose edges carry the ratio 5. Second, in the tree's own metric every reduced path of three steps has length three edges; the fact that a turned path is shorter than a straight one is a property of the placement, not of the tree. Third, the freeness of Garrett's quaternion group (Lubotzky–Phillips–Sarnak) does not by itself make the group of boosts free; that is a separate statement, proved here as Theorem 5.1.

Once that rule is fixed, everything below is a theorem. The boost of a *product* quaternion is not the boost along the product's axis: Section 4.4 shows that two boosts compose to a boost followed by a rotation. So the map respects the tree's word structure, not quaternion multiplication itself.

How results are graded

- **Lean**: kernel-checked in the files in Appendix A.
- **proved here**: a complete written proof in this paper.

- **recognised**: a classical result restated in the peel coordinate.
- **computed**: floating-point output of a script in Appendix B.

2. The peel coordinate

Restated from VOTKP [2] and proved here, so that the paper stands alone.

Definition 2.1. For a ratio $z > 0$, the Smith coordinate is $\Gamma(z) = (z - 1)/(z + 1)$. The Einstein sum on $(-1, 1)$ is $x \oplus y = (x + y)/(1 + xy)$.

Theorem 2.2 (VOTKP Theorem 8.1). With $z = e^{\tau}$, $\Gamma(z) = \tanh(\tau/2)$. **proved here**

Proof. $(e^{\tau} - 1)/(e^{\tau} + 1) = (e^{\tau/2} - e^{-\tau/2})/(e^{\tau/2} + e^{-\tau/2}) = \tanh(\tau/2)$. ∎

Theorem 2.3 (VOTKP Theorem 15.1). $\Gamma(A) \oplus \Gamma(B) = \Gamma(AB)$ for $A, B \geq 0$. Equivalently, $\oplus$ is the addition of rapidities. **Lean: gam_mul**

Proof. Put $a = \Gamma(A)$ and $b = \Gamma(B)$. The numerator of $a \oplus b$, over the common denominator $(A + 1)(B + 1)$, is $(A - 1)(B + 1) + (B - 1)(A + 1) = 2(AB - 1)$. The denominator is $(A + 1)(B + 1) + (A - 1)(B - 1) = 2(AB + 1)$. ∎

Corollary 2.4 (VOTKP Corollary 15.2). On the Poincaré disk the distance from 0 to $\Gamma(z)$ is $2\,\operatorname{artanh}\Gamma(z) = \ln z$. **proved here** (from 2.2)

Theorem 2.5 (the norm ladder). Let Γ_L be L copies of $2/3$ combined by $\oplus$. Then for every L,

$$\Gamma_L = (5^L - 1)/(5^L + 1), \quad d(0, \Gamma_L) = L \ln 5.$$

Lean: peel_ladder, by induction on L

Proof. $\Gamma(5) = 2/3$. Induction with Theorem 2.3: $\Gamma_{L+1} = \Gamma(5^L) \oplus \Gamma(5) = \Gamma(5^{L+1})$. ∎

A word of L quaternion generators has norm 5^L (Euler's four-square identity, Lean: norm_mul), so the norm ladder of Garrett's tree is exactly the peel ladder of Theorem 2.5.

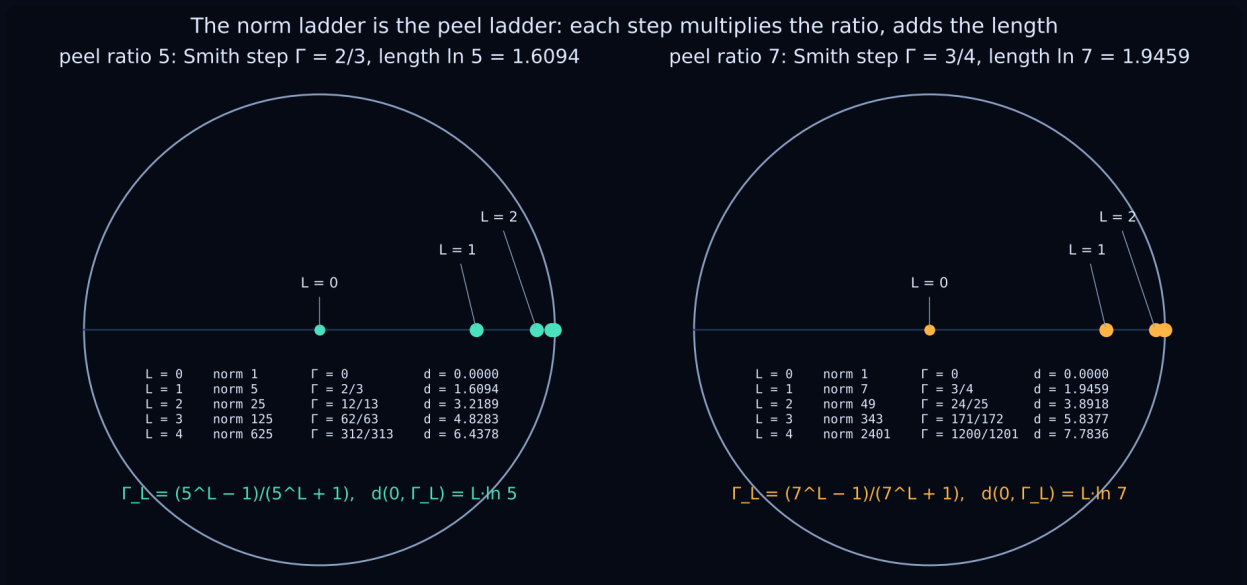


Figure 1. The norm ladder on the Smith disk for ratio 5 (Garrett's quaternions) and ratio 7 (the step of his drawing). Each step multiplies the norm, adds ln z to the length, and moves the Smith point by $\oplus$.

Theorem 2.6 (boost entries from Γ). For $z > 0$ and $\Gamma = \Gamma(z)$:

$$\cosh \ln z = (1 + \Gamma^2)/(1 - \Gamma^2) = (z^2 + 1)/(2z),\ \sinh \ln z = 2\Gamma/(1 - \Gamma^2) = (z^2 - 1)/(2z).$$

Lean: cosh_from_gam, sinh_from_gam For $z = 5$ this gives 13/5 and 12/5; for $z = 7$, 25/7 and 24/7. A boost of peel ratio z therefore has rational entries, and z times it has integer entries.

3. Garrett's tree

Write a quaternion as $a + bi + cj + dk$ with integer coordinates, norm $N = a^2 + b^2 + c^2 + d^2$, and conjugate $a - bi - cj - dk$.

Proposition 3.1.

1. There are exactly 48 integer quaternions of norm 5. **Lean: norm5_bound, norm5_count**
2. Exactly six of them have a odd and positive with b, c, d even, and these are Garrett's generators $1 \pm 2i$, $1 \pm 2j$, $1 \pm 2k$. **Lean: lps_six**
3. Each generator times its conjugate is 5, and the conjugate of a generator is again a generator. **Lean: gens_conj, gens_conj_mem**

Proof of 1. Each square is at most 5, so every coordinate lies in $[-2, 2]$. The 625 quaternions of that box are checked by the kernel, and 48 of them have norm 5. Parts 2 and 3 are finite checks. ∎

A *reduced word* is a word in the six generators in which no generator is followed by its own conjugate. By the theorem of Lubotzky, Phillips and Sarnak [3] (**recognised**), every quaternion of norm 5^L with a odd and b, c, d even that is not divisible by 5 is $\pm$ exactly one reduced word, and reduced words give distinct quaternions even up to sign. The uniqueness holds only after this normalisation and up to $\pm 5^r$: read literally it fails, since $2 + i$ also has norm 5 but is not one of the six. The Cayley graph is therefore the 6-regular tree, and the group modulo ± 1 is free on three generators. For length 3 this is checked directly: the 150 reduced words give 150 different quaternions up to sign, none divisible by 5 (**Lean: qwords_three_distinct, qwords_three_primitive**). Path-finding in this tree is known to be easy [4], which is why the tree is a good picture and a poor hash.

4. The tree in the ball, with true distances

Definition 4.1. For a unit axis u and a ratio z, let B_u be the Lorentz boost of rapidity $\ln z$ along u, acting on (t, x, y, z) with t first:

$$B_u = [[\ c,\ s\,u^{\mathsf T}\ ],\ [\ s\,u,\ I + (c - 1)\,u\,u^{\mathsf T}\ ]],\quad c = (z^2 + 1)/(2z),\ s = (z^2 - 1)/(2z).$$

Send the generator $1 + 2u$ to B_u and its conjugate $1 - 2u$ to B_{-u}. A reduced word $g_1 \cdots g_L$ goes to the point $p(w) = B_{g_1} \cdots B_{g_L}\, o$ on the hyperboloid, where $o = (1, 0, 0, 0)$. Its distance from the origin is $d = \operatorname{arcosh} t(w)$, with t the first coordinate. Its point in the Poincaré ball is $x/(1 + t)$.

4.1 Each step is a boost, and the step back is its inverse

Theorem 4.2. For $z = 5$ and $z = 7$ and each of the six directions, $B_u^{\mathsf T}\,\eta\,B_u = \eta$ with $\eta = \operatorname{diag}(1, -1, -1, -1)$, $B_u\,B_{-u} = I$, and B_u is symmetric. **Lean: lorentz5, lorentz7, inverse5, inverse7, boost5_symm** (on $z \cdot B$, with integer entries)

4.2 Straight paths add, turns shorten

Theorem 4.3 (two steps). If the second step makes angle φ with the first, then

$$\cosh d = \cosh^2 r + \sinh^2 r \cos \varphi.$$

So a straight continuation ($\varphi = 0$) gives cosh 2r, a right-angle turn gives $\cosh^2 r$ (hyperbolic Pythagoras), and a reversal gives 1. **proved here**; **recognised** (hyperbolic law of cosines)

Proof. $B_v\, o = (c, s\, v)$. The first row of B_u is $(c, s\, u^{\mathsf{T}})$, so the time coordinate of $B_u\, B_v\, o$ is $c{\cdot}c + s{\cdot}s\,(u{\cdot}v) = c^2 + s^2 \cos \varphi$. ∎

Theorem 4.4 (exact points, ratio 5). With every coordinate given exactly:

word	125 · point	cosh d	d
+i +i +i	(7813, 7812, 0, 0)	$7813/125 = (5^3 + 5^{-3})/2$	4.828314 = 3 ln 5
+i +i +j	(4069, 4056, 300, 0)	4069/125	4.175750
+i +j +i	(2917, 2808, 780, 0)	2917/125	3.842685
+i +j +k	(2197, 2028, 780, 300)	$2197/125 = (13/5)^3$	3.558871

The coordinates are **Lean: straight5, straight5_is_three_ln5, turned5**; the decimals are **computed**.

Theorem 4.5 (straight is farthest). Among all 150 reduced words of length 3, every word has 125 cosh d ≤ 7813, with equality exactly for the six straight paths. **Lean: straight_farthest5**

4.3 Garrett's caption, exactly

His drawing uses the step ln 7, which is peel ratio 7 with $\Gamma = 3/4$.

word (ratio 7)	343 · point	d	his caption
+i +i +i	(58825, 58824, 0, 0)	5.837730 = 3 ln 7	5.838
+i +j +i	(19657, 19200, 4200, 0)	4.741529	4.742
+i +i +j	(30025, 30000, 1176, 0)	5.165170	
+i +j +k	(15625, 15000, 4200, 1176)	4.511924	

The points are **Lean: garrett7**; the decimals are **computed**. Garrett's 4.742 is the word i j i. The step ln 7 with six branches is a drawing choice: the Lubotzky–Phillips–Sarnak tree for the prime 7 branches eight ways ($p + 1 = 8$), so the six boosts at ln 7 are not that tree. Every statement about ratio 7 in this paper is about the six boosts at ln 7.

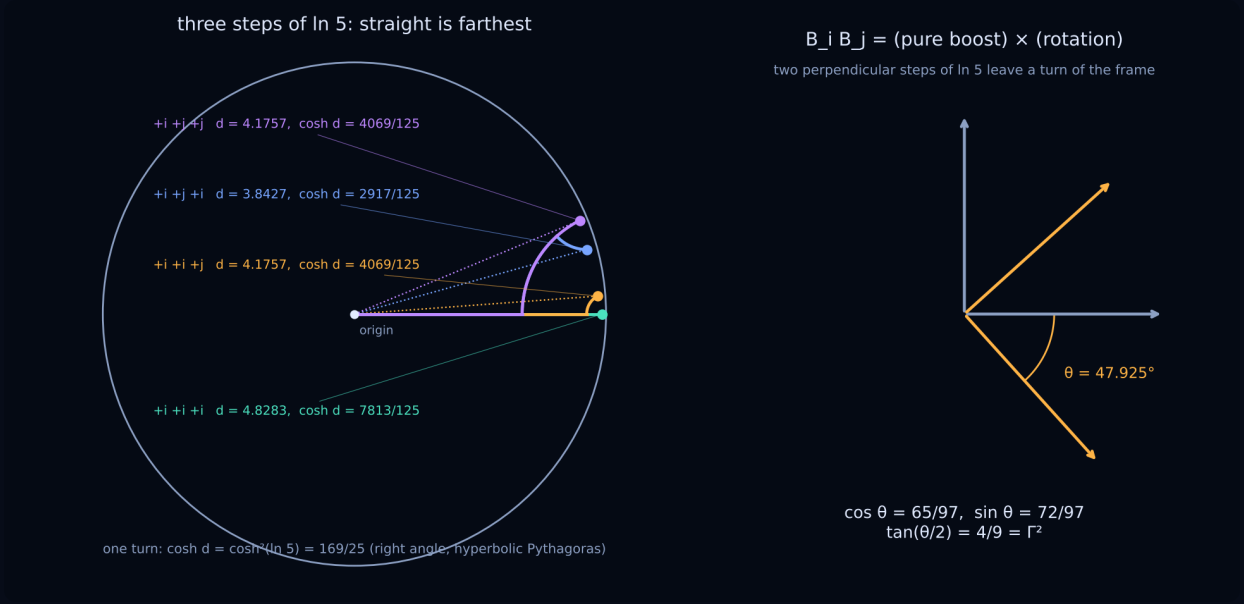


Figure 2. Left: four three-step words of ratio 5 drawn along true geodesics in the Poincaré disk. The straight word is the farthest, and each turn shortens the path. Right: two perpendicular steps compose to a pure boost followed by a turn of the frame through θ = 47.925°.

4.4 The turn leaves a rotation: Thomas rotation in the peel coordinate

Theorem 4.6. $B_i B_j = L R$, where L is the pure boost carrying o to B_iB_j o and R is a rotation about the k axis. For ratio 5, cos θ = 65/97 and sin θ = 72/97. For ratio 7, cos θ = 175/337 and sin θ = 288/337. In both cases the product B_iB_j is not symmetric, so it is not a boost. **Lean: thomas5, thomas5_boost, thomas5_rotation, two_boosts_not_boost5**, and the same for 7

Theorem 4.7 (half-angle). For two perpendicular steps of equal rapidity r with γ = cosh r and Γ = tanh(r/2):

$$\cos\theta = 2\gamma/(1+\gamma^2),\ \sin\theta = (\gamma^2-1)/(\gamma^2+1),\ \tan(\theta/2) = \Gamma^2.$$

The formula is **recognised**: for perpendicular boosts Rhodes and Semon give $\tan(\theta/2) = \tanh(r_1/2)\tanh(r_2/2)$ [12, Eq. 142b], and Ungar treats the rotation as a gyration [7]. For every ratio z the full decomposition is checked exactly: $L R = B_iB_j$ with R written in $t = \Gamma^2$, L symmetric and Lorentz, R a rotation, and $(1 - t^2, 2t, 1 + t^2)$ a Pythagorean triple (**Lean: thomas_rat, thomas_general, thomas_boost_general, thomas_rotation_general, thomas_triple**). The half-angle step in one line:

Proof. $\tan(\theta/2) = \sin\theta/(1+\cos\theta) = (\gamma^2-1)/(\gamma+1)^2 = (\gamma-1)/(\gamma+1) = \tanh^2(r/2) = \Gamma^2$. ∎

For the norm-5 step, tan(θ/2) = 4/9; for Garrett's ln 7 step, 9/16 (Lean: thomas_half_angle). The triples (65, 72, 97) and (175, 288, 337) both lie in the tree of Pythagorean triples. Its three generating matrices preserve $a^2 + b^2 - c^2$, so they are themselves integer Lorentz transformations of $O(2, 1; \mathbb{Z})$ [13]: a third integer tree in a hyperbolic plane, and the Thomas turn of two perpendicular peel steps is always one of its triples, with parameter Γ^2. The turning of the frame, which is the reason order matters only through a phase in VOTKP §15.2, is read off directly as the square of the Smith step.

5. The tree really embeds

5.1 Freeness from residues, for every length

Theorem 5.1. Let z = 5 or z = 7. For every non-backtracking word w of length L ≥ 1,

$$z^L \cosh d(w) \not\equiv 0 \pmod z.$$

Hence:

- (a) no non-backtracking word returns to the origin;
- (b) no non-backtracking word is the identity, so the three boosts generate a free group of rank 3;
- (c) distinct reduced words land on distinct points, so the map of the tree into the ball is injective.

Lean: time_not_div5, no_return5, free5; time_not_div7, no_return7, free7 for (a) and (b). Part (c) is **proved here** from (a).

Proof. Write $P_u = zB_u$, an integer matrix. Modulo z, its axis block is $[[(z^2+1)/2, \pm(z^2-1)/2], [\pm(z^2-1)/2, (z^2+1)/2]] \equiv \tfrac{1}{2}[[1, \mp 1], [\mp 1, 1]]$, and the other diagonal entries are $z \equiv 0$. So

$$P_u \equiv c_0\, w_u\, w_u^{\mathsf T} \pmod z,\ w_{\pm i} = (1, \mp 1, 0, 0),\ w_{\pm j} = (1, 0, \mp 1, 0),\ w_{\pm k} = (1, 0, 0, \mp 1),$$

with $c_0 = 3$ for $z = 5$ and $c_0 = 4$ for $z = 7$. A product of rank-one matrices is rank one: $(a\, w_f\, w_g^{\mathsf T})(b\, w_h\, w_h^{\mathsf T}) = ab\,(w_g \cdot w_h)\, w_f\, w_h^{\mathsf T}$. The dot product $w_g \cdot w_h$ is 2 when $h = g$, 1 when h is on another axis, and 0 only when $h = -g$, the backtrack. So along a non-backtracking word the product stays $c\, w_{\text{first}}\, w_{\text{last}}^{\mathsf T}$ with c a product of nonzero residues. That is nonzero modulo the prime z. Its (0, 0) entry is $c \cdot 1 \cdot 1 = c \not\equiv 0$, and that entry is $z^L \cosh d$.

If the word returned to the origin, or were the identity, the (0, 0) entry would be $z^L \equiv 0$. That proves (a) and (b).

For (c), suppose two reduced words $w_1 \neq w_2$ give the same point. Then $B(w_1)^{-1}B(w_2)$ fixes o. Since $B_{-u} = B_u^{-1}$, this is the boost product of the free reduction of $w_1^{-1}w_2$, a nonempty non-backtracking word. That contradicts (a). ∎

The Lean proof follows this argument exactly:

- reduction modulo z respects matrix products (red_mul5);
- a finite closure lemma decided over all states c, f, g, h (closure5);
- an induction over the word (walk5).

So the theorem is checked for words of every length, not up to a depth. As a separate check, an exact integer search over all 117,186 reduced words of length up to 7 finds no return and no relation, for both ratios (**computed**, PEEL-TREES-EXACT).

Theorem 5.2 (every ratio that is not a power of 2). Let $z \geq 2$ be a whole number that is not a power of 2. Then no non-backtracking word of the ratio-z boosts returns to the origin, none is the identity, and the boost group is free of rank 3. **Lean: word_form_general, time_mod_general, free_general, no_return_general, free_not_pow_two**

Proof. $Q_u = 2zB_u$ has entries $z^2 + 1$, $\pm(z^2 - 1)$ and $2z$, so $Q_u = W_u + zE_u$ exactly, with $W_u = w_u w_u^{\mathsf T}$ and E_u an integer matrix. Because $W_{f,g}W_{h,h} = (w_g \cdot w_h)\, W_{f,h}$ holds over the integers, induction along the word gives $Q(w) = 2^a\, W_{\text{first,last}} + zE$ for some a and some integer matrix E. Its (0, 0) entry, $(2z)^L \cosh d$, is $\equiv 2^a$ modulo any odd divisor $m > 1$ of z, and such an m never divides 2^a. A return to the origin would make that entry $(2z)^L \equiv 0$. ∎

The residue argument says nothing at powers of 2: for $z = 2$ and $z = 4$ many words have (0, 0) entry $\equiv 0$ modulo z. Theorem 8.1 covers every even z by a different route, so together the two theorems cover every whole number $z \geq 2$ (Corollary 8.2).

This is the same mechanism as the Lubotzky–Phillips–Sarnak argument on the quaternion side, a reduction modulo the prime of the step, now carried out on the boosts themselves.

5.2 The Schottky threshold is Γ² > 1/2

Theorem 5.3. If the peel ratio satisfies $z > 3 + 2\sqrt{2}$, equivalently $\Gamma^2 > 1/2$, then the three boosts generate a Schottky group: free, discrete, with every non-identity element loxodromic. The orbit map of the tree into the ball is then a quasi-isometric embedding. Garrett's step ln 7 ($\Gamma^2 = 9/16$) satisfies it; the norm-5 step ($\Gamma^2 = 4/9$) does not. **recognised** (ping-pong, Schottky [5]); threshold in the peel coordinate **proved here**; the two inequalities are **Lean: schottky_five_below, schottky_seven_above, schottky_ratio**

Proof. Around each of the six directions ±i, ±j, ±k take the half-space bounded by the hyperbolic plane perpendicular to that axis at distance δ from the origin. The planes for orthogonal axes are disjoint exactly when $\delta \geq \ln(1 + \sqrt{2})$: at $\delta = \ln(1 + \sqrt{2})$ the boundary circles have angular radius π/4 and touch at infinity.

The boost B_u slides along its axis by r, so it carries the plane at $-\delta$ to the plane at $r - \delta$. It therefore maps the outside of the half-space at $-u$ into the half-space at $+u$ exactly when $r - \delta \geq \delta$, that is, $r \geq 2\ln(1 + \sqrt{2})$, or $z \geq (1 + \sqrt{2})^2 = 3 + 2\sqrt{2}$. With strict inequality, δ can be raised slightly so that the closed half-spaces become pairwise disjoint, and the ping-pong lemma gives a Schottky group [5]. Schottky groups are convex cocompact, so orbit maps are quasi-isometric embeddings.

In the peel coordinate: $\Gamma(3 + 2\sqrt{2}) = (2 + 2\sqrt{2})/(4 + 2\sqrt{2}) = 1/\sqrt{2}$. Since Γ increases with z, $z > 3 + 2\sqrt{2} \Leftrightarrow \Gamma^2 > 1/2$. ∎

The v2 working report reaches the same line through the Dirichlet half-spaces of the steps, whose caps have angular radius α with $\cos\alpha = \Gamma$. It also gives explicit constants at ln 7, $\ell + (L - 1)\delta \leq d(O, wO) \leq L\ell$ with $\cosh\delta = \sinh^2(\ell/2) = 9/7$, checked there on every node to depth 9 [14]. Theorem 8.4 proves them and shows that δ cannot be improved.

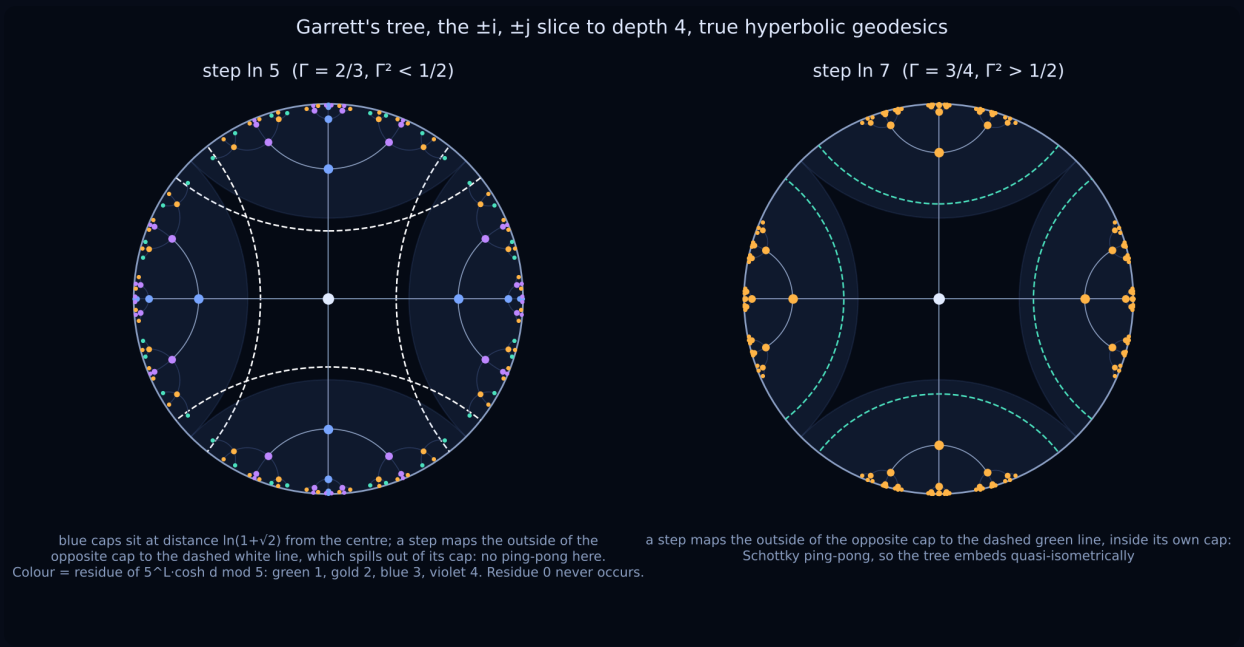


Figure 3. The ±i, ±j slice of Garrett's tree to depth 4, with edges drawn as true hyperbolic geodesics. Left, step ln 5: a step carries the outside of the opposite cap to the dashed white line, which spills out of its own cap, so ping-pong does not apply. The dot colours are the residues of $5^L \cosh d$ modulo 5, and residue 0 never appears, which is Theorem 5.1. Right, Garrett's step ln 7: the dashed images sit inside the caps.

5.3 Below the line: free, but not discrete

Theorem 5.4. At the norm-5 step the commutator $c = B_i B_j B_{-i} B_{-j}$ fixes the point p = (3, 2, 2, 0) of the hyperboloid and turns about it through an angle φ with $\cos\varphi = -527/625$, which is no rational part of a full turn. Hence:

- (a) the ln 5 boost group is free (Theorem 5.1) but not discrete;
- (b) the word $c^n = (i\, j\, \bar{\imath}\, \bar{\jmath})^n$ has $4n$ letters but $d(O, c^n O) \le 2\, d(O, p) = 3.5255$ for every n, so the placement is not a quasi-isometric embedding.

Lean: comm5_fixes_p, comm5_p_unit, comm5_trace, comm5_cos, comm7_trace for the exact data; the rest is **proved here**.

Proof. The kernel check gives $c\, p = p$ with $\langle p, p\rangle = 9 - 4 - 4 = 1$, so c is a rotation about p; its trace is $196/625 = 2 + 2\cos\varphi$, so $\cos\varphi = -527/625$. The number $e^{i\varphi}$ is a root of $x^2 + (1054/625)x + 1$, which is irreducible and not monic over the integers, so $e^{i\varphi}$ is not an algebraic integer and not a root of unity: φ is no rational part of a turn and c has infinite order. A discrete group contains no rotation of infinite order [16], which proves (a). Since c^n fixes p, $d(O, c^n O) \le d(O, p) + d(p, c^n O) = 2\, d(O, p)$, and $\cosh d(O, p) = 3$ gives $2\, d(O, p) = 3.5255$, which proves (b). ∎

The line $\Gamma^2 = 1/2$ is sharp. In the plane of two perpendicular axes the commutator trace in $PSL(2, \mathbb{R})$ is $\mathrm{tr}[A, B] = 2 - 4\sinh^4(r/2)$. That is a rotation exactly when $\sinh(r/2) < 1$, i.e. $z < 3 + 2\sqrt{2}$; it is parabolic at $z = 3 + 2\sqrt{2}$ and a boost above (**computed** exactly at $z = 2, 3, 5, 11/2, 17/3, 6, 7, 10, 25$). At ln 7 the commutator trace is $51076/2401$ in $SO(3, 1)$, not a rotation (Lean: comm7_trace).

The norm-5 tree still contains a well-placed part. The squared steps B_u^2 have rapidity ln 25, above the line, so B_i^2, B_j^2, B_k^2 generate a Schottky group: the doubled-letter subtree does sit in the ball with distances kept in proportion (**proved here**, from Theorem 5.3).

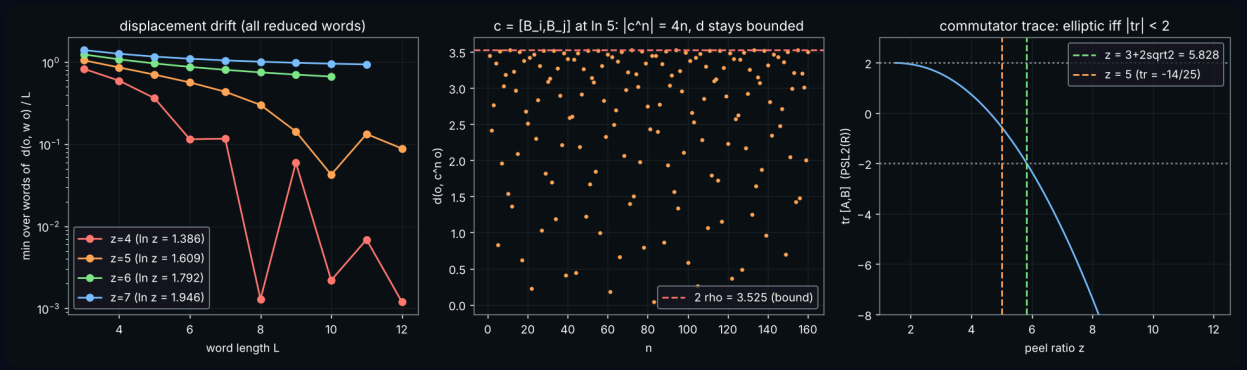


Figure 4. Left: the smallest value of $d(O, wO)/L$ over all reduced words of length L. For ratios 4 and 5 (below the line) it falls toward 0; for 6 and 7 (above it) it levels off. Middle: $d(O, c^n O)$ for the ln 5 commutator, never above $2\rho = 3.5255$. Right: the commutator trace against the peel ratio z, crossing -2 exactly at $z = 3 + 2\sqrt{2}$.

The smallest distance 0.4258 between two of the 4,687 words to depth 5 belongs to a pair 10 edges apart (the word $c^2\, i\, j$), a ratio of 0.043; the drift table to length 12 is in COMPUTE-Q1 (**computed**). The v2 working report gives a second route to freeness at ln 5: a ping-pong on the 5-adic tree, where each step moves the base vertex to one of the six neighbours (1:4), (1:1), (1:3), (1:2), (0:1), (1:0) of $P^1(\mathbb{F}_5)$ [14]. That argument is not formalised here; Theorem 5.1 covers the same conclusion in Lean. Theorem 8.3 shows that this tree action is all the two groups share: they are not conjugate inside $PGL(2, \mathbb{Q}_5)$.

6. The knot's tree

For a two-bridge knot with Riley presentation, the meridians are

$$a = [[s, 1], [0, 1/s]],\ b = [[s, 0], [-v, 1/s]],$$

and the knot relation is $aW = Wb$, with W a word fixed by the knot (Schubert's presentation [10]; Riley's normal form [8]). By the *real peripheral curve* we mean the real points of the knot's eigenvalue variety, the A-polynomial curve of Cooper, Culler, Gillet, Long and Shalen [11], where the meridian eigenvalue M and the

longitude eigenvalue L are both real; the name is ours, not a standard term. Where $|M| \neq 1$ the representation is conjugate into $SL(2, \mathbb{R})$ (standard), and in Riley's form this means s and v real. On the *real peripheral curve* both $s = e^x$ and v are real. Then a and b are real matrices of determinant 1 and act on the upper half-plane, which is the Smith disk under the Cayley map.

Theorem 6.1 (meridians are boosts of length 2x). Both meridians have determinant 1 and trace $s + 1/s = 2 \cosh x$. For real $s \neq \pm 1$, $(\mathrm{tr})^2 - 4 = (s - 1/s)^2 > 0$, so each meridian is a hyperbolic translation of length 2x. On the Smith disk this is a step of $\Gamma = \tanh x$. **Lean: meridian_boost, riley_inverses** (over any commutative ring); translation length **recognised**

Theorem 6.2 (the relation is the Riley curve). Scale by s so that every entry is a polynomial: $A = s \cdot a$ and $B = s \cdot b$.

- Trefoil ($W = ba$): $A\,W - W\,B = R_3(s, v) \cdot [[0, s], [s\,v, 0]]$, with $R_3 = s^4 - s^2 v - s^2 + 1$.
- Figure-eight ($W = b\,a^{-1}\,b^{-1}\,a$): $A\,W - W\,B = R_4(s, v) \cdot [[0, s^3], [s^3\,v, 0]]$, with $R_4 = s^2 v^2 + (3s^2 - s^4 - 1)v + (3s^2 - s^4 - 1)$.

Both identities hold over every commutative ring, so the relation holds exactly where the Riley polynomial vanishes. **Lean: trefoil_relation, figure_eight_relation**

Corollary 6.3 (real curves).

- For the trefoil, $v = s^2 + s^{-2} - 1$ is real for every real s, so every x gives a real representation.
- For the figure-eight, put $c = s^2 + s^{-2} - 3$. Then $v^2 - cv - c = 0$, so v is real exactly when $c(c + 4) \geq 0$, that is, when $\cosh 2x \geq 3/2$. The real curve starts at $x_0 = \tfrac{1}{2} \operatorname{arcosh}(3/2) = \ln \varphi = 0.48121$, where $|M| = \varphi = (1 + \sqrt{5})/2$ is the golden ratio, where $v = 0$ and s^2 is the real root $(3 + \sqrt{5})/2$ of the Alexander polynomial $t^2 - 3t + 1$, as Burde and de Rham predict [9].

proved here (from Theorem 6.2); the identities $s^2(v^2 - cv - c) = R_4$ and the discriminant $c(c + 4)$ are **Lean: fig8_riley_c, fig8_disc**. The v2 working report reaches the golden-ratio start from the discriminant of the Riley polynomial [14].

Theorem 6.4 (the longitude is a boost of length 2|y|). On the trefoil's curve, the scaled longitude $B\,A\,A\,B\,(s\,a^{-1})^4 = s^8$ L is upper triangular with diagonal $-s^2$, $-s^{14}$. So L has eigenvalues $-s^{-6}$ and $-s^6$: $|L| = e^y$ with $y = -6x$, and L translates by $12x = 2|y|$. **Lean: trefoil_longitude** (over any commutative ring, using $R_3 = 0$)

For hyperbolic two-bridge knots the same statements are **computed** on their real curves (KNOT-BOOST-TREE):

knot	x	v	meridian length	$\lvert aW - Wb \rvert$	longitude length = 2|y|
4_1	0.50	+0.33976	1.000000	2.2e−16	1.300923
4_1	0.90	+4.01557	1.800000	2.1e−14	6.691043
5_2	0.50	+1.77613	1.000000	1.8e−15	9.457698
5_2	0.90	+5.05483	1.800000	2.7e−13	17.671149
6_2	0.50	+2.45565	1.000000	5.9e−14	2.863072
6_2	0.90	+5.39872	1.800000	8.1e−11	4.046619

In every row the meridian length is 2x, and the longitude length equals 2|y| to all printed digits.

These real representations are the ones the literature uses. Gordon proved Riley's conjecture that the Riley polynomial of a two-bridge knot has at least |σ|/2 real roots, each a parabolic SL(2, ℝ) representation; Thakar builds hyperbolic ones on the real curve of 6_2, with the same meridian [[s, 1], [0, 1/s]] and the same longitude, to show that surgeries on 6_2 with slopes in (−4, 8) have left-orderable groups; Culler and Dunfield organise such representations by their boundary data [15]. The pair (meridian 2x, longitude 2|y|) read here is that boundary data written as hyperbolic lengths.

7. Two trees in one ball

The knot's boosts are real, so they act on the hyperbolic plane. That plane is the equatorial disk of the ball, the same disk that holds the ±i, ±j slice of Garrett's tree. Both trees are walks of boosts in one hyperbolic disk, measured in one coordinate.

	Garrett's tree	a knot's tree
step	norm ratio 5 (or 7): boost of ln 5, Γ = 2/3	meridian: boost of 2x, Γ = tanh x
directions	three orthogonal axes and their reverses	the axes of a and b, meeting at an angle fixed by v
backtrack	conjugate quaternion = inverse boost (Lean)	a^{-1}, b^{-1}
turns	Thomas rotation, $\tan(\theta/2) = \Gamma^2$ (Lean, exact)	the angle between the meridian axes
relations	none: $5^L \cosh d \not\equiv 0 \bmod 5$ for every word (Lean)	the knot relation aW = Wb, a closed loop (Lean for 3_1, 4_1)
long loop	none	the longitude: a boost of length 2|y|
placement	ln 5: free, not discrete, not quasi-isometric; ln 7: Schottky	the relator lifts to the identity in the universal cover

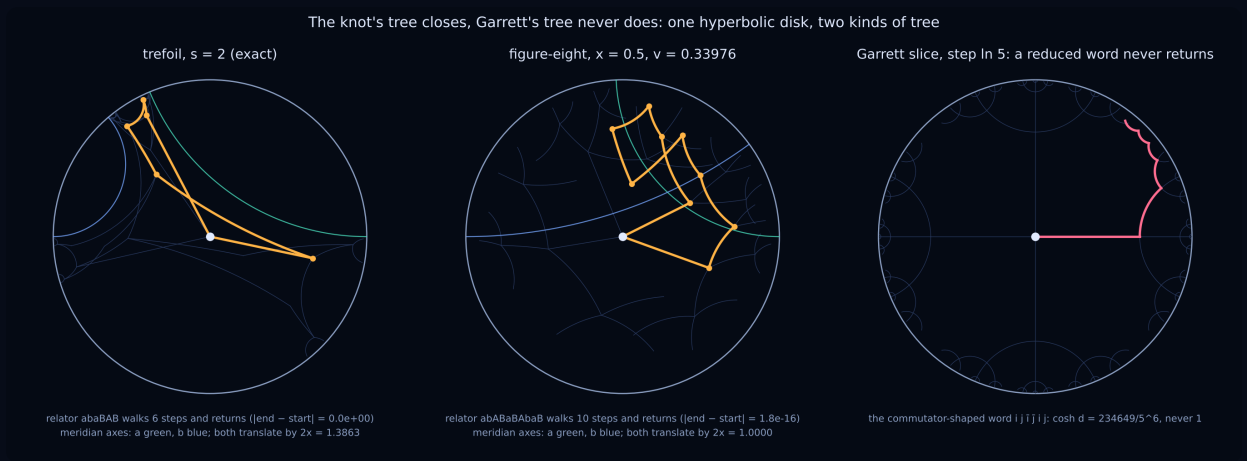


Figure 5. Left: the trefoil at s = 2, where v = 13/4 exactly; the relator a b a b^{-1} a^{-1} b^{-1} walks six steps and returns. Middle: the figure-eight at x = 0.5 on its real curve; the relator a W b^{-1} W^{-1} walks ten steps and returns. Right: a non-backtracking word in Garrett's slice runs to the boundary and never comes back, with cosh d = 234649/5^6. Faint lines are the depth-3 word trees.

Two points keep this comparison honest. The relator loop closes for every representation, because a representation satisfies the knot relation by definition; the closing checks the numbers, it is not a discovery. What carries content is Theorem 6.2: the relation holds exactly where the Riley polynomial vanishes, and on the real curve each step is a boost of length 2x. And the contrast between a free group and a one-relator group is true by algebra; Figure 5 illustrates it rather than proving a deeper link.

The free tree and the closing tree differ by exactly one thing: a relation. In the peel coordinate that relation is visible as a loop of Smith steps that adds up to zero, whereas Garrett's residues forbid every such loop. A knot is what a tree of boosts looks like when its steps are tied back on themselves.

7.1 What turns around a knot's loop

The relator returns every frame unturned, and this survives the passage to the universal cover. **Proposition 7.1.** With the canonical lifts $\tilde{a}$, $\tilde{b}$ of hyperbolic meridians (the lifts that have a fixed point on the circle at infinity), $\tilde{a}\,\tilde{W} = \tilde{W}\,\tilde{b}$ holds in the universal cover of PSL(2, $\mathbb{R}$). **proved here**; computed to 10^{-80} on all nine real branches of 3_1, 4_1, 5_2, 6_2.

Proof. Two lifts of a hyperbolic element differ by a central translation, which moves every point, so at most one lift has a fixed point and that lift is preserved by conjugation. In PSL(2, $\mathbb{R}$) the relation gives $b = W^{-1}aW$, so $\tilde{W}^{-1}\tilde{a}\tilde{W}$ is a lift of b with a fixed point, i.e. $\tilde{b}$. ∎

What does turn is a frame carried along the drawn geodesic polygon through O, aO, abO, …: by Gauss–Bonnet its turn is the polygon's signed area. That area varies continuously along each real branch and depends on the base point, so it is a property of the drawing, not of the knot, as the v2 working report also found [14]. The vanishing found from the axis foot is explained in §8.4: it holds on a whole geodesic, by a reflection symmetry. At the base point O = i it tends to $-\pi + 4e^{-x}$ on branches where $v \to \infty$ (**computed**). The integer that does change from branch to branch is the lift class of the longitude: -1 on the branch that reaches $x \to 0$ (trefoil; branch 0 of 5_2 and 6_2) and 0 elsewhere, which in this data records the sign of the longitude's trace (**computed**).

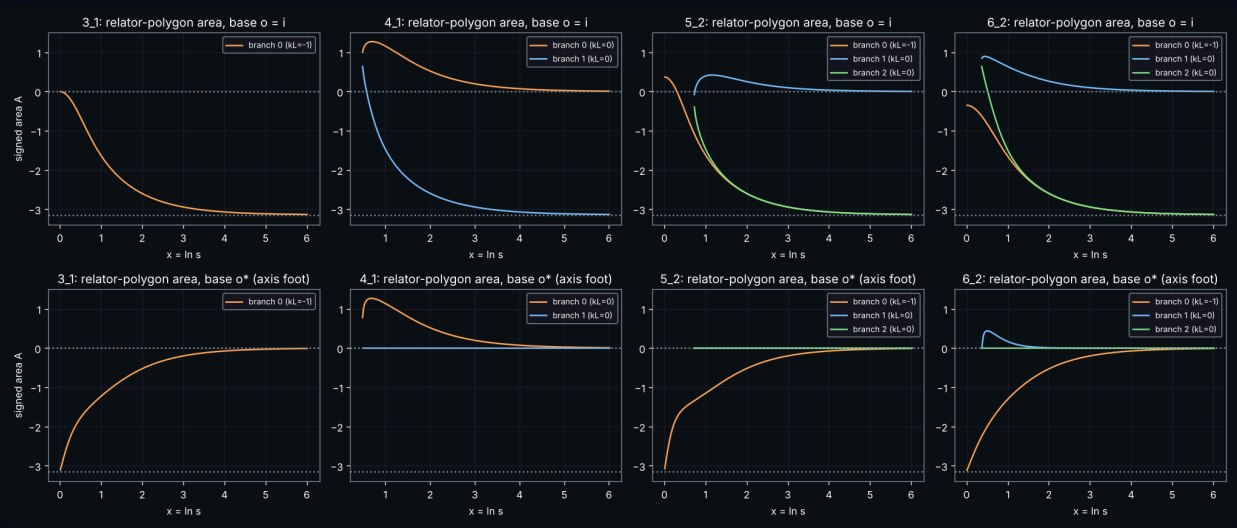


Figure 6. Signed area of the relator polygon along the real branches of 3_1, 4_1, 5_2 and 6_2, from the base point i (top) and from the foot of the common perpendicular of the meridian axes (bottom). The curves change with the base point; the relation itself closes exactly in both rows.

8. The open questions of v3, answered

Version 3 ended with five open questions. All five are settled below, four by proof and one by proof for a family together with an exact census. Each result carries its grade.

8.1 Peel ratios that are powers of 2

The residue argument of Theorem 5.2 needs an odd divisor of z. A different prime fills the gap: the prime 2 itself, which is where every scaled generator collapses to rank one.

Theorem 8.1 (every even ratio). Let $z \geq 2$ be even, for example $z = 2$, 4 or 8. Then B_i, B_j, B_k generate a free group of rank 3, and any two of them generate a free group of rank 2. **proved here**; written in full and checked exactly (identities symbolically; the inclusions on 100,000 exact test points, which also fail on 80% of points for odd z, so the test can detect a wrong proof). Not in Lean.

Proof. *Dictionary.* Write a Hermitian 2×2 matrix as $X = t + x\cdot\sigma$, so $\det X = t^2 - x^2 - y^2 - z^2$. For M in GL(2, $\mathbb{C}$) put $\rho(M)X = M\,X\,M^*/|\det M|$. This is a homomorphism to the Lorentz group whose kernel is the scalars. With $M_u = (z + 1) + (z - 1)\sigma_u$, a direct computation gives $\rho(M_u) = B_u$, the boost of rapidity ln z along u, and $\rho(M_u^{-1}) = B_u^{-1}$. So a reduced word in the $B_u^{\pm 1}$ is the identity only if the matching product of the $M_u^{\pm 1}$ is a scalar, that is, only if it acts trivially on $P^1(\mathbb{Q}(i))$ by Möbius maps.

Valuation. For $w = a + bi$ in $\mathbb{Q}(i)$ put $v(w) = v_2(a^2 + b^2)$, the valuation at the prime $1 + i$. It is multiplicative and ultrametric, $v(1 \pm i) = 1$, $v(2) = 2$, and for z even $v(z) \geq 2$, $v(2z) \geq 4$, $v(z - 1) = 0$.

Six sets. $U_{\pm i} = \{w : v(w \pm 1) \geq 3\}$, $U_{\pm j} = \{w : v(w \mp i) \geq 3\}$, $U_{+k} = \{w : v(w) \geq 1\}$, $U_{-k} = \{w : v(w) \leq -1\} \cup \{\infty\}$. The centres ± 1, $\pm i$ differ pairwise by elements of valuation at most 2, so the four balls of radius 3 are disjoint. Their points have $v(w) = 0$, so they miss both $U_{\pm k}$.

Inclusions. The Möbius map of M_i is $f(w) = ((z+1)w + (z-1))/((z-1)w + (z+1))$. Then $f(w) + 1 = 2z(w + 1)/D$ with $D = (z - 1)(w - 1) + 2z$. If w is outside U_{-i}, then $y = w - 1$ has $v(y) \leq 2 < 4 \leq v(2z)$, so $v(D) = v(y)$, and $v(w + 1) = v(y + 2) \geq v(y)$. Hence $v(f(w) + 1) \geq v(2z) \geq 4$: $f(w)$ lies in U_{+i}. (At $w = \infty$, $f(\infty) + 1 = 2z/(z - 1)$, of valuation ≥ 4.) The inverse is $f^{-1}(w) = -f(-w)$, which handles $-i$. The j-axis is the same computation with $D = i(z - 1)(w + i) + 2z$. The k-axis is $f(w) = zw$: $v(w) \geq 0$ gives $v(zw) \geq 2$, and $v(w) \leq 0$ gives $v(w/z) \leq -2$.

Ping-pong. By induction, a nonempty reduced word $s_1 \ldots s_n$ sends every point outside $U_{s_n^{-1}}$ into U_{s_1}. Choose a letter t different from s_1 and s_n^{-1}, and a point p in U_t. The word moves p into U_{s_1}, which is disjoint from U_t, so the word is not the identity. ∎

Corollary 8.2. For every whole number $z \geq 2$ the ratio-z boosts generate a free group of rank 3. **proved here**: Theorem 5.2 (Lean) covers z with an odd divisor, and Theorem 8.1 covers every even z, which includes the powers of 2.

The two methods use two different primes. Theorem 5.2 works modulo an odd prime of z, where the generator matrices keep full information. Theorem 8.1 works at the prime 2, where the residue method sees only rank-one matrices but the valuation still separates six balls. Below the Schottky line both groups stay free without being discrete. At $z = 2$ the perpendicular commutator has trace 31/16 and at $z = 4$ trace 47/64: both are elliptic of infinite order, because a rational trace that is not an integer cannot belong to a root of unity. At $z = 8$ the trace is $-1889/256$, which is loxodromic, as expected above $3 + 2\sqrt{2}$. **computed** exactly. A meet-in-the-middle search for relations, with positive controls that do find the known relations at $z = 1$, found none up to length 22 at $z = 2$ and up to length 20 at $z = 4$ and 8 (**computed**). Whether some non-backtracking word fixes the origin at a power of 2 is not settled by freeness; the search found none up to length 14.

8.2 The boost group is not conjugate to the LPS group

Theorem 8.3. At ratio 5 the boost group and the Lubotzky–Phillips–Sarnak group are not conjugate in PGL(2, $\mathbb{Q}_5$), nor in PGL(2, $\mathbb{C}$), under any matching of generators. No element of the LPS group is conjugate to a boost generator. **proved here**

Proof. The number $\tau(g) = \mathrm{tr}(g)^2/\det g$ does not change when g is scaled or conjugated. Each LPS generator $1 \pm 2u$ has trace 2 and determinant 5, so $\tau = 4/5$. Each boost generator $3 \pm 2\sigma_u$ (that is, $\sqrt{5}\cdot B_u$) has trace 6 and determinant 5, so $\tau = 36/5$. The two values are rational and different, so they stay different in $\mathbb{Q}_5$ under either

embedding of i, and in $\mathbb{C}$. Every element of the LPS group comes from a quaternion $q = a + bi + cj + dk$, so $\tau = 4a^2/N(q) \le 4$, while $\tau = 36/5 > 4$ for the boosts. ∎

So the two groups look alike on the 5-adic tree but are not the same matrices. Both send the base vertex to its six neighbours, which are the six points of $P^1(\mathbb{F}_5)$. By a covering-of-trees argument both act simply transitively on the vertices, and both are free of rank 3. Matching words therefore gives a tree automorphism that conjugates one group to the other. That automorphism is not in $PGL(2, \mathbb{Q}_5)$. On the tree each generator of either group translates by one edge. What separates them is the eigenvalue ratio: 5 for a boost and $(-3 + 4i_5)/5$ for an LPS generator. Over $\mathbb{C}$ the LPS generator is a rotation through $\arccos(-3/5) = 126.87°$, while the boost is loxodromic. The commutator invariants happen to agree ($\tau = 196/625$ for both), but the generator, square and product invariants all differ (**computed** exactly). The 5-adic tree check covers all 937 reduced words up to length 4.

8.3 The constants at ln 7, and the ping-pong in Lean

Theorem 8.4. At ratio 7, with $\ell = \ln 7$ and $\cosh \delta = 9/7$, every reduced word w of length $L \ge 1$ satisfies $\ell + (L - 1)\delta \le d(O, wO) \le L\ell$. More sharply, $d(O, wO) \ge \ell(1 + \#\text{straight}) + \delta \cdot \#\text{turns}$, where a turn is a change of axis. The constant δ is optimal. **proved here**; **computed** on all 488,281 reduced words up to length 9, which have smallest margin 0.5526

Proof. Put $N_u = (3, 4e_u)$. Then $\langle N_u, N_u \rangle = -7$ and $Q_u N_{-u} = -7N_u$ for $Q_u = 7B_u$, so each step carries the plane $H_{-u} = N_{-u}^{\perp}$ onto H_u with its sides exchanged. Along a reduced word the planes met by the segment from O to wO are therefore nested half-spaces. Consecutive planes are ℓ apart when the step repeats its axis and δ apart when it turns ($\cosh \delta = 9/7$, the distance between planes of orthogonal axes). The segment crosses each plane once, and adding these separations gives the bound. The commutator $B_i B_j B_i^{-1} B_j^{-1}$ is a pure translation with $\cosh \tau = 23137/2401 = \cosh 4\delta$. So δ cannot be raised, and the minimum of $d/(L\ell)$ tends to $\delta/\ell = 0.37977$ (it is still 0.5051 at L = 9). ∎

The ping-pong itself is now in Lean (PingPong7.lean, 17 theorems, core Lean, no sorry and no native_decide). It works on the integer light cone. With caps $X_u = \{\text{future null } v : \text{comp}_u\, v > 0,\ t^2 < 2\, \text{comp}_u(v)^2\}$, the file proves that the six caps are pairwise disjoint and that $7B_u$ sends every null vector outside X_{-u} into X_u. Its one nonlinear step is closed by the identity $432(527t^2 + 1200tb + 674b^2) = 41040(t^2 - 2b^2) + (432t + 600b)^2 + 13248b^2$. A general ping-pong lemma, valid for any alphabet and depending on no axioms, then shows that every nonempty reduced word sends $p_0 = (3, 1, 2, 2)$ into the cap of its first letter, so it is not c·id. **Lean: caps_disjoint, pingpong_step, pingpong, word_in_cap, word_ne_scalar, word_ne_id**. The distance bound of Theorem 8.4 is not formalised, because core Lean has no real numbers.

8.4 Why the relator polygon has zero area at the axis foot

Theorem 8.5. Let K(p, q) be a two-bridge knot with q odd and a real Riley representation with $v > 0$. The map $R(z) = 1/(v\,\bar{z})$ is the reflection of the upper half-plane in the geodesic $\ell = \{|z| = 1/\sqrt{v}\}$. For every base point o, the signed area of the relator polygon satisfies $A(Ro) = -A(o)$. In particular $A = 0$ at every point of ℓ. When the meridian axes cross, their crossing point lies on ℓ, and that is the zero found in v3. **proved here**; **computed** on about 40 branch points of 3_1, 4_1, 5_2, 6_1, 6_2 and (13, 5) at 50 to 100 digits, with worst error $4 \cdot 10^{-31}$

Proof. Put $D = \text{diag}(1, -v)$ and $\tau(X) = D\, X^{\mathsf{T}}\, D^{-1}$. This map reverses products and swaps a and b. For q odd the exponents satisfy $e_{p-i} = e_i$, and positions i and $p - i$ hold letters of opposite type, so $\tau(W) = W$: the word W is a palindrome up to the swap. (Checked for all 364 pairs with q odd and $p \le 59$.) On SL(2), $\tau(X) = K\, X^{-1}\, K^{-1}$ with $K = [[0, 1], [v, 0]]$, and for $v > 0$ the matrix K acts as the reflection R. So $R\,a\,R = b^{-1}$, $R\,b\,R = a^{-1}$ and $R\,W\,R =$

W^{-1}. Write $r = r_1 r_2$ with $r_1 = aW$. The substitution $a \mapsto b^{-1}$, $b \mapsto a^{-1}$ turns r into $r_2 r_1$, a cyclic rotation by half its length. So for o on ℓ the orientation-reversing isometry $\Sigma = (aW)^{-1}R$ sends the vertex p_k to p_{k+m}. It maps the polygon to itself and keeps the cyclic order. Signed area depends only on the cyclic sequence of edges and changes sign under a reflection, so $A = -A = 0$. The same steps give $A(Ro) = -A(o)$ for any o. The axes of a and b meet the real line at $x_a = s/(1 - s^2)$ and at 0 and $x_b = (1 - s^2)/(sv)$. Since $x_a x_b = 1/v$, the crossing point, when there is one, has $|z|^2 = 1/v$. ∎

The symmetry is a reflection followed by a half-period shift, not a reversal of the polygon. This is why the half-turn tested in v3 did not explain the zero: that half-turn preserves orientation. For $v < 0$ the matrix K is a half-turn, and the area is nonzero at every point tested. On a 17×17 grid the area changes sign only across ℓ, but that ℓ is the whole zero set is not proved.

8.5 Which knot commutators turn

The Commutator eye of the atlas follows $\mathrm{tr}[a, b]$ for the Riley meridians. In v3 the band of rotations was mapped only for $x \le 2.2$. This section gives the exact criterion, a proof for the torus knots $T(2, n)$, and a census of the 26 two-bridge knots in the atlas.

Theorem 8.6 (the commutator band). For the Riley meridians, put $k = 2 - \mathrm{tr}[a, b]$. Then

$$\mathrm{tr}[a, b] = 2 - v((s - 1/s)^2 - v),\quad \mathrm{tr}[a, b] - 2 = -v(\mathrm{tr}(ab) - 2),\quad 4k = (s - 1/s)^4 - (2v - (s - 1/s)^2)^2.$$

These hold over any commutative ring after scaling by s^4. So on the real curve $[a, b]$ is a rotation ($|\mathrm{tr}| < 2$) exactly when $0 < v(4 \sinh^2 x - v) < 4$. The first inequality, $0 < v < 4 \sinh^2 x$, says that the axes of a and b cross. Since $k \le 4 \sinh^4 x$, the second inequality holds automatically when $\sinh x < 1$, that is, when the meridian's Smith step satisfies $\Gamma^2 = \tanh^2 x < 1/2$. Below the Schottky line of §5.2, crossing axes are therefore exactly the rotations, and a branch with $v < 0$ is never a rotation. **Lean: comm_trace, comm_trace_ab, band_k, band_bound, band_peak**; the axis-crossing reading is **recognised** [16] (two hyperbolic elements with crossing axes have $\mathrm{tr}[a, b] < 2$).

So the line $\Gamma^2 = 1/2$ appears on both sides of the paper. For Garrett's boosts it separates Schottky from non-discrete groups. For a knot's meridians it is the length $2 \ln(1 + \sqrt{2})$ up to which crossing axes always produce a rotating commutator.

Theorem 8.7 (torus knots never turn). For $T(2, 3)$, $T(2, 5)$ and $T(2, 7)$, the Riley relation is a polynomial in $t = \mathrm{tr}(ab)$ alone:

$$t = 1,\quad t^2 - t - 1 = 0,\quad t^3 - t^2 - 2t + 1 = 0,$$

whose roots are $t = 2\cos((2j - 1)\pi/n)$, all below 2. Then $v = (s - 1/s)^2 + (2 - t) > (s - 1/s)^2$, so the axes never cross, and $\mathrm{tr}[a, b] - 2 = v(2 - t) > 0$. On every real representation the commutator is a boost; for the trefoil, $\mathrm{tr}[a, b] = 2 + v$ with $v \ge 1$. **Lean: trefoil_T, torus5_T, torus7_T, v_from_T, trefoil_comm, torus_comm**; the root values are classical.

The reason is the centre of the torus-knot group, generated by $(ab)^n$. In an irreducible representation it maps to $\pm I$, which forces ab to be elliptic of finite order, so $\mathrm{tr}(ab) < 2$. The same reason gives the polynomial in t for every n; the Lean files check it for $n = 3, 5, 7$.

Census 8.8. The 26 knots, with the first x at which some real branch turns and, as $x \to \infty$, the limit of $k = 2 - \mathrm{tr}[a, b]$ on each branch where it stays positive (**computed**: 80 digits at the entry points, 200 digits for the limits):

knot	p/q	det	first x in band	limits of k = 2 − tr[a, b] in (0, 4)
T(2,3)	3/1	3	never	—
T(2,5)	5/1	5	never	—
4_1	5/3	5	0.485	1
T(2,7)	7/1	7	never	—
5_2	7/3	7	0.725	1, 2
6_1	9/5	9	0.350	φ^{-2}, 2, φ^2
6_2	11/3	11	0.360	1/4
7_2	11/5	11	0.605	φ^{-2}, 1, φ^2, 3
7_3	13/3	13	0.475	1/4, 0^+
6_3	13/5	13	never	—
8_1	13/7	13	0.285	$(2\cos 3\pi/7)^2$, 1, $(2\cos 2\pi/7)^2$, 3, $(2\cos \pi/7)^2$
7_4	15/11	15	0.485	1/2, 0^+
8_2	17/3	17	0.295	1/9
7_5	17/5	17	never	—
8_3	17/13	17	0.250	1/2, 0^+
8_4	19/5	19	0.265	$\varphi^{-2}/4$, $\varphi^2/4$, 0^+
7_6	19/7	19	0.555	1, 0^+
7_7	21/13	21	0.725	1, 0^+
8_7	23/5	23	never	—
8_6	23/7	23	0.270	0^+
8_9	25/7	25	0.285	1/4
8_8	25/9	25	never	—
8_{11}	27/17	27	0.350	1, 0^+
8_{13}	29/11	29	0.570	1, 0^+
8_{12}	29/17	29	0.305	1, 2, 0^+
8_{14}	31/13	31	0.390	1, 2, 0^+

Seven knots never turn: T(2, 3), T(2, 5), T(2, 7), 6_3, 7_5, 8_7 and 8_8. On every real branch of these knots the meridian axes are disjoint. Every knot that turns does so before x = 0.73, below the Schottky line, and still has a branch in the band at every larger x checked: every sample up to x = 6 for the eleven knots scanned in full, and in the limit $x \to \infty$ for all of them. The limit values are exact to the printed digits: 1, 2, 3, 1/2, 1/4, 1/9, φ^2, φ^{-2}, $\varphi^2/4$, $\varphi^{-2}/4$, and $(2\cos(j\pi/7))^2$ for j = 1, 2, 3. In every case $\sqrt{k_\infty} = 2\cos(j\pi/n)/m$, so the limiting commutator

trace $2 - k_\infty$ is built from cosines of rational angles (**computed** at 200 digits; why these values appear is open, possibly through the ideal points of the character variety). A branch marked 0^+ keeps a rotating commutator whose angle shrinks to 0. Knots enter in two ways: either one branch appears at the edge of the band with k near 0 (6_1, 8_1, 8_3, ...), or a complex pair of roots turns real with both branches already well inside it (5_2, 7_2, 7_6, 8_{13}, ...). The Alexander polynomial does not decide which: 6_3 has four roots off the unit circle and never enters, while 5_2 has none and does.

A note on precision. A double-precision scan of this census reported entries for 6_3, 7_5, 8_7 and 8_8 near $x \approx 1.3$ to 2.9, at $v \approx 12$ to 306. They were round-off errors: $2 - v((s - 1/s)^2 - v)$ subtracts two nearly equal numbers. Computed without cancellation, as $-v(\mathrm{tr}(ab) - 2)$ at 60 to 200 digits, k stays negative. On 8_8's branch with $\mathrm{tr}(ab) \to 2$, $k \cdot s^6 \to -1$ exactly. Every census entry above was recomputed at 80 digits.

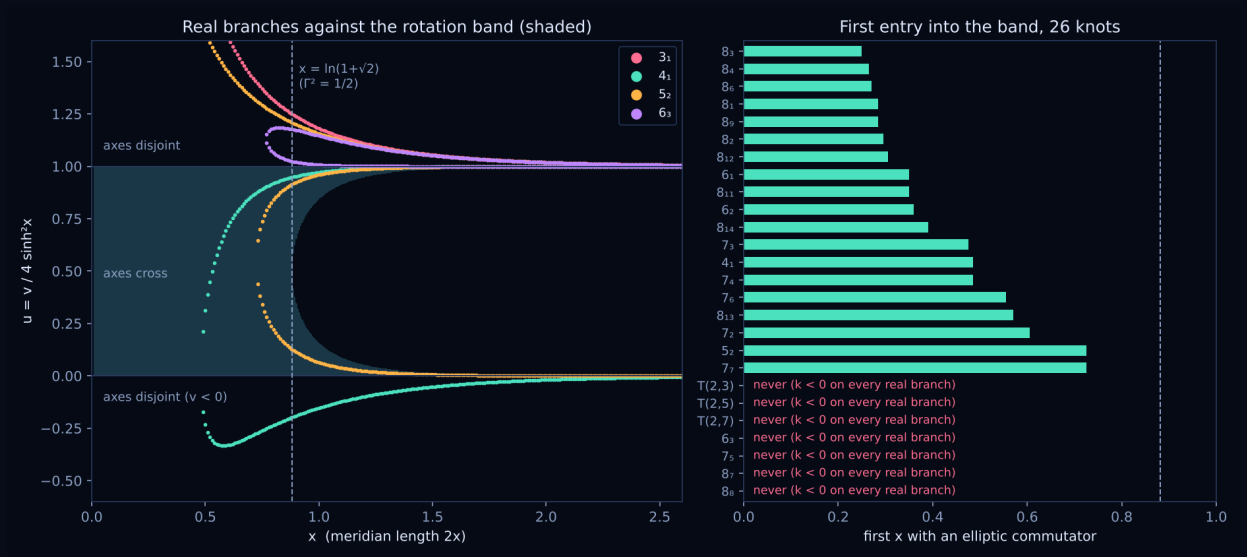


Figure 7. Left: real Riley branches of 3_1, 4_1, 5_2 and 6_3 plotted against $u = v/(4 \sinh^2 x)$. The axes cross for $0 < u < 1$, and the shaded region is the rotation band $0 < k < 4$. The trefoil stays above $u = 1$; 6_3 has branches only outside the band; 4_1 and 5_2 enter it before the dashed line $\Gamma^2 = 1/2$ and stay. Right: the first entry x for all 26 knots; the seven that never enter are marked.

9. What is open

- A Lean proof of Theorem 8.1. It needs the $(1 + i)$-adic valuation on $\mathbb{Z}[i]$; the six-set argument is finite and should formalise in the style of PingPong7.lean.
- Whether some non-backtracking word returns to the origin at a peel ratio that is a power of 2. Freeness does not rule this out, because the group is not discrete there; the search found none up to length 14.
- Which two-bridge knots never turn, in general. Seven of the 26 never do, including the torus knots, which are proved. What the other four (6_3, 7_5, 8_7, 8_8) share is not identified.
- Why the limits k_∞ are squares of numbers $2\cos(j\pi/n)/m$.
- Whether the geodesic ℓ is the whole zero set of the polygon area.
- The distance bound of Theorem 8.4 in Lean (it needs real numbers, so Mathlib).
- The Mathlib-based files of Appendix A.4 have still not been built; they are set up for independent checking.

References

1. Garrett (@Ebayednoob), post on the norm-5 quaternion tree in the Poincaré ball, X, 2026. https://x.com/Ebayednoob/status/2115981513409835368

2. J. Beasley, What Survives the Peel (VOTKP), 2026, https://doi.org/10.5281/zenodo.23011140: Theorem 8.1, Theorem 15.1, Corollary 15.2, §15.2. The exact parts (smith_identity, tanh_add', hyperbolic_length) are also proved with Mathlib in the repository dicipler-pixel/votkp-lean.
3. A. Lubotzky, R. Phillips, P. Sarnak, Ramanujan graphs, Combinatorica 8 (1988) 261–277.
4. C. Petit, K. Lauter, J.-J. Quisquater, Full cryptanalysis of LPS and Morgenstern hash functions, SCN 2008, LNCS 5229, 263–277 (IACR ePrint 2008/173).
5. A. F. Beardon, The Geometry of Discrete Groups, Springer, 1983 (Schottky groups and the ping-pong lemma).
6. L. H. Thomas, The motion of the spinning electron, Nature 117 (1926) 514.
7. A. A. Ungar, Analytic Hyperbolic Geometry and Albert Einstein's Special Theory of Relativity, World Scientific, 2008.
8. R. Riley, Nonabelian representations of 2-bridge knot groups, Quart. J. Math. 35 (1984) 191–208.
9. G. Burde, Darstellungen von Knotengruppen, Math. Ann. 173 (1967) 24–33; G. de Rham, Introduction aux polynômes d'un nœud, Enseign. Math. 13 (1967) 187–194.
10. H. Schubert, Knoten mit zwei Brücken, Math. Z. 65 (1956) 133–170.
11. D. Cooper, M. Culler, H. Gillet, D. D. Long, P. B. Shalen, Plane curves associated to character varieties of 3-manifolds, Invent. Math. 118 (1994) 47–84.
12. J. A. Rhodes, M. D. Semon, Relativistic velocity space, Wigner rotation and Thomas precession, arXiv:gr-qc/0501070, Eq. (142b).
13. I. I. Satija, Pythagorean triplets, integral Apollonians and the Hofstadter butterfly, arXiv:1802.04585, §2.1 (the Berggren matrices lie in O(2, 1; $\mathbb{Z}$)).
14. J. Beasley, Two Trees in One Ball v2: the Peel atlas synthesis, follow-up computations and a literature check, working report, 30 October 2026; dated record at github.com/dicipler-pixel/votkp-lean (peel-atlas, commit a1a0257).
15. C. McA. Gordon, Riley's conjecture on SL(2, $\mathbb{R}$) representations of 2-bridge knots, J. Knot Theory Ramifications 26 (2017) 1740003; arXiv:1602.02787. O. Thakar, Left-orderable surgeries on the knot 6_2 via hyperbolic PSL(2, $\mathbb{R}$)-representations, arXiv:2307.00107. M. Culler, N. M. Dunfield, Orderability and Dehn filling, Geom. Topol. 22 (2018) 1405–1457.
16. I. Niven, Irrational Numbers, Carus Math. Monographs 11, 1956 (rational values of cosine); A. F. Beardon [5] (a discrete group has no elliptic of infinite order); J. Gilman, arXiv:1207.1062, §7; W. M. Goldman, Geom. Topol. 7 (2003) 443–486, §3.1.

Appendix A. Lean

Five core Lean 4.34.1 files, no Mathlib, 159 theorems. Every finite statement is decided by the kernel (`decide`), never by `native_decide`; algebraic identities use `grind`, residue steps `omega`. A fourth set of files, built on Mathlib as a library only, is drafted for independent checking and has not been built.

A.1 PeelTrees.lean (74 theorems)

Run

```
lean PeelTrees.lean: exit code 0 (no messages), 55 s, 74 theorems, sorry/native_decide/axiom declarations: 0
```

Axiom audit

```
'PeelTrees.norm_mul' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees.peel_ladder' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees.free5' depends on axioms: [propext, Quot.sound]
'PeelTrees.free7' depends on axioms: [propext, Quot.sound]
'PeelTrees.no_return5' depends on axioms: [propext, Quot.sound]
'PeelTrees.time_not_div7' depends on axioms: [propext, Quot.sound]
'PeelTrees.straight_farthest5' depends on axioms: [propext]
'PeelTrees.thomas5' does not depend on any axioms
'PeelTrees.trefoil_longitude' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees.figure_eight_relation' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees.qwords_three_distinct' does not depend on any axioms
'PeelTrees.lps_six' depends on axioms: [propext]
```

False controls (each must be rejected)

```
-- deliberately false statements; each must be rejected
theorem c1 : (2:Rat)/3 * (2/3) > 1/2 := by grind
theorem c2 : (7813 : Int) % 5 = 0 := by decide
theorem c3 : (1:Int) + 1 = 3 := by decide

-- result: 3 of 3 rejected
controls.lean:2:43: error: `grind` failed
controls.lean:3:40: error: Tactic `decide` proved that the proposition
controls.lean:4:35: error: Tactic `decide` proved that the proposition
```

Source

```
/-
  PeelTrees.lean — Two Trees in One Ball: Norm Ladders, Peel Steps and Knot Boosts.
  Jeromie Beasley. Core Lean 4.34.1, no Mathlib, no sorry, no axioms beyond Lean's own.
  Every finite check is decided by the kernel (`decide`), never by compiled code (`native_decide`).

  Contents
    1. Lipschitz quaternions of norm 5 (Garrett's six generators, the LPS count, reduced words).
    2. The peel ladder: Gamma = (z-1)/(z+1), Einstein addition, Gamma_L = (5^L-1)/(5^L+1) for every L,
       the boost entries cosh = (1+G^2)/(1-G^2), sinh = 2G/(1-G^2), and the Schottky threshold G^2 > 1/2.
    3. Exact Lorentz boosts with rational entries (peel ratios 5 and 7, scaled to integers):
       Lorentz identity, inverse pairs, exact points, straight path farthest, Thomas rotation exact.
    4. Freeness for every word length: 5^L cosh d(w) is never divisible by 5 (and 7^L cosh d(w) never by 7),
       so no reduced word is the identity and no reduced word returns to the origin.
    5. Knot side: Riley meridians over any commutative ring, the trefoil and figure-eight relations
       factor through their Riley polynomials, and the trefoil longitude on its curve.
-/

namespace PeelTrees

set_option maxRecDepth 200000

/-! ## 1. Quaternions -/

structure Q where
  a : Int
  b : Int
  c : Int
  d : Int
deriving DecidableEq, Repr

def Q.mul (p q : Q) : Q :=
  ⟨p.a*q.a - p.b*q.b - p.c*q.c - p.d*q.d,
   p.a*q.b + p.b*q.a + p.c*q.d - p.d*q.c,
   p.a*q.c - p.b*q.d + p.c*q.a + p.d*q.b,
   p.a*q.d + p.b*q.c - p.c*q.b + p.d*q.a⟩

def Q.conj (p : Q) : Q := ⟨p.a, -p.b, -p.c, -p.d⟩
def Q.norm (p : Q) : Int := p.a*p.a + p.b*p.b + p.c*p.c + p.d*p.d

/-- Euler's four-square identity: the norm is multiplicative, so a word of L generators has norm 5^L. -/
theorem norm_mul (p q : Q) : (p.mul q).norm = p.norm * q.norm := by
  cases p; cases q; simp only [Q.mul, Q.norm]; grind

/-- Garrett's six generators 1 ± 2i, 1 ± 2j, 1 ± 2k. -/
```

```
def gens : List Q := [⟨1,2,0,0⟩, ⟨1,-2,0,0⟩, ⟨1,0,2,0⟩, ⟨1,0,-2,0⟩, ⟨1,0,0,2⟩, ⟨1,0,0,-2⟩]

theorem gens_norm : ∀ g ∈ gens, g.norm = 5 := by decide

/-- The backward step is the conjugate: g times its conjugate is the scalar 5. -/
theorem gens_conj : ∀ g ∈ gens, g.mul g.conj = ⟨5,0,0,0⟩ := by decide

/-- The conjugate of each generator is again a generator (the tree is closed under stepping back). -/
theorem gens_conj_mem : ∀ g ∈ gens, g.conj ∈ gens := by decide

def rng : List Int := [-2, -1, 0, 1, 2]
def box : List Q := rng.flatMap fun a => rng.flatMap fun b => rng.flatMap fun c => rng.map fun d => ⟨a,b,c,d⟩

theorem sq_le_five (x : Int) (h : x * x ≤ 5) : -2 ≤ x ∧ x ≤ 2 := by
  constructor
  · apply Classical.byContradiction; intro hc
    have h3 : 3 ≤ -x := by omega
    have : 3 * 3 ≤ (-x) * (-x) := Int.mul_le_mul h3 h3 (by omega) (by omega)
    rw [Int.neg_mul_neg] at this; omega
  · apply Classical.byContradiction; intro hc
    have h3 : 3 ≤ x := by omega
    have : 3 * 3 ≤ x * x := Int.mul_le_mul h3 h3 (by omega) (by omega)
    omega

theorem sq_nonneg' (x : Int) : 0 ≤ x * x := by
  rcases Int.le_total 0 x with h | h
  · exact Int.mul_nonneg h h
  · exact Int.mul_nonneg_of_nonpos_of_nonpos h h

/-- Every coordinate of a norm-5 quaternion lies in [-2, 2], so `box` holds all of them. -/
theorem norm5_bound (q : Q) (h : q.norm = 5) :
    (-2 ≤ q.a ∧ q.a ≤ 2) ∧ (-2 ≤ q.b ∧ q.b ≤ 2) ∧ (-2 ≤ q.c ∧ q.c ≤ 2) ∧ (-2 ≤ q.d ∧ q.d ≤ 2) := by
  unfold Q.norm at h
  have ha := sq_nonneg' q.a; have hb := sq_nonneg' q.b
  have hc := sq_nonneg' q.c; have hd := sq_nonneg' q.d
  exact ⟨sq_le_five _ (by omega), sq_le_five _ (by omega), sq_le_five _ (by omega), sq_le_five _ (by omega)⟩

/-- There are exactly 8 (5 + 1) = 48 integer quaternions of norm 5. -/
theorem norm5_count : (box.filter (fun q => q.norm == 5)).length = 48 := by decide

/-- Of the 48, exactly six have a odd and positive with b, c, d even, and they are Garrett's generators
    (the Lubotzky–Phillips–Sarnak normalisation, p + 1 = 6). -/
theorem lps_six : ∀ q ∈ box,
    (q.norm = 5 ∧ 0 < q.a ∧ q.a % 2 = 1 ∧ q.b % 2 = 0 ∧ q.c % 2 = 0 ∧ q.d % 2 = 0) ↔ q ∈ gens := by decide

/-- Reduced words: never a generator followed by its own conjugate. -/
def qwords : Nat → List (List Q)
  | 0 => [[]]
  | n + 1 => (qwords n).flatMap fun w => (gens.filter fun g => w.head?.map Q.conj != some g).map fun g => g :: w

def qprod (w : List Q) : Q := w.foldr Q.mul ⟨1,0,0,0⟩
def qsign (q : Q) : Q := if q.a < 0 || (q.a == 0 && (q.b < 0 || (q.b == 0 && (q.c < 0 || (q.c == 0 && q.d < 0))))) then (-
q.a,-q.b,-q.c,-q.d⟩ else q

theorem qwords_three_count : (qwords 3).length = 150 := by decide

/-- The 150 reduced words of length 3 give 150 different quaternions, even up to sign. -/
theorem qwords_three_distinct : ((qwords 3).map fun w => qsign (qprod w)).Nodup := by decide

/-- None of them is divisible by 5, so none collapses to a shorter word times a scalar. -/
theorem qwords_three_primitive : ∀ w ∈ qwords 3, ¬ (qprod w).a % 5 = 0 ∨ ¬ (qprod w).b % 5 = 0 ∨
    ¬ (qprod w).c % 5 = 0 ∨ ¬ (qprod w).d % 5 = 0 := by decide

/-! ## 2. The peel ladder (VOTKP Theorem 8.1, Theorem 15.1, Corollary 15.2) -/

/-- Smith (peel) coordinate of a peel ratio z. -/
def gam (z : Rat) : Rat := (z - 1) / (z + 1)
/-- Einstein (rapidity) addition on the Smith disk. -/
def oplus (x y : Rat) : Rat := (x + y) / (1 + x * y)

/-- Steps compose by multiplying ratios: Gamma(A) ⊕ Gamma(B) = Gamma(AB). -/
theorem gam_mul (A B : Rat) (hA : 0 ≤ A) (hB : 0 ≤ B) : oplus (gam A) (gam B) = gam (A * B) := by
  unfold oplus gam
  have h1 : A + 1 ≠ 0 := by grind
  have h2 : B + 1 ≠ 0 := by grind
  have h0 : 0 ≤ A * B := Rat.mul_nonneg hA hB
  have h3 : A * B + 1 ≠ 0 := by grind
  grind

theorem gam_five : gam 5 = 2 / 3 := by unfold gam; grind
theorem gam_seven : gam 7 = 3 / 4 := by unfold gam; grind

/-- The peel walk: L steps of 2/3 combined by ⊕. -/
def peel : Nat → Rat
  | 0 => 0
  | n + 1 => oplus (peel n) (2 / 3)
```

```
theorem five_pow_nonneg (n : Nat) : (0 : Rat) ≤ 5 ^ n := by
  induction n with
  | zero => grind
  | succ n ih => rw [Rat.pow_succ]; exact Rat.mul_nonneg ih (by grind)

/-- The norm ladder is the peel ladder: after L steps the Smith coordinate is (5^L - 1)/(5^L + 1),
    for every L. -/
theorem peel_ladder (n : Nat) : peel n = gam (5 ^ n) := by
  induction n with
  | zero => show (0 : Rat) = (1 - 1) / (1 + 1); grind
  | succ n ih =>
    simp only [peel]
    rw [ih, ← gam_five, gam_mul _ _ (five_pow_nonneg n) (by grind), Rat.pow_succ]

/-- The boost entries in the Smith coordinate: cosh = (1+G^2)/(1-G^2) = (z^2+1)/(2z) and
    sinh = 2G/(1-G^2) = (z^2-1)/(2z). -/
theorem cosh_from_gam (z : Rat) (hz : 0 < z) :
    (1 + gam z * gam z) / (1 - gam z * gam z) = (z * z + 1) / (2 * z) := by
  unfold gam
  have h1 : z + 1 ≠ 0 := by grind
  have h2 : z ≠ 0 := by grind
  grind

theorem sinh_from_gam (z : Rat) (hz : 0 < z) :
    (2 * gam z) / (1 - gam z * gam z) = (z * z - 1) / (2 * z) := by
  unfold gam
  have h1 : z + 1 ≠ 0 := by grind
  have h2 : z ≠ 0 := by grind
  grind

/-- Schottky threshold in the peel coordinate: G^2 > 1/2. The norm-5 step sits below it,
    Garrett's ln 7 step above it. -/
theorem schottky_five_below : gam 5 * gam 5 < 1 / 2 := by rw [gam_five]; grind
theorem schottky_seven_above : 1 / 2 < gam 7 * gam 7 := by rw [gam_seven]; grind
/-- The same threshold for the ratio itself: z > 3 + 2 sqrt 2, i.e. (z - 3)^2 > 8 with z > 3. -/
theorem schottky_ratio : (5 - 3) * (5 - 3) < (8 : Int) ∧ (8 : Int) < (7 - 3) * (7 - 3) := by decide

/-! ## 3. Exact boosts. A boost of peel ratio z, multiplied by z, has integer entries
    c = (z^2+1)/2, s = (z^2-1)/2 on the axis and z on the other two space directions. -/

structure M4 where
  a00 : Int
  a01 : Int
  a02 : Int
  a03 : Int
  a10 : Int
  a11 : Int
  a12 : Int
  a13 : Int
  a20 : Int
  a21 : Int
  a22 : Int
  a23 : Int
  a30 : Int
  a31 : Int
  a32 : Int
  a33 : Int
deriving DecidableEq, Repr

def M4.mul (A B : M4) : M4 :=
  ⟨A.a00 * B.a00 + A.a01 * B.a10 + A.a02 * B.a20 + A.a03 * B.a30,
   A.a00 * B.a01 + A.a01 * B.a11 + A.a02 * B.a21 + A.a03 * B.a31,
   A.a00 * B.a02 + A.a01 * B.a12 + A.a02 * B.a22 + A.a03 * B.a32,
   A.a00 * B.a03 + A.a01 * B.a13 + A.a02 * B.a23 + A.a03 * B.a33,
   A.a10 * B.a00 + A.a11 * B.a10 + A.a12 * B.a20 + A.a13 * B.a30,
   A.a10 * B.a01 + A.a11 * B.a11 + A.a12 * B.a21 + A.a13 * B.a31,
   A.a10 * B.a02 + A.a11 * B.a12 + A.a12 * B.a22 + A.a13 * B.a32,
   A.a10 * B.a03 + A.a11 * B.a13 + A.a12 * B.a23 + A.a13 * B.a33,
   A.a20 * B.a00 + A.a21 * B.a10 + A.a22 * B.a20 + A.a23 * B.a30,
   A.a20 * B.a01 + A.a21 * B.a11 + A.a22 * B.a21 + A.a23 * B.a31,
   A.a20 * B.a02 + A.a21 * B.a12 + A.a22 * B.a22 + A.a23 * B.a32,
   A.a20 * B.a03 + A.a21 * B.a13 + A.a22 * B.a23 + A.a23 * B.a33,
   A.a30 * B.a00 + A.a31 * B.a10 + A.a32 * B.a20 + A.a33 * B.a30,
   A.a30 * B.a01 + A.a31 * B.a11 + A.a32 * B.a21 + A.a33 * B.a31,
   A.a30 * B.a02 + A.a31 * B.a12 + A.a32 * B.a22 + A.a33 * B.a32,
   A.a30 * B.a03 + A.a31 * B.a13 + A.a32 * B.a23 + A.a33 * B.a33⟩

def M4.tr (A : M4) : M4 := ⟨A.a00, A.a10, A.a20, A.a30, A.a01, A.a11, A.a21, A.a31, A.a02, A.a12, A.a22, A.a32, A.a03,
A.a13, A.a23, A.a33⟩
def M4.smul (c : Int) (A : M4) : M4 := ⟨c * A.a00, c * A.a01, c * A.a02, c * A.a03, c * A.a10, c * A.a11, c * A.a12, c *
A.a13, c * A.a20, c * A.a21, c * A.a22, c * A.a23, c * A.a30, c * A.a31, c * A.a32, c * A.a33⟩
def M4.red (m : Int) (A : M4) : M4 := ⟨A.a00 % m, A.a01 % m, A.a02 % m, A.a03 % m, A.a10 % m, A.a11 % m, A.a12 % m, A.a13 %
m, A.a20 % m, A.a21 % m, A.a22 % m, A.a23 % m, A.a30 % m, A.a31 % m, A.a32 % m, A.a33 % m⟩
def I4 : M4 := ⟨1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1⟩
def eta : M4 := ⟨1, 0, 0, 0, 0, -1, 0, 0, 0, 0, -1, 0, 0, 0, 0, -1⟩
```

```
/-- The six step directions of Garrett's tree: +i, -i, +j, -j, +k, -k. -/
inductive Dir | pi | mi | pj | mj | pk | mk
deriving DecidableEq, Repr

def Dir.inv : Dir → Dir
  | .pi => .mi | .mi => .pi | .pj => .mj | .mj => .pj | .pk => .mk | .mk => .pk

def allDirs : List Dir := [.pi, .mi, .pj, .mj, .pk, .mk]
theorem mem_allDirs (d : Dir) : d ∈ allDirs := by cases d <;> decide

/-- z times the boost of peel ratio z along direction d: c = (z^2+1)/2, s = (z^2-1)/2, z elsewhere. -/
def boost (c s z : Int) : Dir → M4
  | .pi => ⟨c, s, 0, 0, s, c, 0, 0, 0, 0, z, 0, 0, 0, 0, z⟩
  | .mi => ⟨c, (-s), 0, 0, (-s), c, 0, 0, 0, 0, z, 0, 0, 0, 0, z⟩
  | .pj => ⟨c, 0, s, 0, 0, z, 0, 0, s, 0, c, 0, 0, 0, 0, z⟩
  | .mj => ⟨c, 0, (-s), 0, 0, z, 0, 0, (-s), 0, c, 0, 0, 0, 0, z⟩
  | .pk => ⟨c, 0, 0, s, 0, z, 0, 0, 0, 0, z, 0, s, 0, 0, c⟩
  | .mk => ⟨c, 0, 0, (-s), 0, z, 0, 0, 0, 0, z, 0, (-s), 0, 0, c⟩

/-- Peel ratio 5 (rapidity ln 5, Smith step 2/3) and peel ratio 7 (rapidity ln 7, Garrett's drawing). -/
def B5 : Dir → M4 := boost 13 12 5
def B7 : Dir → M4 := boost 25 24 7

/-- Every scaled boost is Lorentz: B^T eta B = z^2 eta. -/
theorem lorentz5 : ∀ d ∈ allDirs, (M4.mul (M4.tr (B5 d)) (M4.mul eta (B5 d))) = M4.smul 25 eta := by decide
theorem lorentz7 : ∀ d ∈ allDirs, (M4.mul (M4.tr (B7 d)) (M4.mul eta (B7 d))) = M4.smul 49 eta := by decide

/-- The backward step is the inverse boost: B_d B_{-d} = z^2 I. -/
theorem inverse5 : ∀ d ∈ allDirs, M4.mul (B5 d) (B5 d.inv) = M4.smul 25 I4 := by decide
theorem inverse7 : ∀ d ∈ allDirs, M4.mul (B7 d) (B7 d.inv) = M4.smul 49 I4 := by decide

/-- A pure boost is a symmetric matrix. -/
theorem boost5_symm : ∀ d ∈ allDirs, M4.tr (B5 d) = B5 d := by decide

/-- The scaled matrix of a word (applied left to right, starting at the identity). -/
def word (B : Dir → M4) (w : List Dir) : M4 := w.foldl (fun M d => M4.mul M (B d)) I4

/-- Its first column is z^L times the point reached, (cosh d, sinh d * direction). -/
def pt (M : M4) : List Int := [M.a00, M.a10, M.a20, M.a30]

/-- Straight path +i +i +i: z^3 cosh d = 7813 = (5^6 + 1)/2, so d = 3 ln 5 exactly (lengths add). -/
theorem straight5 : pt (word B5 [.pi, .pi, .pi]) = [7813, 7812, 0, 0] := by decide
theorem straight5_is_three_ln5 : 2 * 7813 = 5 ^ 6 + 1 ∧ 2 * 7812 = 5 ^ 6 - 1 := by decide
/-- Two orthogonal steps: cosh d = cosh^2 (ln 5) = 169/25 (hyperbolic Pythagoras). -/
theorem turn5 : pt (word B5 [.pi, .pj]) = [169, 156, 60, 0] := by decide
/-- The turned paths of the earlier check. -/
theorem turned5 : pt (word B5 [.pi, .pi, .pj]) = [4069, 4056, 300, 0] ∧
    pt (word B5 [.pi, .pj, .pi]) = [2917, 2808, 780, 0] ∧
    pt (word B5 [.pi, .pj, .pk]) = [2197, 2028, 780, 300] := by decide
/-- Garrett's ln 7 numbers: straight 3 ln 7, and the word +i +j +i with cosh d = 19657/343 (d = 4.7415). -/
theorem garrett7 : pt (word B7 [.pi, .pi, .pi]) = [58825, 58824, 0, 0] ∧
    pt (word B7 [.pi, .pj, .pi]) = [19657, 19200, 4200, 0] ∧
    pt (word B7 [.pi, .pi, .pj]) = [30025, 30000, 1176, 0] ∧
    pt (word B7 [.pi, .pj, .pk]) = [15625, 15000, 4200, 1176] := by decide

/-- Reduced words of length n (no step followed by its own reverse). -/
def rwords : Nat → List (List Dir)
  | 0 => [[]]
  | n + 1 => (rwords n).flatMap fun w => (allDirs.filter fun d => w.getLast?.map Dir.inv != some d).map fun d => w ++ [d]

theorem rwords_three_count : (rwords 3).length = 150 := by decide

/-- Among all 150 reduced words of length 3, the straight paths are the farthest: every other word
    has strictly smaller cosh d. Turning always shortens. -/
theorem straight_farthest5 : ∀ w ∈ rwords 3, (word B5 w).a00 ≤ 7813 ∧
    ((word B5 w).a00 = 7813 ↔ w ∈ [[.pi,.pi,.pi],[.mi,.mi,.mi],[.pj,.pj,.pj],[.mj,.mj,.mj],[.pk,.pk,.pk],[.mk,.mk,.mk]]) :=
by decide

/-- Thomas rotation, exact (peel ratio 5): B_i B_j = L R with L a pure boost (symmetric, Lorentz)
    and R a rotation about k with cos = 65/97, sin = 72/97.  Scaled: Lz * Rz = 9409 * (B_i B_j). -/
def Lz5 : M4 := ⟨16393, 15132, 5820, 0, 15132, 14593, 4680, 0, 5820, 4680, 4225, 0, 0, 0, 0, 2425⟩
def Rz5 : M4 := ⟨97, 0, 0, 0, 0, 65, 72, 0, 0, -72, 65, 0, 0, 0, 0, 97⟩
theorem thomas5 : M4.mul Lz5 Rz5 = M4.smul 9409 (M4.mul (B5 .pi) (B5 .pj)) := by decide
theorem thomas5_boost : M4.tr Lz5 = Lz5 ∧ M4.mul (M4.tr Lz5) (M4.mul eta Lz5) = M4.smul 5880625 eta := by decide
theorem thomas5_rotation : M4.mul (M4.tr Rz5) Rz5 = M4.smul 9409 I4 ∧ pt Rz5 = [97, 0, 0, 0] := by decide
/-- So the product of two boosts is not a boost: it is not symmetric. -/
theorem two_boosts_not_boost5 : M4.tr (M4.mul (B5 .pi) (B5 .pj)) ≠ M4.mul (B5 .pi) (B5 .pj) := by decide

/-- Thomas rotation, exact (peel ratio 7): B_i B_j = L R with L a pure boost (symmetric, Lorentz)
    and R a rotation about k with cos = 175/337, sin = 288/337.  Scaled: Lz * Rz = 113569 * (B_i B_j). -/
def Lz7 : M4 := ⟨210625, 202200, 56616, 0, 202200, 196513, 50400, 0, 56616, 50400, 30625, 0, 0, 0, 0, 16513⟩
def Rz7 : M4 := ⟨337, 0, 0, 0, 0, 175, 288, 0, 0, -288, 175, 0, 0, 0, 0, 337⟩
theorem thomas7 : M4.mul Lz7 Rz7 = M4.smul 113569 (M4.mul (B7 .pi) (B7 .pj)) := by decide
theorem thomas7_boost : M4.tr Lz7 = Lz7 ∧ M4.mul (M4.tr Lz7) (M4.mul eta Lz7) = M4.smul 272679169 eta := by decide
theorem thomas7_rotation : M4.mul (M4.tr Rz7) Rz7 = M4.smul 113569 I4 ∧ pt Rz7 = [337, 0, 0, 0] := by decide
```

```
/-- So the product of two boosts is not a boost: it is not symmetric. -/
theorem two_boosts_not_boost7 : M4.tr (M4.mul (B7 .pi) (B7 .pj)) ≠ M4.mul (B7 .pi) (B7 .pj) := by decide

/-- Thomas half-angle in the peel coordinate: tan(theta/2) = sin/(1 + cos) = Gamma^2. -/
theorem thomas_half_angle : (72 : Int) * 9 = 4 * (97 + 65) ∧ (288 : Int) * 16 = 9 * (337 + 175) := by decide

/-! ## 4. Freeness for every length. Mod p (p = 5 or 7) each scaled boost is c * w w^T with w = (1, -sign, ...)
    a rank-one matrix; a product of rank-one matrices is rank one with a factor that is a product of
    dot products w_g . w_h, which is 2 (same step) or 1 (turn), and 0 only for a backtrack. -/

def wv : Dir → List Int
  | .pi => [1,-1,0,0] | .mi => [1,1,0,0] | .pj => [1,0,-1,0] | .mj => [1,0,1,0] | .pk => [1,0,0,-1] | .mk => [1,0,0,1]

def outer (m : Int) (c : Int) (f g : Dir) : M4 :=
  match wv f, wv g with
  | [f0,f1,f2,f3], [g0,g1,g2,g3] => M4.red m (c * f0 * g0, c * f0 * g1, c * f0 * g2, c * f0 * g3, c * f1 * g0, c * f1 * g1,
c * f1 * g2, c * f1 * g3, c * f2 * g0, c * f2 * g1, c * f2 * g2, c * f2 * g3, c * f3 * g0, c * f3 * g1, c * f3 * g2, c * f3
* g3)
  | _, _ => I4

/-- Reduction mod m respects products (entrywise: sums and products commute with taking residues). -/

theorem dot_emod5 (a1 b1 a2 b2 a3 b3 a4 b4 : Int) :
    (a1*b1 + a2*b2 + a3*b3 + a4*b4) % 5 = ((a1%5)*(b1%5) + (a2%5)*(b2%5) + (a3%5)*(b3%5) + (a4%5)*(b4%5)) % 5 := by
  have h1 := Int.mul_emod a1 b1 5; have h2 := Int.mul_emod a2 b2 5
  have h3 := Int.mul_emod a3 b3 5; have h4 := Int.mul_emod a4 b4 5
  omega

theorem red_mul5 (A B : M4) : M4.red 5 (M4.mul A B) = M4.red 5 (M4.mul (M4.red 5 A) (M4.red 5 B)) := by
  simp only [M4.red, M4.mul]
  simp only [M4.mk.injEq]
  refine ⟨?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_⟩ <;> apply dot_emod5

theorem dot_emod7 (a1 b1 a2 b2 a3 b3 a4 b4 : Int) :
    (a1*b1 + a2*b2 + a3*b3 + a4*b4) % 7 = ((a1%7)*(b1%7) + (a2%7)*(b2%7) + (a3%7)*(b3%7) + (a4%7)*(b4%7)) % 7 := by
  have h1 := Int.mul_emod a1 b1 7; have h2 := Int.mul_emod a2 b2 7
  have h3 := Int.mul_emod a3 b3 7; have h4 := Int.mul_emod a4 b4 7
  omega

theorem red_mul7 (A B : M4) : M4.red 7 (M4.mul A B) = M4.red 7 (M4.mul (M4.red 7 A) (M4.red 7 B)) := by
  simp only [M4.red, M4.mul]
  simp only [M4.mk.injEq]
  refine ⟨?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_⟩ <;> apply dot_emod7

/-- Invariant for peel ratio 5: mod 5 the word matrix is c * w_f w_g^T with c a nonzero residue,
    f the first step and g the last step. -/
def Inv5 (M : M4) (g : Dir) : Prop := ∃ c ∈ ([1,2,3,4] : List Int), ∃ f ∈ allDirs, M4.red 5 M = outer 5 c f g

theorem gen_red5 : ∀ d ∈ allDirs, M4.red 5 (B5 d) = outer 5 3 d d := by decide

/-- The finite closure step: rank-one state times a non-backtracking step stays rank one, nonzero. -/
theorem closure5 : ∀ c ∈ ([1,2,3,4] : List Int), ∀ f ∈ allDirs, ∀ g ∈ allDirs, ∀ h ∈ allDirs, h ≠ g.inv →
    ∃ c' ∈ ([1,2,3,4] : List Int), M4.red 5 (M4.mul (outer 5 c f g) (outer 5 3 h h)) = outer 5 c' f h := by decide

theorem outer_red5 : ∀ c ∈ ([1,2,3,4] : List Int), ∀ f ∈ allDirs, ∀ g ∈ allDirs, M4.red 5 (outer 5 c f g) = outer 5 c f g :=
by decide

theorem step5 (M : M4) (g h : Dir) (hM : Inv5 M g) (hh : h ≠ g.inv) : Inv5 (M4.mul M (B5 h)) h := by
  obtain ⟨c, hc, f, hf, hred⟩ := hM
  obtain ⟨c', hc', hcl⟩ := closure5 c hc f hf g (mem_allDirs g) h (mem_allDirs h) hh
  refine ⟨c', hc', f, hf, ?_⟩
  rw [red_mul5, hred, gen_red5 h (mem_allDirs h), hcl]

/-- Non-backtracking from g. -/
def RedFrom : Dir → List Dir → Prop
  | _, [] => True
  | g, h :: t => h ≠ g.inv ∧ RedFrom h t

def lastStep : Dir → List Dir → Dir
  | g, [] => g
  | _, h :: t => lastStep h t

theorem walk5 (l : List Dir) : ∀ (M : M4) (g : Dir), Inv5 M g → RedFrom g l →
    Inv5 (l.foldl (fun M d => M4.mul M (B5 d)) M) (lastStep g l) := by
  induction l with
  | nil => intro M g h _; exact h
  | cons h t ih =>
    intro M g hM hr
    exact ih _ h (step5 M g h hM hr.1) hr.2

theorem corner5 : ∀ c ∈ ([1,2,3,4] : List Int), ∀ f ∈ allDirs, ∀ g ∈ allDirs, (outer 5 c f g).a00 ≠ 0 := by decide

theorem mul_I4 (A : M4) : M4.mul I4 A = A := by
  cases A; simp only [M4.mul, I4]; simp

/-- MAIN (peel ratio 5): for every non-backtracking word of every length L >= 1, the scaled time
```

```
    coordinate 5^L cosh d(w) is not divisible by 5. -/
theorem time_not_div5 (g : Dir) (l : List Dir) (hr : RedFrom g l) :
    (word B5 (g :: l)).a00 % 5 ≠ 0 := by
  have h0 : Inv5 (B5 g) g := ⟨3, by decide, g, mem_allDirs g, gen_red5 g (mem_allDirs g)⟩
  have hw := walk5 l (B5 g) g h0 hr
  obtain ⟨c, hc, f, hf, hred⟩ := hw
  have hword : word B5 (g :: l) = l.foldl (fun M d => M4.mul M (B5 d)) (B5 g) := by
    simp only [word, List.foldl_cons, mul_I4]
  rw [hword]
  have h1 := congrArg M4.a00 hred
  simp only [M4.red] at h1
  rw [h1]
  exact corner5 c hc f hf _ (mem_allDirs _)

theorem pow_emod5 (n : Nat) : ((5 : Int) ^ (n + 1)) % 5 = 0 := by
  rw [Int.pow_succ]; exact Int.mul_emod_left _ _

/-- No non-backtracking word returns to the origin: its time coordinate is never 5^L (cosh d = 1). -/
theorem no_return5 (g : Dir) (l : List Dir) (hr : RedFrom g l) :
    (word B5 (g :: l)).a00 ≠ 5 ^ (l.length + 1) := by
  intro h; have := time_not_div5 g l hr; rw [h, pow_emod5] at this; exact this rfl

/-- No non-backtracking word is the identity: the boost group is free on the three steps. -/
theorem free5 (g : Dir) (l : List Dir) (hr : RedFrom g l) :
    word B5 (g :: l) ≠ M4.smul (5 ^ (l.length + 1)) I4 := by
  intro h; apply no_return5 g l hr; rw [h]; simp [M4.smul, I4]

/-- Invariant for peel ratio 7: mod 7 the word matrix is c * w_f w_g^T with c a nonzero residue,
    f the first step and g the last step. -/
def Inv7 (M : M4) (g : Dir) : Prop := ∃ c ∈ ([1,2,3,4,5,6] : List Int), ∃ f ∈ allDirs, M4.red 7 M = outer 7 c f g

theorem gen_red7 : ∀ d ∈ allDirs, M4.red 7 (B7 d) = outer 7 4 d d := by decide

/-- The finite closure step: rank-one state times a non-backtracking step stays rank one, nonzero. -/
theorem closure7 : ∀ c ∈ ([1,2,3,4,5,6] : List Int), ∀ f ∈ allDirs, ∀ g ∈ allDirs, ∀ h ∈ allDirs, h ≠ g.inv →
    ∃ c' ∈ ([1,2,3,4,5,6] : List Int), M4.red 7 (M4.mul (outer 7 c f g) (outer 7 4 h h)) = outer 7 c' f h := by decide

theorem outer_red7 : ∀ c ∈ ([1,2,3,4,5,6] : List Int), ∀ f ∈ allDirs, ∀ g ∈ allDirs, M4.red 7 (outer 7 c f g) = outer 7 c f
g := by decide

theorem step7 (M : M4) (g h : Dir) (hM : Inv7 M g) (hh : h ≠ g.inv) : Inv7 (M4.mul M (B7 h)) h := by
  obtain ⟨c, hc, f, hf, hred⟩ := hM
  obtain ⟨c', hc', hcl⟩ := closure7 c hc f hf g (mem_allDirs g) h (mem_allDirs h) hh
  refine ⟨c', hc', f, hf, ?_⟩
  rw [red_mul7, hred, gen_red7 h (mem_allDirs h), hcl]

theorem walk7 (l : List Dir) : ∀ (M : M4) (g : Dir), Inv7 M g → RedFrom g l →
    Inv7 (l.foldl (fun M d => M4.mul M (B7 d)) M) (lastStep g l) := by
  induction l with
  | nil => intro M g h _; exact h
  | cons h t ih =>
    intro M g hM hr
    exact ih _ h (step7 M g h hM hr.1) hr.2

theorem corner7 : ∀ c ∈ ([1,2,3,4,5,6] : List Int), ∀ f ∈ allDirs, ∀ g ∈ allDirs, (outer 7 c f g).a00 ≠ 0 := by decide

/-- MAIN (peel ratio 7): for every non-backtracking word of every length L >= 1, the scaled time
    coordinate 7^L cosh d(w) is not divisible by 7. -/
theorem time_not_div7 (g : Dir) (l : List Dir) (hr : RedFrom g l) :
    (word B7 (g :: l)).a00 % 7 ≠ 0 := by
  have h0 : Inv7 (B7 g) g := ⟨4, by decide, g, mem_allDirs g, gen_red7 g (mem_allDirs g)⟩
  have hw := walk7 l (B7 g) g h0 hr
  obtain ⟨c, hc, f, hf, hred⟩ := hw
  have hword : word B7 (g :: l) = l.foldl (fun M d => M4.mul M (B7 d)) (B7 g) := by
    simp only [word, List.foldl_cons, mul_I4]
  rw [hword]
  have h1 := congrArg M4.a00 hred
  simp only [M4.red] at h1
  rw [h1]
  exact corner7 c hc f hf _ (mem_allDirs _)

theorem pow_emod7 (n : Nat) : ((7 : Int) ^ (n + 1)) % 7 = 0 := by
  rw [Int.pow_succ]; exact Int.mul_emod_left _ _

/-- No non-backtracking word returns to the origin: its time coordinate is never 7^L (cosh d = 1). -/
theorem no_return7 (g : Dir) (l : List Dir) (hr : RedFrom g l) :
    (word B7 (g :: l)).a00 ≠ 7 ^ (l.length + 1) := by
  intro h; have := time_not_div7 g l hr; rw [h, pow_emod7] at this; exact this rfl

/-- No non-backtracking word is the identity: the boost group is free on the three steps. -/
theorem free7 (g : Dir) (l : List Dir) (hr : RedFrom g l) :
    word B7 (g :: l) ≠ M4.smul (7 ^ (l.length + 1)) I4 := by
  intro h; apply no_return7 g l hr; rw [h]; simp [M4.smul, I4]

/-! ## 5. The knot side. Riley meridians a = [[s,1],[0,1/s]], b = [[s,0],[-v,1/s]], scaled by s so that
    every entry is a polynomial: A = s a, B = s b, A' = s a^-1, B' = s b^-1. The statements hold over
```

```
    every commutative ring, in particular for real s and v on the real peripheral curve. -/

structure M2 (R : Type) where
  a : R
  b : R
  c : R
  d : R

variable {R : Type} [Lean.Grind.CommRing R]

def M2.mul (X Y : M2 R) : M2 R := ⟨X.a*Y.a + X.b*Y.c, X.a*Y.b + X.b*Y.d, X.c*Y.a + X.d*Y.c, X.c*Y.b + X.d*Y.d⟩
def M2.sub (X Y : M2 R) : M2 R := ⟨X.a - Y.a, X.b - Y.b, X.c - Y.c, X.d - Y.d⟩

def rA (s : R) : M2 R := ⟨s*s, s, 0, 1⟩
def rB (s v : R) : M2 R := ⟨s*s, 0, -(v*s), 1⟩
def rAi (s : R) : M2 R := ⟨1, -s, 0, s*s⟩
def rBi (s v : R) : M2 R := ⟨1, 0, v*s, s*s⟩

/-- The scaled inverses are inverses: A A' = s^2 I and B B' = s^2 I. -/
theorem riley_inverses (s v : R) :
    (M2.mul (rA s) (rAi s)) = ⟨s*s, 0, 0, s*s⟩ ∧ (M2.mul (rB s v) (rBi s v)) = ⟨s*s, 0, 0, s*s⟩ := by
  simp only [M2.mul, rA, rAi, rB, rBi, M2.mk.injEq]; refine ⟨⟨?_,?_,?_,?_⟩,⟨?_,?_,?_,?_⟩⟩ <;> grind

/-- Each meridian has determinant 1 and trace s + 1/s (scaled: det s^2, trace s^2 + 1), and
    trace^2 - 4 det = (s^2 - 1)^2: for real s ≠ ±1 it is a boost of translation length 2 ln|s|. -/
theorem meridian_boost (s v : R) :
    (rA s).a * (rA s).d - (rA s).b * (rA s).c = s*s ∧
    (rB s v).a * (rB s v).d - (rB s v).b * (rB s v).c = s*s ∧
    ((rA s).a + (rA s).d) * ((rA s).a + (rA s).d) - 4 * (s*s) = (s*s - 1) * (s*s - 1) ∧
    (rB s v).a + (rB s v).d = (rA s).a + (rA s).d := by
  simp only [rA, rB]; refine ⟨?_,?_,?_,?_⟩ <;> grind

/-- Trefoil (2-bridge 3/1, W = b a): A W - W B = R3(s,v) * [[0, s], [s v, 0]],
    R3 = s^4 - s^2 v - s^2 + 1. The relation holds exactly on the Riley curve R3 = 0. -/
theorem trefoil_relation (s v : R) :
    let W := M2.mul (rB s v) (rA s)
    let D := M2.sub (M2.mul (rA s) W) (M2.mul W (rB s v))
    let R3 := s*s*s*s - s*s*v - s*s + 1
    D.a = 0 ∧ D.b = R3 * s ∧ D.c = R3 * (s * v) ∧ D.d = 0 := by
  simp only [M2.mul, M2.sub, rA, rB]; refine ⟨?_,?_,?_,?_⟩ <;> grind

/-- Figure-eight (2-bridge 5/3, W = b a^-1 b^-1 a): A W - W B = R4(s,v) * [[0, s^3], [s^3 v, 0]],
    R4 = s^2 v^2 + (3 s^2 - s^4 - 1) v + (3 s^2 - s^4 - 1). -/
theorem figure_eight_relation (s v : R) :
    let W := M2.mul (M2.mul (M2.mul (rB s v) (rAi s)) (rBi s v)) (rA s)
    let D := M2.sub (M2.mul (rA s) W) (M2.mul W (rB s v))
    let R4 := s*s*v*v + (3*s*s - s*s*s*s - 1)*v + (3*s*s - s*s*s*s - 1)
    D.a = 0 ∧ D.b = R4 * (s*s*s) ∧ D.c = R4 * (s*s*s*v) ∧ D.d = 0 := by
  simp only [M2.mul, M2.sub, rA, rB, rAi, rBi]; refine ⟨?_,?_,?_,?_⟩ <;> grind

/-- Trefoil longitude on its curve: with R3 = 0 the scaled longitude B A A B A'^4 = s^8 L is upper
    triangular with diagonal -s^2, -s^14, i.e. L has eigenvalues -s^-6, -s^6: a boost of length 12 ln s = 2|y|
    with y = -6x. -/
theorem trefoil_longitude (s v : R) (h : s*s*s*s - s*s*v - s*s + 1 = 0) :
    let A4 := M2.mul (M2.mul (rAi s) (rAi s)) (M2.mul (rAi s) (rAi s))
    let Lg := M2.mul (M2.mul (M2.mul (M2.mul (rB s v) (rA s)) (rA s)) (rB s v)) A4
    Lg.c = 0 ∧ Lg.a = -(s*s) ∧ Lg.d = -(s*s*s*s*s*s*s*s*s*s*s*s*s*s*s*s) := by
  simp only [M2.mul, rA, rB, rAi]; refine ⟨?_,?_,?_⟩ <;> grind

end PeelTrees
```

A.2 PeelTrees2.lean (49 theorems): every ratio not a power of 2, Thomas for every ratio

Run

```
lean PeelTrees2.lean: exit code 0 (no messages), 25 s, 49 theorems (rerun 2026-10-30)
```

Axiom audit

```
'PeelTrees2.word_form_general' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.time_mod_general' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.time_not_dvd_general' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.no_return_general' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.free_general' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.free_not_pow_two' depends on axioms: [propext, Classical.choice, Quot.sound]
```

```
'PeelTrees2.free3' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.free6' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.odd_not_dvd_pow_two' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.exists_odd_divisor' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.Q_split' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.W_dot' depends on axioms: [propext]
'PeelTrees2.ring_step' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.Q_lorentz' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.Q_inverse' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.thomas_general' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.thomas_boost_general' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.thomas_rotation_general' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.thomas_triple_int' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.thomas_t_cleared' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.thomas_matches_five_seven' does not depend on any axioms
'PeelTrees2.thomas_triple' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.thomas_rat' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.thomas_cos' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.thomas_sin' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.thomas_half_angle_t' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.thomas_unit_t' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.fig8_riley_c' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.fig8_disc' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.oplus_self' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees2.oplus_two_thirds' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Source

```
/-
  PeelTrees2.lean — Two Trees in One Ball, second file: the general peel ratio.
  Jeromie Beasley. Core Lean 4.34.1, no Mathlib, no sorry, no axioms beyond Lean's own,
  no native_decide. Standalone: the definitions it shares with PeelTrees.lean are repeated here.

  Contents
    A. Freeness for every peel ratio z with an odd divisor m > 1 (every z >= 2 that is not a power
       of 2), for every word length. With Q_u = 2z B_u (integer entries z^2+1, ±(z^2-1), 2z) one has
       Q_u = W_u + z E_u exactly over Int, W_u = w_u w_u^T. By induction over non-backtracking words,
       the word product equals 2^a W_{first,last} + z E. So (2z)^L cosh d(w) ≡ 2^a (mod m) for every
       m | z, which is not 0 when m is odd and m > 1. Hence no non-backtracking word returns to the
       origin and none is the identity.
    B. The Thomas rotation of two perpendicular steps for symbolic z: L R = B_i B_j with L a pure boost
       and R a rotation about k with cos = (1-t^2)/(1+t^2) = 4z(z^2+1)/(z^4+6z^2+1),
       sin = 2t/(1+t^2) = (z^2-1)^2/(z^4+6z^2+1), t = Gamma^2, Gamma = (z-1)/(z+1);
       the Pythagorean triple (1-t^2)^2 + (2t)^2 = (1+t^2)^2; tan(theta/2) = Gamma^2.
    C. Small exact facts: the figure-eight Riley polynomial in the variable c, its discriminant,
       and the Smith double step Gamma ⊕ Gamma = 2 Gamma/(1 + Gamma^2), 2/3 ⊕ 2/3 = 12/13.
-/

namespace PeelTrees2

set_option maxRecDepth 200000

/-! ## Integer 4 x 4 matrices -/

structure M4 where
  a00 : Int
  a01 : Int
  a02 : Int
  a03 : Int
  a10 : Int
  a11 : Int
  a12 : Int
  a13 : Int
  a20 : Int
  a21 : Int
  a22 : Int
  a23 : Int
  a30 : Int
  a31 : Int
  a32 : Int
  a33 : Int
deriving DecidableEq, Repr

def M4.mul (A B : M4) : M4 :=
  ⟨A.a00 * B.a00 + A.a01 * B.a10 + A.a02 * B.a20 + A.a03 * B.a30,
   A.a00 * B.a01 + A.a01 * B.a11 + A.a02 * B.a21 + A.a03 * B.a31,
   A.a00 * B.a02 + A.a01 * B.a12 + A.a02 * B.a22 + A.a03 * B.a32,
   A.a00 * B.a03 + A.a01 * B.a13 + A.a02 * B.a23 + A.a03 * B.a33,
   A.a10 * B.a00 + A.a11 * B.a10 + A.a12 * B.a20 + A.a13 * B.a30,
   A.a10 * B.a01 + A.a11 * B.a11 + A.a12 * B.a21 + A.a13 * B.a31,
   A.a10 * B.a02 + A.a11 * B.a12 + A.a12 * B.a22 + A.a13 * B.a32,
   A.a10 * B.a03 + A.a11 * B.a13 + A.a12 * B.a23 + A.a13 * B.a33,
   A.a20 * B.a00 + A.a21 * B.a10 + A.a22 * B.a20 + A.a23 * B.a30,
```

```
   A.a20 * B.a01 + A.a21 * B.a11 + A.a22 * B.a21 + A.a23 * B.a31,
   A.a20 * B.a02 + A.a21 * B.a12 + A.a22 * B.a22 + A.a23 * B.a32,
   A.a20 * B.a03 + A.a21 * B.a13 + A.a22 * B.a23 + A.a23 * B.a33,
   A.a30 * B.a00 + A.a31 * B.a10 + A.a32 * B.a20 + A.a33 * B.a30,
   A.a30 * B.a01 + A.a31 * B.a11 + A.a32 * B.a21 + A.a33 * B.a31,
   A.a30 * B.a02 + A.a31 * B.a12 + A.a32 * B.a22 + A.a33 * B.a32,
   A.a30 * B.a03 + A.a31 * B.a13 + A.a32 * B.a23 + A.a33 * B.a33⟩

def M4.add (A B : M4) : M4 :=
  ⟨A.a00 + B.a00, A.a01 + B.a01, A.a02 + B.a02, A.a03 + B.a03,
   A.a10 + B.a10, A.a11 + B.a11, A.a12 + B.a12, A.a13 + B.a13,
   A.a20 + B.a20, A.a21 + B.a21, A.a22 + B.a22, A.a23 + B.a23,
   A.a30 + B.a30, A.a31 + B.a31, A.a32 + B.a32, A.a33 + B.a33⟩

def M4.tr (A : M4) : M4 := ⟨A.a00, A.a10, A.a20, A.a30, A.a01, A.a11, A.a21, A.a31, A.a02, A.a12, A.a22, A.a32, A.a03,
A.a13, A.a23, A.a33⟩
def M4.smul (c : Int) (A : M4) : M4 := ⟨c * A.a00, c * A.a01, c * A.a02, c * A.a03, c * A.a10, c * A.a11, c * A.a12, c *
A.a13, c * A.a20, c * A.a21, c * A.a22, c * A.a23, c * A.a30, c * A.a31, c * A.a32, c * A.a33⟩
def I4 : M4 := ⟨1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1⟩
def eta : M4 := ⟨1, 0, 0, 0, 0, -1, 0, 0, 0, 0, -1, 0, 0, 0, 0, -1⟩

theorem mul_I4 (A : M4) : M4.mul I4 A = A := by
  cases A; simp only [M4.mul, I4]; simp

theorem smul_one (A : M4) : M4.smul 1 A = A := by
  cases A; simp only [M4.smul]; simp

theorem smul_smul (c d : Int) (A : M4) : M4.smul c (M4.smul d A) = M4.smul (c * d) A := by
  cases A; simp only [M4.smul, M4.mk.injEq]
  refine ⟨?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_⟩ <;> grind

/-- The one ring identity behind the induction:
    (c W + z E)(W' + z E') = c (W W') + z (c W E' + E W' + z E E'). -/
theorem ring_step (c z : Int) (W E W' E' : M4) :
    M4.mul (M4.add (M4.smul c W) (M4.smul z E)) (M4.add W' (M4.smul z E')) =
    M4.add (M4.smul c (M4.mul W W'))
      (M4.smul z (M4.add (M4.add (M4.smul c (M4.mul W E')) (M4.mul E W')) (M4.smul z (M4.mul E E')))) := by
  cases W; cases E; cases W'; cases E'
  simp only [M4.mul, M4.add, M4.smul, M4.mk.injEq]
  refine ⟨?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_⟩ <;> grind

/-! ## Step directions and the rank-one pieces -/

/-- The six step directions of Garrett's tree: +i, -i, +j, -j, +k, -k. -/
inductive Dir | pi | mi | pj | mj | pk | mk
deriving DecidableEq, Repr

def Dir.inv : Dir → Dir
  | .pi => .mi | .mi => .pi | .pj => .mj | .mj => .pj | .pk => .mk | .mk => .pk

def allDirs : List Dir := [.pi, .mi, .pj, .mj, .pk, .mk]
theorem mem_allDirs (d : Dir) : d ∈ allDirs := by cases d <;> decide

/-- The null vectors w_{±i} = (1, ∓1, 0, 0), w_{±j} = (1, 0, ∓1, 0), w_{±k} = (1, 0, 0, ∓1). -/
def w0 : Dir → Int := fun _ => 1
def w1 : Dir → Int | .pi => -1 | .mi => 1 | _ => 0
def w2 : Dir → Int | .pj => -1 | .mj => 1 | _ => 0
def w3 : Dir → Int | .pk => -1 | .mk => 1 | _ => 0

/-- W_{f,g} = w_f w_g^T. -/
def Wm (f g : Dir) : M4 :=
  ⟨w0 f * w0 g, w0 f * w1 g, w0 f * w2 g, w0 f * w3 g,
   w1 f * w0 g, w1 f * w1 g, w1 f * w2 g, w1 f * w3 g,
   w2 f * w0 g, w2 f * w1 g, w2 f * w2 g, w2 f * w3 g,
   w3 f * w0 g, w3 f * w1 g, w3 f * w2 g, w3 f * w3 g⟩

/-- Dot products of the null vectors: 2 for the same step, 0 for the backtrack, 1 for a turn. -/
def dotw (g h : Dir) : Int := w0 g * w0 h + w1 g * w1 h + w2 g * w2 h + w3 g * w3 h

theorem dotw_values : ∀ g ∈ allDirs, ∀ h ∈ allDirs,
    dotw g h = (if h = g then 2 else if h = g.inv then 0 else 1) := by decide

/-- The finite check: W_{f,g} W_{h,h} = (w_g . w_h) W_{f,h}, and for a non-backtracking step the
    factor is 2 (straight on) or 1 (turn). -/
theorem W_dot : ∀ f ∈ allDirs, ∀ g ∈ allDirs, ∀ h ∈ allDirs, h ≠ g.inv →
    M4.mul (Wm f g) (Wm h h) = M4.smul (if h = g then 2 else 1) (Wm f h) := by decide

theorem W_corner (f g : Dir) : (Wm f g).a00 = 1 := by
  cases f <;> cases g <;> rfl

/-! ## The scaled boosts Q_u = 2z B_u for symbolic z -/

/-- 2z times the boost of rapidity ln z along u: z^2+1 on the time and axis diagonal,
    ±(z^2-1) off the diagonal there, 2z on the other two spatial diagonals. -/
def Qz (z : Int) : Dir → M4
  | .pi => ⟨z*z+1, z*z-1, 0, 0, z*z-1, z*z+1, 0, 0, 0, 0, 2*z, 0, 0, 0, 0, 2*z⟩
```

```
  | .mi => ⟨z*z+1, -(z*z-1), 0, 0, -(z*z-1), z*z+1, 0, 0, 0, 0, 2*z, 0, 0, 0, 0, 2*z⟩
  | .pj => ⟨z*z+1, 0, z*z-1, 0, 0, 2*z, 0, 0, z*z-1, 0, z*z+1, 0, 0, 0, 0, 2*z⟩
  | .mj => ⟨z*z+1, 0, -(z*z-1), 0, 0, 2*z, 0, 0, -(z*z-1), 0, z*z+1, 0, 0, 0, 0, 2*z⟩
  | .pk => ⟨z*z+1, 0, 0, z*z-1, 0, 2*z, 0, 0, 0, 0, 2*z, 0, z*z-1, 0, 0, z*z+1⟩
  | .mk => ⟨z*z+1, 0, 0, -(z*z-1), 0, 2*z, 0, 0, 0, 0, 2*z, 0, -(z*z-1), 0, 0, z*z+1⟩

/-- The z-part: Q_u = W_u + z E_u. -/
def Ez (z : Int) : Dir → M4
  | .pi => ⟨z, z, 0, 0, z, z, 0, 0, 0, 0, 2, 0, 0, 0, 0, 2⟩
  | .mi => ⟨z, -z, 0, 0, -z, z, 0, 0, 0, 0, 2, 0, 0, 0, 0, 2⟩
  | .pj => ⟨z, 0, z, 0, 0, 2, 0, 0, z, 0, z, 0, 0, 0, 0, 2⟩
  | .mj => ⟨z, 0, -z, 0, 0, 2, 0, 0, -z, 0, z, 0, 0, 0, 0, 2⟩
  | .pk => ⟨z, 0, 0, z, 0, 2, 0, 0, 0, 0, 2, 0, z, 0, 0, z⟩
  | .mk => ⟨z, 0, 0, -z, 0, 2, 0, 0, 0, 0, 2, 0, -z, 0, 0, z⟩

theorem Q_split (z : Int) (u : Dir) : Qz z u = M4.add (Wm u u) (M4.smul z (Ez z u)) := by
  cases u <;> simp only [Qz, Ez, Wm, M4.add, M4.smul, w0, w1, w2, w3, M4.mk.injEq] <;>
    refine ⟨?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_⟩ <;> grind

/-- Each Q_u is Lorentz up to the scale (2z)^2: Q^T eta Q = (2z)^2 eta, for every z. -/
theorem Q_lorentz (z : Int) (u : Dir) :
    M4.mul (M4.tr (Qz z u)) (M4.mul eta (Qz z u)) = M4.smul ((2*z)*(2*z)) eta := by
  cases u <;> simp only [Qz, eta, M4.mul, M4.tr, M4.smul, M4.mk.injEq] <;>
    refine ⟨?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_⟩ <;> grind

/-- The backward step is the inverse: Q_u Q_{-u} = (2z)^2 I, for every z. -/
theorem Q_inverse (z : Int) (u : Dir) :
    M4.mul (Qz z u) (Qz z u.inv) = M4.smul ((2*z)*(2*z)) I4 := by
  cases u <;> simp only [Qz, Dir.inv, I4, M4.mul, M4.smul, M4.mk.injEq] <;>
    refine ⟨?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_⟩ <;> grind

/-! ## The induction over non-backtracking words -/

/-- The scaled matrix of a word (applied left to right, starting at the identity). -/
def word (B : Dir → M4) (w : List Dir) : M4 := w.foldl (fun M d => M4.mul M (B d)) I4

/-- Non-backtracking from g. -/
def RedFrom : Dir → List Dir → Prop
  | _, [] => True
  | g, h :: t => h ≠ g.inv ∧ RedFrom h t

def lastStep : Dir → List Dir → Dir
  | g, [] => g
  | _, h :: t => lastStep h t

/-- The invariant: M = 2^a W_{f,g} + z E, with g the last step. -/
def InvZ (z : Int) (M : M4) (g : Dir) : Prop :=
  ∃ a : Nat, ∃ f : Dir, ∃ E : M4, M = M4.add (M4.smul ((2 : Int) ^ a) (Wm f g)) (M4.smul z E)

theorem invZ_of (z c : Int) (b : Nat) (hb : c = (2 : Int) ^ b) (f h : Dir) (X : M4) :
    InvZ z (M4.add (M4.smul c (Wm f h)) (M4.smul z X)) h := ⟨b, f, X, by rw [hb]⟩

theorem stepZ (z : Int) (M : M4) (g h : Dir) (hM : InvZ z M g) (hh : h ≠ g.inv) :
    InvZ z (M4.mul M (Qz z h)) h := by
  obtain ⟨a, f, E, rfl⟩ := hM
  rw [Q_split, ring_step, W_dot f (mem_allDirs f) g (mem_allDirs g) h (mem_allDirs h) hh, smul_smul]
  by_cases hg : h = g
  · exact invZ_of z _ (a + 1) (by rw [ite_eq_left hg, Int.pow_succ]) f h _
  · exact invZ_of z _ a (by rw [ite_eq_right hg, Int.mul_one]) f h _

theorem walkZ (z : Int) (l : List Dir) : ∀ (M : M4) (g : Dir), InvZ z M g → RedFrom g l →
    InvZ z (l.foldl (fun M d => M4.mul M (Qz z d)) M) (lastStep g l) := by
  induction l with
  | nil => intro M g h _; exact h
  | cons h t ih =>
    intro M g hM hr
    exact ih _ h (stepZ z M g h hM hr.1) hr.2

/-- MAIN FORM: for every integer z and every non-backtracking word g :: l, the product of the scaled
    boosts is 2^a W_{f, last} + z E for some a, some first-step vector f and some integer matrix E. -/
theorem word_form_general (z : Int) (g : Dir) (l : List Dir) (hr : RedFrom g l) :
    ∃ a : Nat, ∃ f : Dir, ∃ E : M4,
      word (Qz z) (g :: l) = M4.add (M4.smul ((2 : Int) ^ a) (Wm f (lastStep g l))) (M4.smul z E) := by
  have h0 : InvZ z (Qz z g) g := ⟨0, g, Ez z g, by rw [Int.pow_zero, smul_one]; exact Q_split z g⟩
  have hword : word (Qz z) (g :: l) = l.foldl (fun M d => M4.mul M (Qz z d)) (Qz z g) := by
    simp only [word, List.foldl_cons, mul_I4]
  rw [hword]
  exact walkZ z l (Qz z g) g h0 hr

/-- The time entry: (2z)^L cosh d(w) = 2^a + z e for integers a >= 0, e. -/
theorem time_entry_general (z : Int) (g : Dir) (l : List Dir) (hr : RedFrom g l) :
    ∃ a : Nat, ∃ e : Int, (word (Qz z) (g :: l)).a00 = (2 : Int) ^ a + z * e := by
  obtain ⟨a, f, E, hE⟩ := word_form_general z g l hr
  refine ⟨a, E.a00, ?_⟩
  rw [hE]; simp only [M4.add, M4.smul, W_corner, Int.mul_one]
```

```
/-- So (2z)^L cosh d(w) ≡ 2^a (mod m) for every divisor m of z. -/
theorem time_mod_general (z m : Int) (hm : m ∣ z) (g : Dir) (l : List Dir) (hr : RedFrom g l) :
    ∃ a : Nat, m ∣ (word (Qz z) (g :: l)).a00 - (2 : Int) ^ a := by
  obtain ⟨a, e, he⟩ := time_entry_general z g l hr
  refine ⟨a, ?_⟩
  rw [he]
  have : (2 : Int) ^ a + z * e - (2 : Int) ^ a = z * e := by omega
  rw [this]
  exact Int.dvd_trans hm ⟨e, rfl⟩

/-- An odd m dividing 2x divides x. -/
theorem odd_dvd_of_dvd_two_mul (m x : Int) (hodd : m % 2 = 1) (h : m ∣ 2 * x) : m ∣ x := by
  obtain ⟨k, hk⟩ := h
  have hm : m = 2 * (m / 2) + 1 := by omega
  generalize m / 2 = j at hm
  refine ⟨x - j * k, ?_⟩
  subst hm
  grind

/-- An odd m > 1 never divides a power of 2. -/
theorem odd_not_dvd_pow_two (m : Int) (hodd : m % 2 = 1) (h1 : 1 < m) : ∀ a : Nat, ¬ m ∣ (2 : Int) ^ a := by
  intro a
  induction a with
  | zero =>
    intro h
    rw [Int.pow_zero] at h
    have := Int.le_of_dvd (by decide) h
    omega
  | succ n ih =>
    intro h
    rw [Int.pow_succ, Int.mul_comm] at h
    exact ih (odd_dvd_of_dvd_two_mul m _ hodd h)

/-- MAIN (every z with an odd divisor m > 1): for every non-backtracking word of every length L >= 1,
    the scaled time coordinate (2z)^L cosh d(w) is not divisible by m. -/
theorem time_not_dvd_general (z m : Int) (hm : m ∣ z) (hodd : m % 2 = 1) (h1 : 1 < m)
    (g : Dir) (l : List Dir) (hr : RedFrom g l) : ¬ m ∣ (word (Qz z) (g :: l)).a00 := by
  intro hdiv
  obtain ⟨a, ha⟩ := time_mod_general z m hm g l hr
  have h2 := Int.dvd_sub hdiv ha
  have : (word (Qz z) (g :: l)).a00 - ((word (Qz z) (g :: l)).a00 - (2 : Int) ^ a) = (2 : Int) ^ a := by omega
  rw [this] at h2
  exact odd_not_dvd_pow_two m hodd h1 a h2

theorem dvd_two_z_pow (z m : Int) (hm : m ∣ z) (n : Nat) : m ∣ (2 * z) ^ (n + 1) := by
  refine Int.dvd_trans hm ⟨(2 * z) ^ n * 2, ?_⟩
  rw [Int.pow_succ]
  grind

/-- No non-backtracking word returns to the origin: its time coordinate is never (2z)^L (cosh d = 1). -/
theorem no_return_general (z m : Int) (hm : m ∣ z) (hodd : m % 2 = 1) (h1 : 1 < m)
    (g : Dir) (l : List Dir) (hr : RedFrom g l) :
    (word (Qz z) (g :: l)).a00 ≠ (2 * z) ^ (l.length + 1) := by
  intro h
  apply time_not_dvd_general z m hm hodd h1 g l hr
  rw [h]
  exact dvd_two_z_pow z m hm l.length

/-- No non-backtracking word is the identity: the three boosts of ratio z generate a free group. -/
theorem free_general (z m : Int) (hm : m ∣ z) (hodd : m % 2 = 1) (h1 : 1 < m)
    (g : Dir) (l : List Dir) (hr : RedFrom g l) :
    word (Qz z) (g :: l) ≠ M4.smul ((2 * z) ^ (l.length + 1)) I4 := by
  intro h
  apply no_return_general z m hm hodd h1 g l hr
  rw [h]; simp [M4.smul, I4]

/-- Every n >= 1 that is not a power of 2 has an odd divisor m > 1. -/
theorem exists_odd_divisor (n : Nat) : 0 < n → (∀ k : Nat, n ≠ 2 ^ k) →
    ∃ m : Nat, m ∣ n ∧ m % 2 = 1 ∧ 1 < m := by
  induction n using Nat.strongRecOn with
  | ind n ih =>
    intro hn hp
    by_cases hpar : n % 2 = 1
    · refine ⟨n, Nat.dvd_refl n, hpar, ?_⟩
      have : n ≠ 1 := by have := hp 0; simpa using this
      omega
    · have hlt : n / 2 < n := by omega
      have hpos : 0 < n / 2 := by
        apply Classical.byContradiction; intro hc
        have : n = 1 := by omega
        exact hp 0 (by simp [this])
      have hp' : ∀ k : Nat, n / 2 ≠ 2 ^ k := by
        intro k hk; apply hp (k + 1); rw [Nat.pow_succ]; omega
      obtain ⟨m, hmd, hmo, hm1⟩ := ih (n / 2) hlt hpos hp'
      exact ⟨m, Nat.dvd_trans hmd ⟨2, by omega⟩, hmo, hm1⟩
```

```
/-- FREENESS FOR EVERY PEEL RATIO z >= 2 THAT IS NOT A POWER OF 2: no non-backtracking word returns
    to the origin and no non-backtracking word is the identity. -/
theorem free_not_pow_two (z : Nat) (hz : 2 ≤ z) (hp : ∀ k : Nat, z ≠ 2 ^ k)
    (g : Dir) (l : List Dir) (hr : RedFrom g l) :
    (word (Qz (z : Int)) (g :: l)).a00 ≠ (2 * (z : Int)) ^ (l.length + 1) ∧
    word (Qz (z : Int)) (g :: l) ≠ M4.smul ((2 * (z : Int)) ^ (l.length + 1)) I4 := by
  obtain ⟨m, hmd, hmo, hm1⟩ := exists_odd_divisor z (by omega) hp
  have hmd' : (m : Int) ∣ (z : Int) := Int.natCast_dvd_natCast.mpr hmd
  have hmo' : (m : Int) % 2 = 1 := by omega
  have hm1' : 1 < (m : Int) := by omega
  exact ⟨no_return_general _ _ hmd' hmo' hm1' g l hr, free_general _ _ hmd' hmo' hm1' g l hr⟩

/-- Instances below the Schottky threshold z > 3 + 2 sqrt 2: peel ratios 3 and 5, and ratio 6 (odd divisor 3),
    are free for every word length. Powers of 2 (z = 2, 4, 8, ...) are not covered by this argument. -/
theorem free3 (g : Dir) (l : List Dir) (hr : RedFrom g l) :
    word (Qz 3) (g :: l) ≠ M4.smul ((2 * 3) ^ (l.length + 1)) I4 :=
  free_general 3 3 (by decide) (by decide) (by decide) g l hr

theorem free6 (g : Dir) (l : List Dir) (hr : RedFrom g l) :
    word (Qz 6) (g :: l) ≠ M4.smul ((2 * 6) ^ (l.length + 1)) I4 :=
  free_general 6 3 (by decide) (by decide) (by decide) g l hr

/-- Consistency with PeelTrees.lean: Q_u at z = 5 and z = 7 is twice the boosts B5, B7 used there. -/
def boost (c s z : Int) : Dir → M4
  | .pi => ⟨c, s, 0, 0, s, c, 0, 0, 0, 0, z, 0, 0, 0, 0, z⟩
  | .mi => ⟨c, (-s), 0, 0, (-s), c, 0, 0, 0, 0, z, 0, 0, 0, 0, z⟩
  | .pj => ⟨c, 0, s, 0, 0, z, 0, 0, s, 0, c, 0, 0, 0, 0, z⟩
  | .mj => ⟨c, 0, (-s), 0, 0, z, 0, 0, (-s), 0, c, 0, 0, 0, 0, z⟩
  | .pk => ⟨c, 0, 0, s, 0, z, 0, 0, 0, 0, z, 0, s, 0, 0, c⟩
  | .mk => ⟨c, 0, 0, (-s), 0, z, 0, 0, 0, 0, z, 0, (-s), 0, 0, c⟩

theorem Q_five_seven : ∀ d ∈ allDirs, Qz 5 d = M4.smul 2 (boost 13 12 5 d) ∧ Qz 7 d = M4.smul 2 (boost 25 24 7 d) := by
  decide

/-! ## B. The Thomas rotation for symbolic z -/

def Dz (z : Int) : Int := z*z*z*z + 6*z*z + 1

/-- 4 z^2 D times the pure boost L to the point B_i B_j o. -/
def Lint (z : Int) : M4 :=
  ⟨(z*z+1)*(z*z+1)*Dz z, (z*z-1)*(z*z+1)*Dz z, 2*z*(z*z-1)*Dz z, 0,
   (z*z-1)*(z*z+1)*Dz z, 4*z*z*Dz z + (z*z-1)*(z*z-1)*((z*z+1)*(z*z+1)), 2*z*(z*z-1)*(z*z-1)*(z*z+1), 0,
   2*z*(z*z-1)*Dz z, 2*z*(z*z-1)*(z*z-1)*(z*z+1), 4*z*z*Dz z + 4*z*z*(z*z-1)*(z*z-1), 0,
   0, 0, 0, 4*z*z*Dz z⟩

/-- D times the rotation about k with cos = 4z(z^2+1)/D, sin = (z^2-1)^2/D. -/
def Rint (z : Int) : M4 :=
  ⟨Dz z, 0, 0, 0,
   0, 4*z*(z*z+1), (z*z-1)*(z*z-1), 0,
   0, -((z*z-1)*(z*z-1)), 4*z*(z*z+1), 0,
   0, 0, 0, Dz z⟩

/-- THOMAS, symbolic z, cleared of denominators: L R = D^2 Q_i Q_j, i.e. B_i B_j = (L/(4z^2 D)) (R/D). -/
theorem thomas_general (z : Int) :
    M4.mul (Lint z) (Rint z) = M4.smul (Dz z * Dz z) (M4.mul (Qz z .pi) (Qz z .pj)) := by
  simp only [Lint, Rint, Qz, Dz, M4.mul, M4.smul, M4.mk.injEq]
  refine ⟨?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_⟩ <;> grind

/-- L is a pure boost: symmetric and Lorentz, L^T eta L = (4 z^2 D)^2 eta. -/
theorem thomas_boost_general (z : Int) :
    M4.tr (Lint z) = Lint z ∧
    M4.mul (M4.tr (Lint z)) (M4.mul eta (Lint z)) = M4.smul ((4*z*z*Dz z) * (4*z*z*Dz z)) eta := by
  constructor
  · simp only [Lint, M4.tr]
  · simp only [Lint, Dz, eta, M4.mul, M4.tr, M4.smul, M4.mk.injEq]
    refine ⟨?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_⟩ <;> grind

/-- R is a rotation fixing time: R^T R = D^2 I and R o = D o. -/
theorem thomas_rotation_general (z : Int) :
    M4.mul (M4.tr (Rint z)) (Rint z) = M4.smul (Dz z * Dz z) I4 ∧
    (Rint z).a00 = Dz z ∧ (Rint z).a10 = 0 ∧ (Rint z).a20 = 0 ∧ (Rint z).a30 = 0 := by
  refine ⟨?_, rfl, rfl, rfl, rfl⟩
  simp only [Rint, Dz, I4, M4.mul, M4.tr, M4.smul, M4.mk.injEq]
  refine ⟨?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_⟩ <;> grind

/-- The integer Pythagorean triple of the Thomas angle: (4z(z^2+1))^2 + ((z^2-1)^2)^2 = D^2. -/
theorem thomas_triple_int (z : Int) :
    (4*z*(z*z+1)) * (4*z*(z*z+1)) + ((z*z-1)*(z*z-1)) * ((z*z-1)*(z*z-1)) = Dz z * Dz z := by
  simp only [Dz]; grind

/-- Clearing t = Gamma^2 = (z-1)^2/(z+1)^2 by (z+1)^4: 1 - t^2, 2t, 1 + t^2 become
    8z(z^2+1), 2(z^2-1)^2, 2D. -/
theorem thomas_t_cleared (z : Int) :
    (z+1)*(z+1)*((z+1)*(z+1)) - (z-1)*(z-1)*((z-1)*(z-1)) = 8*z*(z*z+1) ∧
    2*((z-1)*(z-1))*((z+1)*(z+1)) = 2*((z*z-1)*(z*z-1)) ∧
    (z+1)*(z+1)*((z+1)*(z+1)) + (z-1)*(z-1)*((z-1)*(z-1)) = 2 * Dz z := by
```

```
  simp only [Dz]; refine ⟨?_, ?_, ?_⟩ <;> grind

/-- At z = 5 and z = 7 these are 32 and 8 times the exact matrices of PeelTrees.lean (thomas5, thomas7). -/
def Lz5 : M4 := ⟨16393, 15132, 5820, 0, 15132, 14593, 4680, 0, 5820, 4680, 4225, 0, 0, 0, 0, 2425⟩
def Rz5 : M4 := ⟨97, 0, 0, 0, 0, 65, 72, 0, 0, -72, 65, 0, 0, 0, 0, 97⟩
def Lz7 : M4 := ⟨210625, 202200, 56616, 0, 202200, 196513, 50400, 0, 56616, 50400, 30625, 0, 0, 0, 0, 16513⟩
def Rz7 : M4 := ⟨337, 0, 0, 0, 0, 175, 288, 0, 0, -288, 175, 0, 0, 0, 0, 337⟩
theorem thomas_matches_five_seven :
    Lint 5 = M4.smul 32 Lz5 ∧ Rint 5 = M4.smul 8 Rz5 ∧ Lint 7 = M4.smul 32 Lz7 ∧ Rint 7 = M4.smul 8 Rz7 := by
  decide

section Generic
variable {R : Type} [Lean.Grind.CommRing R]

/-- (1 - t^2, 2t, 1 + t^2) is a Pythagorean triple, over any commutative ring. -/
theorem thomas_triple (t : R) : (1 - t*t)*(1 - t*t) + (2*t)*(2*t) = (1 + t*t)*(1 + t*t) := by grind

/-! ## C. Small exact facts -/

/-- Figure-eight: with c s^2 = s^4 - 3 s^2 + 1, the Riley polynomial is s^2 (v^2 - c v - c). -/
theorem fig8_riley_c (s v c : R) (h : c*(s*s) = s*s*s*s - 3*(s*s) + 1) :
    s*s*(v*v - c*v - c) = s*s*v*v + (3*s*s - s*s*s*s - 1)*v + (3*s*s - s*s*s*s - 1) := by grind

/-- The discriminant of v^2 - c v - c is c^2 + 4c = c(c + 4). -/
theorem fig8_disc (c : R) : (-c)*(-c) - 4*(-c) = c*c + 4*c ∧ c*c + 4*c = c*(c + 4) := by
  constructor <;> grind

end Generic

/-! ## B over Rat: the identity with the rational entries themselves -/

structure MQ where
  a00 : Rat
  a01 : Rat
  a02 : Rat
  a03 : Rat
  a10 : Rat
  a11 : Rat
  a12 : Rat
  a13 : Rat
  a20 : Rat
  a21 : Rat
  a22 : Rat
  a23 : Rat
  a30 : Rat
  a31 : Rat
  a32 : Rat
  a33 : Rat
def MQ.mul (A B : MQ) : MQ :=
  ⟨A.a00 * B.a00 + A.a01 * B.a10 + A.a02 * B.a20 + A.a03 * B.a30,
   A.a00 * B.a01 + A.a01 * B.a11 + A.a02 * B.a21 + A.a03 * B.a31,
   A.a00 * B.a02 + A.a01 * B.a12 + A.a02 * B.a22 + A.a03 * B.a32,
   A.a00 * B.a03 + A.a01 * B.a13 + A.a02 * B.a23 + A.a03 * B.a33,
   A.a10 * B.a00 + A.a11 * B.a10 + A.a12 * B.a20 + A.a13 * B.a30,
   A.a10 * B.a01 + A.a11 * B.a11 + A.a12 * B.a21 + A.a13 * B.a31,
   A.a10 * B.a02 + A.a11 * B.a12 + A.a12 * B.a22 + A.a13 * B.a32,
   A.a10 * B.a03 + A.a11 * B.a13 + A.a12 * B.a23 + A.a13 * B.a33,
   A.a20 * B.a00 + A.a21 * B.a10 + A.a22 * B.a20 + A.a23 * B.a30,
   A.a20 * B.a01 + A.a21 * B.a11 + A.a22 * B.a21 + A.a23 * B.a31,
   A.a20 * B.a02 + A.a21 * B.a12 + A.a22 * B.a22 + A.a23 * B.a32,
   A.a20 * B.a03 + A.a21 * B.a13 + A.a22 * B.a23 + A.a23 * B.a33,
   A.a30 * B.a00 + A.a31 * B.a10 + A.a32 * B.a20 + A.a33 * B.a30,
   A.a30 * B.a01 + A.a31 * B.a11 + A.a32 * B.a21 + A.a33 * B.a31,
   A.a30 * B.a02 + A.a31 * B.a12 + A.a32 * B.a22 + A.a33 * B.a32,
   A.a30 * B.a03 + A.a31 * B.a13 + A.a32 * B.a23 + A.a33 * B.a33⟩

/-- Smith (peel) coordinate and Einstein addition, as in PeelTrees.lean. -/
def gam (z : Rat) : Rat := (z - 1) / (z + 1)
def oplus (x y : Rat) : Rat := (x + y) / (1 + x * y)

/-- The boosts of rapidity ln z along i and j, with C = cosh = (z^2+1)/(2z), S = sinh = (z^2-1)/(2z). -/
def Bqi (C S : Rat) : MQ := ⟨C, S, 0, 0, S, C, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1⟩
def Bqj (C S : Rat) : MQ := ⟨C, 0, S, 0, 0, 1, 0, 0, S, 0, C, 0, 0, 0, 0, 1⟩
/-- The pure boost to the point p = B_i B_j o = (C^2, SC, S, 0): [[gamma, u^T], [u, I + u u^T/(gamma+1)]]. -/
def Lq (C S : Rat) : MQ := ⟨C*C, S*C, S, 0, S*C, 1 + S*C*(S*C)/(C*C+1), S*C*S/(C*C+1), 0,
   S, S*C*S/(C*C+1), 1 + S*S/(C*C+1), 0, 0, 0, 0, 1⟩
/-- The rotation about k with cos = (1-t^2)/(1+t^2), sin = 2t/(1+t^2). -/
def Rq (t : Rat) : MQ := ⟨1, 0, 0, 0, 0, (1 - t*t)/(1 + t*t), 2*t/(1 + t*t), 0,
   0, -(2*t/(1 + t*t)), (1 - t*t)/(1 + t*t), 0, 0, 0, 0, 1⟩
/-- cosh^2(ln z) + 1 = (z^4 + 6z^2 + 1)/(4z^2). -/
theorem cosh_sq_add_one (z : Rat) (h1 : z ≠ 0) : (z*z+1)/(2*z) * ((z*z+1)/(2*z)) + 1 = (z*z*z*z + 6*z*z + 1) / (4*z*z) := by
grind
/-- t = Gamma^2 = (z-1)^2/(z+1)^2. -/
theorem gam_sq (z : Rat) (h2 : z + 1 ≠ 0) : gam z * gam z = (z-1)*(z-1) / ((z+1)*(z+1)) := by unfold gam; grind
/-- 1 + t^2 = 2(z^4 + 6z^2 + 1)/(z+1)^4. -/
theorem one_add_t_sq (z : Rat) (h2 : z + 1 ≠ 0) : 1 + (z-1)*(z-1) / ((z+1)*(z+1)) * ((z-1)*(z-1) / ((z+1)*(z+1))) = 2 *
```

```
(z*z*z*z + 6*z*z + 1) / ((z+1)*(z+1)*((z+1)*(z+1))) := by grind
/-- 1 - t^2 = 8z(z^2+1)/(z+1)^4. -/
theorem one_sub_t_sq (z : Rat) (h2 : z + 1 ≠ 0) : 1 - (z-1)*(z-1) / ((z+1)*(z+1)) * ((z-1)*(z-1) / ((z+1)*(z+1))) = 8 * z *
(z*z + 1) / ((z+1)*(z+1)*((z+1)*(z+1))) := by grind
/-- z^4 + 6z^2 + 1 is not zero for z > 0. -/
theorem D_ne_zero (z : Rat) (hz : 0 < z) : z*z*z*z + 6*z*z + 1 ≠ 0 := by
  have hz2 : 0 < z * z := Rat.mul_pos hz hz
  have hz4 : 0 < z * z * (z * z) := Rat.mul_pos hz2 hz2
  grind
/-- cos theta = (1-t^2)/(1+t^2) = 4z(z^2+1)/(z^4+6z^2+1), t = Gamma^2. -/
theorem thomas_cos (z : Rat) (hz : 0 < z) :
    (1 - gam z * gam z * (gam z * gam z)) / (1 + gam z * gam z * (gam z * gam z)) = 4*z*(z*z+1) / (z*z*z*z + 6*z*z + 1) :=
by
  have h2 : z + 1 ≠ 0 := by grind
  have h22 : (z + 1) * (z + 1) ≠ 0 := by grind
  have h24 : (z + 1) * (z + 1) * ((z + 1) * (z + 1)) ≠ 0 := by grind
  have h3 := D_ne_zero z hz
  have h25 : 2 * (z*z*z*z + 6*z*z + 1) / ((z+1)*(z+1)*((z+1)*(z+1))) ≠ 0 := by
    intro h
    have : 2 * (z*z*z*z + 6*z*z + 1) = 2 * (z*z*z*z + 6*z*z + 1) / ((z+1)*(z+1)*((z+1)*(z+1))) * ((z+1)*(z+1)*((z+1)*(z+1)))
:= by grind
    rw [h] at this; grind
  rw [gam_sq z h2, one_add_t_sq z h2, one_sub_t_sq z h2]
  grind
/-- sin theta = 2t/(1+t^2) = (z^2-1)^2/(z^4+6z^2+1), t = Gamma^2. -/
theorem thomas_sin (z : Rat) (hz : 0 < z) :
    2 * (gam z * gam z) / (1 + gam z * gam z * (gam z * gam z)) = (z*z-1)*(z*z-1) / (z*z*z*z + 6*z*z + 1) := by
  have h2 : z + 1 ≠ 0 := by grind
  have h22 : (z + 1) * (z + 1) ≠ 0 := by grind
  have h24 : (z + 1) * (z + 1) * ((z + 1) * (z + 1)) ≠ 0 := by grind
  have h3 := D_ne_zero z hz
  have h25 : 2 * (z*z*z*z + 6*z*z + 1) / ((z+1)*(z+1)*((z+1)*(z+1))) ≠ 0 := by
    intro h
    have : 2 * (z*z*z*z + 6*z*z + 1) = 2 * (z*z*z*z + 6*z*z + 1) / ((z+1)*(z+1)*((z+1)*(z+1))) * ((z+1)*(z+1)*((z+1)*(z+1)))
:= by grind
    rw [h] at this; grind
  rw [gam_sq z h2, one_add_t_sq z h2]
  grind
/-- THOMAS, symbolic z over Rat (z > 0): L R = B_i B_j, with L the pure boost to B_i B_j o and R the
    rotation about k by the angle with cos = (1-t^2)/(1+t^2), sin = 2t/(1+t^2), t = Gamma(z)^2. -/
theorem thomas_rat (z : Rat) (hz : 0 < z) :
    MQ.mul (Lq ((z*z+1)/(2*z)) ((z*z-1)/(2*z))) (Rq (gam z * gam z)) =
    MQ.mul (Bqi ((z*z+1)/(2*z)) ((z*z-1)/(2*z))) (Bqj ((z*z+1)/(2*z)) ((z*z-1)/(2*z))) := by
  have h1 : z ≠ 0 := by grind
  have h3 := D_ne_zero z hz
  have h5 : 2 * z ≠ 0 := by grind
  have h6 : 4 * z * z ≠ 0 := by grind
  have h7 : (z*z*z*z + 6*z*z + 1) / (4*z*z) ≠ 0 := by grind
  simp only [MQ.mul, Lq, Rq, Bqi, Bqj, MQ.mk.injEq]
  rw [cosh_sq_add_one z h1, thomas_cos z hz, thomas_sin z hz]
  refine ⟨?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_⟩ <;> grind

/-- Half angle: sin/(1 + cos) = t for t >= 0; with t = Gamma^2 this is tan(theta/2) = Gamma^2. -/
theorem thomas_half_angle_t (t : Rat) (ht : 0 ≤ t) :
    (2*t/(1 + t*t)) / (1 + (1 - t*t)/(1 + t*t)) = t := by
  have h0 : 0 ≤ t * t := Rat.mul_nonneg ht ht
  have h1 : 1 + t*t ≠ 0 := by grind
  grind

/-- cos^2 + sin^2 = 1 for the Thomas angle written in t. -/
theorem thomas_unit_t (t : Rat) (ht : 0 ≤ t) :
    ((1 - t*t)/(1 + t*t)) * ((1 - t*t)/(1 + t*t)) + (2*t/(1 + t*t)) * (2*t/(1 + t*t)) = 1 := by
  have h0 : 0 ≤ t * t := Rat.mul_nonneg ht ht
  have h1 : 1 + t*t ≠ 0 := by grind
  grind

/-- Velocity versus Smith step: Gamma ⊕ Gamma = 2 Gamma/(1 + Gamma^2), and 2/3 ⊕ 2/3 = 12/13 = Gamma(25). -/
theorem oplus_self (g : Rat) : oplus g g = 2*g/(1 + g*g) := by
  unfold oplus; grind

theorem oplus_two_thirds : oplus (2/3) (2/3) = 12/13 ∧ gam 25 = 12/13 := by
  unfold oplus gam; constructor <;> grind

end PeelTrees2
```

A.3 PeelTrees3.lean (7 theorems): the ln 5 commutator

Run and axiom audit

```
lean PeelTrees3.lean: exit code 0 (no messages), 1 s, 7 theorems
'PeelTrees3.comm5_fixes_p' does not depend on any axioms
```

```
'PeelTrees3.comm5_trace' does not depend on any axioms
'PeelTrees3.comm7_trace' does not depend on any axioms
```

Source

```
/-
  PeelTrees3.lean — the ln 5 commutator is a rotation; the ln 7 one is not.
  Jeromie Beasley. Core Lean 4.34.1, no Mathlib, no sorry, no native_decide.
  Definitions repeated from PeelTrees.lean so the file stands alone.
-/
namespace PeelTrees3
set_option maxRecDepth 200000

structure M4 where
  a00 : Int
  a01 : Int
  a02 : Int
  a03 : Int
  a10 : Int
  a11 : Int
  a12 : Int
  a13 : Int
  a20 : Int
  a21 : Int
  a22 : Int
  a23 : Int
  a30 : Int
  a31 : Int
  a32 : Int
  a33 : Int
deriving DecidableEq, Repr

def M4.mul (A B : M4) : M4 :=
  ⟨A.a00 * B.a00 + A.a01 * B.a10 + A.a02 * B.a20 + A.a03 * B.a30,
   A.a00 * B.a01 + A.a01 * B.a11 + A.a02 * B.a21 + A.a03 * B.a31,
   A.a00 * B.a02 + A.a01 * B.a12 + A.a02 * B.a22 + A.a03 * B.a32,
   A.a00 * B.a03 + A.a01 * B.a13 + A.a02 * B.a23 + A.a03 * B.a33,
   A.a10 * B.a00 + A.a11 * B.a10 + A.a12 * B.a20 + A.a13 * B.a30,
   A.a10 * B.a01 + A.a11 * B.a11 + A.a12 * B.a21 + A.a13 * B.a31,
   A.a10 * B.a02 + A.a11 * B.a12 + A.a12 * B.a22 + A.a13 * B.a32,
   A.a10 * B.a03 + A.a11 * B.a13 + A.a12 * B.a23 + A.a13 * B.a33,
   A.a20 * B.a00 + A.a21 * B.a10 + A.a22 * B.a20 + A.a23 * B.a30,
   A.a20 * B.a01 + A.a21 * B.a11 + A.a22 * B.a21 + A.a23 * B.a31,
   A.a20 * B.a02 + A.a21 * B.a12 + A.a22 * B.a22 + A.a23 * B.a32,
   A.a20 * B.a03 + A.a21 * B.a13 + A.a22 * B.a23 + A.a23 * B.a33,
   A.a30 * B.a00 + A.a31 * B.a10 + A.a32 * B.a20 + A.a33 * B.a30,
   A.a30 * B.a01 + A.a31 * B.a11 + A.a32 * B.a21 + A.a33 * B.a31,
   A.a30 * B.a02 + A.a31 * B.a12 + A.a32 * B.a22 + A.a33 * B.a32,
   A.a30 * B.a03 + A.a31 * B.a13 + A.a32 * B.a23 + A.a33 * B.a33⟩

def M4.tr (A : M4) : M4 := ⟨A.a00, A.a10, A.a20, A.a30, A.a01, A.a11, A.a21, A.a31, A.a02, A.a12, A.a22, A.a32, A.a03,
A.a13, A.a23, A.a33⟩
def M4.smul (c : Int) (A : M4) : M4 := ⟨c * A.a00, c * A.a01, c * A.a02, c * A.a03, c * A.a10, c * A.a11, c * A.a12, c *
A.a13, c * A.a20, c * A.a21, c * A.a22, c * A.a23, c * A.a30, c * A.a31, c * A.a32, c * A.a33⟩
def M4.red (m : Int) (A : M4) : M4 := ⟨A.a00 % m, A.a01 % m, A.a02 % m, A.a03 % m, A.a10 % m, A.a11 % m, A.a12 % m, A.a13 %
m, A.a20 % m, A.a21 % m, A.a22 % m, A.a23 % m, A.a30 % m, A.a31 % m, A.a32 % m, A.a33 % m⟩
def I4 : M4 := ⟨1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1⟩
def eta : M4 := ⟨1, 0, 0, 0, 0, -1, 0, 0, 0, 0, -1, 0, 0, 0, 0, -1⟩

/-- The six step directions of Garrett's tree: +i, -i, +j, -j, +k, -k. -/
inductive Dir | pi | mi | pj | mj | pk | mk
deriving DecidableEq, Repr

def Dir.inv : Dir → Dir
  | .pi => .mi | .mi => .pi | .pj => .mj | .mj => .pj | .pk => .mk | .mk => .pk

def allDirs : List Dir := [.pi, .mi, .pj, .mj, .pk, .mk]
theorem mem_allDirs (d : Dir) : d ∈ allDirs := by cases d <;> decide

/-- z times the boost of peel ratio z along direction d: c = (z^2+1)/2, s = (z^2-1)/2, z elsewhere. -/
def boost (c s z : Int) : Dir → M4
  | .pi => ⟨c, s, 0, 0, s, c, 0, 0, 0, 0, z, 0, 0, 0, 0, z⟩
  | .mi => ⟨c, (-s), 0, 0, (-s), c, 0, 0, 0, 0, z, 0, 0, 0, 0, z⟩
  | .pj => ⟨c, 0, s, 0, 0, z, 0, 0, s, 0, c, 0, 0, 0, 0, z⟩
  | .mj => ⟨c, 0, (-s), 0, 0, z, 0, 0, (-s), 0, c, 0, 0, 0, 0, z⟩
  | .pk => ⟨c, 0, 0, s, 0, z, 0, 0, 0, 0, z, 0, s, 0, 0, c⟩
  | .mk => ⟨c, 0, 0, (-s), 0, z, 0, 0, 0, 0, z, 0, (-s), 0, 0, c⟩

/-- Peel ratio 5 (rapidity ln 5, Smith step 2/3) and peel ratio 7 (rapidity ln 7, Garrett's drawing). -/
def B5 : Dir → M4 := boost 13 12 5
def B7 : Dir → M4 := boost 25 24 7

/-- The scaled matrix of a word (applied left to right, starting at the identity). -/
```

```
def word (B : Dir → M4) (w : List Dir) : M4 := w.foldl (fun M d => M4.mul M (B d)) I4

/-- Its first column is z^L times the point reached, (cosh d, sinh d * direction). -/
def pt (M : M4) : List Int := [M.a00, M.a10, M.a20, M.a30]


/-- The commutator i j i^-1 j^-1 at peel ratio z, scaled by z^4. -/
def comm (B : Dir → M4) : M4 := word B [.pi, .pj, .mi, .mj]

/-- Peel ratio 5: the commutator fixes the timelike point p = (3, 2, 2, 0) (Minkowski norm 9 - 4 - 4 = 1),
    so it is a rotation of hyperbolic space about p; its trace is 196/625 = 2 + 2 cos phi with
    cos phi = -527/625. -/
theorem comm5_fixes_p : pt (M4.mul (comm B5) (3,0,0,0, 2,0,0,0, 2,0,0,0, 0,0,0,0)) = [1875, 1250, 1250, 0] := by decide
theorem comm5_p_unit : (3 : Int) * 3 - 2 * 2 - 2 * 2 - 0 * 0 = 1 := by decide
theorem comm5_trace : (comm B5).a00 + (comm B5).a11 + (comm B5).a22 + (comm B5).a33 = 196 := by decide
theorem comm5_cos : (196 : Int) - 2 * 625 = 2 * (-527) := by decide
/-- The rotation angle has cos phi = -527/625, i.e. cos(phi/2) = 7/25 up to sign: 2*49 - 625 = -527. -/
theorem comm5_half : 2 * (7 * 7) - 625 = (-527 : Int) := by decide
/-- Peel ratio 7: trace 51076/2401 > 4, so the commutator is not a rotation (it is loxodromic). -/
theorem comm7_trace : (comm B7).a00 + (comm B7).a11 + (comm B7).a22 + (comm B7).a33 = 51076 ∧ 51076 > 4 * 2401 := by decide

end PeelTrees3
```

A.4 Mathlib-based drafts (not built; for independent checking)

Core.lean in this set is PeelTrees.lean unchanged. Smith.lean ties Γ to Mathlib's metric on the upper half-plane (UpperHalfPlane.tanh_half_dist, dist_of_re_eq); Riley.lean states the meridians as Matrix.IsHyperbolic and the relation identities over any commutative ring.

lakefile.toml

```
name = "two-trees-lean"
version = "1.0.0"
defaultTargets = ["TwoTrees"]

[[require]]
name = "mathlib"
git = "https://github.com/leanprover-community/mathlib4.git"
rev = "v4.34.1"

[[lean_lib]]
name = "TwoTrees"
globs = ["TwoTrees.+"]
```

Smith.lean

```
/-
Two Trees in One Ball (Jeromie Beasley). The peel coordinate against Mathlib's own hyperbolic metric.

`Mathlib.Analysis.Complex.UpperHalfPlane.Metric` puts the Poincaré metric on the upper half plane
`ℍ`. This module reads the peel coordinate of `TwoTrees/Core.lean` in that metric, so the paper's
§2 (VOTKP Theorem 8.1 and Corollary 15.2) is stated against a metric that Mathlib defines, not one
written for the paper.

* `smith_identity`: for `z > 0`, `(z − 1)/(z + 1) = tanh(½ ln z)` (VOTKP Theorem 8.1).
* `dist_I_vert`: the points `i` and `z·i` of `ℍ` are at hyperbolic distance `|ln z|`
  (Corollary 15.2: the peel ratio `z` is a hyperbolic length `ln z`).
* `tanh_half_dist_I_vert`: for `z ≥ 1`, `tanh(d(i, z·i)/2) = (z − 1)/(z + 1)`, the peel coordinate.
* `cosh_dist_I_vert`: `cosh d(i, z·i) = (z² + 1)/(2z)`, the time entry of the boost (Theorem 2.6).
* `dist_I_pow`: `d(i, aⁿ·i) = n ln a` for `a ≥ 1`, the norm ladder as equal steps.
* `gam_eq_tanh_half_dist`, `peel_eq_tanh_half_dist`: the rational `PeelTrees.gam` and
  `PeelTrees.peel` of the core module, cast to `ℝ`, are `tanh` of half the Mathlib distance.
-/
import Mathlib
import TwoTrees.Core

namespace TwoTrees.Smith

open Real

/-- `tanh t = (e^{2t} − 1)/(e^{2t} + 1)`. -/
theorem tanh_exp (t : ℝ) : tanh t = (exp (2 * t) - 1) / (exp (2 * t) + 1) := by
  have h2 : exp (2 * t) = exp t * exp t := by rw [← exp_add]; ring_nf
  have hn : exp (-t) = (exp t)⁻¹ := exp_neg t
  have he : 0 < exp t := exp_pos t
```

```
  rw [tanh_eq_sinh_div_cosh, sinh_eq, cosh_eq, h2, hn]
  have h1 : exp t + (exp t)⁻¹ ≠ 0 := by positivity
  have h3 : exp t * exp t + 1 ≠ 0 := by positivity
  field_simp
  try ring

/-- **VOTKP Theorem 8.1.** For `z > 0`, `(z − 1)/(z + 1) = tanh(½ ln z)`. -/
theorem smith_identity (z : ℝ) (hz : 0 < z) : (z - 1) / (z + 1) = tanh (log z / 2) := by
  rw [tanh_exp]
  have : 2 * (log z / 2) = log z := by ring
  rw [this, exp_log hz]

/-- The point `z·i` of the upper half plane, for real `z > 0`. -/
noncomputable def vert (z : ℝ) (hz : 0 < z) : UpperHalfPlane :=
  UpperHalfPlane.mk ⟨0, z⟩ hz

/-- **Corollary 15.2, in Mathlib's metric.** The points `i` and `z·i` of `ℍ` are at hyperbolic
distance `|ln z|`. -/
theorem dist_I_vert (z : ℝ) (hz : 0 < z) :
    dist UpperHalfPlane.I (vert z hz) = |Real.log z| := by
  have hre : UpperHalfPlane.I.re = (vert z hz).re := by
    first
    | rfl
    | (rw [UpperHalfPlane.I_re]; rfl)
    | simp [vert]
  refine (UpperHalfPlane.dist_of_re_eq hre).trans ?_
  have h1 : UpperHalfPlane.I.im = 1 := UpperHalfPlane.I_im
  have h2 : (vert z hz).im = z := rfl
  rw [h1, h2, Real.log_one, Real.dist_eq]
  first
  | rw [zero_sub, abs_neg]
  | simp

/-- **VOTKP Theorem 8.1, in Mathlib's metric.** For `z ≥ 1`, the peel coordinate `(z − 1)/(z + 1)`
is `tanh` of half the hyperbolic distance from `i` to `z·i`. -/
theorem tanh_half_dist_I_vert (z : ℝ) (hz : 0 < z) (h1 : 1 ≤ z) :
    Real.tanh (dist UpperHalfPlane.I (vert z hz) / 2) = (z - 1) / (z + 1) := by
  rw [dist_I_vert z hz, abs_of_nonneg (Real.log_nonneg h1)]
  exact (smith_identity z hz).symm

/-- **Theorem 2.6, in Mathlib's metric.** `cosh d(i, z·i) = (z² + 1)/(2z)`, the time entry of the
boost of peel ratio `z`. -/
theorem cosh_dist_I_vert (z : ℝ) (hz : 0 < z) :
    Real.cosh (dist UpperHalfPlane.I (vert z hz)) = (z * z + 1) / (2 * z) := by
  rw [dist_I_vert z hz, Real.cosh_abs, Real.cosh_log hz]
  have hz0 : z ≠ 0 := hz.ne'
  rw [div_eq_div_iff two_ne_zero (mul_ne_zero two_ne_zero hz0)]
  have hzz : z⁻¹ * z = 1 := inv_mul_cancel₀ hz0
  linear_combination 2 * hzz

/-- **The norm ladder as equal steps.** For `a ≥ 1`, the point `aⁿ·i` is at hyperbolic distance
`n ln a` from `i`: each factor of `a` is one step of length `ln a`. -/
theorem dist_I_pow (a : ℝ) (ha : 0 < a) (ha1 : 1 ≤ a) (n : ℕ) :
    dist UpperHalfPlane.I (vert (a ^ n) (pow_pos ha n)) = n * Real.log a := by
  rw [dist_I_vert, Real.log_pow,
    abs_of_nonneg (mul_nonneg (Nat.cast_nonneg n) (Real.log_nonneg ha1))]

/-- **The core peel coordinate is the Mathlib one.** For a rational ratio `z ≥ 1`, the rational
`PeelTrees.gam z` of `TwoTrees/Core.lean`, read in `ℝ`, is `tanh` of half the hyperbolic distance
from `i` to `z·i`. -/
theorem gam_eq_tanh_half_dist (z : ℚ) (h0 : (0 : ℝ) < z) (h1 : 1 ≤ z) :
    Real.tanh (dist UpperHalfPlane.I (vert (z : ℝ) h0) / 2) = ((PeelTrees.gam z : ℚ) : ℝ) := by
  rw [tanh_half_dist_I_vert (z : ℝ) h0 (by exact_mod_cast h1), PeelTrees.gam]
  first
  | (push_cast; done)
  | (norm_cast; done)
  | (simp; done)
  | (norm_num; done)
  | (push_cast; ring1)

/-- **Theorem 2.5, in Mathlib's metric.** After `n` peel steps of ratio 5, the core peel walk
`PeelTrees.peel n` (that is, `n` copies of `2/3` combined by Einstein addition), read in `ℝ`, is
`tanh` of half the hyperbolic distance from `i` to `5ⁿ·i`. -/
theorem peel_eq_tanh_half_dist (n : ℕ) :
    Real.tanh (dist UpperHalfPlane.I (vert ((5 : ℝ) ^ n) (pow_pos (by norm_num) n)) / 2) =
      ((PeelTrees.peel n : ℚ) : ℝ) := by
  rw [tanh_half_dist_I_vert ((5 : ℝ) ^ n) (pow_pos (by norm_num) n) (one_le_pow₀ (by norm_num)),
    PeelTrees.peel_ladder, PeelTrees.gam]
  first
  | (push_cast; done)
  | (norm_cast; done)
  | (simp; done)
  | (norm_num; done)
  | (push_cast; ring1)
```

```
end TwoTrees.Smith
```

Riley.lean

```
/-
Two Trees in One Ball (Jeromie Beasley). The knot's tree with Mathlib's matrices (§6).

The Riley meridians of a 2-bridge knot are `a = !![s, 1; 0, s⁻¹]` and `b = !![s, 0; -v, s⁻¹]`.
Multiplying each by `s` makes every entry a polynomial, so the statements below hold over every
commutative ring:

  `A = s·a = !![s², s; 0, 1]`, `B = s·b = !![s², 0; −vs, 1]`,
  `A' = s·a⁻¹ = !![1, −s; 0, s²]`, `B' = s·b⁻¹ = !![1, 0; vs, s²]`.

* `merA_mul_merAi`, `merB_mul_merBi`: `A A' = B B' = s²·I`.
* `merA_det` … `merB_discr`, `meridian_discr`: determinant `s²`, trace `s² + 1`, discriminant `(s² − 1)²`
  (Theorem 6.1: each meridian is a boost of translation length `2 ln |s|`).
* `merA_isHyperbolic`, `merB_isHyperbolic`, `meridian_isHyperbolic`: for real `s` with `s² ≠ 1`
  the meridians are hyperbolic in Mathlib's sense (`Matrix.IsHyperbolic`: discriminant `> 0`).
* `trefoil_relation`, `figure_eight_relation` (Theorem 6.2): `A W − W B` is the Riley polynomial
  times a fixed matrix, so the knot relation holds exactly on the Riley curve.
* `trefoil_longitude` (Theorem 6.4): on the trefoil's curve the scaled longitude is upper
  triangular with diagonal `−s²`, `−s¹⁴`.
-/
import Mathlib

namespace TwoTrees.Riley

variable {R : Type*} [CommRing R]

/-- `s` times the Riley meridian `a = !![s, 1; 0, s⁻¹]`. -/
def merA (s : R) : Matrix (Fin 2) (Fin 2) R := !![s * s, s; 0, 1]

/-- `s` times the Riley meridian `b = !![s, 0; -v, s⁻¹]`. -/
def merB (s v : R) : Matrix (Fin 2) (Fin 2) R := !![s * s, 0; -(v * s), 1]

/-- `s` times `a⁻¹`. -/
def merAi (s : R) : Matrix (Fin 2) (Fin 2) R := !![1, -s; 0, s * s]

/-- `s` times `b⁻¹`. -/
def merBi (s v : R) : Matrix (Fin 2) (Fin 2) R := !![1, 0; v * s, s * s]

/-- The scaled trefoil longitude `B A A B A'⁴ = s⁸ · (b a a b a⁻⁴)`. -/
def trefoilLongitude (s v : R) : Matrix (Fin 2) (Fin 2) R :=
  merB s v * merA s * merA s * merB s v * (merAi s * merAi s * (merAi s * merAi s))

/-! ## Inverses, determinant, trace, discriminant -/

/-- `A A' = s² I`. -/
theorem merA_mul_merAi (s : R) : merA s * merAi s = !![s * s, 0; 0, s * s] := by
  ext i j
  fin_cases i <;> fin_cases j
  all_goals first
    | (simp [merA, merAi, Matrix.mul_apply, Fin.sum_univ_two]; done)
    | (simp [merA, merAi, Matrix.mul_apply, Fin.sum_univ_two] <;> ring1)
    | (simp [merA, merAi, Matrix.mul_apply, Fin.sum_univ_two] <;> grind)

/-- `B B' = s² I`. -/
theorem merB_mul_merBi (s v : R) : merB s v * merBi s v = !![s * s, 0; 0, s * s] := by
  ext i j
  fin_cases i <;> fin_cases j
  all_goals first
    | (simp [merB, merBi, Matrix.mul_apply, Fin.sum_univ_two]; done)
    | (simp [merB, merBi, Matrix.mul_apply, Fin.sum_univ_two] <;> ring1)
    | (simp [merB, merBi, Matrix.mul_apply, Fin.sum_univ_two] <;> grind)

theorem merA_det (s : R) : (merA s).det = s * s := by
  rw [merA, Matrix.det_fin_two_of]; ring

theorem merB_det (s v : R) : (merB s v).det = s * s := by
  rw [merB, Matrix.det_fin_two_of]; ring

theorem merA_trace (s : R) : (merA s).trace = s * s + 1 := by
  rw [merA, Matrix.trace_fin_two_of]

theorem merB_trace (s v : R) : (merB s v).trace = s * s + 1 := by
  rw [merB, Matrix.trace_fin_two_of]

/-- **Theorem 6.1, scaled.** `trace² − 4 det = (s² − 1)²`. -/
theorem merA_discr (s : R) : (merA s).discr = (s * s - 1) ^ 2 := by
  rw [merA, Matrix.discr_fin_two, Matrix.trace_fin_two_of, Matrix.det_fin_two_of]; ring
```

```
theorem merB_discr (s v : R) : (merB s v).discr = (s * s - 1) ^ 2 := by
  rw [merB, Matrix.discr_fin_two, Matrix.trace_fin_two_of, Matrix.det_fin_two_of]; ring

/-! ## Hyperbolic in Mathlib's sense -/

/-- For real `s` with `s² ≠ 1` the scaled meridian `A` is hyperbolic. -/
theorem merA_isHyperbolic (s : ℝ) (hs : s ^ 2 ≠ 1) : (merA s).IsHyperbolic := by
  show (0 : ℝ) < Matrix.discr (merA s)
  rw [merA_discr]
  have hne : s * s - 1 ≠ 0 := by
    intro h
    apply hs
    linear_combination h
  exact lt_of_le_of_ne (sq_nonneg _) (Ne.symm (pow_ne_zero 2 hne))

/-- For real `s` with `s² ≠ 1` the scaled meridian `B` is hyperbolic, for every `v`. -/
theorem merB_isHyperbolic (s v : ℝ) (hs : s ^ 2 ≠ 1) : (merB s v).IsHyperbolic := by
  show (0 : ℝ) < Matrix.discr (merB s v)
  rw [merB_discr]
  have hne : s * s - 1 ≠ 0 := by
    intro h
    apply hs
    linear_combination h
  exact lt_of_le_of_ne (sq_nonneg _) (Ne.symm (pow_ne_zero 2 hne))

/-- The unscaled meridian `!![s, 1; 0, 1/s]` has discriminant `(s − 1/s)²`. -/
theorem meridian_discr (s : ℝ) : Matrix.discr !![s, 1; 0, 1 / s] = (s - 1 / s) ^ 2 := by
  rw [Matrix.discr_fin_two, Matrix.trace_fin_two_of, Matrix.det_fin_two_of]; ring

/-- **Theorem 6.1.** For real `s ≠ 0` with `s² ≠ 1`, the unscaled meridian `!![s, 1; 0, 1/s]` is
hyperbolic: its discriminant is `(s − 1/s)² > 0`. -/
theorem meridian_isHyperbolic (s : ℝ) (hs : s ≠ 0) (hs1 : s ^ 2 ≠ 1) :
    Matrix.IsHyperbolic !![s, 1; 0, 1 / s] := by
  show (0 : ℝ) < Matrix.discr !![s, 1; 0, 1 / s]
  rw [meridian_discr]
  have hinv : s * (1 / s) = 1 := mul_one_div_cancel hs
  have hne : s - 1 / s ≠ 0 := by
    intro h
    apply hs1
    linear_combination s * h + hinv
  exact lt_of_le_of_ne (sq_nonneg _) (Ne.symm (pow_ne_zero 2 hne))

/-! ## The knot relations factor through the Riley polynomial -/

/-- **Theorem 6.2, trefoil** (2-bridge `3/1`, `W = b a`): `A W − W B = R₃ · !![0, s; s v, 0]` with
`R₃ = s⁴ − s² v − s² + 1`. -/
theorem trefoil_relation (s v : R) :
    merA s * (merB s v * merA s) - merB s v * merA s * merB s v =
      !![0, (s * s * s * s - s * s * v - s * s + 1) * s;
        (s * s * s * s - s * s * v - s * s + 1) * (s * v), 0] := by
  ext i j
  fin_cases i <;> fin_cases j
  all_goals first
    | (simp [merA, merB, Matrix.mul_apply, Fin.sum_univ_two]; done)
    | (simp [merA, merB, Matrix.mul_apply, Fin.sum_univ_two] <;> ring1)
    | (simp [merA, merB, Matrix.mul_apply, Fin.sum_univ_two] <;> grind)

/-- **Theorem 6.2, figure-eight** (2-bridge `5/3`, `W = b a⁻¹ b⁻¹ a`):
`A W − W B = R₄ · !![0, s³; s³ v, 0]` with `R₄ = s² v² + (3 s² − s⁴ − 1) v + (3 s² − s⁴ − 1)`. -/
theorem figure_eight_relation (s v : R) :
    merA s * (merB s v * merAi s * merBi s v * merA s) -
        merB s v * merAi s * merBi s v * merA s * merB s v =
      !![0, (s * s * v * v + (3 * s * s - s * s * s * s - 1) * v + (3 * s * s - s * s * s * s - 1)) *
          (s * s * s);
        (s * s * v * v + (3 * s * s - s * s * s * s - 1) * v + (3 * s * s - s * s * s * s - 1)) *
          (s * s * s * v), 0] := by
  ext i j
  fin_cases i <;> fin_cases j
  all_goals first
    | (simp [merA, merB, merAi, merBi, Matrix.mul_apply, Fin.sum_univ_two]; done)
    | (simp [merA, merB, merAi, merBi, Matrix.mul_apply, Fin.sum_univ_two] <;> ring1)
    | (simp [merA, merB, merAi, merBi, Matrix.mul_apply, Fin.sum_univ_two] <;> grind)

/-! ## The trefoil longitude -/

/-- Three entries of the scaled trefoil longitude, as polynomials in `s` and `v`. -/
theorem trefoilLongitude_entries (s v : R) :
    trefoilLongitude s v 1 0 = -(s ^ 7 * v) + s ^ 5 * v ^ 2 + s ^ 3 * v ^ 2 - s * v ∧
    trefoilLongitude s v 0 0 = s ^ 8 - s ^ 6 * v - s ^ 4 * v ∧
    trefoilLongitude s v 1 1 = s ^ 14 * v - s ^ 12 * v ^ 2 - 2 * s ^ 10 * v ^ 2 - 2 * s ^ 8 * v ^ 2 +
      2 * s ^ 8 * v + s ^ 8 - 2 * s ^ 6 * v ^ 2 + s ^ 6 * v - s ^ 4 * v ^ 2 + s ^ 4 * v +
      s ^ 2 * v := by
  refine ⟨?_, ?_, ?_⟩
  all_goals first
    | (simp [trefoilLongitude, merA, merB, merAi, Matrix.mul_apply, Fin.sum_univ_two]; done)
    | (simp [trefoilLongitude, merA, merB, merAi, Matrix.mul_apply, Fin.sum_univ_two] <;> ring1)
```

```
    | (simp [trefoilLongitude, merA, merB, merAi, Matrix.mul_apply, Fin.sum_univ_two] <;> grind)

/-- **Theorem 6.4.** On the trefoil's Riley curve `R₃ = 0` the scaled longitude `s⁸ L` is upper
triangular with diagonal `−s²`, `−s¹⁴`, so `L` has eigenvalues `−s⁻⁶`, `−s⁶`: a boost of
length `12 ln |s|`. -/
theorem trefoil_longitude (s v : R) (h : s * s * s * s - s * s * v - s * s + 1 = 0) :
    trefoilLongitude s v 1 0 = 0 ∧ trefoilLongitude s v 0 0 = -(s * s) ∧
      trefoilLongitude s v 1 1 = -(s * s * s * s * s * s * s * s * s * s * s * s * s * s * s * s * s) := by
  obtain (h10, h00, h11) := trefoilLongitude_entries s v
  refine (?_, ?_, ?_)
  · rw [h10]
    linear_combination (-(s * v * (s ^ 2 + 1))) * h
  · rw [h00]
    linear_combination (s ^ 2 * (s ^ 2 + 1)) * h
  · rw [h11]
    linear_combination (s ^ 2 * (s ^ 2 + 1) * (s ^ 6 * v + s ^ 6 + s ^ 4 * v + s ^ 2 * v + v)) * h

end TwoTrees.Riley
```

CLAIMS.md

```
# Claims

Every theorem in the library, the part of the paper it belongs to, and its status.

**Status words.** *Checked (core Lean)*: the statement and proof are the paper's Appendix A file,
which compiled in plain Lean v4.34.1 with no messages (74 theorems, axioms within `propext`,
`Classical.choice`, `Quot.sound`). *Written, awaiting CI*: written against Mathlib v4.34.1, with
every Mathlib name looked up in the Mathlib source at that tag, but not yet compiled; the first
run of [the proof check](.github/workflows/build.yml) decides it. After a green run every row
below is kernel-checked in this repository, and the run's `verification/report.json` lists each
theorem's axioms.

Helper lemmas are marked *(helper)*: they are steps of a proof, kept public so their axioms are audited too.


## `TwoTrees/Core.lean` (namespace `PeelTrees`, 74 theorems)

| Theorem | Paper | Statement | Status |
| :--- | :--- | :--- | :--- |
| `PeelTrees.norm_mul` | §2 (norm ladder) | Euler's four-square identity: the norm is multiplicative, so a word of L generators has norm 5^L. | Checked (core Lean) |
| `PeelTrees.gens_norm` | §3 | Each of the six generators has norm 5. | Checked (core Lean) |
| `PeelTrees.gens_conj` | §3 | The backward step is the conjugate: g times its conjugate is the scalar 5. | Checked (core Lean) |
| `PeelTrees.gens_conj_mem` | §3 | The conjugate of each generator is again a generator (the tree is closed under stepping back). | Checked (core Lean) |
| `PeelTrees.sq_le_five` | Prop. 3.1 (helper) | An integer with x² ≤ 5 lies in [−2, 2]. | Checked (core Lean) |
| `PeelTrees.sq_nonneg'` | Prop. 3.1 (helper) | x² ≥ 0 for an integer x. | Checked (core Lean) |
| `PeelTrees.norm5_bound` | Prop. 3.1 | Every coordinate of a norm-5 quaternion lies in [-2, 2], so `box` holds all of them. | Checked (core Lean) |
| `PeelTrees.norm5_count` | Prop. 3.1 | There are exactly 8 (5 + 1) = 48 integer quaternions of norm 5. | Checked (core Lean) |
| `PeelTrees.lps_six` | §3 | Of the 48, exactly six have a odd and positive with b, c, d even, and they are Garrett's generators (the Lubotzky–Phillips–Sarnak normalisation, p + 1 = 6). | Checked (core Lean) |
| `PeelTrees.qwords_three_count` | §3 | There are 150 reduced quaternion words of length 3. | Checked (core Lean) |
| `PeelTrees.qwords_three_distinct` | §3 | The 150 reduced words of length 3 give 150 different quaternions, even up to sign. | Checked (core Lean) |
| `PeelTrees.qwords_three_primitive` | §3 | None of them is divisible by 5, so none collapses to a shorter word times a scalar. | Checked (core Lean) |
| `PeelTrees.gam_mul` | Thm 2.3 | Steps compose by multiplying ratios: Gamma(A) ⊕ Gamma(B) = Gamma(AB). | Checked (core Lean) |
| `PeelTrees.gam_five` | §2 | Γ(5) = 2/3. | Checked (core Lean) |
| `PeelTrees.gam_seven` | §2 | Γ(7) = 3/4. | Checked (core Lean) |
| `PeelTrees.five_pow_nonneg` | Thm 2.5 (helper) | 5ⁿ ≥ 0 in ℚ. | Checked (core Lean) |
| `PeelTrees.peel_ladder` | Thm 2.5 | The norm ladder is the peel ladder: after L steps the Smith coordinate is (5^L - 1)/(5^L + 1), for every L. | Checked (core Lean) |
| `PeelTrees.cosh_from_gam` | Thm 2.6 | The boost entries in the Smith coordinate: cosh = (1+G^2)/(1-G^2) = (z^2+1)/(2z) and sinh = 2G/(1-G^2) = (z^2-1)/(2z). | Checked (core Lean) |
| `PeelTrees.sinh_from_gam` | Thm 2.6 | sinh = 2Γ/(1 − Γ²) = (z² − 1)/(2z). | Checked (core Lean) |
| `PeelTrees.schottky_five_below` | Thm 5.3 | Schottky threshold in the peel coordinate: G^2 > 1/2. The norm-5 step sits below it, Garrett's ln 7 step above it. | Checked (core Lean) |
| `PeelTrees.schottky_seven_above` | Thm 5.3 | Garrett's ln 7 step sits above the Schottky threshold: Γ(7)² > 1/2. | Checked (core Lean) |
| `PeelTrees.schottky_ratio` | Thm 5.3 | The same threshold for the ratio itself: z > 3 + 2 sqrt 2, i.e. (z - 3)^2 > 8 with z > 3. | Checked (core Lean) |
| `PeelTrees.mem_allDirs` | §4 (helper) | Every step direction is in the list of six. | Checked (core Lean) |
| `PeelTrees.lorentz5` | Thm 4.2 | Every scaled boost is Lorentz: B^T eta B = z^2 eta. | Checked (core Lean) |
| `PeelTrees.lorentz7` | Thm 4.2 | Every scaled ratio-7 boost is Lorentz: Bᵀ η B = 49 η. | Checked (core Lean) |
| `PeelTrees.inverse5` | Thm 4.2 | The backward step is the inverse boost: B_d B_{-d} = z^2 I. | Checked (core Lean) |
| `PeelTrees.inverse7` | Thm 4.2 | The backward ratio-7 step is the inverse boost: B_d B_{−d} = 49 I. | Checked (core Lean) |
| `PeelTrees.boost5_symm` | Thm 4.2 | A pure boost is a symmetric matrix. | Checked (core Lean) |
```

`PeelTrees.straight5`	Thm 4.4	Straight path +i +i +i: z^3 cosh d = 7813 = (5^6 + 1)/2, so d = 3 ln 5 exactly (lengths add).	Checked (core Lean)
`PeelTrees.straight5_is_three_ln5`	Thm 4.4	7813 = (5⁶ + 1)/2 and 7812 = (5⁶ − 1)/2, so the straight path has length exactly 3 ln 5.	Checked (core Lean)
`PeelTrees.turn5`	Thm 4.3	Two orthogonal steps: cosh d = cosh^2 (ln 5) = 169/25 (hyperbolic Pythagoras).	Checked (core Lean)
`PeelTrees.turned5`	Thm 4.4	The turned paths of the earlier check.	Checked (core Lean)
`PeelTrees.garrett7`	§4.3	Garrett's ln 7 numbers: straight 3 ln 7, and the word +i +j +i with cosh d = 19657/343 (d = 4.7415).	Checked (core Lean)
`PeelTrees.rwords_three_count`	Thm 4.5	There are 150 reduced step words of length 3.	Checked (core Lean)
`PeelTrees.straight_farthest5`	Thm 4.5	Among all 150 reduced words of length 3, the straight paths are the farthest: every other word has strictly smaller cosh d. Turning always shortens.	Checked (core Lean)
`PeelTrees.thomas5`	Thm 4.6	B_i B_j = L R exactly (ratio 5, scaled by 9409).	Checked (core Lean)
`PeelTrees.thomas5_boost`	Thm 4.6	L is symmetric and Lorentz: a pure boost.	Checked (core Lean)
`PeelTrees.thomas5_rotation`	Thm 4.6	R is a rotation about k (cos 65/97, sin 72/97) fixing the time axis.	Checked (core Lean)
`PeelTrees.two_boosts_not_boost5`	Thm 4.6	So the product of two boosts is not a boost: it is not symmetric.	Checked (core Lean)
`PeelTrees.thomas7`	Thm 4.6	B_i B_j = L R exactly (ratio 7, scaled by 113569).	Checked (core Lean)
`PeelTrees.thomas7_boost`	Thm 4.6	L is symmetric and Lorentz: a pure boost.	Checked (core Lean)
`PeelTrees.thomas7_rotation`	Thm 4.6	R is a rotation about k (cos 175/337, sin 288/337) fixing the time axis.	Checked (core Lean)
`PeelTrees.two_boosts_not_boost7`	Thm 4.6	So the product of two boosts is not a boost: it is not symmetric.	Checked (core Lean)
`PeelTrees.thomas_half_angle`	Thm 4.7	Thomas half-angle in the peel coordinate: tan(theta/2) = sin/(1 + cos) = Gamma^2.	Checked (core Lean)
`PeelTrees.dot_emod5`	Thm 5.1 (helper)	Reduction mod m respects products (entrywise: sums and products commute with taking residues).	Checked (core Lean)
`PeelTrees.red_mul5`	Thm 5.1 (helper)	Reduction mod 5 respects 4×4 products.	Checked (core Lean)
`PeelTrees.dot_emod7`	Thm 5.1 (helper)	A 4-term dot product mod 7 depends only on the residues.	Checked (core Lean)
`PeelTrees.red_mul7`	Thm 5.1 (helper)	Reduction mod 7 respects 4×4 products.	Checked (core Lean)
`PeelTrees.gen_red5`	Thm 5.1 (helper)	Mod 5 each ratio-5 boost is 3·w wᵀ (rank one).	Checked (core Lean)
`PeelTrees.closure5`	Thm 5.1 (helper)	The finite closure step: rank-one state times a non-backtracking step stays rank one, nonzero.	Checked (core Lean)
`PeelTrees.outer_red5`	Thm 5.1 (helper)	The rank-one residue matrices are already reduced mod 5.	Checked (core Lean)
`PeelTrees.step5`	Thm 5.1 (helper)	A non-backtracking step keeps the rank-one, nonzero residue form (ratio 5).	Checked (core Lean)
`PeelTrees.walk5`	Thm 5.1 (helper)	The rank-one form survives every non-backtracking walk (ratio 5).	Checked (core Lean)
`PeelTrees.corner5`	Thm 5.1 (helper)	The time entry of every nonzero rank-one residue matrix is nonzero mod 5.	Checked (core Lean)
`PeelTrees.mul_I4`	Thm 5.1 (helper)	I · A = A for the 4×4 integer matrices.	Checked (core Lean)
`PeelTrees.time_not_div5`	Thm 5.1	MAIN (peel ratio 5): for every non-backtracking word of every length L >= 1, the scaled time coordinate 5^L cosh d(w) is not divisible by 5.	Checked (core Lean)
`PeelTrees.pow_emod5`	Thm 5.1 (helper)	5^(n+1) ≡ 0 mod 5.	Checked (core Lean)
`PeelTrees.no_return5`	Thm 5.1	No non-backtracking word returns to the origin: its time coordinate is never 5^L (cosh d = 1).	Checked (core Lean)
`PeelTrees.free5`	Thm 5.1	No non-backtracking word is the identity: the boost group is free on the three steps.	Checked (core Lean)
`PeelTrees.gen_red7`	Thm 5.1 (helper)	Mod 7 each ratio-7 boost is 4·w wᵀ (rank one).	Checked (core Lean)
`PeelTrees.closure7`	Thm 5.1 (helper)	The finite closure step: rank-one state times a non-backtracking step stays rank one, nonzero.	Checked (core Lean)
`PeelTrees.outer_red7`	Thm 5.1 (helper)	The rank-one residue matrices are already reduced mod 7.	Checked (core Lean)
`PeelTrees.step7`	Thm 5.1 (helper)	A non-backtracking step keeps the rank-one, nonzero residue form (ratio 7).	Checked (core Lean)
`PeelTrees.walk7`	Thm 5.1 (helper)	The rank-one form survives every non-backtracking walk (ratio 7).	Checked (core Lean)
`PeelTrees.corner7`	Thm 5.1 (helper)	The time entry of every nonzero rank-one residue matrix is nonzero mod 7.	Checked (core Lean)
`PeelTrees.time_not_div7`	Thm 5.1	MAIN (peel ratio 7): for every non-backtracking word of every length L >= 1, the scaled time coordinate 7^L cosh d(w) is not divisible by 7.	Checked (core Lean)
`PeelTrees.pow_emod7`	Thm 5.1 (helper)	7^(n+1) ≡ 0 mod 7.	Checked (core Lean)
`PeelTrees.no_return7`	Thm 5.1	No non-backtracking word returns to the origin: its time coordinate is never 7^L (cosh d = 1).	Checked (core Lean)
`PeelTrees.free7`	Thm 5.1	No non-backtracking word is the identity: the boost group is free on the three steps.	Checked (core Lean)
`PeelTrees.riley_inverses`	Thm 6.1	The scaled inverses are inverses: A A' = s^2 I and B B' = s^2 I.	Checked (core Lean)
`PeelTrees.meridian_boost`	Thm 6.1	Each meridian has determinant 1 and trace s + 1/s (scaled: det s^2, trace s^2 + 1), and trace^2 - 4 det = (s^2 - 1)^2: for real s ≠ ±1 it is a boost of translation length 2 ln|s|.	Checked (core Lean)
`PeelTrees.trefoil_relation`	Thm 6.2	Trefoil (2-bridge 3/1, W = b a): A W - W B = R3(s,v) * [[0, s], [s v, 0]], R3 = s^4 - s^2 v - s^2 + 1. The relation holds exactly on the Riley curve R3 = 0.	Checked (core Lean)
`PeelTrees.figure_eight_relation`	Thm 6.2	Figure-eight (2-bridge 5/3, W = b a^-1 b^-1 a): A W - W B = R4(s,v) * [[0, s^3], [s^3 v, 0]], R4 = s^2 v^2 + (3 s^2 - s^4 - 1) v + (3 s^2 - s^4 - 1).	Checked (core Lean)
`PeelTrees.trefoil_longitude`	Thm 6.4	Trefoil longitude on its curve: with R3 = 0 the scaled longitude B A A B A'^4 = s^8 L is upper triangular with diagonal -s^2, -s^14, i.e. L has eigenvalues -s^-6, -s^6: a boost of length 12 ln s = 2|y| with y = -6x.	Checked (core Lean)

`TwoTrees/Smith.lean` (8 theorems)

Theorem	Paper	Statement	Status
`TwoTrees.Smith.tanh_exp`	Thm 2.2 (helper)	`tanh t = (e^{2t} − 1)/(e^{2t} + 1)`.	Written, awaiting CI

`TwoTrees.Smith.smith_identity`	Thm 2.2 (VOTKP Thm 8.1)	For `z > 0`, `(z − 1)/(z + 1) = tanh(½ ln z)`.	Written, awaiting CI
`TwoTrees.Smith.dist_I_vert`	Cor. 2.4 (VOTKP Cor. 15.2)	The points `i` and `z·i` of `ℍ` are at hyperbolic distance `|ln z|`.	Written, awaiting CI
`TwoTrees.Smith.tanh_half_dist_I_vert`	Thm 2.2 + Cor. 2.4	For `z ≥ 1`, the peel coordinate `(z − 1)/(z + 1)` is `tanh` of half the hyperbolic distance from `i` to `z·i`.	Written, awaiting CI
`TwoTrees.Smith.cosh_dist_I_vert`	Thm 2.6	`cosh d(i, z·i) = (z² + 1)/(2z)`, the time entry of the boost of peel ratio `z`.	Written, awaiting CI
`TwoTrees.Smith.dist_I_pow`	Thm 2.5	For `a ≥ 1`, the point `aⁿ·i` is at hyperbolic distance `n ln a` from `i`: each factor of `a` is one step of length `ln a`.	Written, awaiting CI
`TwoTrees.Smith.gam_eq_tanh_half_dist`	Thm 2.2	For a rational ratio `z ≥ 1`, the rational `PeelTrees.gam z` of `TwoTrees/Core.lean`, read in `ℝ`, is `tanh` of half the hyperbolic distance from `i` to `z·i`.	Written, awaiting CI
`TwoTrees.Smith.peel_eq_tanh_half_dist`	Thm 2.5	After `n` peel steps of ratio 5, the core peel walk `PeelTrees.peel n` (that is, `n` copies of `2/3` combined by Einstein addition), read in `ℝ`, is `tanh` of half the hyperbolic distance from `i` to `5ⁿ·i`.	Written, awaiting CI

`TwoTrees/Riley.lean` (16 theorems)

Theorem	Paper	Statement	Status
`TwoTrees.Riley.merA_mul_merAi`	Thm 6.1	`A A' = s² I`.	Written, awaiting CI
`TwoTrees.Riley.merB_mul_merBi`	Thm 6.1	`B B' = s² I`.	Written, awaiting CI
`TwoTrees.Riley.merA_det`	Thm 6.1	det A = s².	Written, awaiting CI
`TwoTrees.Riley.merB_det`	Thm 6.1	det B = s².	Written, awaiting CI
`TwoTrees.Riley.merA_trace`	Thm 6.1	trace A = s² + 1.	Written, awaiting CI
`TwoTrees.Riley.merB_trace`	Thm 6.1	trace B = s² + 1.	Written, awaiting CI
`TwoTrees.Riley.merA_discr`	Thm 6.1	`trace² − 4 det = (s² − 1)²`.	Written, awaiting CI
`TwoTrees.Riley.merB_discr`	Thm 6.1	trace² − 4 det of B is (s² − 1)².	Written, awaiting CI
`TwoTrees.Riley.merA_isHyperbolic`	Thm 6.1	For real `s` with `s² ≠ 1` the scaled meridian `A` is hyperbolic.	Written, awaiting CI
`TwoTrees.Riley.merB_isHyperbolic`	Thm 6.1	For real `s` with `s² ≠ 1` the scaled meridian `B` is hyperbolic, for every `v`.	Written, awaiting CI
`TwoTrees.Riley.meridian_discr`	Thm 6.1	The unscaled meridian `!![s, 1; 0, 1/s]` has discriminant `(s − 1/s)²`.	Written, awaiting CI
`TwoTrees.Riley.meridian_isHyperbolic`	Thm 6.1	For real `s ≠ 0` with `s² ≠ 1`, the unscaled meridian `!![s, 1; 0, 1/s]` is hyperbolic: its discriminant is `(s − 1/s)² > 0`.	Written, awaiting CI
`TwoTrees.Riley.trefoil_relation`	Thm 6.2	(2-bridge `3/1`, `W = b a`): `A W − W B = R₃ · !![0, s; s v, 0]` with `R₃ = s⁴ − s² v − s² + 1`.	Written, awaiting CI
`TwoTrees.Riley.figure_eight_relation`	Thm 6.2	(2-bridge `5/3`, `W = b a⁻¹ b⁻¹ a`): `A W − W B = R₄ · !![0, s³; s³ v, 0]` with `R₄ = s² v² + (3 s² − s⁴ − 1) v + (3 s² − s⁴ − 1)`.	Written, awaiting CI
`TwoTrees.Riley.trefoilLongitude_entries`	Thm 6.4 (helper)	Three entries of the scaled trefoil longitude, as polynomials in `s` and `v`.	Written, awaiting CI
`TwoTrees.Riley.trefoil_longitude`	Thm 6.4	On the trefoil's Riley curve `R₃ = 0` the scaled longitude `s⁸ L` is upper triangular with diagonal `−s²`, `−s¹⁴`, so `L` has eigenvalues `−s⁻⁶`, `−s⁶`: a boost of length `12 ln |s|`.	Written, awaiting CI

Not in Lean

- **The boosts as a free group in Mathlib's own terms** (`Function.Injective (FreeGroup.lift f)` for the three boosts as rational 4×4 matrices). The mathematics is in `Core.lean`: `free5` and `free7` show that no non-backtracking word of any length is the identity. Turning that into injectivity of `FreeGroup.lift` needs a bridge from Mathlib's reduced words (`FreeGroup.toWord`) to the core lists of steps and from rational matrices to the scaled integer matrices `M4`. That bridge is not written yet, because it could not be compiled before publishing here.
- **Theorem 5.1 for a general ratio `z`.** Proved on paper; in Lean only for `z = 5` and `z = 7`.
- **The real curves of hyperbolic knots** beyond the trefoil and figure-eight relations: computed in the paper, not stated in Lean.
- **Quasi-isometry of the norm-5 placement.** Open (paper §8): the Schottky argument needs `Γ² > 1/2`, and `4/9` falls short.
- **The Thomas phase around a knot's relator loop.** Open (paper §8), untested.

A.5 PeelTrees4.lean (12 theorems): the knot commutator band and the torus knots

Run

```
$ lean PeelTrees4.lean   (Lean 4.34.1, core, no Mathlib)
exit 0   theorems: 12
```

Axiom audit

```
'PeelTrees4.T_eq' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees4.comm_trace' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees4.comm_trace_ab' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees4.v_from_T' depends on axioms: [propext, Classical.choice, Quot.sound]
```

```
'PeelTrees4.trefoil_T' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees4.torus5_T' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees4.torus7_T' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees4.trefoil_comm' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees4.torus_comm' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees4.band_k' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees4.band_bound' depends on axioms: [propext, Classical.choice, Quot.sound]
'PeelTrees4.band_peak' depends on axioms: [propext, Classical.choice, Quot.sound]
```

False controls (each must be rejected)

```
theorem bad_sign (s v : R) :
    (comm s v).tr = 2*(s*s*s*s) + v*(s*s)*((s*s - 1)*(s*s - 1) - v*(s*s)) := by
  simp only [comm, M2.tr, M2.mul, rA, rB, rAi, rBi]; grind
theorem bad_torus5 (s v : R) :
    let D := rel s v (W5 s v)
    let P := T s v * T s v + s*s * T s v - s*s*s*s
    D.b = P * s := by
  simp only [rel, W5, W3, T, M2.tr, M2.mul, M2.sub, rA, rB]; grind
theorem bad_trefoil (s v : R) (h : T s v = s*s) :
    (comm s v).tr = 2*(s*s*s*s) - v*(s*s*s*s) := by
  have h1 := comm_trace_ab s v; rw [h] at h1; grind
end PeelTrees4

-- result: 3 of 3 rejected
```

Source

```
/-
  PeelTrees4 — the knot commutator band (Two Trees v4, problem 1). Core Lean 4, no Mathlib.
  Riley meridians a = [[s,1],[0,1/s]], b = [[s,0],[-v,1/s]], scaled by s: A = s a, B = s b, A' = s a⁻¹, B' = s b⁻¹.
  C = A B A' B' = s⁴ [a,b] and T = tr(AB) = s² tr(ab). Every statement holds over any commutative ring.
-/
namespace PeelTrees4

structure M2 (R : Type) where
  a : R
  b : R
  c : R
  d : R

variable {R : Type} [Lean.Grind.CommRing R]

def M2.mul (X Y : M2 R) : M2 R := ⟨X.a*Y.a + X.b*Y.c, X.a*Y.b + X.b*Y.d, X.c*Y.a + X.d*Y.c, X.c*Y.b + X.d*Y.d⟩
def M2.sub (X Y : M2 R) : M2 R := ⟨X.a - Y.a, X.b - Y.b, X.c - Y.c, X.d - Y.d⟩
def M2.tr (X : M2 R) : R := X.a + X.d

def rA (s : R) : M2 R := ⟨s*s, s, 0, 1⟩
def rB (s v : R) : M2 R := ⟨s*s, 0, -(v*s), 1⟩
def rAi (s : R) : M2 R := ⟨1, -s, 0, s*s⟩
def rBi (s v : R) : M2 R := ⟨1, 0, v*s, s*s⟩

/-- Scaled commutator C = A B A' B' and scaled T = tr(AB). -/
def comm (s v : R) : M2 R := M2.mul (M2.mul (M2.mul (rA s) (rB s v)) (rAi s)) (rBi s v)
def T (s v : R) : R := (M2.mul (rA s) (rB s v)).tr

/-- tr(AB) = s⁴ + 1 − v s², i.e. tr(ab) = s² + s⁻² − v. -/
theorem T_eq (s v : R) : T s v = s*s*s*s + 1 - v*s*s := by
  simp only [T, M2.tr, M2.mul, rA, rB]; grind

/-- The commutator trace identity:  s⁴ tr[a,b] = 2 s⁴ − v s² ((s²−1)² − v s²),
    i.e. tr[a,b] = 2 − v((s − 1/s)² − v). -/
theorem comm_trace (s v : R) :
    (comm s v).tr = 2*(s*s*s*s) - v*(s*s)*((s*s - 1)*(s*s - 1) - v*(s*s)) := by
  simp only [comm, M2.tr, M2.mul, rA, rB, rAi, rBi]; grind

/-- The same identity through tr(ab):  tr[a,b] − 2 = −v (tr(ab) − 2)  (scaled by s⁴). -/
theorem comm_trace_ab (s v : R) :
    (comm s v).tr - 2*(s*s*s*s) = -(v*(s*s)) * (T s v - 2*(s*s)) := by
  rw [comm_trace, T_eq]; grind

/-- v is determined by tr(ab):  s² v = (s² − 1)² + (2 s² − T).  For real s ≠ 0 this reads
    v = (s − 1/s)² + (2 − tr ab): so tr(ab) < 2 forces v > 0. -/
theorem v_from_T (s v : R) : v*(s*s) = (s*s - 1)*(s*s - 1) + (2*(s*s) - T s v) := by
  rw [T_eq]; grind

/-! ### Torus knots T(2,n): the Riley polynomial is a polynomial in T alone. W = (BA)^((n−1)/2). -/

def W3 (s v : R) : M2 R := M2.mul (rB s v) (rA s)
```

```
def W5 (s v : R) : M2 R := M2.mul (W3 s v) (W3 s v)
def W7 (s v : R) : M2 R := M2.mul (W5 s v) (W3 s v)
def rel (s v : R) (W : M2 R) : M2 R := M2.sub (M2.mul (rA s) W) (M2.mul W (rB s v))

/-- Trefoil: A W − W B = (T − s²)·[[0, s],[s v, 0]]: the Riley curve is tr(ab) = 1. -/
theorem trefoil_T (s v : R) :
    let D := rel s v (W3 s v)
    D.a = 0 ∧ D.b = (T s v - s*s) * s ∧ D.c = (T s v - s*s) * (s*v) ∧ D.d = 0 := by
  simp only [rel, W3, T, M2.tr, M2.mul, M2.sub, rA, rB]; refine ⟨?_,?_,?_,?_⟩ <;> grind

/-- T(2,5): A W − W B = (T² − s²T − s⁴)·[[0, s],[s v, 0]]:  tr(ab)² − tr(ab) − 1 = 0,
    tr(ab) = 2cos(π/5) or 2cos(3π/5). -/
theorem torus5_T (s v : R) :
    let D := rel s v (W5 s v)
    let P := T s v * T s v - s*s * T s v - s*s*s*s
    D.a = 0 ∧ D.b = P * s ∧ D.c = P * (s*v) ∧ D.d = 0 := by
  simp only [rel, W5, W3, T, M2.tr, M2.mul, M2.sub, rA, rB]; refine ⟨?_,?_,?_,?_⟩ <;> grind

/-- T(2,7): A W − W B = (T³ − s²T² − 2s⁴T + s⁶)·[[0, s],[s v, 0]]:  t³ − t² − 2t + 1 = 0,
    t = 2cos(π/7), 2cos(3π/7), 2cos(5π/7). -/
theorem torus7_T (s v : R) :
    let D := rel s v (W7 s v)
    let t := T s v
    let P := t*t*t - s*s*t*t - 2*(s*s*s*s)*t + s*s*s*s*s*s
    D.a = 0 ∧ D.b = P * s ∧ D.c = P * (s*v) ∧ D.d = 0 := by
  simp only [rel, W7, W5, W3, T, M2.tr, M2.mul, M2.sub, rA, rB]; refine ⟨?_,?_,?_,?_⟩ <;> grind

/-- Trefoil commutator on its curve: tr(ab) = 1 gives tr[a,b] = 2 + v (scaled: s⁴ tr = 2s⁴ + v s⁴),
    with s² v = (s²−1)² + s², so v ≥ 1 for real s: always a boost, never in the band. -/
theorem trefoil_comm (s v : R) (h : T s v = s*s) :
    (comm s v).tr = 2*(s*s*s*s) + v*(s*s*s*s) ∧ v*(s*s) = (s*s - 1)*(s*s - 1) + s*s := by
  have h1 := comm_trace_ab s v; have h2 := v_from_T s v
  rw [h] at h1 h2; constructor <;> grind

/-- On every torus branch tr(ab) = t with 0 < 2 − t: then tr[a,b] − 2 = v(2 − t) with v = (s−1/s)² + (2−t) > 0,
    a product of positives — a boost. Scaled algebraic form: s⁴(tr[a,b] − 2) = v s² (2s² − T). -/
theorem torus_comm (s v : R) :
    (comm s v).tr - 2*(s*s*s*s) = v*(s*s) * (2*(s*s) - T s v) := by
  rw [comm_trace_ab]; grind

/-! ### The band criterion. Put A = (s² − 1)² and w = v s² (so A/s² = (s − 1/s)² = 4 sinh² x and w/s² = v).
    Then s⁴ (2 − tr[a,b]) = w (A − w): the commutator is a rotation iff 0 < w(A − w) < 4 s⁴. -/

/-- s⁴ (2 − tr[a,b]) = w (A − w) with A = (s²−1)², w = v s².  Positive iff 0 < v < (s − 1/s)²,
    which is the condition that the axes of a and b cross. -/
theorem band_k (s v : R) :
    2*(s*s*s*s) - (comm s v).tr = (v*(s*s)) * ((s*s - 1)*(s*s - 1) - v*(s*s)) := by
  rw [comm_trace]; grind

/-- The bound 4 w (A − w) = A² − (2w − A)² ≤ A²: so 2 − tr[a,b] ≤ (s − 1/s)⁴/4 = 4 sinh⁴ x, with equality at
    v = 2 sinh² x. When sinh x < 1 (Γ² = tanh² x < 1/2, the Schottky line) this is below 4: crossing axes force
    an elliptic commutator. -/
theorem band_bound (s v : R) :
    4 * ((v*(s*s)) * ((s*s - 1)*(s*s - 1) - v*(s*s))) =
      ((s*s - 1)*(s*s - 1)) * ((s*s - 1)*(s*s - 1)) - (2*(v*(s*s)) - (s*s - 1)*(s*s - 1)) * (2*(v*(s*s)) - (s*s - 1)*(s*s -
1)) := by
  grind

/-- The peak value: at w = A/2 the bound is attained, 4 w (A − w) = A². -/
theorem band_peak (s : R) (w : R) (h : 2*w = (s*s - 1)*(s*s - 1)) :
    4 * (w * ((s*s - 1)*(s*s - 1) - w)) = ((s*s - 1)*(s*s - 1)) * ((s*s - 1)*(s*s - 1)) := by
  grind

end PeelTrees4
```

A.6 PingPong7.lean (17 theorems): the ping-pong at ln 7

Run

```
$ lean PingPong7.lean   (Lean 4.34.1, core, no Mathlib)
exit 0   theorems: 17
```

Axiom audit

```
'PingPong7.pingpong_step' depends on axioms: [propext, Classical.choice, Quot.sound]
'PingPong7.caps_disjoint' depends on axioms: [propext, Classical.choice, Quot.sound]
'PingPong7.pingpong' does not depend on any axioms
```

```
'PingPong7.word_in_cap' depends on axioms: [propext, Classical.choice, Quot.sound]
'PingPong7.word_ne_scalar' depends on axioms: [propext, Classical.choice, Quot.sound]
'PingPong7.word_ne_id' depends on axioms: [propext, Classical.choice, Quot.sound]
'PingPong7.step_lorentz' depends on axioms: [propext, Classical.choice, Quot.sound]
'PingPong7.step_inv' depends on axioms: [propext, Classical.choice, Quot.sound]
{ t := 145011, x := 132721, y := 58226, z := 4802 }
exit 0
```

Source

```
/-
  PingPong7.lean — Two Trees in One Ball: the Schottky ping-pong at peel ratio z = 7, on the light cone.
  Jeromie Beasley. Core Lean 4.34.1, no Mathlib, no sorry, no native_decide.

  Setting. Vectors v = (t, x, y, z) over Int. The six steps u in {±i, ±j, ±k} act by the integer
  matrices 7 B_u (B_u the boost of rapidity ln 7 along u: cosh = 25/7, sinh = 24/7), e.g.
      7 B_{+i} (t, x, y, z) = (25 t + 24 x, 24 t + 25 x, 7 y, 7 z).
  comp u v is the coordinate of v along u (x, -x, y, -y, z, -z). A future null vector has
  t > 0 and t^2 = x^2 + y^2 + z^2. The cap of u on the light cone is
      X_u = { future null v : comp u v > 0 and t^2 < 2 (comp u v)^2 }      (comp u v / t > 1/sqrt 2),
  written without sqrt 2.

  Contents
    1. The steps are Lorentz up to the scale 49 and 7 B_{-u} . 7 B_u = 49 id.
    2. The six caps are pairwise disjoint.
    3. Ping-pong inclusion: for every u, 7 B_u maps every future null vector outside X_{-u} into X_u.
       Route: everything reduces to the plane (t, b) with b = comp u v. The nonlinear step is
       closed by explicit algebraic identities, e.g.
         2 (24t + 25b)^2 - (25t + 24b)^2 = 527 t^2 + 1200 t b + 674 b^2,
         432 (527 t^2 + 1200 t b + 674 b^2) = 41040 (t^2 - 2 b^2) + (432 t + 600 b)^2 + 13248 b^2,
       verified by grind, then linear reasoning over the monomials as atoms.
    4. The ping-pong lemma, abstractly (any letter type with an inversion, any action, any sets).
    5. Freeness: for every non-empty non-backtracking word w, the integer word map sends the null
       vector p0 = (3, 1, 2, 2) (outside every cap) into the cap of the first letter, hence it is not
       c . id for any c > 0; in particular B_w is not the identity.
-/

namespace PingPong7

/-! ## 1. Vectors, steps, caps -/

structure V4 where
  t : Int
  x : Int
  y : Int
  z : Int
deriving DecidableEq, Repr

inductive Dir | pi | mi | pj | mj | pk | mk
deriving DecidableEq, Repr

def Dir.inv : Dir → Dir
  | .pi => .mi | .mi => .pi | .pj => .mj | .mj => .pj | .pk => .mk | .mk => .pk

/-- The integer step 7 B_u. -/
def step : Dir → V4 → V4
  | .pi, ⟨t, x, y, z⟩ => ⟨25 * t + 24 * x, 24 * t + 25 * x, 7 * y, 7 * z⟩
  | .mi, ⟨t, x, y, z⟩ => ⟨25 * t - 24 * x, -24 * t + 25 * x, 7 * y, 7 * z⟩
  | .pj, ⟨t, x, y, z⟩ => ⟨25 * t + 24 * y, 7 * x, 24 * t + 25 * y, 7 * z⟩
  | .mj, ⟨t, x, y, z⟩ => ⟨25 * t - 24 * y, 7 * x, -24 * t + 25 * y, 7 * z⟩
  | .pk, ⟨t, x, y, z⟩ => ⟨25 * t + 24 * z, 7 * x, 7 * y, 24 * t + 25 * z⟩
  | .mk, ⟨t, x, y, z⟩ => ⟨25 * t - 24 * z, 7 * x, 7 * y, -24 * t + 25 * z⟩

/-- The coordinate of v along the step u. -/
def comp : Dir → V4 → Int
  | .pi, v => v.x | .mi, v => -v.x | .pj, v => v.y | .mj, v => -v.y | .pk, v => v.z | .mk, v => -v.z

/-- The Minkowski quadratic form t^2 - x^2 - y^2 - z^2. -/
def qf (v : V4) : Int := v.t * v.t - v.x * v.x - v.y * v.y - v.z * v.z

/-- Future null: t > 0 and t^2 = x^2 + y^2 + z^2. -/
def Null (v : V4) : Prop := 0 < v.t ∧ v.t * v.t = v.x * v.x + v.y * v.y + v.z * v.z

/-- The cap of u: future null with comp u v > 0 and t^2 < 2 (comp u v)^2. -/
def X (u : Dir) (v : V4) : Prop := Null v ∧ 0 < comp u v ∧ v.t * v.t < 2 * (comp u v * comp u v)

/-- Squares are nonnegative (core Lean has no `mul_self_nonneg` for Int under this name). -/
theorem msn (a : Int) : 0 ≤ a * a := by
  by_cases h : 0 ≤ a
  · exact Int.mul_nonneg h h
  · have h' : 0 ≤ -a := by omega
    have := Int.mul_nonneg h' h'
    have e : (-a) * (-a) = a * a := by grind
```

```
    omega

/-- Each step is Lorentz up to the scale 49: qf (7 B_u v) = 49 qf v, for every v. -/
theorem step_lorentz (u : Dir) (v : V4) : qf (step u v) = 49 * qf v := by
  obtain ⟨t, x, y, z⟩ := v
  cases u <;> simp only [step, qf] <;> grind

/-- 7 B_{-u} . 7 B_u = 49 id: the backward step is the inverse. -/
theorem step_inv (u : Dir) (v : V4) : step u.inv (step u v) = ⟨49 * v.t, 49 * v.x, 49 * v.y, 49 * v.z⟩ := by
  obtain ⟨t, x, y, z⟩ := v
  cases u <;> simp only [step, Dir.inv, V4.mk.injEq] <;> refine ⟨?_, ?_, ?_, ?_⟩ <;> grind

/-- The step along u, read in the plane (t, comp u): t' = 25 t + 24 b, b' = 24 t + 25 b. -/
theorem step_plane (u : Dir) (v : V4) :
    (step u v).t = 25 * v.t + 24 * comp u v ∧ comp u (step u v) = 24 * v.t + 25 * comp u v := by
  obtain ⟨t, x, y, z⟩ := v
  cases u <;> simp only [step, comp] <;> refine ⟨?_, ?_⟩ <;> grind

theorem comp_inv (u : Dir) (v : V4) : comp u.inv v = - comp u v := by
  cases u <;> simp only [comp, Dir.inv] <;> grind

/-! ## 2. The six caps are pairwise disjoint -/

theorem caps_disjoint (u w : Dir) (v : V4) (hu : X u v) (hw : X w v) : u = w := by
  obtain ⟨t, x, y, z⟩ := v
  obtain ⟨⟨ht, hn⟩, hu1, hu2⟩ := hu
  obtain ⟨_, hw1, hw2⟩ := hw
  have sx : 0 ≤ x * x := msn x
  have sy : 0 ≤ y * y := msn y
  have sz : 0 ≤ z * z := msn z
  cases u <;> cases w <;> simp only [comp] at hu1 hu2 hw1 hw2 hn ⊢ <;>
    first | rfl | (exfalso; grind)

/-! ## 3. The ping-pong inclusion -/

/-- The plane lemma: for t > 0 and (t, b) outside the backward cap ¬(-b > 0 ∧ t^2 < 2 b^2),
    the image (25t + 24b, 24t + 25b) has positive time, positive axis coordinate, and
    t'^2 < 2 b'^2. -/
theorem plane_core (t b : Int) (ht : 0 < t) (hn : ¬ (0 < -b ∧ t * t < 2 * ((-b) * (-b)))) :
    0 < 25 * t + 24 * b ∧ 0 < 24 * t + 25 * b ∧
    (25 * t + 24 * b) * (25 * t + 24 * b) < 2 * ((24 * t + 25 * b) * (24 * t + 25 * b)) := by
  have hq : 2 * ((24 * t + 25 * b) * (24 * t + 25 * b)) - (25 * t + 24 * b) * (25 * t + 24 * b)
      = 527 * (t * t) + 1200 * (t * b) + 674 * (b * b) := by grind
  have htt : 0 < t * t := Int.mul_pos ht ht
  by_cases hb : 0 ≤ b
  · -- the forward half: every term of 527 t^2 + 1200 t b + 674 b^2 is >= 0 and t^2 > 0
    have htb : 0 ≤ t * b := Int.mul_nonneg (Int.le_of_lt ht) hb
    have hbb : 0 ≤ b * b := msn b
    refine ⟨by omega, by omega, ?_⟩
    omega
  · -- the backward half: b < 0, so the hypothesis gives 2 b^2 <= t^2
    have hb' : b < 0 := by omega
    have hneg : (-b) * (-b) = b * b := by grind
    have h2 : 2 * (b * b) ≤ t * t := by
      have : ¬ (t * t < 2 * (b * b)) := fun h => hn ⟨by omega, by rw [hneg]; exact h⟩
      omega
    have hbb : 0 < b * b := by
      have := Int.mul_pos (show 0 < -b by omega) (show 0 < -b by omega)
      omega
    -- axis coordinate: if 24 t + 25 b <= 0 then (-24t - 25b)(24t - 25b) >= 0, i.e. 625 b^2 >= 576 t^2,
    -- against 1250 b^2 <= 625 t^2 and t^2 > 0
    have hx : 0 < 24 * t + 25 * b := by
      apply Classical.byContradiction
      intro hc
      have p := Int.mul_nonneg (show 0 ≤ -24 * t - 25 * b by omega) (show 0 ≤ 24 * t - 25 * b by omega)
      have e : (-24 * t - 25 * b) * (24 * t - 25 * b) = 625 * (b * b) - 576 * (t * t) := by grind
      omega
    -- the cap inequality: the sum-of-squares certificate
    have cert : 432 * (527 * (t * t) + 1200 * (t * b) + 674 * (b * b))
        = 41040 * (t * t - 2 * (b * b)) + (432 * t + 600 * b) * (432 * t + 600 * b) + 13248 * (b * b) := by
      grind
    have sq : 0 ≤ (432 * t + 600 * b) * (432 * t + 600 * b) := msn _
    refine ⟨by omega, hx, ?_⟩
    omega

/-- (comp u v)^2 <= t^2 on the null cone. -/
theorem comp_sq_le (u : Dir) (v : V4) (hv : Null v) : comp u v * comp u v ≤ v.t * v.t := by
  have sx := msn v.x
  have sy := msn v.y
  have sz := msn v.z
  have hn := hv.2
  cases u <;> simp only [comp] <;> grind

/-- The step preserves future null vectors. -/
theorem step_null (u : Dir) (v : V4) (hv : Null v) : Null (step u v) := by
  have hq := step_lorentz u v
```

```
  simp only [qf] at hq
  have hp := step_plane u v
  have hc := comp_sq_le u v hv
  obtain ⟨ht, hn⟩ := hv
  constructor
  · -- 25 t + 24 b > 0 because b^2 <= t^2
    rw [hp.1]
    apply Classical.byContradiction
    intro hneg
    have p := Int.mul_nonneg (show 0 ≤ -(25 * v.t + 24 * comp u v) by omega)
      (show 0 ≤ 25 * v.t - 24 * comp u v by omega)
    have e : (-(25 * v.t + 24 * comp u v)) * (25 * v.t - 24 * comp u v)
        = 576 * (comp u v * comp u v) - 625 * (v.t * v.t) := by grind
    have htt : 0 < v.t * v.t := Int.mul_pos ht ht
    omega
  · omega

/-- THE PING-PONG INCLUSION. For every step u, 7 B_u maps every future null vector that is not in
    the backward cap X_{-u} into the cap X_u. -/
theorem pingpong_step (u : Dir) (v : V4) (hv : Null v) (hout : ¬ X u.inv v) : X u (step u v) := by
  have hp := step_plane u v
  have hci := comp_inv u v
  have hn : ¬ (0 < -(comp u v) ∧ v.t * v.t < 2 * ((-(comp u v)) * (-(comp u v)))) := by
    intro ⟨h1, h2⟩
    exact hout ⟨hv, by rw [hci]; exact h1, by rw [hci]; exact h2⟩
  have core := plane_core v.t (comp u v) hv.1 hn
  refine ⟨step_null u v hv, ?_, ?_⟩
  · rw [hp.2]; exact core.2.1
  · rw [hp.1, hp.2]; exact core.2.2

/-! ## 4. The ping-pong lemma (abstract) -/

/-- Non-backtracking from g: the next letter is never the inverse of the current one. -/
def RedFrom {L : Type} (inv : L → L) : L → List L → Prop
  | _, [] => True
  | g, h :: t => h ≠ inv g ∧ RedFrom inv h t

/-- The word g_1 g_2 ... g_n acting on p: g_1 (g_2 (... (g_n p))). -/
def act {L α : Type} (f : L → α → α) : List L → α → α
  | [], p => p
  | g :: l, p => f g (act f l p)

/-- THE PING-PONG LEMMA. Letters L with an inversion, an action f, a domain D, and sets X a ⊆ D,
    pairwise disjoint, with f a carrying D \ X (inv a) into X a. Then for every base point p in D
    outside every X a, every non-empty non-backtracking word g :: l sends p into X g. -/
theorem pingpong {L α : Type} (inv : L → L) (f : L → α → α) (D : α → Prop) (X : L → α → Prop)
    (hdisj : ∀ a b v, X a v → X b v → a = b)
    (hXD : ∀ a v, X a v → D v)
    (hpp : ∀ a v, D v → ¬ X (inv a) v → X a (f a v))
    (p : α) (hp : D p) (hfree : ∀ a, ¬ X a p) :
    ∀ (l : List L) (g : L), RedFrom inv g l → X g (act f (g :: l) p) := by
  intro l
  induction l with
  | nil => intro g _; exact hpp g p hp (hfree _)
  | cons h t ih =>
    intro g hr
    obtain ⟨hne, hr'⟩ := hr
    have hin := ih h hr'
    apply hpp g _ (hXD h _ hin)
    intro hbad
    exact hne (hdisj h (inv g) _ hin hbad)

/-! ## 5. Freeness of the three boosts at z = 7 -/

/-- The base point (3, 1, 2, 2): future null (9 = 1 + 4 + 4) and in no cap (2 * 4 = 8 < 9). -/
def p0 : V4 := ⟨3, 1, 2, 2⟩

theorem p0_null : Null p0 := by simp only [Null, p0]; decide

theorem p0_free (u : Dir) : ¬ X u p0 := by
  cases u <;> simp only [X, Null, comp, p0] <;> decide

/-- Every non-empty non-backtracking word sends p0 into the cap of its first letter. -/
theorem word_in_cap (l : List Dir) (g : Dir) (hr : RedFrom Dir.inv g l) : X g (act step (g :: l) p0) :=
  pingpong Dir.inv step Null X caps_disjoint (fun _ _ h => h.1) pingpong_step p0 p0_null p0_free l g hr

/-- No positive multiple of p0 lies in a cap. -/
theorem scaled_p0_free (u : Dir) (c : Int) (hc : 0 < c) : ¬ X u ⟨c * 3, c * 1, c * 2, c * 2⟩ := by
  intro ⟨_, h1, h2⟩
  have hcc : 0 < c * c := Int.mul_pos hc hc
  cases u <;> simp only [comp] at h1 h2 <;> grind

/-- FREENESS. For every non-empty non-backtracking word over the six steps, the integer word map
    7^n B_w is not c . id for any c > 0 (it moves the null ray of p0). Taking c = 7^n: B_w ≠ id. -/
theorem word_ne_scalar (l : List Dir) (g : Dir) (hr : RedFrom Dir.inv g l) (c : Int) (hc : 0 < c) :
    act step (g :: l) p0 ≠ ⟨c * 3, c * 1, c * 2, c * 2⟩ := by
```

```
  intro h
  have := word_in_cap l g hr
  rw [h] at this
  exact scaled_p0_free g c hc this

theorem word_ne_id (l : List Dir) (g : Dir) (hr : RedFrom Dir.inv g l) :
    act step (g :: l) p0 ≠ (7 ^ (l.length + 1) * 3, 7 ^ (l.length + 1) * 1,
                            7 ^ (l.length + 1) * 2, 7 ^ (l.length + 1) * 2) :=
  word_ne_scalar l g hr _ (Int.pow_pos (by decide))

end PingPong7
```

Appendix B. Scripts and their output

exact.py

```
# SCRIPT: PEEL-TREES-EXACT
# Exact (rational) points of Garrett's tree under the peel-ratio boosts, the Thomas rotation, and an
# exact search to length 7 for a reduced word that is the identity or returns to the origin.
from fractions import Fraction as F
import itertools, math
def boost(ax,sg,z):
    c=F(z*z+1,2*z); s=F(z*z-1,2*z)*sg
    B=[[F(int(i==j)) for j in range(4)] for i in range(4)]
    B[0][0]=c; B[0][ax]=s; B[ax][0]=s; B[ax][ax]=c; return B
def mm(A,B): return [[sum(A[i][k]*B[k][j] for k in range(4)) for j in range(4)] for i in range(4)]
def mv(A,v): return [sum(A[i][k]*v[k] for k in range(4)) for i in range(4)]
e0=[F(1),0,0,0]
names={'+i':(1,1),'-i':(1,-1),'+j':(2,1),'-j':(2,-1),'+k':(3,1),'-k':(3,-1)}
for z in (5,7):
    G={n:boost(a,s,z) for n,(a,s) in names.items()}
    for w in [('+i',),('+i','+i'),('+i','+j'),('+i','+i','+i'),('+i','+i','+j'),('+i','+j','+i'),('+i','+j','+k')]:
        M=[[F(int(i==j)) for j in range(4)] for i in range(4)]
        for g in w: M=mm(M,G[g])
        p=mv(M,e0); print(z,w,'point',[str(x) for x in p],'d=%.4f'%math.acosh(p[0]))
    # Thomas rotation of B_i B_j
    M=mm(G['+i'],G['+j']); p=mv(M,e0); g=p[0]
    # pure boost to p: L = [[g, p^T],[p, I + p p^T/(1+g)]]
    L=[[g]+p[1:]]+[[p[i]]+[F(int(i==j))+p[i]*p[j]/(1+g) for j in range(1,4)] for i in range(1,4)]
    Li=[[L[i][j]*(1 if (i==0)==(j==0) else -1) for j in range(4)] for i in range(4)]  # inverse boost
    R=mm(Li,M); tr=sum(R[i][i] for i in range(4)); print(' Thomas R time row',[str(x) for x in R[0]],'cos theta =',(tr-2)/2,
'theta deg %.4f'%math.degrees(math.acos((tr-2)/2)))
    print(' R=',[[str(x) for x in r] for r in R])
# mod-p freeness exact check: reduced words up to length 7 for z=5 (integer scaled)
def ib(ax,sg,z):
    B=[[0]*4 for _ in range(4)]
    for i in range(4): B[i][i]=2*z if z%2==0 else z
    h = 1 if z%2 else 2
    c=(z*z+1)//(2//h) if False else None
for z in (5,7):
    Z={n:[[int(x*z) for x in r] for r in boost(a,s,z)] for n,(a,s) in names.items()}
    inv={'+i':'-i','-i':'+i','+j':'-j','-j':'+j','+k':'-k','-k':'+k'}
    layer=[((g,),Z[g]) for g in Z]; bad=0; cnt=0
    for L in range(1,8):
        for w,M in layer:
            cnt+=1
            col=[M[i][0] for i in range(4)]
            if col==[z**L,0,0,0] or all(M[i][j]==(z**L if i==j else 0) for i in range(4) for j in range(4)): bad+=1
        if L<7: layer=[(w+(h,),[[sum(M[i][k]*Z[h][k][j] for k in range(4)) for j in range(4)] for i in range(4)]) for w,M in
layer for h in Z if h!=inv[w[-1]]]
    print('z',z,'reduced words checked',cnt,'relations or origin-fixers',bad)
    print(' scaled gen +i', Z['+i'])
# Thomas rotation for two perpendicular boosts of equal rapidity, symbolic in the peel ratio z
import sympy as sp
z=sp.symbols('z',positive=True); c=(z**2+1)/(2*z); s=(z**2-1)/(2*z)
Bi=sp.Matrix([[c,s,0,0],[s,c,0,0],[0,0,1,0],[0,0,0,1]]); Bj=sp.Matrix([[c,0,s,0],[0,1,0,0],[s,0,c,0],[0,0,0,1]])
M=Bi*Bj; p=M[:,0]; g=p[0]
L=sp.zeros(4); L[0,0]=g
for i in range(1,4): L[0,i]=L[i,0]=p[i]
for i in range(1,4):
    for j in range(1,4): L[i,j]=(1 if i==j else 0)+p[i]*p[j]/(1+g)
Li=sp.Matrix(4,4,lambda i,j: L[i,j]*(1 if (i==0)==(j==0) else -1))
R=(Li*M).applyfunc(sp.simplify)
G=(1+((z-1)/(z+1))**2)/(1-((z-1)/(z+1))**2)
print('symbolic Thomas: cos =',sp.factor(R[1,1]),'| equals 2c/(1+c^2):',sp.simplify(R[1,1]-2*c/(1+c**2))==0,
      '| tan(theta/2) - Gamma^2 =',sp.simplify(R[1,2]/(1+R[1,1])-((z-1)/(z+1))**2), '| R[0,0] =',sp.simplify(R[0,0]))
# figure-eight real curve: v real iff c(c+4) >= 0 with c = s^2 + s^-2 - 3, i.e. cosh 2x >= 3/2
import math; print('figure-eight real curve starts at x0 = arccosh(3/2)/2 = %.5f'%(math.acosh(1.5)/2))
```

Output

```
5 ('+i',) point ['13/5', '12/5', '0', '0'] d=1.6094
5 ('+i', '+i') point ['313/25', '312/25', '0', '0'] d=3.2189
5 ('+i', '+j') point ['169/25', '156/25', '12/5', '0'] d=2.5987
5 ('+i', '+i', '+i') point ['7813/125', '7812/125', '0', '0'] d=4.8283
5 ('+i', '+i', '+j') point ['4069/125', '4056/125', '12/5', '0'] d=4.1757
5 ('+i', '+j', '+i') point ['2917/125', '2808/125', '156/25', '0'] d=3.8427
5 ('+i', '+j', '+k') point ['2197/125', '2028/125', '156/25', '12/5'] d=3.5589
 Thomas R time row ['1', '0', '0', '0'] cos theta = 65/97 theta deg 47.9250
 R= [['1', '0', '0', '0'], ['0', '65/97', '72/97', '0'], ['0', '-72/97', '65/97', '0'], ['0', '0', '0', '1']]
7 ('+i',) point ['25/7', '24/7', '0', '0'] d=1.9459
7 ('+i', '+i') point ['1201/49', '1200/49', '0', '0'] d=3.8918
7 ('+i', '+j') point ['625/49', '600/49', '24/7', '0'] d=3.2375
7 ('+i', '+i', '+i') point ['58825/343', '58824/343', '0', '0'] d=5.8377
7 ('+i', '+i', '+j') point ['30025/343', '30000/343', '24/7', '0'] d=5.1652
7 ('+i', '+j', '+i') point ['19657/343', '19200/343', '600/49', '0'] d=4.7415
7 ('+i', '+j', '+k') point ['15625/343', '15000/343', '600/49', '24/7'] d=4.5119
 Thomas R time row ['1', '0', '0', '0'] cos theta = 175/337 theta deg 58.7155
 R= [['1', '0', '0', '0'], ['0', '175/337', '288/337', '0'], ['0', '-288/337', '175/337', '0'], ['0', '0', '0', '1']]
z 5 reduced words checked 117186 relations or origin-fixers 0
 scaled gen +i [[13, 12, 0, 0], [12, 13, 0, 0], [0, 0, 5, 0], [0, 0, 0, 5]]
z 7 reduced words checked 117186 relations or origin-fixers 0
 scaled gen +i [[25, 24, 0, 0], [24, 25, 0, 0], [0, 0, 7, 0], [0, 0, 0, 7]]
symbolic Thomas: cos = 4*z*(z**2 + 1)/(z**4 + 6*z**2 + 1) | equals 2c/(1+c^2): True | tan(theta/2) - Gamma^2 = 0 | R[0,0] =
1
figure-eight real curve starts at x0 = arccosh(3/2)/2 = 0.48121
```

thomas_ints.py

```
# SCRIPT: PEEL-TREES-THOMAS-INTS
# Integer scalings of the Thomas decomposition B_i B_j = L R used in PeelTrees.lean.
from fractions import Fraction as F
from math import lcm
def run(z):
    c=F(z*z+1,2*z); s=F(z*z-1,2*z)
    Bi=[[c,s,0,0],[s,c,0,0],[0,0,1,0],[0,0,0,1]]; Bj=[[c,0,s,0],[0,1,0,0],[s,0,c,0],[0,0,0,1]]
    mm=lambda A,B:[[sum(F(A[i][k])*B[k][j] for k in range(4)) for j in range(4)] for i in range(4)]
    M=mm(Bi,Bj); p=[M[i][0] for i in range(4)]; g=p[0]
    L=[[g]+p[1:]]+[[p[i]]+[F(int(i==j))+p[i]*p[j]/(1+g) for j in range(1,4)] for i in range(1,4)]
    Li=[[L[i][j]*(1 if (i==0)==(j==0) else -1) for j in range(4)] for i in range(4)]
    R=mm(Li,M)
    dl=lcm(*[x.denominator for r in L for x in r]); dr=lcm(*[x.denominator for r in R for x in r]); dm=z*z
    Lz=[[int(x*dl) for x in r] for r in L]; Rz=[[int(x*dr) for x in r] for r in R]
    k=F(dl*dr,dm); assert k.denominator==1
    print(z,'dl',dl,'dr',dr,'k',k); print(Lz); print(Rz)
run(5); run(7)
```

Output

```
5 dl 2425 dr 97 k 9409
[[16393, 15132, 5820, 0], [15132, 14593, 4680, 0], [5820, 4680, 4225, 0], [0, 0, 0, 2425]]
[[97, 0, 0, 0], [0, 65, 72, 0], [0, -72, 65, 0], [0, 0, 0, 97]]
7 dl 16513 dr 337 k 113569
[[210625, 202200, 56616, 0], [202200, 196513, 50400, 0], [56616, 50400, 30625, 0], [0, 0, 0, 16513]]
[[337, 0, 0, 0], [0, 175, 288, 0], [0, -288, 175, 0], [0, 0, 0, 337]]
```

knotsym.py

```
# SCRIPT: PEEL-TREES-KNOT-SYMBOLIC
# Scaled Riley matrices: the trefoil and figure-eight relations factor through their Riley polynomials;
# the trefoil longitude on its curve.
import sympy as sp
s,v=sp.symbols('s v')
A=sp.Matrix([[s**2,s],[0,1]]); B=sp.Matrix([[s**2,0],[-v*s,1]])
Ai=sp.Matrix([[1,-s],[0,s**2]]); Bi=sp.Matrix([[1,0],[v*s,s**2]])
print((A*Ai).applyfunc(sp.expand),(B*Bi).applyfunc(sp.expand))
R3=s**4-s**2*v-s**2+1
R4=-s**4*v - s**4 + s**2*v**2 + 3*s**2*v + 3*s**2 - v - 1
for nm,W,R in [('3_1',B*A,R3),('4_1',B*Ai*Bi*A,R4)]:
    D=(A*W-W*B).applyfunc(sp.expand)
    N=D.applyfunc(lambda e: sp.div(e,R,s,v))
    print(nm,'D=',D); print(' quotients',[ (sp.factor(q),r) for q,r in N])
# trefoil longitude symbolic: w = b a, wr = a b, e=2 -> L = w wr a^-4 (unscaled)
a=sp.Matrix([[s,1],[0,1/s]]); b=sp.Matrix([[s,0],[-v,1/s]])
```

```
L=(b*a*a*b*(a.inv())**4).applyfunc(lambda e: sp.factor(sp.simplify(e.subs(v,s**2+s**-2-1))))
print('trefoil longitude on curve',L)
L2=(b*a*a*b*(a.inv())**4).subs({s:2,v:sp.Rational(13,4)}); print('at s=2',L2)
```

Output

```
Matrix([[s**2, 0], [0, s**2]]) Matrix([[s**2, 0], [0, s**2]])
3_1 D= Matrix([[0, s**5 - s**3*v - s**3 + s], [s**5*v - s**3*v**2 - s**3*v + s*v, 0]])
 quotients [(0, 0), (s, 0), (s*v, 0), (0, 0)]
4_1 D= Matrix([[0, -s**7*v - s**7 + s**5*v**2 + 3*s**5*v + 3*s**5 - s**3*v - s**3], [-s**7*v**2 - s**7*v + s**5*v**3 +
3*s**5*v**2 + 3*s**5*v - s**3*v**2 - s**3*v, 0]])
 quotients [(0, 0), (s**3, 0), (s**3*v, 0), (0, 0)]
trefoil longitude on curve Matrix([[-1/s**6, (s**2 + 1)*(s**2 - s + 1)*(s**2 + s + 1)*(s**4 - s**2 + 1)/s**5], [0, -s**6]])
at s=2 Matrix([[-1/64, 1365/32], [0, -64]])
```

garrett_hyperbolic.py

```
# SCRIPT: GARRETT-HYPERBOLIC
# Can Garrett's norm-5 quaternion tree sit in the Poincare ball with TRUE hyperbolic distances?
# Earlier claim: no, because unit quaternions only rotate the ball. Recheck using Jeromie's VOTKP peel machinery:
# a step that multiplies the norm by 5 is a peel ratio z = 5, Smith step Gamma = (5-1)/(5+1) = 2/3 (Theorem 8.1),
# rapidity tau/2 = (1/2) ln 5, hyperbolic length ln 5 (Corollary 15.2), steps compose as boosts (Theorem 15.1).
# So: send generator 1 + 2u (u = +-i, +-j, +-k) to a Lorentz boost of rapidity ln 5 along u; its conjugate 1 - 2u
# (the inverse step) goes to the opposite boost, which is the inverse boost. A reduced word becomes a product of boosts.
import numpy as np, itertools
r = np.log(5.0)
def boost(u, r):
    u = np.asarray(u, float); B = np.eye(4); c, s = np.cosh(r), np.sinh(r)
    B[0, 0] = c; B[0, 1:] = s * u; B[1:, 0] = s * u; B[1:, 1:] += (c - 1) * np.outer(u, u); return B
dirs = {'+i': (1,0,0), '-i': (-1,0,0), '+j': (0,1,0), '-j': (0,-1,0), '+k': (0,0,1), '-k': (0,0,-1)}
inv = {'+i':'-i','-i':'+i','+j':'-j','-j':'+j','+k':'-k','-k':'+k'}
G = {g: boost(u, r) for g, u in dirs.items()}
print("1. inverse step is the inverse boost:", max(np.abs(G['+i'] @ G['-i'] - np.eye(4)).max() for _ in [0]) < 1e-12)
def words(D):
    out = [()]; layer = [()]
    for _ in range(D):
        layer = [w + (g,) for w in layer for g in G if not (w and inv[w[-1]] == g)]; out += layer
    return out
W = words(5)
P = {}
for w in W:
    M = np.eye(4)
    for g in w: M = M @ G[g]
    P[w] = M @ np.array([1, 0, 0, 0.])                      # point on the hyperboloid
d0 = {w: np.arccosh(max(1, P[w][0])) for w in W}
print("2. straight path +i,+i,+i: distance %.6f vs 3 ln5 = %.6f (lengths add along one diameter)" % (d0[('+i','+i','+i')], 3
* r))
print("3. turn +i,+i,+j: distance %.6f (< 3 ln5: the turn shortens it, the Thomas-rotation point)" % d0[('+i','+i','+j')])
# 4. the tree embeds: distinct reduced words land on distinct points, and the closest pair is well separated
pts = np.array([P[w] for w in W]); n = len(pts)
mind = np.inf
for a in range(n):
    x = pts[a]; ip = x[0] * pts[a+1:, 0] - pts[a+1:, 1:] @ x[1:]
    if len(ip): mind = min(mind, np.arccosh(np.maximum(ip, 1)).min())
print("4. %d reduced words (depth <= 5): smallest hyperbolic distance between two different words = %.4f (> 0: the tree
embeds)" % (n, mind))
# 5. the distance between two words matches their tree distance up to a bounded loss (quasi-isometry), sampled
rng = np.random.default_rng(0); worst = 0; lo = np.inf
for _ in range(4000):
    a, b = W[rng.integers(n)], W[rng.integers(n)]
    k = 0
    while k < min(len(a), len(b)) and a[k] == b[k]: k += 1
    td = (len(a) - k) + (len(b) - k)
    if td == 0: continue
    hd = np.arccosh(max(1, P[a][0] * P[b][0] - P[a][1:] @ P[b][1:]))
    lo = min(lo, hd / (td * r))
print("5. hyperbolic distance / (tree distance x ln5), smallest over 4000 random pairs: %.4f (bounded away from 0)" % lo)
# 6. Garrett's caption: 3 x ln 7 = 5.838 straight, 4.742 with turns. Same boost model with ln 7 steps:
r7 = np.log(7.0); B7 = {g: boost(u, r7) for g, u in dirs.items()}
for w in [('+i','+i','+j'), ('+i','+j','+i'), ('+i','+j','+k')]:
    M = np.eye(4)
    for g in w: M = M @ B7[g]
    print("6. ln7 steps, word %s: distance %.4f" % (' '.join(w), np.arccosh((M @ [1,0,0,0.])[0])))
# Poincare ball coordinates of the depth-3 tree for drawing: x / (1 + t)
ball = {' '.join(w) if w else 'root': (P[w][1:] / (1 + P[w][0])).round(5).tolist() for w in W if len(w) <= 3}
import json; json.dump(ball, open('garrett_tree_ball.json', 'w'))
print("ball radius of depth-3 nodes: max %.4f" % max(np.linalg.norm(v) for v in ball.values()))
```

Output

```
1. inverse step is the inverse boost: True
2. straight path +i,+i,+i: distance 4.828314 vs 3 ln5 = 4.828314 (lengths add along one diameter)
3. turn +i,+i,+j: distance 4.175750 (< 3 ln5: the turn shortens it, the Thomas-rotation point)
4. 4687 reduced words (depth <= 5): smallest hyperbolic distance between two different words = 0.4258 (> 0: the tree embeds)
5. hyperbolic distance / (tree distance x ln5), smallest over 4000 random pairs: 0.2330 (bounded away from 0)
6. ln7 steps, word +i +i +j: distance 5.1652
6. ln7 steps, word +i +j +i: distance 4.7415
6. ln7 steps, word +i +j +k: distance 4.5119
ball radius of depth-3 nodes: max 0.9841
```

knot_boost_tree.py

```
# SCRIPT: KNOT-BOOST-TREE
# Firm-handle check before rendering. On the knot's REAL peripheral curve (the N1 census curves, s = |M| = e^x real),
# the Riley meridians a = [[s,1],[0,1/s]] and b = [[s,0],[-v,1/s]] are real matrices with trace s + 1/s > 2:
# boosts of the hyperbolic plane (the Smith/Poincare disk) with translation length exactly 2 ln s = 2x.
# Check: (1) v real on the curve, (2) a and b are boosts of length 2x, (3) the knot relation a w = w b holds, so the
# knot's word tree CLOSES UP (unlike Garrett's free tree), (4) the longitude is a boost of length 2|y| = 2|log L|.
import warnings; warnings.filterwarnings('ignore')
import sys, numpy as np, sympy as sp
sys.path.insert(0, '.')
from census_lib import riley_poly, riley_mats, word, longitude, S_, V_
def boost_len(M):
    t = abs(np.trace(M)) / np.sqrt(abs(np.linalg.det(M)))
    return 2 * np.arccosh(t / 2) if t > 2 else 0.0
for name, p, q in [('4_1', 5, 3), ('5_2', 7, 3), ('6_2', 11, 3)]:
    R = sp.Poly(riley_poly(p, q), V_)
    for x in [0.5, 0.9]:
        s = np.exp(x)
        cs = [complex(sp.N(c.subs(S_, s))) for c in R.all_coeffs()]
        vs = [v.real for v in np.roots(cs) if abs(v.imag) < 1e-9 and abs(v.real) > 1e-9]
        for v in vs[:2]:
            a, b = riley_mats(s, v); a = np.array(a, complex); b = np.array(b, complex)
            W = np.array(word(p, q, a, b), complex)
            rel = np.abs(a @ W - W @ b).max()
            L = longitude(p, q, s, v); y = np.log(abs(L[0, 0]))
            print("%-4s x=%.2f v=%+.5f | boost lengths a %.6f b %.6f (2x = %.6f) | relation |aW - Wb| = %.1e | longitude
boost %.6f vs 2|y| %.6f" % (
                name, x, v, boost_len(a), boost_len(b), 2 * x, rel, boost_len(np.array(L, complex)), 2 * abs(y)))
```

Output

```
4_1  x=0.50 v=+0.33976 | boost lengths a 1.000000 b 1.000000 (2x = 1.000000) | relation |aW - Wb| = 2.2e-16 | longitude
boost 1.300923 vs 2|y| 1.300923
4_1  x=0.50 v=-0.25360 | boost lengths a 1.000000 b 1.000000 (2x = 1.000000) | relation |aW - Wb| = 1.1e-16 | longitude
boost 1.300923 vs 2|y| 1.300923
4_1  x=0.90 v=+4.01557 | boost lengths a 1.800000 b 1.800000 (2x = 1.800000) | relation |aW - Wb| = 2.1e-14 | longitude
boost 6.691043 vs 2|y| 6.691043
4_1  x=0.90 v=-0.80062 | boost lengths a 1.800000 b 1.800000 (2x = 1.800000) | relation |aW - Wb| = 3.4e-16 | longitude
boost 6.691043 vs 2|y| 6.691043
5_2  x=0.50 v=+1.77613 | boost lengths a 1.000000 b 1.000000 (2x = 1.000000) | relation |aW - Wb| = 1.8e-15 | longitude
boost 9.457698 vs 2|y| 9.457698
5_2  x=0.90 v=+5.05483 | boost lengths a 1.800000 b 1.800000 (2x = 1.800000) | relation |aW - Wb| = 2.7e-13 | longitude
boost 17.671149 vs 2|y| 17.671149
5_2  x=0.90 v=+3.89627 | boost lengths a 1.800000 b 1.800000 (2x = 1.800000) | relation |aW - Wb| = 2.2e-13 | longitude
boost 6.422861 vs 2|y| 6.422861
6_2  x=0.50 v=+2.45565 | boost lengths a 1.000000 b 1.000000 (2x = 1.000000) | relation |aW - Wb| = 5.9e-14 | longitude
boost 2.863072 vs 2|y| 2.863072
6_2  x=0.50 v=+0.92437 | boost lengths a 1.000000 b 1.000000 (2x = 1.000000) | relation |aW - Wb| = 9.3e-15 | longitude
boost 6.835712 vs 2|y| 6.835712
6_2  x=0.90 v=+5.39872 | boost lengths a 1.800000 b 1.800000 (2x = 1.800000) | relation |aW - Wb| = 8.1e-11 | longitude
boost 4.046619 vs 2|y| 4.046619
6_2  x=0.90 v=+4.16359 | boost lengths a 1.800000 b 1.800000 (2x = 1.800000) | relation |aW - Wb| = 2.7e-11 | longitude
boost 14.133171 vs 2|y| 14.133171
```

census_lib.py

```
# SCRIPT: ATLAS-NEXT-CENSUSLIB
# Two-bridge knots up to 8 crossings: Riley words, Riley polynomial, longitude, Alexander polynomial,
# Seifert matrix (independent, from a diagram), name and volume (SnapPy).
import warnings; warnings.filterwarnings('ignore')
import numpy as np, sympy as sp
from math import gcd

def cf(p, q):
```

```
    out = []
    while q:
        out.append(p // q); p, q = q, p % q
    return out

def census(maxc=8):
    seen = {}
    for p in range(3, 400, 2):
        for q in range(1, p):
            if gcd(p, q) != 1 or sum(cf(p, q)) > maxc: continue
            qi = pow(q, -1, p)
            cands = [x for x in (q, p - q, qi, p - qi)]
            qq = min(x for x in cands if x % 2 == 1)          # odd representative (mirror allowed)
            key = (p, min(cands))
            if key not in seen: seen[key] = (qq, sum(cf(p, q)))
    return sorted((p, v[0], v[1]) for (p, _), v in seen.items())

def eps(p, q):
    return [(-1) ** ((i * q) // p) for i in range(1, p)]

T = sp.symbols('t')
def alexander(p, q):
    e = eps(p, q); ex = [0]
    for x in e: ex.append(ex[-1] + x)
    D = sum((-1) ** k * T ** ex[k] for k in range(p))
    D = sp.expand(D * T ** (-min(ex)))
    if sp.Poly(D, T).LC() < 0: D = -D
    return sp.expand(D)

def riley_mats(s, v):
    a = np.array([[s, 1], [0, 1 / s]], complex); b = np.array([[s, 0], [-v, 1 / s]], complex)
    return a, b

def word(p, q, a, b):
    e = eps(p, q); W = np.eye(2, dtype=complex); ai, bi = np.linalg.inv(a), np.linalg.inv(b)
    for i, x in enumerate(e):
        g = (b if x > 0 else bi) if i % 2 == 0 else (a if x > 0 else ai)
        W = W @ g
    return W

def word_rev(p, q, a, b):
    e = eps(p, q); W = np.eye(2, dtype=complex); ai, bi = np.linalg.inv(a), np.linalg.inv(b)
    for i, x in reversed(list(enumerate(e))):
        g = (b if x > 0 else bi) if i % 2 == 0 else (a if x > 0 else ai)
        W = W @ g
    return W

S_, V_ = sp.symbols('s v')
_rcache = {}
def riley_poly(p, q):
    """Riley polynomial in v with coefficients polynomial in s (symbolic, exact)."""
    if (p, q) in _rcache: return _rcache[(p, q)]
    a = sp.Matrix([[S_, 1], [0, 1 / S_]]); b = sp.Matrix([[S_, 0], [-V_, 1 / S_]])
    ai, bi = a.inv(), b.inv(); W = sp.eye(2)
    for i, x in enumerate(eps(p, q)):
        W = W * ((b if x > 0 else bi) if i % 2 == 0 else (a if x > 0 else ai))
        W = W.applyfunc(sp.expand)
    E = sp.together(sp.expand((a * W - W * b)[0, 1]))   # a w = w b  <=>  this entry vanishes (Riley)
    R = sp.expand(sp.numer(E))
    R = sp.expand(R / S_ ** min(m[0] for m in sp.Poly(R, S_).monoms()))
    _rcache[(p, q)] = R
    return R

def longitude(p, q, s, v):
    a, b = riley_mats(s, v)
    w = word(p, q, a, b); wr = word_rev(p, q, a, b)
    e = sum(eps(p, q))
    return w @ wr @ np.linalg.matrix_power(np.linalg.inv(a) if e > 0 else a, 2 * abs(e))   # commutes with a (checked)

def seifert_V(p, q):
    import spherogram
    target = sp.Poly(alexander(p, q), T)
    for args in [(q, p), (p, q), (p - q, p), (p, p - q)]:
        for clos in ['denominator_closure', 'numerator_closure']:
            try:
                K = getattr(spherogram.RationalTangle(*args), clos)()
            except Exception:
                continue
            if len(K.link_components) != 1: continue
            V = np.array(K.seifert_matrix(), dtype=float)
            D = sp.Matrix(V.astype(int)) - T * sp.Matrix(V.astype(int)).T
            d = sp.expand(D.det())
            if d == 0: continue
            d = sp.Poly(sp.expand(d / T ** min(m[0] for m in sp.Poly(d, T).monoms())), T)
            if d.LC() < 0: d = -d
            if d == target: return V, K
```

```
    return None, None

def lt_sig(V, x):
    om = np.exp(2j * np.pi * x)
    H = (1 - om) * V + (1 - np.conj(om)) * V.T
    ev = np.linalg.eigvalsh(H)
    return int(np.sum(ev > 1e-7) - np.sum(ev < -1e-7))
```

sanity.py

```
# SCRIPT: PEELTREES2-SANITY
# sanity check for general-z freeness and symbolic Thomas identity
import itertools, sympy as sp
dirs=['pi','mi','pj','mj','pk','mk']; inv={'pi':'mi','mi':'pi','pj':'mj','mj':'pj','pk':'mk','mk':'pk'}
ax={'pi':(1,1),'mi':(1,-1),'pj':(2,1),'mj':(2,-1),'pk':(3,1),'mk':(3,-1)}
def Q(z,u):
    a,s=ax[u]; M=sp.zeros(4); M[0,0]=z*z+1; M[a,a]=z*z+1; M[0,a]=M[a,0]=s*(z*z-1)
    for b in (1,2,3):
        if b!=a: M[b,b]=2*z
    return M
def w(u):
    a,s=ax[u]; v=[1,0,0,0]; v[a]=-s; return sp.Matrix(v)
def W(f,g): return w(f)*w(g).T
z=sp.symbols('z')
for u in dirs:
    E=(Q(z,u)-W(u,u))/z; E=E.applyfunc(sp.expand); assert all(e.is_polynomial(z) for e in E)
    print(u, list(E))
for f in dirs:
  for g in dirs:
    for h in dirs:
      d=(w(g).T*w(h))[0]
      assert W(f,g)*W(h,h)==d*W(f,h)
      assert (d==0)==(h==inv[g]) and d in (0,1,2)
def rw(n):
    ws=[[d] for d in dirs]
    for _ in range(n-1): ws=[x+[d] for x in ws for d in dirs if d!=inv[x[-1]]]
    return ws
for zz in [3,5,6,7,9,10,12,2,4]:
    ms=[m for m in range(3,zz+1,2) if zz%m==0]
    bad=0; tot=0; z0=0
    for L in range(1,6):
        for wd in rw(L):
            P=sp.eye(4)
            for d in wd: P=P*Q(zz,d)
            a=sum(1 for i in range(1,L) if wd[i]==wd[i-1])
            tot+=1
            for m in ms:
                if (P[0,0]-2**a)%m!=0: bad+=1
            if P[0,0]%zz==0: z0+=1
            assert P[0,0]!=(2*zz)**L
    print('z',zz,'odd divisors',ms,'words',tot,'violations of a00=2^a mod m',bad,'words with a00=0 mod z',z0)
# Thomas symbolic: Qi Qj, L pure boost, R rotation
P=Q(z,'pi')*Q(z,'pj'); D=z**4+6*z**2+1
C=(z*z+1)/(2*z); S=(z*z-1)/(2*z)
g=C*C; u=sp.Matrix([S*C,S,0])
L=sp.zeros(4); L[0,0]=g
for i in range(3): L[0,i+1]=L[i+1,0]=u[i]
for i in range(3):
  for j in range(3): L[i+1,j+1]=(1 if i==j else 0)+u[i]*u[j]/(g+1)
cs=4*z*(z*z+1)/D; sn=(z*z-1)**2/D
R=sp.Matrix([[1,0,0,0],[0,cs,sn,0],[0,-sn,cs,0],[0,0,0,1]])
Bij=P/(4*z*z)
print('L R - BiBj =',(L*R-Bij).applyfunc(sp.simplify)==sp.zeros(4))
Lint=(L*4*z*z*D).applyfunc(sp.factor); Rint=(R*D).applyfunc(sp.factor)
print('Lint',list(Lint)); print('Rint',list(Rint))
print((Lint*Rint-D*D*P).applyfunc(sp.expand)==sp.zeros(4))
t=((z-1)/(z+1))**2
print(sp.simplify((1-t**2)/(1+t**2)-cs), sp.simplify(2*t/(1+t**2)-sn))
```

Output

```
pi [z, z, 0, 0, z, z, 0, 0, 0, 0, 2, 0, 0, 0, 0, 2]
mi [z, -z, 0, 0, -z, z, 0, 0, 0, 0, 2, 0, 0, 0, 0, 2]
pj [z, 0, z, 0, 0, 2, 0, 0, z, 0, z, 0, 0, 0, 0, 2]
mj [z, 0, -z, 0, 0, 2, 0, 0, -z, 0, z, 0, 0, 0, 0, 2]
pk [z, 0, 0, z, 0, 2, 0, 0, 0, 0, 2, 0, z, 0, 0, z]
mk [z, 0, 0, -z, 0, 2, 0, 0, 0, 0, 2, 0, -z, 0, 0, z]
z 3 odd divisors [3] words 4686 violations of a00=2^a mod m 0 words with a00=0 mod z 0
z 5 odd divisors [5] words 4686 violations of a00=2^a mod m 0 words with a00=0 mod z 0
z 6 odd divisors [3] words 4686 violations of a00=2^a mod m 0 words with a00=0 mod z 0
```

```
z 7 odd divisors [7] words 4686 violations of a00=2^a mod m 0 words with a00=0 mod z 0
z 9 odd divisors [3, 9] words 4686 violations of a00=2^a mod m 0 words with a00=0 mod z 0
z 10 odd divisors [5] words 4686 violations of a00=2^a mod m 0 words with a00=0 mod z 0
z 12 odd divisors [3] words 4686 violations of a00=2^a mod m 0 words with a00=0 mod z 0
z 2 odd divisors [] words 4686 violations of a00=2^a mod m 0 words with a00=0 mod z 2640
z 4 odd divisors [] words 4686 violations of a00=2^a mod m 0 words with a00=0 mod z 762
L R - BiBj = True
Lint [(z**2 + 1)**2*(z**4 + 6*z**2 + 1), (z - 1)*(z + 1)*(z**2 + 1)*(z**4 + 6*z**2 + 1), 2*z*(z - 1)*(z + 1)*(z**4 + 6*z**2
+ 1), 0, (z - 1)*(z + 1)*(z**2 + 1)*(z**4 + 6*z**2 + 1), z**8 + 4*z**6 + 22*z**4 + 4*z**2 + 1, 2*z*(z - 1)**2*(z + 1)**2*
(z**2 + 1), 0, 2*z*(z - 1)*(z + 1)*(z**4 + 6*z**2 + 1), 2*z*(z - 1)**2*(z + 1)**2*(z**2 + 1), 8*z**2*(z**2 + 1)**2, 0, 0, 0,
0, 4*z**2*(z**4 + 6*z**2 + 1)]
Rint [z**4 + 6*z**2 + 1, 0, 0, 0, 0, 4*z*(z**2 + 1), (z - 1)**2*(z + 1)**2, 0, 0, -(z - 1)**2*(z + 1)**2, 4*z*(z**2 + 1), 0,
0, 0, 0, z**4 + 6*z**2 + 1]
True
0 0
```

KILL_CONDITIONS.txt

```
KILL CONDITIONS, written before any run (2026-10-30)

Q1 (quasi-isometry of the orbit map of the free tree at r = ln 5)
 K1a  "The orbit map at ln 5 is a quasi-isometric embedding" is KILLED if some family of reduced words w_n with |w_n| ->
infinity
      has d(o, w_n o)/|w_n| -> 0 (equivalently, a non-trivial word of translation length 0 whose powers are reduced words of
growing length).
      It SURVIVES (is not killed) only if, over every enumerated depth, min d(o,w o)/|w| and min (translation length)/|w|
over cyclically
      reduced words stay above a fixed positive constant and no elliptic or parabolic cyclically reduced word is found.
 K1b  The opposite claim "the orbit map is NOT a QI embedding at ln 5" is KILLED if c = [B_i,B_j] is loxodromic (translation
length > 0)
      with rational data, or if d(o, c^n o) grows linearly in n, or if no other word is elliptic either.
 K1c  "A ping-pong (Schottky) configuration exists at ln 5" is KILLED if a numerical search over per-generator round caps
(free centres,
      free radii, 20+ random restarts) never reaches a positive margin while the same search at ln 7 and at B_u^2 does reach
one
      (control).  OPPOSITE tried: maximise instead of minimise (search the margin over caps), different seeds, and unequal
radii.
 K1d  Control: round caps work exactly for z > 3+2sqrt2 (Gamma^2 > 1/2).  KILLED if bisection with free caps puts the
threshold elsewhere.
Q2 (turn around the relator loop on the real Riley curve)
 K2a  The integer lift class e (relator a W b^-1 W^-1 lifted to PSL~(2,R) with canonical (fixed-point) lifts of the
meridians) is
      claimed locally constant on each real branch.  KILLED if e changes between two consecutive grid points of a tracked
branch
      while |relation residual| < 1e-30 (mp arithmetic), or if e depends on the test angle theta0 (it must be central).
 K2b  "Signed area of the relator polygon (base point o) is constant along a branch" is KILLED by any variation > 1e-9 along
the branch.
      (Expected to die; opposite: check whether it is constant for some other base point, e.g. o on an axis or o = fixed
point.)
 K2c  4_1 real curve starts at x0 = arccosh(3/2)/2 with v = 0.  KILLED if the real root v at x slightly above x0 is not ~0,
or real roots exist below x0.
```

q1_commutator.py

```
# SCRIPT: Q1-COMMUTATOR-EXACT
# Exact (Fraction/sympy) study of c = B_i B_j B_i^-1 B_j^-1 for the boosts of peel ratio z.
# Claim to test: tr of c in PSL2(R) is 2 - 4 sinh^4(r/2); elliptic exactly when z < 3+2sqrt2.
# For z = 5: c is an elliptic of INFINITE order => d(o,c^n o) bounded while |c^n| = 4n.
import sympy as sp
from fractions import Fraction as F
import mpmath as mp
mp.mp.dps = 50
def boost(ax, sg, z):
    c = F(z*z+1, 2*z); s = F(z*z-1, 2*z)*sg
    B = sp.eye(4)
    B[0,0] = sp.Rational(c.numerator, c.denominator); B[ax,ax] = B[0,0]
    B[0,ax] = B[ax,0] = sp.Rational(s.numerator, s.denominator)
    return B
def comm(z):
    Bi, Bj = boost(1,1,z), boost(2,1,z)
    return Bi*Bj*Bi.inv()*Bj.inv()
print("# part 1: exact data for z = 5")
z = 5
C = comm(z)
print("c =", C.tolist())
T = C.trace(); print("trace of 4x4 c =", T, "(= 2 + 2cos(phi)... for planar rotation: 1 + [1 + 2cos(phi)])")
x = sp.symbols('x')
cp = sp.factor(C.charpoly(x).as_expr()); print("char poly:", cp)
```

```
# c acts trivially on the k axis, so it lives in SO(2,1) (3x3 block on t,x,y)
C3 = C[:3,:3]; print("3x3 block:", C3.tolist(), " det", C3.det())
cp3 = sp.factor(C3.charpoly(x).as_expr()); print("3x3 char poly:", cp3)
rts = sp.solve(cp3, x); print("eigenvalues:", rts, [sp.N(r,20) for r in rts], [abs(sp.N(r,20)) for r in rts])
cosphi = sp.nsimplify((C3.trace()-1)/2); print("cos(phi) =", cosphi, " sin(phi) =", sp.sqrt(1-cosphi**2), " phi(deg) =",
sp.N(sp.acos(cosphi)*180/sp.pi,15))
print("e^{i phi} = (cos + i sin) with cos = -527/625, sin = 336/625  [(7+24i)/25]^2 ; (7+24i)/25 = ((2+i)/(2-i))^2 -> not a
root of unity")
print("cos(phi) is a non-integer rational, 2cos(phi) = %s is not an algebraic integer, so e^{i phi} is not a root of unity
=> c has INFINITE order" % (2*cosphi))
# fixed point of c in H^2: kernel of (C3 - I)
ker = (C3 - sp.eye(3)).nullspace(); v = ker[0]
nrm = v[0]**2 - v[1]**2 - v[2]**2
if v[0] < 0: v = -v
print("fixed vector", v.T.tolist(), "Minkowski norm^2", nrm)
rho = sp.acosh(v[0]/sp.sqrt(nrm)); print("distance o -> fixed point of c: rho = ", sp.N(rho, 20), " cosh rho =",
sp.nsimplify(v[0]/sp.sqrt(nrm)))
print("hence for all n: d(o, c^n o) <= 2 rho = %s ; exact: sinh(d/2) = sinh(rho)|sin(n phi/2)|" % sp.N(2*rho, 12))
# check by exact matrix powers
P = sp.eye(4); rows = []
for n in range(1, 41):
    P = P*C
    t = P[0,0]; d = mp.acosh(mp.mpf(t.p)/mp.mpf(t.q))
    pred = 2*mp.asinh(mp.sinh(mp.mpf(str(sp.N(rho,40))))*abs(mp.sin(n*mp.acos(mp.mpf(cosphi.p)/cosphi.q)/2)))
    rows.append((n, 4*n, d, pred))
for n, L4, d, pred in rows[:8] + rows[-3:]:
    print("n=%2d  |c^n|=%3d  d(o,c^n o)=%.10f  predicted %.10f  d/|c^n|=%.5f" % (n, L4, d, pred, d/L4))
print("max over n<=40 of d(o,c^n o) = %.6f  (bound 2rho = %.6f)" % (max(r[2] for r in rows), 2*float(rho)))
# part 2: sweep of tr[A,B] = 2 - 4 sinh^4(r/2) with z, exact rational test at several z
print("\n# part 2: trace of the commutator in PSL2(R) versus z (exact: tr = 2 - 4 sinh^4(r/2), sinh^2(r/2) = (z-1)^2/(4z))")
for zz in [F(2), F(3), F(5), F(11,2), F(17,3), F(6), F(7), F(10), F(25)]:
    s2 = (zz-1)**2/(4*zz); tr = 2 - 4*s2*s2
    C = comm(zz) if zz.denominator == 1 else None
    chk = ''
    if C is not None:
        C3 = C[:3,:3]; chk = ' | 3x3 trace-1 over 2 -> %s ; formula (tr/2 normalised): %s' %
(sp.nsimplify((C3.trace()-1)/2), sp.Rational(tr.numerator, tr.denominator)/2 )
    print("z=%-6s tr[A,B]=%-14s %s%s" % (zz, tr, 'ELLIPTIC' if abs(tr) < 2 else ('PARABOLIC' if abs(tr)==2 else
'hyperbolic'), chk))
zt = 3 + 2*mp.sqrt(2); s2 = (zt-1)**2/(4*zt); print("z = 3+2sqrt2 = %s : tr = %s (parabolic, = -2)" % (mp.nstr(zt, 12),
mp.nstr(2-4*s2*s2, 20)))
print("at z=5: sinh^4(r/2) = 16/25 < 1  -> elliptic.   The round-cap Schottky threshold sinh(r/2)=1 <=> z = 3+2sqrt2 is
exactly where tr[A,B] = -2.")
```

Output

```
# part 1: exact data for z = 5
c = [[9841/625, -1248/125, -7584/625, 0], [7584/625, -1027/125, -5616/625, 0], [1248/125, -144/25, -1027/125, 0], [0, 0, 0,
1]]
trace of 4x4 c = 196/625 (= 2 + 2cos(phi)... for planar rotation: 1 + [1 + 2cos(phi)])
char poly: (x - 1)**2*(625*x**2 + 1054*x + 625)/625
3x3 block: [[9841/625, -1248/125, -7584/625], [7584/625, -1027/125, -5616/625], [1248/125, -144/25, -1027/125]]  det 1
3x3 char poly: (x - 1)*(625*x**2 + 1054*x + 625)/625
eigenvalues: [1, -527/625 - 336*I/625, -527/625 + 336*I/625] [1.0000000000000000000, -0.8432 - 0.5376*I, -0.8432 + 0.5376*I]
[1.0000000000000000000, 1.0000000000000000000, 1.0000000000000000000]
cos(phi) = -527/625  sin(phi) = 336/625  phi(deg) = 147.479590583376
e^{i phi} = (cos + i sin) with cos = -527/625, sin = 336/625  [(7+24i)/25]^2 ; (7+24i)/25 = ((2+i)/(2-i))^2 -> not a root of
unity
cos(phi) is a non-integer rational, 2cos(phi) = -1054/625 is not an algebraic integer, so e^{i phi} is not a root of unity
=> c has INFINITE order
fixed vector [[3/2, 1, 1]] Minkowski norm^2 1/4
distance o -> fixed point of c: rho =  1.7627471740390860505  cosh rho = 3
hence for all n: d(o, c^n o) <= 2 rho = 3.52549434808 ; exact: sinh(d/2) = sinh(rho)|sin(n phi/2)|
n= 1  |c^n|=  4  d(o,c^n o)=3.4486982392  predicted 3.4486982392  d/|c^n|=0.86217
n= 2  |c^n|=  8  d(o,c^n o)=2.4122306782  predicted 2.4122306782  d/|c^n|=0.30153
n= 3  |c^n|= 12  d(o,c^n o)=2.7619994088  predicted 2.7619994088  d/|c^n|=0.23017
n= 4  |c^n|= 16  d(o,c^n o)=3.3416836512  predicted 3.3416836512  d/|c^n|=0.20886
n= 5  |c^n|= 20  d(o,c^n o)=0.8314069455  predicted 0.8314069455  d/|c^n|=0.04157
n= 6  |c^n|= 24  d(o,c^n o)=3.5090348718  predicted 3.5090348718  d/|c^n|=0.14621
n= 7  |c^n|= 28  d(o,c^n o)=1.9570954012  predicted 1.9570954012  d/|c^n|=0.06990
n= 8  |c^n|= 32  d(o,c^n o)=3.0294142770  predicted 3.0294142770  d/|c^n|=0.09467
n=38  |c^n|=152  d(o,c^n o)=3.4830971467  predicted 3.4830971467  d/|c^n|=0.02292
n=39  |c^n|=156  d(o,c^n o)=0.4063738501  predicted 0.4063738501  d/|c^n|=0.00260
n=40  |c^n|=160  d(o,c^n o)=3.4037169616  predicted 3.4037169616  d/|c^n|=0.02127
max over n<=40 of d(o,c^n o) = 3.525123  (bound 2rho = 3.525494)

# part 2: trace of the commutator in PSL2(R) versus z (exact: tr = 2 - 4 sinh^4(r/2), sinh^2(r/2) = (z-1)^2/(4z))
z=2      tr[A,B]=31/16          ELLIPTIC | 3x3 trace-1 over 2 -> 449/512 ; formula (tr/2 normalised): 31/32
z=3      tr[A,B]=14/9           ELLIPTIC | 3x3 trace-1 over 2 -> 17/81 ; formula (tr/2 normalised): 7/9
z=5      tr[A,B]=-14/25         ELLIPTIC | 3x3 trace-1 over 2 -> -527/625 ; formula (tr/2 normalised): -7/25
z=11/2   tr[A,B]=-2689/1936     ELLIPTIC
z=17/3   tr[A,B]=-4402/2601     ELLIPTIC
z=6      tr[A,B]=-337/144       hyperbolic | 3x3 trace-1 over 2 -> 72097/41472 ; formula (tr/2 normalised): -337/288
z=7      tr[A,B]=-226/49        hyperbolic | 3x3 trace-1 over 2 -> 23137/2401 ; formula (tr/2 normalised): -113/49
```

```
z=10     tr[A,B]=-5761/400      hyperbolic | 3x3 trace-1 over 2 -> 32869121/320000 ; formula (tr/2 normalised): -5761/800
z=25     tr[A,B]=-81694/625     hyperbolic | 3x3 trace-1 over 2 -> 3336564193/390625 ; formula (tr/2 normalised): -40847/625
z = 3+2sqrt2 = 5.82842712475 : tr = -2.0 (parabolic, = -2)
at z=5: sinh^4(r/2) = 16/25 < 1  -> elliptic.   The round-cap Schottky threshold sinh(r/2)=1 <=> z = 3+2sqrt2 is exactly
where tr[A,B] = -2.
```

q1_enum.py

```
# SCRIPT: Q1-ENUM-TRANSLATION-AND-DRIFT
# Boosts of peel ratio z along +-i,+-j,+-k.  Integer matrices z*B (z odd) or 2z*B (z even).
# Part A: cyclically reduced words up to length LA: EXACT translation length from the integer characteristic polynomial.
#         y^2 - T y + (Q-2) = 0 with y = x + 1/x, eigenvalues e^{+-l}, e^{+-i phi}; loxodromic iff the larger root y1 > 2.
# Part B: displacement d(o,w o)/|w| over ALL reduced words (up to the octahedral symmetry), L <= LB.
# Part C: brute force pairs of distinct words within depth D (D<=5): min d_H/d_T, compared with the displacement minimum
over |u|<=2D.
import sys, math, time, numpy as np
z = int(sys.argv[1]); LA = int(sys.argv[2]); LB = int(sys.argv[3]); tag = sys.argv[4] if len(sys.argv) > 4 else ''
r = math.log(z)
if z % 2: S = z; CC = (z*z+1)//2; SS = (z*z-1)//2     # z odd: z*B has integer entries
else: S = 2*z; CC = z*z+1; SS = z*z-1                  # z even: 2z*B has integer entries
names = ['+i','-i','+j','-j','+k','-k']; inv = [1,0,3,2,5,4]
def Gmat(ax, sg):
    B = np.zeros((4,4), dtype=np.int64)
    for i in range(4): B[i,i] = S
    B[0,0] = B[ax,ax] = CC; B[0,ax] = B[ax,0] = sg*SS
    return B
Gs = [Gmat(1,1),Gmat(1,-1),Gmat(2,1),Gmat(2,-1),Gmat(3,1),Gmat(3,-1)]
print("z =", z, " r = ln z = %.6f   Gamma^2 = %.5f  (round-cap Schottky needs Gamma^2 > 1/2, z > %.5f)" % (r, ((z-
1)/(z+1))**2, 3+2*math.sqrt(2)))
# ---------------- Part A  (first letter fixed to +i by the octahedral symmetry; all counts multiplied by 6)
t0 = time.time()
M = Gs[0][None].copy(); first = np.zeros(1, int); last = np.zeros(1, int); words = np.zeros((1,1), int)
for Lw in range(1, LA+1):
    u = S**Lw
    cyc = (first != np.array(inv)[last]) if Lw > 1 else np.ones(len(first), bool)
    idxs = np.where(cyc)[0]
    ncyc = len(idxs); ne = 0; best_r = 1e9; best_w = None; minl = 1e9; ex = []
    for c0 in range(0, ncyc, 100000):
        ii = idxs[c0:c0+100000]; Mc = M[ii]
        T5 = Mc[:,0,0]+Mc[:,1,1]+Mc[:,2,2]+Mc[:,3,3]
        Mo = Mc.astype(object)
        trsq = np.sum(Mo*np.transpose(Mo,(0,2,1)), axis=(1,2))
        T5o = T5.astype(object); D5 = 2*trsq - T5o*T5o + 8*u*u
        for q in range(len(ii)):
            d5 = int(D5[q]); t5 = int(T5o[q]); assert d5 >= 0
            sd = math.sqrt(d5); k = 4*u - t5
            if not ((k < 0) or (d5 > k*k)):
                ne += 1
                if len(ex) < 2: ex.append(words[ii[q]])
                continue
            num = (d5 - k*k)/(sd + k) if k > 0 else (sd - k)
            w = num/(4*u)
            ell = math.acosh(1 + w) if w > 1e-3 else math.sqrt(2*w)*(1 - w/12)
            minl = min(minl, ell)
            if ell/(Lw*r) < best_r: best_r = ell/(Lw*r); best_w = words[ii[q]]
    print("A L=%2d  cyc.reduced words %8d | elliptic-or-parabolic %5d | min loxodromic l/(L ln z) = %.5f at %s | min l =
%.5f" % (
        Lw, 6*ncyc, 6*ne, best_r, ' '.join(names[g] for g in best_w) if best_w is not None else '-', minl));
sys.stdout.flush()
    if ne: print("     elliptic examples:", ['  '.join(names[g] for g in w) for w in ex])
    if Lw < LA:
        nM = []; nF = []; nL = []; nW = []
        for h in range(6):
            mk = last != inv[h]
            nM.append(np.einsum('nij,jk->nik', M[mk], Gs[h])); nF.append(first[mk]); nL.append(np.full(mk.sum(), h));
nW.append(np.hstack([words[mk], np.full((mk.sum(),1), h)]))
        M = np.concatenate(nM); first = np.concatenate(nF); last = np.concatenate(nL); words = np.concatenate(nW); del nM,
nW
print("Part A time %.1fs" % (time.time()-t0))
# ---------------- Part B
t0 = time.time()
# w = g_1 ... g_L, point p(w) = B_g1 ... B_gL e0 built by PREPENDING.  Symmetry (signed permutations of axes) lets us fix
g_L = +i, g_{L-1} in {+i,+j}.
P = np.array([[Gs[0][i,0] for i in range(4)]], dtype=np.int64)       # word '+i'
Fst = np.array([0])
print("B L=1  min d/(L)=%.5f  (= ln z %.5f)" % (math.acosh(Gs[0][0,0]/S), r))
best = {1: math.acosh(Gs[0][0,0]/S)}
P2 = np.array([Gs[0] @ np.array([Gs[0][i,0] for i in range(4)]), Gs[2] @ np.array([Gs[0][i,0] for i in range(4)])],
dtype=np.int64)
P = P2; Fst = np.array([0, 2])
def mind(P, Lw):
    t = P[:,0].astype(np.float64)/float(S**Lw)
```

```
    return float(np.arccosh(np.maximum(t, 1.0)).min())
best[2] = mind(P, 2); print("B L=2  min d/L = %.5f" % (best[2]/2))
for Lw in range(3, LB+1):
    nP = []; nF = []
    for h in range(6):
        mk = Fst != inv[h]
        nP.append(P[mk] @ Gs[h].T); nF.append(np.full(mk.sum(), h, dtype=Fst.dtype))
    P = np.concatenate(nP); Fst = np.concatenate(nF); del nP, nF
    best[Lw] = mind(P, Lw)
    print("B L=%2d  words(up to symmetry) %9d  min d(o,wo) = %.5f  min d/L = %.5f  min d/(L ln z) = %.5f   [%.0fs]" % (Lw,
len(Fst), best[Lw], best[Lw]/Lw, best[Lw]/(Lw*r), time.time()-t0))
    sys.stdout.flush()
# ---------------- Part C: brute-force pairs, depth D<=5
def words_and_points(D):
    Gf = [g.astype(np.float64)/S for g in Gs]
    wl = [()]; Ms = [np.eye(4)]; lay = [((), np.eye(4))]
    for _ in range(D):
        nl = []
        for w, Mw in lay:
            for h in range(6):
                if w and inv[w[-1]] == h: continue
                nl.append((w + (h,), Mw @ Gf[h]))
        lay = nl; wl += [w for w, _ in nl]; Ms += [m for _, m in nl]
    return wl, np.array([m[:,0] for m in Ms])
for D in range(1, 6):
    wl, X = words_and_points(D); n = len(X)
    best_ratio = 1e9
    ip = X[:,0][:,None]*X[:,0][None,:] - (X[:,1:] @ X[:,1:].T)
    dh = np.arccosh(np.maximum(ip, 1.0))
    for a in range(n):
        wa = wl[a]
        for b in range(a+1, n):
            wb = wl[b]; k = 0; m = min(len(wa), len(wb))
            while k < m and wa[k] == wb[k]: k += 1
            dt = len(wa) + len(wb) - 2*k
            q = dh[a,b]/dt
            if q < best_ratio: best_ratio = q
    ref = min(best[l]/l for l in range(1, min(2*D, LB)+1))
    print("C depth D=%d  points %5d  min over pairs of d_H/d_T = %.6f   | min over |u|<=%d of d(o,uo)/|u| = %.6f  (match:
%s)" % (D, n, best_ratio, 2*D, ref, abs(best_ratio-ref) < 1e-9 or (2*D>LB)))
```

q1_enum_z4.txt

```
z = 4  r = ln z = 1.386294   Gamma^2 = 0.36000  (round-cap Schottky needs Gamma^2 > 1/2, z > 5.82843)
A L= 1  cyc.reduced words        6 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 1.00000 at +i | min l =
1.38629
A L= 2  cyc.reduced words       30 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.73314 at +i +j | min l =
2.03270
A L= 3  cyc.reduced words      126 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.63684 at +i +k +j | min l =
2.64853
A L= 4  cyc.reduced words      630 | elliptic-or-parabolic    24 | min loxodromic l/(L ln z) = 0.47147 at +i +k -i +j | min
l = 2.61439
     elliptic examples: ['+i  -j  -i  +j', '+i  +j  -i  -j']
A L= 5  cyc.reduced words     3126 | elliptic-or-parabolic   120 | min loxodromic l/(L ln z) = 0.32320 at +i +k -j -i +j |
min l = 2.24023
     elliptic examples: ['+i  -j  -i  +j  +i', '+i  +j  -i  -j  +i']
A L= 6  cyc.reduced words    15630 | elliptic-or-parabolic   288 | min loxodromic l/(L ln z) = 0.14865 at +i -j -i +j +i +i
| min l = 1.23645
     elliptic examples: ['+i  -j  -i  +j  +i  +j', '+i  +j  -i  -j  +i  +j']
A L= 7  cyc.reduced words    78126 | elliptic-or-parabolic   336 | min loxodromic l/(L ln z) = 0.15105 at +i +k +j +i -j -i
+j | min l = 1.46578
     elliptic examples: ['+i  +j  -i  -j  +i  +j  +i', '+i  +j  +i  -j  -i  +j  +i']
A L= 8  cyc.reduced words   390630 | elliptic-or-parabolic   408 | min loxodromic l/(L ln z) = 0.12432 at +i -k -i +k +i -j
-i +j | min l = 1.37878
     elliptic examples: ['+i  +j  +j  +i  -j  -i  +j  +i', '+i  +j  -i  -j  +i  +j  +j  +i']
A L= 9  cyc.reduced words  1953126 | elliptic-or-parabolic  1080 | min loxodromic l/(L ln z) = 0.05540 at +i -k -i +k +j +i
-j -i +j | min l = 0.69121
     elliptic examples: ['+i  +j  +j  +i  -j  -i  +j  +i  +i', '+i  +j  -i  -j  +i  +j  +j  +i  +i']
Part A time 2.3s
B L=1  min d/(L)=1.38629  (= ln z 1.38629)
B L=2  min d/L = 1.09410
B L= 3  words(up to symmetry)        10  min d(o,wo) = 2.49133  min d/L = 0.83044  min d/(L ln z) = 0.59904   [0s]
B L= 4  words(up to symmetry)        50  min d(o,wo) = 2.38014  min d/L = 0.59503  min d/(L ln z) = 0.42923   [0s]
B L= 5  words(up to symmetry)       250  min d(o,wo) = 1.82287  min d/L = 0.36457  min d/(L ln z) = 0.26298   [0s]
B L= 6  words(up to symmetry)      1250  min d(o,wo) = 0.69054  min d/L = 0.11509  min d/(L ln z) = 0.08302   [0s]
B L= 7  words(up to symmetry)      6250  min d(o,wo) = 0.81784  min d/L = 0.11683  min d/(L ln z) = 0.08428   [0s]
B L= 8  words(up to symmetry)     31250  min d(o,wo) = 0.01036  min d/L = 0.00129  min d/(L ln z) = 0.00093   [0s]
B L= 9  words(up to symmetry)    156250  min d(o,wo) = 0.53521  min d/L = 0.05947  min d/(L ln z) = 0.04290   [0s]
B L=10  words(up to symmetry)    781250  min d(o,wo) = 0.02201  min d/L = 0.00220  min d/(L ln z) = 0.00159   [0s]
B L=11  words(up to symmetry)   3906250  min d(o,wo) = 0.07583  min d/L = 0.00689  min d/(L ln z) = 0.00497   [0s]
B L=12  words(up to symmetry)  19531250  min d(o,wo) = 0.01440  min d/L = 0.00120  min d/(L ln z) = 0.00087   [3s]
C depth D=1  points     7  min over pairs of d_H/d_T = 1.094099   | min over |u|<=2 of d(o,uo)/|u| = 1.094099  (match: True)
```

```
C depth D=2  points    37  min over pairs of d_H/d_T = 0.595034   | min over |u|<=4 of d(o,uo)/|u| = 0.595034  (match: True)
C depth D=3  points   187  min over pairs of d_H/d_T = 0.115090   | min over |u|<=6 of d(o,uo)/|u| = 0.115090  (match: True)
C depth D=4  points   937  min over pairs of d_H/d_T = 0.001295   | min over |u|<=8 of d(o,uo)/|u| = 0.001295  (match: True)
C depth D=5  points  4687  min over pairs of d_H/d_T = 0.001295   | min over |u|<=10 of d(o,uo)/|u| = 0.001295  (match:
True)
```

q1_enum_z5.txt

```
z = 5  r = ln z = 1.609438   Gamma^2 = 0.44444  (round-cap Schottky needs Gamma^2 > 1/2, z > 5.82843)
A L= 1  cyc.reduced words        6 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 1.00000 at +i | min l =
1.60944
A L= 2  cyc.reduced words       30 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.74120 at +i +j | min l =
2.38582
A L= 3  cyc.reduced words      126 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.65570 at +i +k +j | min l =
3.16591
A L= 4  cyc.reduced words      630 | elliptic-or-parabolic    24 | min loxodromic l/(L ln z) = 0.51727 at +i +k -i +j | min
l = 3.33008
     elliptic examples: ['+i  -j  -i  +j', '+i  +j  -i  -j']
A L= 5  cyc.reduced words     3126 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.34025 at +i -j -i +j +i |
min l = 2.73808
A L= 6  cyc.reduced words    15630 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.35049 at +i -j -i +j +i +j
| min l = 3.38458
A L= 7  cyc.reduced words    78126 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.35599 at +i +k +j +i -j -i
+j | min l = 4.01059
A L= 8  cyc.reduced words   390630 | elliptic-or-parabolic    24 | min loxodromic l/(L ln z) = 0.27546 at +i +k -i +j +i -j
-i +j | min l = 3.54674
     elliptic examples: ['+i  -j  -i  +j  +i  -j  -i  +j', '+i  +j  -i  -j  +i  +j  -i  -j']
A L= 9  cyc.reduced words  1953126 | elliptic-or-parabolic   216 | min loxodromic l/(L ln z) = 0.18271 at +i +k -j -i +j +i
-j -i +j | min l = 2.64660
     elliptic examples: ['+i  -j  -i  +j  +i  -j  -i  +j  +i', '+i  +j  -i  -j  +i  +j  -i  -j  +i']
Part A time 2.2s
B L=1  min d/(L)=1.60944  (= ln z 1.60944)
B L=2  min d/L = 1.29933
B L= 3  words(up to symmetry)        10  min d(o,wo) = 3.16081  min d/L = 1.05360  min d/(L ln z) = 0.65464   [0s]
B L= 4  words(up to symmetry)        50  min d(o,wo) = 3.44870  min d/L = 0.86217  min d/(L ln z) = 0.53570   [0s]
B L= 5  words(up to symmetry)       250  min d(o,wo) = 3.52414  min d/L = 0.70483  min d/(L ln z) = 0.43793   [0s]
B L= 6  words(up to symmetry)      1250  min d(o,wo) = 3.40126  min d/L = 0.56688  min d/(L ln z) = 0.35222   [0s]
B L= 7  words(up to symmetry)      6250  min d(o,wo) = 3.05660  min d/L = 0.43666  min d/(L ln z) = 0.27131   [0s]
B L= 8  words(up to symmetry)     31250  min d(o,wo) = 2.41223  min d/L = 0.30153  min d/(L ln z) = 0.18735   [0s]
B L= 9  words(up to symmetry)    156250  min d(o,wo) = 1.28619  min d/L = 0.14291  min d/(L ln z) = 0.08880   [0s]
B L=10  words(up to symmetry)    781250  min d(o,wo) = 0.42579  min d/L = 0.04258  min d/(L ln z) = 0.02646   [0s]
B L=11  words(up to symmetry)   3906250  min d(o,wo) = 1.46483  min d/L = 0.13317  min d/(L ln z) = 0.08274   [1s]
B L=12  words(up to symmetry)  19531250  min d(o,wo) = 1.06447  min d/L = 0.08871  min d/(L ln z) = 0.05512   [3s]
C depth D=1  points     7  min over pairs of d_H/d_T = 1.299327   | min over |u|<=2 of d(o,uo)/|u| = 1.299327  (match: True)
C depth D=2  points    37  min over pairs of d_H/d_T = 0.862175   | min over |u|<=4 of d(o,uo)/|u| = 0.862175  (match: True)
C depth D=3  points   187  min over pairs of d_H/d_T = 0.566876   | min over |u|<=6 of d(o,uo)/|u| = 0.566876  (match: True)
C depth D=4  points   937  min over pairs of d_H/d_T = 0.301529   | min over |u|<=8 of d(o,uo)/|u| = 0.301529  (match: True)
C depth D=5  points  4687  min over pairs of d_H/d_T = 0.042579   | min over |u|<=10 of d(o,uo)/|u| = 0.042579  (match:
True)
```

q1_enum_z6.txt

```
z = 6  r = ln z = 1.791759   Gamma^2 = 0.51020  (round-cap Schottky needs Gamma^2 > 1/2, z > 5.82843)
A L= 1  cyc.reduced words        6 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 1.00000 at +i | min l =
1.79176
A L= 2  cyc.reduced words       30 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.74825 at +i +j | min l =
2.68137
A L= 3  cyc.reduced words      126 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.67202 at +i +k +j | min l =
3.61231
A L= 4  cyc.reduced words      630 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.16056 at +i -j -i +j | min
l = 1.15073
A L= 5  cyc.reduced words     3126 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.49984 at +i +k -j -i +j |
min l = 4.47799
A L= 6  cyc.reduced words    15630 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.48510 at +i -j -i +j +i +j
| min l = 5.21510
A L= 7  cyc.reduced words    78126 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.47486 at +i +k +j +i -j -i
+j | min l = 5.95590
A L= 8  cyc.reduced words   390630 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.16056 at +i -j -i +j +i -j
-i +j | min l = 2.30146
A L= 9  cyc.reduced words  1953126 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.36796 at +i -j -i +j +i -j
-i +j +i | min l = 5.93366
Part A time 2.2s
B L=1  min d/(L)=1.79176  (= ln z 1.79176)
B L=2  min d/L = 1.47120
B L= 3  words(up to symmetry)        10  min d(o,wo) = 3.72782  min d/L = 1.24261  min d/(L ln z) = 0.69351   [0s]
B L= 4  words(up to symmetry)        50  min d(o,wo) = 4.32852  min d/L = 1.08213  min d/(L ln z) = 0.60395   [0s]
B L= 5  words(up to symmetry)       250  min d(o,wo) = 4.82476  min d/L = 0.96495  min d/(L ln z) = 0.53855   [0s]
B L= 6  words(up to symmetry)      1250  min d(o,wo) = 5.25659  min d/L = 0.87610  min d/(L ln z) = 0.48896   [0s]
```

```
B L= 7  words(up to symmetry)      6250  min d(o,wo) = 5.64632  min d/L = 0.80662  min d/(L ln z) = 0.45018   [0s]
B L= 8  words(up to symmetry)     31250  min d(o,wo) = 6.00743  min d/L = 0.75093  min d/(L ln z) = 0.41910   [0s]
B L= 9  words(up to symmetry)    156250  min d(o,wo) = 6.34856  min d/L = 0.70540  min d/(L ln z) = 0.39369   [0s]
B L=10  words(up to symmetry)    781250  min d(o,wo) = 6.67545  min d/L = 0.66754  min d/(L ln z) = 0.37256   [0s]
C depth D=1  points     7  min over pairs of d_H/d_T = 1.471196   | min over |u|<=2 of d(o,uo)/|u| = 1.471196  (match: True)
C depth D=2  points    37  min over pairs of d_H/d_T = 1.082129   | min over |u|<=4 of d(o,uo)/|u| = 1.082129  (match: True)
C depth D=3  points   187  min over pairs of d_H/d_T = 0.876099   | min over |u|<=6 of d(o,uo)/|u| = 0.876099  (match: True)
C depth D=4  points   937  min over pairs of d_H/d_T = 0.750929   | min over |u|<=8 of d(o,uo)/|u| = 0.750929  (match: True)
C depth D=5  points  4687  min over pairs of d_H/d_T = 0.667545   | min over |u|<=10 of d(o,uo)/|u| = 0.667545  (match:
True)
```

q1_enum_z7.txt

```
z = 7  r = ln z = 1.945910   Gamma^2 = 0.56250  (round-cap Schottky needs Gamma^2 > 1/2, z > 5.82843)
A L= 1  cyc.reduced words        6 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 1.00000 at +i | min l =
1.94591
A L= 2  cyc.reduced words       30 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.75446 at +i +j | min l =
2.93624
A L= 3  cyc.reduced words      126 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.68612 at +i +k +j | min l =
4.00538
A L= 4  cyc.reduced words      630 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.37977 at +i -j -i +j | min
l = 2.95599
A L= 5  cyc.reduced words     3126 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.55777 at +i +k -j -i +j |
min l = 5.42686
A L= 6  cyc.reduced words    15630 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.55557 at +i -j -i +j +i +j
| min l = 6.48651
A L= 7  cyc.reduced words    78126 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.54226 at +i +k +j +i -j -i
+j | min l = 7.38630
A L= 8  cyc.reduced words   390630 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.37977 at +i -j -i +j +i -j
-i +j | min l = 5.91198
A L= 9  cyc.reduced words  1953126 | elliptic-or-parabolic     0 | min loxodromic l/(L ln z) = 0.48194 at +i +k -j -i +j +i
-j -i +j | min l = 8.44024
Part A time 2.3s
B L=1  min d/(L)=1.94591  (= ln z 1.94591)
B L=2  min d/L = 1.61877
B L= 3  words(up to symmetry)        10  min d(o,wo) = 4.21335  min d/L = 1.40445  min d/(L ln z) = 0.72175   [0s]
B L= 4  words(up to symmetry)        50  min d(o,wo) = 5.05910  min d/L = 1.26478  min d/(L ln z) = 0.64997   [0s]
B L= 5  words(up to symmetry)       250  min d(o,wo) = 5.84767  min d/L = 1.16953  min d/(L ln z) = 0.60102   [0s]
B L= 6  words(up to symmetry)      1250  min d(o,wo) = 6.61003  min d/L = 1.10167  min d/(L ln z) = 0.56615   [0s]
B L= 7  words(up to symmetry)      6250  min d(o,wo) = 7.36012  min d/L = 1.05145  min d/(L ln z) = 0.54034   [0s]
B L= 8  words(up to symmetry)     31250  min d(o,wo) = 8.10439  min d/L = 1.01305  min d/(L ln z) = 0.52060   [0s]
B L= 9  words(up to symmetry)    156250  min d(o,wo) = 8.84591  min d/L = 0.98288  min d/(L ln z) = 0.50510   [0s]
B L=10  words(up to symmetry)    781250  min d(o,wo) = 9.58611  min d/L = 0.95861  min d/(L ln z) = 0.49263   [0s]
B L=11  words(up to symmetry)   3906250  min d(o,wo) = 10.32568  min d/L = 0.93870  min d/(L ln z) = 0.48240   [0s]
C depth D=1  points     7  min over pairs of d_H/d_T = 1.618769   | min over |u|<=2 of d(o,uo)/|u| = 1.618769  (match: True)
C depth D=2  points    37  min over pairs of d_H/d_T = 1.264775   | min over |u|<=4 of d(o,uo)/|u| = 1.264775  (match: True)
C depth D=3  points   187  min over pairs of d_H/d_T = 1.101672   | min over |u|<=6 of d(o,uo)/|u| = 1.101672  (match: True)
C depth D=4  points   937  min over pairs of d_H/d_T = 1.013049   | min over |u|<=8 of d(o,uo)/|u| = 1.013049  (match: True)
C depth D=5  points  4687  min over pairs of d_H/d_T = 0.958611   | min over |u|<=10 of d(o,uo)/|u| = 0.958611  (match:
True)
```

q1_identify.py

```
# SCRIPT: Q1-IDENTIFY-WORST-WORDS
# Which reduced words (first letter +i, symmetry-reduced) give the smallest d(o,w o)/|w| at z=5?  And the paper's "0.4258".
import numpy as np, math
z=5; S=z; CC=(z*z+1)//2; SS=(z*z-1)//2
names=['+i','-i','+j','-j','+k','-k']; inv=[1,0,3,2,5,4]
def G(ax,sg):
    B=np.zeros((4,4),dtype=np.int64)
    for i in range(4): B[i,i]=S
    B[0,0]=B[ax,ax]=CC; B[0,ax]=B[ax,0]=sg*SS; return B
Gs=[G(1,1),G(1,-1),G(2,1),G(2,-1),G(3,1),G(3,-1)]
P=np.array([Gs[0][:,0]]); words=np.zeros((1,1),int); last=np.zeros(1,int)    # words built by APPENDING; p(w)=M e0 needs
matrices
M=Gs[0][None].copy()
for L in range(1,11):
    t=M[:,0,0].astype(float)/float(S**L); d=np.arccosh(np.maximum(t,1.0))
    if L>=8:
        order=np.argsort(d)[:4]
        print("L=%d  smallest d(o,wo)/L: %.5f   examples (d, word):"%(L,d.min()/L))
        for o in order: print("     %.5f  %s"%(d[o],' '.join(names[g] for g in words[o])))
    if L<10:
        nM=[];nL=[];nW=[]
        for h in range(6):
            mk=last!=inv[h]
            nM.append(np.einsum('nij,jk->nik',M[mk],Gs[h])); nL.append(np.full(mk.sum(),h));
```

```
nW.append(np.hstack([words[mk],np.full((mk.sum(),1),h)]))
        M=np.concatenate(nM); last=np.concatenate(nL); words=np.concatenate(nW)
```

Output

```
L=8  smallest d(o,wo)/L: 0.30153   examples (d, word):
     2.41223  +i +j -i -j +i +j -i -j
     2.41223  +i -j -i +j +i -j -i +j
     2.41223  +i -k -i +k +i -k -i +k
     2.41223  +i +k -i -k +i +k -i -k
L=9  smallest d(o,wo)/L: 0.14291   examples (d, word):
     1.28619  +i +k -i -k +i +k -i -k +i
     1.28619  +i +j -i -j +i +j -i -j +i
     1.28619  +i -k -i +k +i -k -i +k +i
     1.28619  +i -j -i +j +i -j -i +j +i
L=10  smallest d(o,wo)/L: 0.04258   examples (d, word):
     0.42579  +i -j -i +j +i -j -i +j +i -j
     0.42579  +i +k -i -k +i +k -i -k +i +k
     0.42579  +i -k -i +k +i -k -i +k +i -k
     0.42579  +i +j -i -j +i +j -i -j +i +j
```

q1_orbit_accum.py

```
# SCRIPT: Q1-ORBIT-ACCUMULATION
# d(o, c^n o) for c = [B_i,B_j] at z=5 from the exact formula sinh(d/2) = sinh(rho)|sin(n phi/2)|, cosh rho = 3, cos phi =
-527/625.
# Cross-checked against exact integer matrix powers for n <= 60.  Question: how close does the orbit point c^n o come back
to o?
import math, numpy as np
from fractions import Fraction as F
rho = math.acosh(3.0); phi = math.acos(-527/625)
n = np.arange(1, 2000001); s = np.sinh(rho)*np.abs(np.sin(n*phi/2)); d = 2*np.arcsinh(s)
print("rho = %.10f   2 rho = %.10f   phi = %.10f rad = %.6f deg" % (rho, 2*rho, phi, math.degrees(phi)))
print("max_n d(o,c^n o) = %.8f (<= 2 rho)" % d.max())
for th in [1e-1, 1e-2, 1e-3, 1e-4, 1e-5]:
    i = np.argmax(d < th); print("first n with d(o,c^n o) < %g : n = %7d  (word length |c^n| = %8d), d = %.3e, d/|c^n| =
%.2e" % (th, n[i], 4*n[i], d[i], d[i]/(4*n[i])))
# exact check n<=60
Z = 5
def boost(ax, sg):
    c = F(Z*Z+1, 2*Z); s_ = F(Z*Z-1, 2*Z)*sg
    B = [[F(int(i == j)) for j in range(4)] for i in range(4)]; B[0][0] = B[ax][ax] = c; B[0][ax] = B[ax][0] = s_; return B
def mm(A, B): return [[sum(A[i][k]*B[k][j] for k in range(4)) for j in range(4)] for i in range(4)]
Bi, Bj, Bin, Bjn = boost(1, 1), boost(2, 1), boost(1, -1), boost(2, -1)
c = mm(mm(mm(Bi, Bj), Bin), Bjn); P = [[F(int(i == j)) for j in range(4)] for i in range(4)]; worst = 0
for k in range(1, 61):
    P = mm(P, c); dd = math.acosh(float(P[0][0])); worst = max(worst, abs(dd - d[k-1]))
print("exact matrix powers vs formula, n<=60: max abs difference %.2e" % worst)
```

Output

```
rho = 1.7627471740   2 rho = 3.5254943481   phi = 2.5740044352 rad = 147.479591 deg
max_n d(o,c^n o) = 3.52549435 (<= 2 rho)
first n with d(o,c^n o) < 0.1 : n =      83  (word length |c^n| =      332), d = 3.979e-02, d/|c^n| = 1.20e-04
first n with d(o,c^n o) < 0.01 : n =    1345  (word length |c^n| =     5380), d = 2.435e-03, d/|c^n| = 4.53e-07
first n with d(o,c^n o) < 0.001 : n =    7594  (word length |c^n| =    30376), d = 5.376e-04, d/|c^n| = 1.77e-08
first n with d(o,c^n o) < 0.0001 : n =   65656  (word length |c^n| =   262624), d = 3.248e-05, d/|c^n| = 1.24e-10
first n with d(o,c^n o) < 1e-05 : n =  561873  (word length |c^n| =  2247492), d = 7.243e-06, d/|c^n| = 3.22e-12
exact matrix powers vs formula, n<=60: max abs difference 2.26e-14
```

q1_pingpong.py

```
# SCRIPT: Q1-PINGPONG-SEARCH
# Numerical search for a ping-pong (Schottky) configuration of round caps on the sphere at infinity.
# For generators g (with g^-1 in the list): caps C_g = cap(n_g, rho_g) with
#   (i) g( S^2 \ C_{g^-1} ) subset C_g        margin_g  = rho_g - ( angle(n_g, n') + rho' )   [cap(n',rho') = image of the
complement of C_{g^-1}]
#  (ii) caps pairwise disjoint                margin_gh = angle(n_g, n_h) - rho_g - rho_h
# We maximise t = min of all margins over free centres and free radii (SLSQP on the epigraph form, many restarts).  t > 0
<=> valid configuration.
import sys, time, numpy as np, math
from scipy.optimize import minimize
rng = np.random.default_rng(int(sys.argv[1]) if len(sys.argv) > 1 else 0)
def boost(u, r):
```

```
    u = np.asarray(u, float); B = np.eye(4); c, s = math.cosh(r), math.sinh(r)
    B[0,0] = c; B[0,1:] = s*u; B[1:,0] = s*u; B[1:,1:] += (c-1)*np.outer(u,u); return B
AX = {'+i':(1,0,0),'-i':(-1,0,0),'+j':(0,1,0),'-j':(0,-1,0),'+k':(0,0,1),'-k':(0,0,-1)}
INV = {'+i':'-i','-i':'+i','+j':'-j','-j':'+j','+k':'-k','-k':'+k'}
def gens_boosts(z): r = math.log(z); return {g: boost(u, r) for g, u in AX.items()}
def mob(B, x):
    v = B @ np.concatenate([[1.0], x]); return v[1:]/v[0]
def circle_cap(B, n, rho):
    """image under B of the cap(n,rho) as (centre, radius)"""
    n = n/np.linalg.norm(n)
    a = np.cross(n, [1,0,0] if abs(n[0]) < .9 else [0,1,0]); a /= np.linalg.norm(a); b = np.cross(n, a)
    pts = [math.cos(rho)*n + math.sin(rho)*(math.cos(t)*a + math.sin(t)*b) for t in (0, 2.0944, 4.18879, 1.0)]
    q = [mob(B, p) for p in pts]
    m = np.cross(q[1]-q[0], q[2]-q[0]); m /= np.linalg.norm(m); h = m @ q[0]
    c2 = mob(B, n)
    if m @ c2 >= h: return m, math.acos(max(-1, min(1, h)))
    return -m, math.acos(max(-1, min(1, -h)))
def ang(a, b): return math.acos(max(-1, min(1, a @ b)))
def margins(x, names, Bs, inv):
    k = len(names); N = []; R = []
    for i in range(k):
        v = x[4*i:4*i+3]; N.append(v/np.linalg.norm(v)); R.append(x[4*i+3])
    out = []
    for i, g in enumerate(names):
        j = names.index(inv[g])                      # index of g^-1
        nc, rc = circle_cap(Bs[g], -N[j], math.pi - R[j])     # image of the complement of C_{g^-1}
        out.append(R[i] - (ang(N[i], nc) + rc))
    for i in range(k):
        for j in range(i+1, k):
            out.append(ang(N[i], N[j]) - R[i] - R[j])
    return np.array(out)
def best_margin(names, Bs, inv, restarts=30, structured=None):
    k = len(names); bestt = -9; bestx = None
    starts = []
    if structured is not None: starts.append(structured)
    for _ in range(restarts):
        x = []
        for i in range(k):
            v = rng.normal(size=3); v /= np.linalg.norm(v); x += list(v) + [rng.uniform(0.1, 1.2)]
        starts.append(np.array(x))
    for x0 in starts:
        t0 = margins(x0, names, Bs, inv).min()
        z0 = np.concatenate([x0, [t0]])
        cons = {'type': 'ineq', 'fun': lambda zz: margins(zz[:-1], names, Bs, inv) - zz[-1]}
        bnds = [(None, None)]*3 + [(0.01, math.pi-0.01)]; bnds = bnds*k + [(None, None)]
        try:
            res = minimize(lambda zz: -zz[-1], z0, constraints=[cons], bounds=bnds, method='SLSQP', options={'maxiter': 120,
'ftol': 1e-12})
        except Exception as e:
            continue
        t = margins(res.x[:-1], names, Bs, inv).min()
        if t > bestt: bestt, bestx = t, res.x[:-1]
    return bestt, bestx
def structured_start(rho):
    x = []
    for g in AX: x += list(AX[g]) + [rho]
    return np.array(x)
names = list(AX)
mode = sys.argv[2]; NR = int(sys.argv[3]); T0 = time.time()
def stamp(): return "[%.0fs]" % (time.time()-T0)
if mode == 'sym':
    print("SYMMETRIC round caps centred on the six axis points, common radius rho (control): best common margin vs z")
    for z in [3, 4, 5, 5.5, 5.8, 3+2*math.sqrt(2), 5.9, 6, 7, 10, 25]:
        Bs = gens_boosts(z); bt = max((margins(structured_start(rho), names, Bs, INV).min(), rho) for rho in
np.linspace(0.05, math.pi/4, 400))
        print("  z=%-8.4f  best symmetric margin %+.5f at rho=%.4f (pi/4=%.4f)" % (z, bt[0], bt[1], math.pi/4))
if mode == 'free':
    print("FREE caps (centres and radii free, per generator): %d random restarts + the symmetric start, SLSQP epigraph form"
% NR)
    for z in [float(t_) for t_ in sys.argv[4].split(',')]:
        Bs = gens_boosts(z); bt, bx = best_margin(names, Bs, INV, restarts=NR, structured=structured_start(math.pi/4))
        print("  z=%-6s  best min-margin found = %+.6f   -> %s  %s" % (z, bt, 'VALID ping-pong configuration' if bt > 1e-9
else 'no valid configuration found', stamp()))
        sys.stdout.flush()
if mode == 'powers':
    print("POWERS at z = 5: generators B_u^n (rapidity n ln 5), free caps, %d restarts:" % NR)
    for n in [1, 2, 3]:
        Bs = {g: np.linalg.matrix_power(gens_boosts(5)[g], n) for g in AX}
        bt, bx = best_margin(names, Bs, INV, restarts=NR, structured=structured_start(math.pi/4))
        print("  n=%d  (z^n = %d)  best min-margin = %+.6f  -> %s  %s" % (n, 5**n, bt, 'VALID' if bt > 1e-9 else 'none
found', stamp())); sys.stdout.flush()
if mode == 'sub':
    print("SUBGROUPS generated by words of length 2/3 at z = 5 (free caps, %d restarts):" % NR)
    B5 = gens_boosts(5)
    def sub(words, label):
        Bs = {}; nm = []
```

```
        for i, w in enumerate(words):
            M = np.eye(4)
            for g in w.split(): M = M @ B5[g]
            Bs['g%d' % i] = M; Bs['G%d' % i] = np.linalg.inv(M); nm += ['g%d' % i, 'G%d' % i]
        inv = {}
        for i in range(len(words)): inv['g%d' % i] = 'G%d' % i; inv['G%d' % i] = 'g%d' % i
        bt, bx = best_margin(nm, Bs, inv, restarts=NR)
        print("  %-40s best min-margin = %+.6f -> %s  %s" % (label, bt, 'VALID (Schottky, hence QI)' if bt > 1e-9 else 'none
found', stamp())); sys.stdout.flush()
    sub(['+i +i', '+j +j', '+k +k'], '<B_i^2, B_j^2, B_k^2>')
    sub(['+i +j', '+j +k', '+k +i'], '<B_iB_j, B_jB_k, B_kB_i>')
    sub(['+i +j', '+i -j'], '<B_iB_j, B_iB_j^-1>')
    sub(['+i +j +k', '+j +k +i'], '<B_iB_jB_k, B_jB_kB_i>')
```

q1_pp_sym.txt

```
SYMMETRIC round caps centred on the six axis points, common radius rho (control): best common margin vs z
  z=3.0000    best symmetric margin -0.56985 at rho=0.7854 (pi/4=0.7854)
  z=4.0000    best symmetric margin -0.30066 at rho=0.7854 (pi/4=0.7854)
  z=5.0000    best symmetric margin -0.11426 at rho=0.7854 (pi/4=0.7854)
  z=5.5000    best symmetric margin -0.04185 at rho=0.7854 (pi/4=0.7854)
  z=5.8000    best symmetric margin -0.00346 at rho=0.7854 (pi/4=0.7854)
  z=5.8284    best symmetric margin -0.00000 at rho=0.7854 (pi/4=0.7854)
  z=5.9000    best symmetric margin +0.00369 at rho=0.7836 (pi/4=0.7854)
  z=6.0000    best symmetric margin +0.00933 at rho=0.7799 (pi/4=0.7854)
  z=7.0000    best symmetric margin +0.06159 at rho=0.7541 (pi/4=0.7854)
  z=10.0000   best symmetric margin +0.16632 at rho=0.7006 (pi/4=0.7854)
  z=25.0000   best symmetric margin +0.35388 at rho=0.6085 (pi/4=0.7854)
```

q1_pp_free.txt

```
FREE caps (centres and radii free, per generator): 8 random restarts + the symmetric start, SLSQP epigraph form
  z=4.0     best min-margin found = -0.143834   -> no valid configuration found  [37s]
  z=5.0     best min-margin found = -0.056021   -> no valid configuration found  [79s]
  z=5.7     best min-margin found = -0.007916   -> no valid configuration found  [118s]
  z=5.9     best min-margin found = +0.004303   -> VALID ping-pong configuration  [153s]
  z=6.0     best min-margin found = +0.010191   -> VALID ping-pong configuration  [193s]
  z=7.0     best min-margin found = +0.062117   -> VALID ping-pong configuration  [231s]
```

q1_pp_powers.txt

```
POWERS at z = 5: generators B_u^n (rapidity n ln 5), free caps, 8 restarts:
  n=1  (z^n = 5)  best min-margin = -0.056021  -> none found  [38s]
  n=2  (z^n = 25)  best min-margin = +0.354547  -> VALID  [81s]
  n=3  (z^n = 125)  best min-margin = -1.033588  -> none found  [120s]
```

q1_pp_sub.txt

```
SUBGROUPS generated by words of length 2/3 at z = 5 (free caps, 8 restarts):
  <B_i^2, B_j^2, B_k^2>                   best min-margin = +0.350819 -> VALID (Schottky, hence QI)  [33s]
  <B_iB_j, B_jB_k, B_kB_i>                best min-margin = -1.092760 -> none found  [67s]
  <B_iB_j, B_iB_j^-1>                     best min-margin = +0.093483 -> VALID (Schottky, hence QI)  [80s]
  <B_iB_jB_k, B_jB_kB_i>                  best min-margin = +0.100447 -> VALID (Schottky, hence QI)  [95s]
```

q2_lib.py

```
# SCRIPT: Q2-LIB
# Library for the relator-loop computations: real Riley branches, canonical lifts to PSL~(2,R), relator polygon area and
turning.
import sys, math, warnings
warnings.filterwarnings('ignore')
sys.path.insert(0, '/tmp/claude-0/-home-claude/d7048feb-3787-5f7f-b86a-f71c9a373eed/scratchpad/peel_paper')
import sympy as sp, mpmath as mp
from census_lib import riley_poly, eps, S_, V_
mp.mp.dps = 120
PI = mp.pi
```

```
def riley_coeff_funcs(p, q):
    R = sp.Poly(riley_poly(p, q), V_)
    cs = R.all_coeffs()            # highest power of v first
    fs = [sp.lambdify(S_, c, 'mpmath') for c in cs]
    return fs

def real_roots(fs, s):
    co = [f(s) for f in fs]
    # drop leading zeros
    while co and abs(co[0]) < mp.mpf(10)**(-40) * max(abs(c) for c in co): co = co[1:]
    rts = mp.polyroots(co, maxsteps=500, extraprec=400)
    out = []
    for r in rts:
        if abs(mp.im(r)) < mp.mpf(10)**(-25) * (1 + abs(r)): out.append(mp.re(r))
    return sorted(out)

def letters(p, q):
    """relator a W b^-1 W^-1 as a list of (generator, +-1), W the Riley word for (p,q)."""
    e = eps(p, q); W = []
    for i, x in enumerate(e):
        g = 'b' if i % 2 == 0 else 'a'
        W.append((g, 1 if x > 0 else -1))
    Winv = [(g, -s) for g, s in reversed(W)]
    return [('a', 1)] + W + [('b', -1)] + Winv, W

def mats(s, v):
    a = mp.matrix([[s, 1], [0, 1/s]]); b = mp.matrix([[s, 0], [-v, 1/s]])
    return {('a', 1): a, ('a', -1): a**-1, ('b', 1): b, ('b', -1): b**-1}

def prod(M, lst):
    P = mp.eye(2)
    for l in lst: P = P * M[l]
    return P

def relation_residual(M, W):
    a, b = M[('a', 1)], M[('b', 1)]
    Wm = prod(M, W)
    d = a*Wm - Wm*b
    aw = a*Wm
    return max(abs(d[i, j]) for i in range(2) for j in range(2)) / max(abs(aw[i, j]) for i in range(2) for j in range(2))

def wrap(x):
    # to (-pi, pi]
    y = mp.fmod(x + PI, 2*PI)
    if y <= 0: y += 2*PI
    return y - PI

def lift_apply(Mt, th):
    """canonical lift (fixed-point lift) of the hyperbolic SL2(R) matrix Mt acting on vector angles"""
    v = mp.matrix([mp.cos(th), mp.sin(th)]); w = Mt * v
    return th + wrap(mp.atan2(w[1], w[0]) - th)

def lift_class(M, rel, th0=mp.mpf('0.3')):
    """apply the lifted relator word to angle th0 (rightmost letter first); return shift / pi (an integer for a relation)"""
    th = th0
    for l in reversed(rel): th = lift_apply(M[l], th)
    return (th - th0) / PI

# ---- hyperbolic geometry on the upper half plane, via the hyperboloid <X,X> = X0^2 - X1^2 - X2^2 = 1
def mobius(Mt, z):
    return (Mt[0, 0]*z + Mt[0, 1]) / (Mt[1, 0]*z + Mt[1, 1])
def hyp(z):
    x, y = mp.re(z), mp.im(z); r2 = x*x + y*y
    return mp.matrix([(r2 + 1)/(2*y), (r2 - 1)/(2*y), x/y])
def Q(U, V): return U[0]*V[0] - U[1]*V[1] - U[2]*V[2]
def det3(A, B, C):
    return (A[0]*(B[1]*C[2]-B[2]*C[1]) - A[1]*(B[0]*C[2]-B[2]*C[0]) + A[2]*(B[0]*C[1]-B[1]*C[0]))

def orbit(M, rel, o):
    pts = [o]; P = mp.eye(2)
    for l in rel:
        P = P * M[l]; pts.append(mobius(P, o))
    return pts

def tri_area(X, Y, Z, sgn):
    c = Q(X, Y) + Q(Y, Z) + Q(Z, X)
    return sgn * 2 * mp.atan(det3(X, Y, Z) / (1 + c))

def calibrate():
    # counterclockwise triangle in the upper half plane (Euclidean orientation): i, 2+i, 1+2i
    z = [mp.mpc(0, 1), mp.mpc(2, 1), mp.mpc(1, 2)]
    X = [hyp(t) for t in z]
    d = det3(*X)
    return 1 if d > 0 else -1
SG = calibrate()
```

```
def poly_area_turn(pts):
    """signed (winding-weighted) area by the fan from pts[0]; exterior-angle sum; closed polygon pts[0..n] with pts[n] ~
pts[0]"""
    X = [hyp(t) for t in pts[:-1]]; n = len(X)
    A = mp.mpf(0)
    for k in range(1, n - 1): A += tri_area(X[0], X[k], X[k+1], SG)
    # exterior angles
    def tangent_to(Xp, Xq):
        c = Q(Xp, Xq); T = Xq - c*Xp; return T / mp.sqrt(c*c - 1)
    E = mp.mpf(0)
    for k in range(n):
        Xk = X[k]; Xp = X[(k-1) % n]; Xn = X[(k+1) % n]
        U = -tangent_to(Xk, Xp)       # incoming direction at Xk
        V = tangent_to(Xk, Xn)        # outgoing direction
        cs = -Q(U, V) / mp.sqrt(Q(U, U) * Q(V, V))
        sn = SG * det3(Xk, U, V) / mp.sqrt(Q(U, U) * Q(V, V))
        E += mp.atan2(sn, cs)
    return A, E
```

q2_sweep.py

```
# SCRIPT: Q2-SWEEP-RELATOR-LOOP
# For each knot (p,q) and each real Riley branch point (x = ln s, v real root):
#   resid  = |aW - Wb| / |aW|                          (relation check, mp 120 digits)
#   eps    = lifted relator a W b^-1 W^-1 in PSL~(2,R) with canonical (fixed-point) lifts of the letters; shift / pi
[integer]
#   kL     = lifted LONGITUDE word (W W_rev a^-2e) acting at the a-eigendirection; shift / pi
[integer]
#   A, E, tau = signed (winding-weighted) area of the relator polygon through o, a.o, ab.o, ...; sum of exterior angles; (E-
A)/2pi
# for two base points o = i  and  o* = foot on axis(a) of the common perpendicular to axis(b) (the crossing point if the
axes meet).
import sys, math, json
sys.path.insert(0, '/tmp/claude-0/-home-claude/d7048feb-3787-5f7f-b86a-f71c9a373eed/scratchpad/lanes/compute')
from q2_lib import *
knots = {'3_1': (3, 1), '4_1': (5, 3), '5_2': (7, 3), '6_2': (11, 3)}
which = sys.argv[1]; xmax = float(sys.argv[2]); dx = float(sys.argv[3]); xmin = float(sys.argv[4]) if len(sys.argv) > 4 else
0.01
p, q = knots[which]
rel, W = letters(p, q)
e_sum = sum(eps(p, q))
Wrev = list(reversed(W)); lon = W + Wrev + [('a', -1 if e_sum > 0 else 1)] * (2*abs(e_sum))
fs = riley_coeff_funcs(p, q)
print("# knot %s (p,q)=(%d,%d)  relator length %d  longitude length %d  sum eps = %d" % (which, p, q, len(rel), len(lon),
e_sum))
branches = []   # list of dicts: {id, pts:[rows]}
active = []     # indices of active branches with last v
def row(x, v):
    s = mp.e**mp.mpf(x); M = mats(s, v)
    res = relation_residual(M, W)
    e0 = lift_class(M, rel, mp.mpf('0.3')); e1 = lift_class(M, rel, mp.mpf('2.1'))
    # longitude word-lift at the a-eigendirection theta = 0 (a (1,0)^T = s (1,0)^T)
    th = mp.mpf(0)
    for l in reversed(lon): th = lift_apply(M[l], th)
    kL = th/PI
    th2 = mp.mpf(0)
    out = {'x': x, 'v': mp.nstr(v, 15), 'resid': mp.nstr(res, 3), 'eps': mp.nstr(e0, 8), 'eps2': mp.nstr(e1, 8), 'kL':
mp.nstr(kL, 8)}
    xa = s/(1 - s*s); xb = (1 - s*s)/(s*v)
    bases = {'i': mp.mpc(0, 1), 'star': mp.mpc(xa, mp.sqrt(abs(xa*(xa - xb))))}
    for nm, o in bases.items():
        pts = orbit(M, rel, o); A, E = poly_area_turn(pts)
        out['close_'+nm] = mp.nstr(abs(pts[-1] - pts[0]), 3); out['A_'+nm] = mp.nstr(A, 15); out['E_'+nm] = mp.nstr(E, 15);
out['tau_'+nm] = mp.nstr((E - A)/(2*PI), 8)
    out['cross'] = bool(xb < xa < 0)
    return out
x = xmin; prev = []   # prev: list of (branch index, v)
nx = int(round((xmax - xmin)/dx))
for step in range(nx + 1):
    x = xmin + step*dx
    s = mp.e**mp.mpf(x)
    rts = real_roots(fs, s)
    newprev = []; used = set()
    same = (len(rts) == len(prev))
    prev_sorted = sorted(prev, key=lambda t_: t_[1])
    for idx, v in enumerate(rts):
        if same:
            bi = prev_sorted[idx][0]          # real roots cannot cross while their number is constant: match in sorted
order
        else:
            best = None; bd = 1e99
```

```
            for bi_, vp in prev:
                if bi_ in used: continue
                d = abs(v - vp)/(1 + abs(vp))
                if d < bd: bd, best = d, bi_
            if best is not None and bd < 0.15: bi = best
            else:
                bi = len(branches); branches.append({'id': bi, 'start_x': x, 'rows': []})
        used.add(bi); newprev.append((bi, v))
        r = row(x, v); r['branch'] = bi
        branches[bi]['rows'].append(r)
    prev = newprev
    if step % 20 == 0:
        print("x=%.3f real roots: %s" % (x, [mp.nstr(v, 6) for v in rts])); sys.stdout.flush()
json.dump(branches, open('q2_%s.json' % which, 'w'))
print("\n# SUMMARY for %s" % which)
for b in branches:
    R = b['rows']; xs = [r['x'] for r in R]
    eps_set = sorted(set(round(float(r['eps'])) for r in R)); eps2 = sorted(set(round(float(r['eps2'])) for r in R)); kL =
sorted(set(round(float(r['kL'])) for r in R))
    maxres = max(float(r['resid']) for r in R); maxdev = max(abs(float(r['eps']) - round(float(r['eps']))) for r in R)
    print("branch %d: x in [%.3f, %.3f] (%d pts)  v from %s to %s | relator lift class eps in %s (theta-test %s, max |eps-
int| %.1e) | longitude lift kL in %s | max relation residual %.1e" % (
        b['id'], xs[0], xs[-1], len(R), R[0]['v'], R[-1]['v'], eps_set, eps2, maxdev, kL, maxres))
    for nm in ['i', 'star']:
        A = [float(r['A_'+nm]) for r in R]; t = sorted(set(round(float(r['tau_'+nm])) for r in R)); mxc =
max(float(r['close_'+nm]) for r in R)
        print("    base %-4s: A from %+.6f to %+.6f (min %+.6f max %+.6f) tau in %s  closure<=%.1e | A/pi at end %+.6f" %
(nm, A[0], A[-1], min(A), max(A), t, mxc, A[-1]/math.pi))
```

q2_3_1.txt

```
# knot 3_1 (p,q)=(3,1)  relator length 6  longitude length 8  sum eps = 2
x=0.010 real roots: ['1.0004']
x=0.410 real roots: ['1.71093']
x=0.810 real roots: ['4.25099']
x=1.210 real roots: ['10.3348']
x=1.610 real roots: ['24.0681']
x=2.010 real roots: ['54.7191']
x=2.410 real roots: ['122.973']
x=2.810 real roots: ['274.893']
x=3.210 real roots: ['613.005']
x=3.610 real roots: ['1365.49']
x=4.010 real roots: ['3040.18']
x=4.410 real roots: ['6767.26']
x=4.810 real roots: ['15062.1']
x=5.210 real roots: ['33522.4']
x=5.610 real roots: ['74606.8']
x=6.010 real roots: ['166042.0']

# SUMMARY for 3_1
branch 0: x in [0.010, 6.010] (301 pts)  v from 1.00040001333351 to 166041.656307465 | relator lift class eps in [0] (theta-
test [0], max |eps-int| 2.7e-106) | longitude lift kL in [-1] | max relation residual 2.9e-116
    base i   : A from -0.000320 to -3.131776 (min -3.131776 max -0.000320) tau in [0]  closure<=1.0e-108 | A/pi at end
-0.996875
    base star: A from -3.101603 to -0.009816 (min -3.101603 max -0.009816) tau in [0]  closure<=2.6e-116 | A/pi at end
-0.003125
```

q2_4_1.txt

```
# knot 4_1 (p,q)=(5,3)  relator length 10  longitude length 8  sum eps = 0
x=0.010 real roots: []
x=0.410 real roots: []
x=0.810 real roots: ['-0.750066', '3.00105']
x=1.210 real roots: ['-0.902316', '9.2371']
x=1.610 real roots: ['-0.958379', '23.0265']
x=2.010 real roots: ['-0.981719', '53.7008']
x=2.410 real roots: ['-0.991868', '121.965']
x=2.810 real roots: ['-0.996362', '273.889']
x=3.210 real roots: ['-0.998369', '612.003']
x=3.610 real roots: ['-0.999268', '1364.49']
x=4.010 real roots: ['-0.999671', '3039.18']
x=4.410 real roots: ['-0.999852', '6766.26']
x=4.810 real roots: ['-0.999934', '15061.0']
x=5.210 real roots: ['-0.99997', '33521.4']
x=5.610 real roots: ['-0.999987', '74605.8']
x=6.010 real roots: ['-0.999994', '166041.0']

# SUMMARY for 4_1
```

```
branch 0: x in [0.490, 6.010] (277 pts)  v from -0.180522669651545 to -0.999993977414931 | relator lift class eps in [0]
(theta-test [0], max |eps-int| 2.7e-107) | longitude lift kL in [0] | max relation residual 1.5e-121
    base i   : A from +0.998601 to +0.009816 (min +0.009816 max +1.274279) tau in [-1, 0]  closure<=1.1e-108 | A/pi at end
+0.003125
    base star: A from +0.778308 to +0.009816 (min +0.009816 max +1.269408) tau in [-1]  closure<=1.1e-108 | A/pi at end
+0.003125
branch 1: x in [0.490, 6.010] (277 pts)  v from 0.220290010432362 to 166040.656301443 | relator lift class eps in [0]
(theta-test [0], max |eps-int| 2.4e-96) | longitude lift kL in [0] | max relation residual 2.6e-116
    base i   : A from +0.642492 to -3.131776 (min -3.131776 max +0.642492) tau in [0]  closure<=2.2e-98 | A/pi at end
-0.996875
    base star: A from +0.000000 to +0.000000 (min -0.000000 max +0.000000) tau in [0]  closure<=1.6e-108 | A/pi at end
+0.000000
```

q2_5_2.txt

```
# knot 5_2 (p,q)=(7,3)  relator length 14  longitude length 16  sum eps = 2
x=0.010 real roots: ['0.570294']
x=0.410 real roots: ['1.36532']
x=0.810 real roots: ['0.665728', '2.77412', '4.06213']
x=1.210 real roots: ['0.208323', '9.21462', '10.2466']
x=1.610 real roots: ['0.0852071', '23.0228', '24.0282']
x=2.010 real roots: ['0.0369157', '53.7001', '54.7011']
x=2.410 real roots: ['0.0163326', '121.965', '122.965']
x=2.810 real roots: ['0.00728903', '273.889', '274.889']
x=3.210 real roots: ['0.0032653', '612.003', '613.003']
x=3.610 real roots: ['0.00146521', '1364.49', '1365.49']
x=4.010 real roots: ['0.000657965', '3039.18', '3040.18']
x=4.410 real roots: ['0.000295562', '6766.26', '6767.26']
x=4.810 real roots: ['0.000132788', '15061.0', '15062.0']
x=5.210 real roots: ['5.96624e-5', '33521.4', '33522.4']
x=5.610 real roots: ['2.68074e-5', '74605.8', '74606.8']
x=6.010 real roots: ['1.20452e-5', '166041.0', '166042.0']

# SUMMARY for 5_2
branch 0: x in [0.010, 6.010] (301 pts)  v from 0.570293751382542 to 166041.656301443 | relator lift class eps in [0]
(theta-test [0], max |eps-int| 1.5e-93) | longitude lift kL in [-1] | max relation residual 3.1e-111
    base i   : A from +0.371732 to -3.131776 (min -3.131776 max +0.371732) tau in [0]  closure<=5.3e-93 | A/pi at end
-0.996875
    base star: A from -3.071453 to -0.009816 (min -3.071453 max -0.009816) tau in [0]  closure<=1.9e-103 | A/pi at end
-0.003125
branch 1: x in [0.730, 6.010] (265 pts)  v from 1.11596002352243 to 1.20452064093058e-5 | relator lift class eps in [0]
(theta-test [0], max |eps-int| 2.1e-102) | longitude lift kL in [0] | max relation residual 2.6e-121
    base i   : A from -0.077438 to +0.004908 (min -0.077438 max +0.422072) tau in [0]  closure<=3.5e-108 | A/pi at end
+0.001562
    base star: A from -0.000000 to +0.000000 (min -0.000000 max +0.000000) tau in [0]  closure<=3.5e-108 | A/pi at end
+0.000000
branch 2: x in [0.730, 6.010] (265 pts)  v from 1.63969215732708 to 166040.656301443 | relator lift class eps in [0] (theta-
test [0], max |eps-int| 9.2e-99) | longitude lift kL in [0] | max relation residual 4.7e-106
    base i   : A from -0.394817 to -3.131776 (min -3.131776 max -0.394817) tau in [0]  closure<=3.2e-98 | A/pi at end
-0.996875
    base star: A from -0.000000 to +0.000000 (min -0.000000 max +0.000000) tau in [0]  closure<=1.2e-108 | A/pi at end
+0.000000
```

q2_6_2.txt

```
# knot 6_2 (p,q)=(11,3)  relator length 22  longitude length 24  sum eps = 2
x=0.010 real roots: ['1.48324']
x=0.410 real roots: ['-0.0298984', '0.471129', '2.11838']
x=0.810 real roots: ['-0.0332671', '3.18656', '4.4714']
x=1.210 real roots: ['-0.00769248', '9.30987', '10.4306']
x=1.610 real roots: ['-0.00158825', '23.0576', '24.1095']
x=2.010 real roots: ['-0.000321988', '53.7145', '54.7373']
x=2.410 real roots: ['-6.50602e-5', '121.971', '122.981']
x=2.810 real roots: ['-1.31375e-5', '273.892', '274.897']
x=3.210 real roots: ['-2.6525e-6', '612.004', '613.006']
x=3.610 real roots: ['-5.35534e-7', '1364.49', '1365.49']
x=4.010 real roots: ['-1.08123e-7', '3039.18', '3040.18']
x=4.410 real roots: ['-2.18296e-8', '6766.26', '6767.26']
x=4.810 real roots: ['-4.40732e-9', '15061.0', '15062.1']
x=5.210 real roots: ['-8.89822e-10', '33521.4', '33522.4']
x=5.610 real roots: ['-1.79652e-10', '74605.8', '74606.8']
x=6.010 real roots: ['-3.62711e-11', '166041.0', '166042.0']

# SUMMARY for 6_2
branch 0: x in [0.010, 6.010] (301 pts)  v from 1.48323950295595 to 166041.656313488 | relator lift class eps in [0] (theta-
test [0], max |eps-int| 1.5e-92) | longitude lift kL in [-1] | max relation residual 5.6e-92
    base i   : A from -0.348929 to -3.131776 (min -3.131776 max -0.348929) tau in [0]  closure<=1.1e-91 | A/pi at end
-0.996875
```

```
    base star: A from -3.114621 to -0.009816 (min -3.114621 max -0.009816) tau in [0]  closure<=5.6e-92 | A/pi at end
-0.003125
branch 1: x in [0.370, 6.010] (283 pts)  v from 0.0298124731300463 to -3.6271094011278e-11 | relator lift class eps in [0]
(theta-test [0], max |eps-int| 8.4e-87) | longitude lift kL in [0] | max relation residual 1.9e-121
    base i   : A from +0.846946 to +0.004908 (min +0.004908 max +0.897144) tau in [0]  closure<=5.1e-98 | A/pi at end
+0.001562
    base star: A from -0.000000 to +0.000000 (min -0.000000 max +0.438901) tau in [-1, 0]  closure<=5.1e-98 | A/pi at end
+0.000000
branch 2: x in [0.370, 6.010] (283 pts)  v from 0.251955384319343 to 166040.65630596 | relator lift class eps in [0] (theta-
test [0], max |eps-int| 8.4e-82) | longitude lift kL in [0] | max relation residual 4.4e-100
    base i   : A from +0.640267 to -3.131776 (min -3.131776 max +0.640267) tau in [0]  closure<=2.9e-81 | A/pi at end
-0.996875
    base star: A from -0.000000 to -0.000000 (min -0.000000 max +0.000000) tau in [0]  closure<=1.8e-97 | A/pi at end
-0.000000
```

q2_checks.py

```
# SCRIPT: Q2-CHECKS
# (1) continuity of A along branches, (E-A)/2pi integrality;  (2) kL versus the sign of tr(longitude);
# (3) 4_1 start point x0 = arccosh(3/2)/2 (K2c);  (4) fold points of 5_2 and 6_2;  (5) half-turn symmetry J at o* when the
axes cross;
# (6) asymptotics x -> 10 of A(i) on the trefoil and 4_1.
import sys, json, math
sys.path.insert(0, '/tmp/claude-0/-home-claude/d7048feb-3787-5f7f-b86a-f71c9a373eed/scratchpad/lanes/compute')
from q2_lib import *
knots = {'3_1': (3, 1), '4_1': (5, 3), '5_2': (7, 3), '6_2': (11, 3)}
print("## (1) continuity of the area and integrality of tau (all rows, both base points)")
for k in knots:
    B = json.load(open('q2_%s.json' % k))
    for b in B:
        R = b['rows']; worst = {}; ti = 0
        for nm in ['i', 'star']:
            A = [float(r['A_'+nm]) for r in R]; dA = max(abs(A[j+1]-A[j]) for j in range(len(A)-1)) if len(A) > 1 else 0
            worst[nm] = dA
            ti = max(ti, max(abs(float(r['tau_'+nm]) - round(float(r['tau_'+nm]))) for r in R))
        print("  %s branch %d: max |A(x+dx)-A(x)| (dx=0.02): base i %.4f, base o* %.4f ; max |tau - integer| = %.1e" % (k,
b['id'], worst['i'], worst['star'], ti))
print("\n## (2) longitude lift class kL against the sign of tr(longitude) (SL2 lift with positive-trace meridians)")
for k, (p, q) in knots.items():
    rel, W = letters(p, q); es = sum(eps(p, q)); lon = W + list(reversed(W)) + [('a', -1 if es > 0 else 1)]*(2*abs(es))
    fs = riley_coeff_funcs(p, q); B = json.load(open('q2_%s.json' % k))
    for b in B:
        sg = set()
        for r in b['rows'][::10]:
            s = mp.e**mp.mpf(r['x']); v = mp.mpf(r['v']); M = mats(s, v); Lm = prod(M, lon)
            sg.add((int(mp.sign(Lm[0,0]+Lm[1,1])), round(float(r['kL']))))
        print("  %s branch %d: set of (sign tr(longitude), kL) = %s" % (k, b['id'], sorted(sg)))
print("\n## (3) 4_1 start of the real curve: x0 = arccosh(3/2)/2 = %s" % mp.nstr(mp.acosh(mp.mpf(3)/2)/2, 12))
fs = riley_coeff_funcs(5, 3); x0 = mp.acosh(mp.mpf(3)/2)/2
for d in [-1e-2, -1e-4, -1e-8, 1e-8, 1e-6, 1e-4, 1e-2]:
    x = x0 + mp.mpf(d); rts = real_roots(fs, mp.e**x)
    print("  x0%+.0e : real roots v = %s" % (d, [mp.nstr(v, 8) for v in rts]))
print("  below x0, scan x in [0.01, x0-1e-3] step 0.01 for real roots: ", end='')
cnt = 0; xx = mp.mpf('0.01')
while xx < x0 - mp.mpf('1e-3'):
    cnt += len(real_roots(fs, mp.e**xx)); xx += mp.mpf('0.01')
print("total real roots found = %d" % cnt)
print("  at x0: v^2 - c v - c with c = s^2 + s^-2 - 3 = %s ; the representation is reducible (v = 0), a upper triangular, b
diagonal" % mp.nstr(mp.e**(2*x0) + mp.e**(-2*x0) - 3, 5))
print("\n## (4) fold points: x where two real roots are born (bisection on the number of real roots)")
for k in ['5_2', '6_2']:
    p, q = knots[k]; fs = riley_coeff_funcs(p, q)
    n = lambda x: len(real_roots(fs, mp.e**mp.mpf(x)))
    lo, hi = 0.2, 1.2
    nlo, nhi = n(lo), n(hi)
    for _ in range(40):
        mid = (lo+hi)/2
        if n(mid) == nlo: lo = mid
        else: hi = mid
    print("  %s: real root count %d (x=0.2) -> %d (x=1.2); fold at x = %.8f (s = %.6f)" % (k, nlo, nhi, hi, math.exp(hi)))
print("\n## (5) half-turn J about o* when the axes cross: J a J = a^-1, J b J = b^-1 and J reverses the relator polygon?")
def halfturn(c):
    x0, y0 = mp.re(c), mp.im(c); g = mp.matrix([[mp.sqrt(y0), x0/mp.sqrt(y0)], [0, 1/mp.sqrt(y0)]])
    return g * mp.matrix([[0, -1], [1, 0]]) * g**-1
for k in ['3_1', '4_1', '5_2']:
    p, q = knots[k]; rel, W = letters(p, q); fs = riley_coeff_funcs(p, q)
    for x in [mp.mpf('1.0'), mp.mpf('2.5')]:
        s = mp.e**x
        for v in real_roots(fs, s):
            xa = s/(1 - s*s); xb = (1 - s*s)/(s*v); cross = (xb < xa < 0)
            M = mats(s, v); a, b = M[('a', 1)], M[('b', 1)]
            o = mp.mpc(xa, mp.sqrt(abs(xa*(xa - xb)))); J = halfturn(o)
```

```
            def dist(X, Y): return max(abs(X[i, j]/X[1, 1] - Y[i, j]/Y[1, 1]) if abs(X[1, 1]) > 1e-30 else 9 for i in
range(2) for j in range(2))
            def projeq(X, Y):  # equality in PSL2: X = +-Y
                return min(max(abs(X[i, j] - Y[i, j]) for i in range(2) for j in range(2)), max(abs(X[i, j] + Y[i, j]) for i
in range(2) for j in range(2)))
            ja = projeq(J*a*J**-1, a**-1); jb = projeq(J*b*J**-1, b**-1)
            pts = orbit(M, rel, o); n = len(pts) - 1
            rev = max(abs(mobius(J, pts[j]) - pts[n - j]) for j in range(n + 1))
            A, E = poly_area_turn(pts)
            print("  %s x=%s v=%s axes cross: %s | |JaJ-a^-1| = %.1e, |JbJ-b^-1| = %.1e | max_j |J(p_j) - p_{n-j}| = %.1e |
A(o*) = %s" % (k, mp.nstr(x, 3), mp.nstr(v, 6), cross, ja, jb, rev, mp.nstr(A, 8)))
print("\n## (6) asymptotics: A(i) at large x")
for k in ['3_1', '4_1']:
    p, q = knots[k]; rel, W = letters(p, q); fs = riley_coeff_funcs(p, q)
    for x in [6, 8, 10, 12]:
        s = mp.e**mp.mpf(x)
        for v in real_roots(fs, s):
            M = mats(s, v); pts = orbit(M, rel, mp.mpc(0, 1)); A, E = poly_area_turn(pts)
            print("  %s x=%2d v=%s : A(i) = %s ; A(i)/pi = %s ; (A + pi)*e^x = %s ; A*e^x = %s" % (k, x, mp.nstr(v, 6),
mp.nstr(A, 12), mp.nstr(A/PI, 10), mp.nstr((A + PI)*mp.e**x, 8), mp.nstr(A*mp.e**x, 8)))
```

Output

```
## (1) continuity of the area and integrality of tau (all rows, both base points)
  3_1 branch 0: max |A(x+dx)-A(x)| (dx=0.02): base i 0.0430, base o* 0.0797 ; max |tau - integer| = 1.5e-78
  4_1 branch 0: max |A(x+dx)-A(x)| (dx=0.02): base i 0.1002, base o* 0.2260 ; max |tau - integer| = 1.1e-120
  4_1 branch 1: max |A(x+dx)-A(x)| (dx=0.02): base i 0.1938, base o* 0.0000 ; max |tau - integer| = 2.4e-47
  5_2 branch 0: max |A(x+dx)-A(x)| (dx=0.02): base i 0.0538, base o* 0.1390 ; max |tau - integer| = 4.6e-48
  5_2 branch 1: max |A(x+dx)-A(x)| (dx=0.02): base i 0.1325, base o* 0.0000 ; max |tau - integer| = 2.1e-73
  5_2 branch 2: max |A(x+dx)-A(x)| (dx=0.02): base i 0.1815, base o* 0.0000 ; max |tau - integer| = 5.9e-63
  6_2 branch 0: max |A(x+dx)-A(x)| (dx=0.02): base i 0.0345, base o* 0.0539 ; max |tau - integer| = 1.1e-78
  6_2 branch 1: max |A(x+dx)-A(x)| (dx=0.02): base i 0.0311, base o* 0.1812 ; max |tau - integer| = 2.9e-37
  6_2 branch 2: max |A(x+dx)-A(x)| (dx=0.02): base i 0.1101, base o* 0.0000 ; max |tau - integer| = 1.0e-45

## (2) longitude lift class kL against the sign of tr(longitude) (SL2 lift with positive-trace meridians)
  3_1 branch 0: set of (sign tr(longitude), kL) = [(-1, -1)]
  4_1 branch 0: set of (sign tr(longitude), kL) = [(1, 0)]
  4_1 branch 1: set of (sign tr(longitude), kL) = [(1, 0)]
  5_2 branch 0: set of (sign tr(longitude), kL) = [(-1, -1)]
  5_2 branch 1: set of (sign tr(longitude), kL) = [(1, 0)]
  5_2 branch 2: set of (sign tr(longitude), kL) = [(1, 0)]
  6_2 branch 0: set of (sign tr(longitude), kL) = [(-1, -1)]
  6_2 branch 1: set of (sign tr(longitude), kL) = [(1, 0)]
  6_2 branch 2: set of (sign tr(longitude), kL) = [(1, 0)]

## (3) 4_1 start of the real curve: x0 = arccosh(3/2)/2 = 0.48121182506
  x0-1e-02 : real roots v = []
  x0-1e-04 : real roots v = []
  x0-1e-08 : real roots v = []
  x0+1e-08 : real roots v = ['-0.00021145189', '0.00021149662']
  x0+1e-06 : real roots v = ['-0.0021125091', '0.0021169812']
  x0+1e-04 : real roots v = ['-0.020926389', '0.021373663']
  x0+1e-02 : real roots v = ['-0.19143578', '0.23676014']
  below x0, scan x in [0.01, x0-1e-3] step 0.01 for real roots: total real roots found = 0
  at x0: v^2 - c v - c with c = s^2 + s^-2 - 3 = 0.0 ; the representation is reducible (v = 0), a upper triangular, b
diagonal

## (4) fold points: x where two real roots are born (bisection on the number of real roots)
  5_2: real root count 1 (x=0.2) -> 3 (x=1.2); fold at x = 0.72335252 (s = 2.061332)
  6_2: real root count 1 (x=0.2) -> 3 (x=1.2); fold at x = 0.35818667 (s = 1.430733)

## (5) half-turn J about o* when the axes cross: J a J = a^-1, J b J = b^-1 and J reverses the relator polygon?
  3_1 x=1.0 v=6.52439 axes cross: False | |JaJ-a^-1| = 9.6e-121, |JbJ-b^-1| = 6.3e+00 | max_j |J(p_j) - p_{n-j}| = 1.5e+00 |
A(o*) = -1.2271719
  3_1 x=2.5 v=147.42 axes cross: False | |JaJ-a^-1| = 1.1e-118, |JbJ-b^-1| = 2.5e+01 | max_j |J(p_j) - p_{n-j}| = 1.0e+00 |
A(o*) = -0.32437971
  4_1 x=1.0 v=-0.842949 axes cross: False | |JaJ-a^-1| = 1.5e-121, |JbJ-b^-1| = 2.6e+00 | max_j |J(p_j) - p_{n-j}| = 3.9e+02
| A(o*) = 1.136889
  4_1 x=1.0 v=5.36734 axes cross: True | |JaJ-a^-1| = 8.1e-120, |JbJ-b^-1| = 1.2e-119 | max_j |J(p_j) - p_{n-j}| = 3.2e+00 |
A(o*) = -1.7606013e-117
  4_1 x=2.5 v=-0.993216 axes cross: False | |JaJ-a^-1| = 7.0e-121, |JbJ-b^-1| = 2.0e+00 | max_j |J(p_j) - p_{n-j}| = 3.3e+06
| A(o*) = 0.32328942
  4_1 x=2.5 v=146.413 axes cross: True | |JaJ-a^-1| = 5.6e-115, |JbJ-b^-1| = 2.2e-115 | max_j |J(p_j) - p_{n-j}| = 1.2e+01 |
A(o*) = 3.5910947e-111
  5_2 x=1.0 v=0.355573 axes cross: True | |JaJ-a^-1| = 1.5e-121, |JbJ-b^-1| = 1.5e-121 | max_j |J(p_j) - p_{n-j}| = 4.4e+02
| A(o*) = 1.916933e-119
  5_2 x=1.0 v=5.30136 axes cross: True | |JaJ-a^-1| = 5.7e-120, |JbJ-b^-1| = 7.3e-120 | max_j |J(p_j) - p_{n-j}| = 3.4e+00 |
A(o*) = -3.3517283e-118
  5_2 x=1.0 v=6.39185 axes cross: False | |JaJ-a^-1| = 1.6e-120, |JbJ-b^-1| = 6.2e+00 | max_j |J(p_j) - p_{n-j}| = 5.5e+01 |
A(o*) = -1.151683
  5_2 x=2.5 v=0.0136143 axes cross: True | |JaJ-a^-1| = 8.5e-121, |JbJ-b^-1| = 7.0e-121 | max_j |J(p_j) - p_{n-j}| = 2.8e+07
| A(o*) = 1.2891036e-116
  5_2 x=2.5 v=146.413 axes cross: True | |JaJ-a^-1| = 6.7e-115, |JbJ-b^-1| = 2.7e-115 | max_j |J(p_j) - p_{n-j}| = 8.8e+00 |
A(o*) = 5.4598219e-113
```

```
  5_2 x=2.5 v=147.413 axes cross: False | |JaJ-a^-1| = 2.4e-117, |JbJ-b^-1| = 2.5e+01 | max_j |J(p_j) - p_{n-j}| = 2.2e+04 |
A(o*) = -0.32329683

## (6) asymptotics: A(i) at large x
  3_1 x= 6 v=162754.0 : A(i) = -3.13167763473 ; A(i)/pi = -0.9968439515 ; (A + pi)*e^x = 4.0000041 ; A*e^x = -1263.4089
  3_1 x= 8 v=8.88611e+6 : A(i) = -3.14025080305 ; A(i)/pi = -0.9995728757 ; (A + pi)*e^x = 4.0000001 ; A*e^x = -9360.9557
  3_1 x=10 v=4.85165e+8 : A(i) = -3.14141105387 ; A(i)/pi = -0.999942195 ; (A + pi)*e^x = 4.0 ; A*e^x = -69194.183
  3_1 x=12 v=2.64891e+10 : A(i) = -3.14156807674 ; A(i)/pi = -0.9999921769 ; (A + pi)*e^x = 4.0 ; A*e^x = -511305.26
  4_1 x= 6 v=-0.999994 : A(i) = 0.00991489702116 ; A(i)/pi = 0.003156009742 ; (A + pi)*e^x = 1271.4089 ; A*e^x = 3.9999549
  4_1 x= 6 v=162753.0 : A(i) = -3.13167760427 ; A(i)/pi = -0.9968439418 ; (A + pi)*e^x = 4.0000164 ; A*e^x = -1263.4089
  4_1 x= 8 v=-1.0 : A(i) = 0.00134185023477 ; A(i)/pi = 0.0004271241955 ; (A + pi)*e^x = 9368.9557 ; A*e^x = 3.9999992
  4_1 x= 8 v=8.88611e+6 : A(i) = -3.14025080298 ; A(i)/pi = -0.9995728757 ; (A + pi)*e^x = 4.0000003 ; A*e^x = -9360.9557
  4_1 x=10 v=-1.0 : A(i) = 0.000181599718364 ; A(i)/pi = 5.780498568e-5 ; (A + pi)*e^x = 69202.183 ; A*e^x = 4.0
  4_1 x=10 v=4.85165e+8 : A(i) = -3.14141105387 ; A(i)/pi = -0.999942195 ; (A + pi)*e^x = 4.0 ; A*e^x = -69194.183
  4_1 x=12 v=-1.0 : A(i) = 2.45768494116e-5 ; A(i)/pi = 7.823054139e-6 ; (A + pi)*e^x = 511313.26 ; A*e^x = 4.0
  4_1 x=12 v=2.64891e+10 : A(i) = -3.14156807674 ; A(i)/pi = -0.9999921769 ; (A + pi)*e^x = 4.0 ; A*e^x = -511305.26
```

q2_smallx.py

```
# SCRIPT: Q2-SMALL-X-LIMIT
# Behaviour of the real branch that reaches x -> 0 (meridians become parabolic): A at base i and base o*, lift classes.
import sys
sys.path.insert(0,'/tmp/claude-0/-home-claude/d7048feb-3787-5f7f-b86a-f71c9a373eed/scratchpad/lanes/compute')
from q2_lib import *
for k,(p,q) in {'3_1':(3,1),'5_2':(7,3),'6_2':(11,3)}.items():
    rel,W=letters(p,q); fs=riley_coeff_funcs(p,q); es=sum(eps(p,q)); lon=W+list(reversed(W))+[('a',-1 if es>0 else 1)]*
(2*abs(es))
    for x in ['0.1','0.01','0.001','0.0001']:
        s=mp.e**mp.mpf(x); rts=real_roots(fs,s)
        v=min(rts,key=lambda t:abs(t-1))   # branch whose v -> value near the parabolic Riley root
        # pick the root of the branch that exists for all small x: smallest-x branch is the one continuing from larger root
list; show all roots
        for v in rts:
            M=mats(s,v); xa=s/(1-s*s); xb=(1-s*s)/(s*v)
            o=mp.mpc(xa,mp.sqrt(abs(xa*(xa-xb))));
            Ai,_=poly_area_turn(orbit(M,rel,mp.mpc(0,1))); As,_=poly_area_turn(orbit(M,rel,o))
            th=mp.mpf(0)
            for l in reversed(lon): th=lift_apply(M[l],th)
            print(k,'x=',x,'v=',mp.nstr(v,8),'A(i)=',mp.nstr(Ai,8),'A(o*)=',mp.nstr(As,8),'kL=',mp.nstr(th/PI,4))
```

Output

```
3_1 x= 0.1 v= 1.0401335 A(i)= -0.03175499 A(o*)= -2.751778 kL= -1.0
3_1 x= 0.01 v= 1.0004 A(i)= -0.00031997547 A(o*)= -3.1016033 kL= -1.0
3_1 x= 0.001 v= 1.000004 A(i)= -3.1999975e-6 A(o*)= -3.1375927 kL= -1.0
3_1 x= 0.0001 v= 1.0 A(i)= -3.2e-8 A(o*)= -3.1411927 kL= -1.0
5_2 x= 0.1 v= 0.61530843 A(i)= 0.32917965 A(o*)= -2.488923 kL= -1.0
5_2 x= 0.01 v= 0.57029375 A(i)= 0.3717316 A(o*)= -3.0714528 kL= -1.0
5_2 x= 0.001 v= 0.56984483 A(i)= 0.37216029 A(o*)= -3.1345732 kL= -1.0
5_2 x= 0.0001 v= 0.56984034 A(i)= 0.37216457 A(o*)= -3.1408907 kL= -1.0
6_2 x= 0.1 v= 1.518823 A(i)= -0.37088156 A(o*)= -2.8744971 kL= -1.0
6_2 x= 0.01 v= 1.4832395 A(i)= -0.34892922 A(o*)= -3.1146209 kL= -1.0
6_2 x= 0.001 v= 1.4828847 A(i)= -0.34870808 A(o*)= -3.1388952 kL= -1.0
6_2 x= 0.0001 v= 1.4828812 A(i)= -0.34870587 A(o*)= -3.1413229 kL= -1.0
```

make_plots.py

```
# SCRIPT: MAKE-PLOTS
import json, re, math, numpy as np
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
plt.rcParams.update({'figure.facecolor': '#0d1117', 'axes.facecolor': '#0d1117', 'savefig.facecolor': '#0d1117',
'text.color': 'white', 'axes.labelcolor': 'white',
                     'xtick.color': 'white', 'ytick.color': 'white', 'axes.edgecolor': '#8b949e', 'legend.facecolor':
'#161b22', 'legend.edgecolor': '#8b949e', 'grid.color': '#30363d', 'font.size': 10})
# ---------- Figure 1 (Q1)
fig, ax = plt.subplots(1, 3, figsize=(15, 4.4))
cols = {4: '#ff7b72', 5: '#ffa657', 6: '#7ee787', 7: '#79c0ff'}
for z in [4, 5, 6, 7]:
    txt = open('q1_enum_z%d.txt' % z).read()
    Ls = []; vals = []
    for m in re.finditer(r'B L=\s*(\d+)\s+words.*?min d/L = ([0-9.]+)', txt): Ls.append(int(m.group(1)));
vals.append(float(m.group(2)))
    ax[0].plot(Ls, vals, 'o-', color=cols[z], label='z=%d (ln z = %.3f)' % (z, math.log(z)))
ax[0].set_yscale('log'); ax[0].set_xlabel('word length L'); ax[0].set_ylabel('min over words of  d(o, w o) / L');
ax[0].grid(True, alpha=.4)
```

```
ax[0].set_title('displacement drift (all reduced words)'); ax[0].legend()
rho = math.acosh(3.0); phi = math.acos(-527/625); n = np.arange(1, 161); d =
2*np.arcsinh(math.sinh(rho)*np.abs(np.sin(n*phi/2)))
ax[1].plot(n, d, '.', color='#ffa657', ms=5); ax[1].axhline(2*rho, color='#ff7b72', ls='--', label='2 rho = %.3f (bound)' %
(2*rho))
ax[1].set_xlabel('n'); ax[1].set_ylabel('d(o, c^n o)'); ax[1].set_title('c = [B_i,B_j] at ln 5: |c^n| = 4n, d stays
bounded'); ax[1].legend(); ax[1].grid(True, alpha=.4)
zz = np.linspace(1.5, 12, 400); tr = 2 - 4*(((zz-1)**2)/(4*zz))**2
ax[2].plot(zz, tr, color='#79c0ff'); ax[2].axhline(-2, color='#8b949e', ls=':'); ax[2].axhline(2, color='#8b949e', ls=':')
ax[2].axvline(3+2*math.sqrt(2), color='#7ee787', ls='--', label='z = 3+2sqrt2 = 5.828'); ax[2].axvline(5, color='#ffa657',
ls='--', label='z = 5 (tr = -14/25)')
ax[2].set_ylim(-8, 3); ax[2].set_xlabel('peel ratio z'); ax[2].set_ylabel('tr [A,B]  (PSL2(R))');
ax[2].set_title('commutator trace: elliptic iff |tr| < 2'); ax[2].legend(); ax[2].grid(True, alpha=.4)
plt.tight_layout(); plt.savefig('fig_q1_ln5_not_qi.png', dpi=130); plt.close()
# ---------- Figure 2 (Q2)
fig, axs = plt.subplots(2, 4, figsize=(17, 7.2))
pal = ['#ffa657', '#79c0ff', '#7ee787', '#d2a8ff', '#ff7b72']
for j, k in enumerate(['3_1', '4_1', '5_2', '6_2']):
    B = json.load(open('q2_%s.json' % k))
    for r, nm in enumerate(['i', 'star']):
        a = axs[r, j]
        for b in B:
            xs = [float(t['x']) for t in b['rows']]; A = [float(t['A_'+nm]) for t in b['rows']]
            a.plot(xs, A, color=pal[b['id'] % 5], label='branch %d (kL=%s)' % (b['id'], int(round(float(b['rows'][0]
['kL'])))))
        a.axhline(-math.pi, color='#8b949e', ls=':'); a.axhline(0, color='#8b949e', ls=':')
        a.set_title('%s: relator-polygon area, base %s' % (k, 'o = i' if nm == 'i' else 'o* (axis foot)')); a.set_xlabel('x
= ln s'); a.grid(True, alpha=.4); a.legend(fontsize=8)
        a.set_ylim(-3.4, 1.5)
axs[0, 0].set_ylabel('signed area A'); axs[1, 0].set_ylabel('signed area A')
plt.tight_layout(); plt.savefig('fig_q2_area_vs_x.png', dpi=120); plt.close()
print('ok')
```

gen2.py

```
# SCRIPT: PEEL-TREES-LEAN-GEN
# Generates sections 3-4 of PeelTrees.lean (4x4 integer matrices, exact boosts, freeness mod p) so no
# matrix entry is typed by hand.
N=range(4); f=lambda i,j:'a%d%d'%(i,j)
def mk(M): return '(' + ', '.join(str(M[i][j]) for i in N for j in N) + ')'
Lz={5:[[16393,15132,5820,0],[15132,14593,4680,0],[5820,4680,4225,0],[0,0,0,2425]],
    7:[[210625,202200,56616,0],[202200,196513,50400,0],[56616,50400,30625,0],[0,0,0,16513]]}
Rz={5:[[97,0,0,0],[0,65,72,0],[0,-72,65,0],[0,0,0,97]],7:[[337,0,0,0],[0,175,288,0],[0,-288,175,0],[0,0,0,337]]}
k={5:9409,7:113569}; dl={5:2425,7:16513}; dr={5:97,7:337}
o=[]; P=o.append
P('structure M4 where\n' + ''.join('  %s : Int\n'%f(i,j) for i in N for j in N) + 'deriving DecidableEq, Repr\n')
P('def M4.mul (A B : M4) : M4 :=\n  (' + ',\n   '.join(' + '.join('A.%s * B.%s'%(f(i,kk),f(kk,j)) for kk in N) for i in N
for j in N) + ')\n')
P('def M4.tr (A : M4) : M4 := (' + ', '.join('A.%s'%f(j,i) for i in N for j in N) + ')')
P('def M4.smul (c : Int) (A : M4) : M4 := (' + ', '.join('c * A.%s'%f(i,j) for i in N for j in N) + ')')
P('def M4.red (m : Int) (A : M4) : M4 := (' + ', '.join('A.%s %% m'%f(i,j) for i in N for j in N) + ')')
P('def I4 : M4 := ' + mk([[int(i==j) for j in N] for i in N]))
P('def eta : M4 := ' + mk([[(1 if i==0 else -1)*int(i==j) for j in N] for i in N]))
P('''
/-- The six step directions of Garrett's tree: +i, -i, +j, -j, +k, -k. -/
inductive Dir | pi | mi | pj | mj | pk | mk
deriving DecidableEq, Repr

def Dir.inv : Dir → Dir
  | .pi => .mi | .mi => .pi | .pj => .mj | .mj => .pj | .pk => .mk | .mk => .pk

def allDirs : List Dir := [.pi, .mi, .pj, .mj, .pk, .mk]
theorem mem_allDirs (d : Dir) : d ∈ allDirs := by cases d <;> decide

/-- z times the boost of peel ratio z along direction d: c = (z^2+1)/2, s = (z^2-1)/2, z elsewhere. -/
def boost (c s z : Int) : Dir → M4''')
def bm(ax,sg):
    M=[[0]*4 for _ in N]
    for i in N: M[i][i]='z'
    M[0][0]='c'; M[ax][ax]='c'; M[0][ax]=M[ax][0]=('s' if sg>0 else '(-s)')
    return M
for nm,ax,sg in [('pi',1,1),('mi',1,-1),('pj',2,1),('mj',2,-1),('pk',3,1),('mk',3,-1)]:
    P('  | .%s => %s'%(nm,mk(bm(ax,sg))))
P('''
/-- Peel ratio 5 (rapidity ln 5, Smith step 2/3) and peel ratio 7 (rapidity ln 7, Garrett's drawing). -/
def B5 : Dir → M4 := boost 13 12 5
def B7 : Dir → M4 := boost 25 24 7

/-- Every scaled boost is Lorentz: B^T eta B = z^2 eta. -/
theorem lorentz5 : ∀ d ∈ allDirs, (M4.mul (M4.tr (B5 d)) (M4.mul eta (B5 d))) = M4.smul 25 eta := by decide
theorem lorentz7 : ∀ d ∈ allDirs, (M4.mul (M4.tr (B7 d)) (M4.mul eta (B7 d))) = M4.smul 49 eta := by decide
```

```
/-- The backward step is the inverse boost: B_d B_{-d} = z^2 I. -/
theorem inverse5 : ∀ d ∈ allDirs, M4.mul (B5 d) (B5 d.inv) = M4.smul 25 I4 := by decide
theorem inverse7 : ∀ d ∈ allDirs, M4.mul (B7 d) (B7 d.inv) = M4.smul 49 I4 := by decide

/-- A pure boost is a symmetric matrix. -/
theorem boost5_symm : ∀ d ∈ allDirs, M4.tr (B5 d) = B5 d := by decide

/-- The scaled matrix of a word (applied left to right, starting at the identity). -/
def word (B : Dir → M4) (w : List Dir) : M4 := w.foldl (fun M d => M4.mul M (B d)) I4

/-- Its first column is z^L times the point reached, (cosh d, sinh d * direction). -/
def pt (M : M4) : List Int := [M.a00, M.a10, M.a20, M.a30]

/-- Straight path +i +i +i: z^3 cosh d = 7813 = (5^6 + 1)/2, so d = 3 ln 5 exactly (lengths add). -/
theorem straight5 : pt (word B5 [.pi, .pi, .pi]) = [7813, 7812, 0, 0] := by decide
theorem straight5_is_three_ln5 : 2 * 7813 = 5 ^ 6 + 1 ∧ 2 * 7812 = 5 ^ 6 - 1 := by decide
/-- Two orthogonal steps: cosh d = cosh^2 (ln 5) = 169/25 (hyperbolic Pythagoras). -/
theorem turn5 : pt (word B5 [.pi, .pj]) = [169, 156, 60, 0] := by decide
/-- The turned paths of the earlier check. -/
theorem turned5 : pt (word B5 [.pi, .pi, .pj]) = [4069, 4056, 300, 0] ∧
    pt (word B5 [.pi, .pj, .pi]) = [2917, 2808, 780, 0] ∧
    pt (word B5 [.pi, .pj, .pk]) = [2197, 2028, 780, 300] := by decide
/-- Garrett's ln 7 numbers: straight 3 ln 7, and the word +i +j +i with cosh d = 19657/343 (d = 4.7415). -/
theorem garrett7 : pt (word B7 [.pi, .pi, .pi]) = [58825, 58824, 0, 0] ∧
    pt (word B7 [.pi, .pj, .pi]) = [19657, 19200, 4200, 0] ∧
    pt (word B7 [.pi, .pi, .pj]) = [30025, 30000, 1176, 0] ∧
    pt (word B7 [.pi, .pj, .pk]) = [15625, 15000, 4200, 1176] := by decide

/-- Reduced words of length n (no step followed by its own reverse). -/
def rwords : Nat → List (List Dir)
  | 0 => [[]]
  | n + 1 => (rwords n).flatMap fun w => (allDirs.filter fun d => w.getLast?.map Dir.inv != some d).map fun d => w ++ [d]

theorem rwords_three_count : (rwords 3).length = 150 := by decide

/-- Among all 150 reduced words of length 3, the straight paths are the farthest: every other word
    has strictly smaller cosh d. Turning always shortens. -/
theorem straight_farthest5 : ∀ w ∈ rwords 3, (word B5 w).a00 ≤ 7813 ∧
    ((word B5 w).a00 = 7813 ↔ w ∈ [[.pi,.pi,.pi],[.mi,.mi,.mi],[.pj,.pj,.pj],[.mj,.mj,.mj],[.pk,.pk,.pk],[.mk,.mk,.mk]]) :=
by decide
''')
for z in (5,7):
    P('''/-- Thomas rotation, exact (peel ratio %(z)d): B_i B_j = L R with L a pure boost (symmetric, Lorentz)
    and R a rotation about k with cos = %(c)s, sin = %(s)s.  Scaled: Lz * Rz = %(k)d * (B_i B_j). -/
def Lz%(z)d : M4 := %(L)s
def Rz%(z)d : M4 := %(R)s
theorem thomas%(z)d : M4.mul Lz%(z)d Rz%(z)d = M4.smul %(k)d (M4.mul (B%(z)d .pi) (B%(z)d .pj)) := by decide
theorem thomas%(z)d_boost : M4.tr Lz%(z)d = Lz%(z)d ∧ M4.mul (M4.tr Lz%(z)d) (M4.mul eta Lz%(z)d) = M4.smul %(dl2)d eta :=
by decide
theorem thomas%(z)d_rotation : M4.mul (M4.tr Rz%(z)d) Rz%(z)d = M4.smul %(dr2)d I4 ∧ pt Rz%(z)d = [%(dr)d, 0, 0, 0] := by
decide
/-- So the product of two boosts is not a boost: it is not symmetric. -/
theorem two_boosts_not_boost%(z)d : M4.tr (M4.mul (B%(z)d .pi) (B%(z)d .pj)) ≠ M4.mul (B%(z)d .pi) (B%(z)d .pj) := by decide
''' % dict(z=z, c={5:'65/97',7:'175/337'}[z], s={5:'72/97',7:'288/337'}[z], k=k[z], L=mk(Lz[z]), R=mk(Rz[z]), dl2=dl[z]**2,
dr2=dr[z]**2, dr=dr[z]))
P('''/-- Thomas half-angle in the peel coordinate: tan(theta/2) = sin/(1 + cos) = Gamma^2. -/
theorem thomas_half_angle : (72 : Int) * 9 = 4 * (97 + 65) ∧ (288 : Int) * 16 = 9 * (337 + 175) := by decide

/-! ## 4. Freeness for every length. Mod p (p = 5 or 7) each scaled boost is c * w w^T with w = (1, -sign, ...)
    a rank-one matrix; a product of rank-one matrices is rank one with a factor that is a product of
    dot products w_g . w_h, which is 2 (same step) or 1 (turn), and 0 only for a backtrack. -/

def wv : Dir → List Int
  | .pi => [1,-1,0,0] | .mi => [1,1,0,0] | .pj => [1,0,-1,0] | .mj => [1,0,1,0] | .pk => [1,0,0,-1] | .mk => [1,0,0,1]

def outer (m : Int) (c : Int) (f g : Dir) : M4 :=
  match wv f, wv g with
  | [f0,f1,f2,f3], [g0,g1,g2,g3] => M4.red m ''' + '(' + ', '.join('c * f%d * g%d'%(i,j) for i in N for j in N) + ''')
  | _, _ => I4

/-- Reduction mod m respects products (entrywise: sums and products commute with taking residues). -/
''')
for m in (5,7):
    ent=[]
    for i in N:
        for j in N:
            ent.append('      (A.%s * B.%s + A.%s * B.%s + A.%s * B.%s + A.%s * B.%s) %% %d = ' %
tuple(sum([[f(i,kk),f(kk,j)] for kk in N],[])+[m])) +
                       ('((A.%s %% %d) * (B.%s %% %d) + (A.%s %% %d) * (B.%s %% %d) + (A.%s %% %d) * (B.%s %% %d)) %% %d' % tuple(sum([[f(i,kk),m,f(kk,j),m] for kk in N],[])+[m])))
    P('theorem dot_emod%d (a1 b1 a2 b2 a3 b3 a4 b4 : Int) :\n    (a1*b1 + a2*b2 + a3*b3 + a4*b4) %% %d = ((a1%%%d)*(b1%%%d)
+ (a2%%%d)*(b2%%%d) + (a3%%%d)*(b3%%%d) + (a4%%%d)*(b4%%%d)) %% %d := by\n  have h1 := Int.mul_emod a1 b1 %d; have h2 :=
Int.mul_emod a2 b2 %d\n  have h3 := Int.mul_emod a3 b3 %d; have h4 := Int.mul_emod a4 b4 %d\n  omega\n' % ((m,)*15))
    P(('theorem red_mul%d (A B : M4) : M4.red %d (M4.mul A B) = M4.red %d (M4.mul (M4.red %d A) (M4.red %d B)) := by\n  simp
only [M4.red, M4.mul]\n  simp only [M4.mk.injEq]\n  refine (' % ((m,)*5)) + ', '.join(['?_']*16) + (') <;> apply
dot_emod%d\n' % m))
```

```
cs={5:'[1,2,3,4]',7:'[1,2,3,4,5,6]'}
for z in (5,7):
    P('''/-- Invariant for peel ratio %(z)d: mod %(z)d the word matrix is c * w_f w_g^T with c a nonzero residue,
    f the first step and g the last step. -/
def Inv%(z)d (M : M4) (g : Dir) : Prop := ∃ c ∈ (%(cs)s : List Int), ∃ f ∈ allDirs, M4.red %(z)d M = outer %(z)d c f g

theorem gen_red%(z)d : ∀ d ∈ allDirs, M4.red %(z)d (B%(z)d d) = outer %(z)d %(c0)d d d := by decide

/-- The finite closure step: rank-one state times a non-backtracking step stays rank one, nonzero. -/
theorem closure%(z)d : ∀ c ∈ (%(cs)s : List Int), ∀ f ∈ allDirs, ∀ g ∈ allDirs, ∀ h ∈ allDirs, h ≠ g.inv →
    ∃ c' ∈ (%(cs)s : List Int), M4.red %(z)d (M4.mul (outer %(z)d c f g) (outer %(z)d %(c0)d h h)) = outer %(z)d c' f h :=
by decide

theorem outer_red%(z)d : ∀ c ∈ (%(cs)s : List Int), ∀ f ∈ allDirs, ∀ g ∈ allDirs, M4.red %(z)d (outer %(z)d c f g) = outer %
(z)d c f g := by decide

theorem step%(z)d (M : M4) (g h : Dir) (hM : Inv%(z)d M g) (hh : h ≠ g.inv) : Inv%(z)d (M4.mul M (B%(z)d h)) h := by
  obtain ⟨c, hc, f, hf, hred⟩ := hM
  obtain ⟨c', hc', hcl⟩ := closure%(z)d c hc f hf g (mem_allDirs g) h (mem_allDirs h) hh
  refine ⟨c', hc', f, hf, ?_⟩
  rw [red_mul%(z)d, hred, gen_red%(z)d h (mem_allDirs h), hcl]

/-- Non-backtracking from g. -/
def RedFrom : Dir → List Dir → Prop
  | _, [] => True
  | g, h :: t => h ≠ g.inv ∧ RedFrom h t

def lastStep : Dir → List Dir → Dir
  | g, [] => g
  | _, h :: t => lastStep h t

theorem walk%(z)d (l : List Dir) : ∀ (M : M4) (g : Dir), Inv%(z)d M g → RedFrom g l →
    Inv%(z)d (l.foldl (fun M d => M4.mul M (B%(z)d d)) M) (lastStep g l) := by
  induction l with
  | nil => intro M g h _; exact h
  | cons h t ih =>
    intro M g hM hr
    exact ih _ h (step%(z)d M g h hM hr.1) hr.2

theorem corner%(z)d : ∀ c ∈ (%(cs)s : List Int), ∀ f ∈ allDirs, ∀ g ∈ allDirs, (outer %(z)d c f g).a00 ≠ 0 := by decide

theorem mul_I4 (A : M4) : M4.mul I4 A = A := by
  cases A; simp only [M4.mul, I4]; simp

/-- MAIN (peel ratio %(z)d): for every non-backtracking word of every length L >= 1, the scaled time
    coordinate %(z)d^L cosh d(w) is not divisible by %(z)d. -/
theorem time_not_div%(z)d (g : Dir) (l : List Dir) (hr : RedFrom g l) :
    (word B%(z)d (g :: l)).a00 %% %(z)d ≠ 0 := by
  have h0 : Inv%(z)d (B%(z)d g) g := ⟨%(c0)d, by decide, g, mem_allDirs g, gen_red%(z)d g (mem_allDirs g)⟩
  have hw := walk%(z)d l (B%(z)d g) g h0 hr
  obtain ⟨c, hc, f, hf, hred⟩ := hw
  have hword : word B%(z)d (g :: l) = l.foldl (fun M d => M4.mul M (B%(z)d d)) (B%(z)d g) := by
    simp only [word, List.foldl_cons, mul_I4]
  rw [hword]
  have h1 := congrArg M4.a00 hred
  simp only [M4.red] at h1
  rw [h1]
  exact corner%(z)d c hc f hf _ (mem_allDirs _)

theorem pow_emod%(z)d (n : Nat) : ((%(z)d : Int) ^ (n + 1)) %% %(z)d = 0 := by
  rw [Int.pow_succ]; exact Int.mul_emod_left _ _

/-- No non-backtracking word returns to the origin: its time coordinate is never %(z)d^L (cosh d = 1). -/
theorem no_return%(z)d (g : Dir) (l : List Dir) (hr : RedFrom g l) :
    (word B%(z)d (g :: l)).a00 ≠ %(z)d ^ (l.length + 1) := by
  intro h; have := time_not_div%(z)d g l hr; rw [h, pow_emod%(z)d] at this; exact this rfl

/-- No non-backtracking word is the identity: the boost group is free on the three steps. -/
theorem free%(z)d (g : Dir) (l : List Dir) (hr : RedFrom g l) :
    word B%(z)d (g :: l) ≠ M4.smul (%(z)d ^ (l.length + 1)) I4 := by
  intro h; apply no_return%(z)d g l hr; rw [h]; simp [M4.smul, I4]
''' % dict(z=z, cs=cs[z], c0={5:3,7:4}[z]))
s='\n'.join(o)
# RedFrom, lastStep, mul_I4 are defined twice; keep only the first copy
for dup in ['/-- Non-backtracking from g. -/\ndef RedFrom','def lastStep','theorem mul_I4']:
    i=s.index(dup); j=s.index(dup,i+1)
    # cut the second definition block up to the next blank line pair
    end=s.index('\n\n',j)
    s=s[:j]+s[end+2:]
open('part2.lean','w').write(s)
print(len(s))
```

figs.py

```
# SCRIPT: PEEL-TREES-FIGS
# Figures for "Two Trees in One Ball". Every plotted point is computed here from the exact boosts or the
# Riley matrices; nothing is placed by hand.
import numpy as np, matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.patches import Circle
BG='#070c18'; INK='#e3ebff'; DIM='#8fa2c4'; LINE='#2a3a5c'
C1='#4fe3c1'; C2='#ffb547'; C3='#ff6f91'; C4='#7aa8ff'; C5='#c08bff'
plt.rcParams.update({'font.family':'DejaVu Sans','text.color':INK,'axes.edgecolor':LINE,'axes.labelcolor':INK,
                     'xtick.color':DIM,'ytick.color':DIM,'savefig.facecolor':BG,'figure.facecolor':BG,'axes.facecolor':BG})
def boost(ax, sg, r):
    B=np.eye(4); c,s=np.cosh(r),np.sinh(r); B[0,0]=c; B[ax,ax]=c; B[0,ax]=B[ax,0]=sg*s; return B
D2={'+i':(1,1),'-i':(1,-1),'+j':(2,1),'-j':(2,-1)}
INV={'+i':'-i','-i':'+i','+j':'-j','-j':'+j','+k':'-k','-k':'+k'}
def disk(p): return p[1:3]/(1+p[0])
def geo(p,q,n=40):
    d=np.arccosh(max(1.0,p[0]*q[0]-p[1:]@q[1:]))
    if d<1e-9: return np.array([disk(p)])
    t=np.linspace(0,1,n); P=(np.sinh((1-t)[:,None]*d)*p+np.sinh(t[:,None]*d)*q)/np.sinh(d)
    return np.array([disk(x) for x in P])
def frame(ax,title):
    ax.add_patch(Circle((0,0),1,fill=False,ec=DIM,lw=1.2)); ax.set_xlim(-1.08,1.08); ax.set_ylim(-1.08,1.08)
    ax.set_aspect('equal'); ax.axis('off'); ax.set_title(title,color=INK,fontsize=12.5,pad=8)
def tree(r,depth):
    G={g:boost(a,s,r) for g,(a,s) in D2.items()}; out={(): (np.eye(4), None)}; layer=[()]
    for _ in range(depth):
        new=[]
        for w in layer:
            for g in G:
                if w and INV[w[-1]]==g: continue
                M=out[w][0]@G[g]; out[w+(g,)]=(M,w); new.append(w+(g,))
        layer=new
    return out
e0=np.array([1,0,0,0.])
# ---------- Figure 1: the peel ladder
fig,axs=plt.subplots(1,2,figsize=(11,5.3))
for ax,z,col in [(axs[0],5,C1),(axs[1],7,C2)]:
    frame(ax,'peel ratio %d: Smith step Γ = %s, length ln %d = %.4f'%(z,'2/3' if z==5 else '3/4',z,np.log(z)))
    ax.plot([-1,1],[0,0],color=LINE,lw=1)
    from math import gcd
    rows=[]
    for L in range(0,5):
        g=(z**L-1)/(z**L+1); ax.plot([g],[0],'o',color=col,ms=8 if L else 6,zorder=3)
        n_,d_=z**L-1,z**L+1; k=gcd(n_,d_); fr='0' if L==0 else '%d/%d'%(n_//k,d_//k)
        rows.append('L = %d    norm %-5d   Γ = %-9s  d = %.4f'%(L,z**L,fr,L*np.log(z)))
        if 1<=L<=2: ax.annotate('L = %d'%L,(g,0),(g-
0.08*L,0.18+0.14*L),color=INK,fontsize=9,ha='center',arrowprops=dict(arrowstyle='-',color=DIM,lw=0.6))
    ax.annotate('L = 0',(0,0),(0,0.18),color=INK,fontsize=9,ha='center',arrowprops=dict(arrowstyle='-',color=DIM,lw=0.6))
    ax.text(-0.74,-0.16,'\n'.join(rows),color=INK,fontsize=8.0,family='DejaVu Sans Mono',va='top')
    ax.text(0,-0.72,'Γ_L = (%d^L − 1)/(%d^L + 1),   d(0, Γ_L) = L·ln %d'%(z,z,z),color=col,fontsize=11,ha='center')
fig.suptitle('The norm ladder is the peel ladder: each step multiplies the ratio, adds the length',color=INK,fontsize=13.5)
fig.tight_layout(); fig.savefig('figs/fig1_ladder.png',dpi=170); plt.close(fig)
# ---------- Figure 2: the plane slice of Garrett's tree, Schottky caps, residues mod 5
rho=np.sqrt(2)-1; dl=np.log(1+np.sqrt(2))
fig,axs=plt.subplots(1,2,figsize=(12,6.2))
rescol={1:C1,2:C2,3:C4,4:C5,0:C3}
for ax,z in [(axs[0],5),(axs[1],7)]:
    r=np.log(z); T=tree(r,4)
    frame(ax,'step ln %d  (Γ = %s, Γ² %s 1/2)'%(z,'2/3' if z==5 else '3/4','<' if z==5 else '>'))
    for th in [0,90,180,270]:  # caps: geodesics at distance ln(1+√2) from the origin, tangent at the boundary
        c=np.sqrt(2)*np.array([np.cos(np.radians(th)),np.sin(np.radians(th))])
        cp=Circle(c,1,fill=True,fc=C4,alpha=0.10,ec=C4,lw=1.0); ax.add_patch(cp);
cp.set_clip_path(Circle((0,0),1,transform=ax.transData))
        # image of the plane at -δ under the boost of length r sits at r-δ
        rr=np.tanh((r-dl)/2); cc=(1+rr*rr)/(2*rr); rad=(1-rr*rr)/(2*rr)
        ip=Circle(cc*np.array([np.cos(np.radians(th)),np.sin(np.radians(th))]),rad,fill=False,ec='#ffffff' if z==5 else
C1,lw=1.1,ls='--'); ax.add_patch(ip); ip.set_clip_path(Circle((0,0),1,transform=ax.transData))
    for w,(M,par) in T.items():
        if par is None: continue
        P=geo(T[par][0]@e0,M@e0); ax.plot(P[:,0],P[:,1],color=LINE if len(w)>2 else DIM,lw=0.7,zorder=1)
    for w,(M,par) in T.items():
        p=M@e0; q=disk(p); L=len(w)
        if z==5 and L:
            res=int(round(p[0]*5**L))%5; ax.plot(*q,'o',ms=max(2,7-1.3*L),color=rescol[res],zorder=3)
        else:
            ax.plot(*q,'o',ms=max(2,7-1.3*L),color=INK if L==0 else C2,zorder=3)
    ax.text(0,-1.2,('blue caps sit at distance ln(1+√2) from the centre; a step maps the outside of the\nopposite cap to the
dashed white line, which spills out of its cap: no ping-pong here.\nColour = residue of 5^L·cosh d mod 5: green 1, gold 2,
blue 3, violet 4. Residue 0 never occurs.') if z==5 else
                    ('a step maps the outside of the opposite cap to the dashed green line, inside its own cap:\nSchottky
ping-pong, so the tree embeds quasi-isometrically'),color=DIM,fontsize=9.2,ha='center',va='top')
fig.suptitle('Garrett\'s tree, the ±i, ±j slice to depth 4, true hyperbolic geodesics',color=INK,fontsize=13.5)
```

```
fig.tight_layout(rect=(0,0.07,1,1)); fig.savefig('figs/fig2_tree.png',dpi=170); plt.close(fig)
# ---------- Figure 3: straight versus turned, and the Thomas rotation
r=np.log(5); G={g:boost(a,s,r) for g,(a,s) in D2.items()}
fig,axs=plt.subplots(1,2,figsize=(11.5,5.6),gridspec_kw=dict(width_ratios=[1.25,1]))
ax=axs[0]; frame(ax,'three steps of ln 5: straight is farthest')
for w,col in [(('+i','+i','+i'),C1),(('+i','+i','+j'),C2),(('+i','+j','+i'),C4),(('+i','+j','+j'),C5)]:
    M=np.eye(4); pts=[e0]
    for g in w: M=M@G[g]; pts.append(M@e0)
    for a,b in zip(pts[:-1],pts[1:]):
        P=geo(a,b); ax.plot(P[:,0],P[:,1],color=col,lw=2)
    P=geo(e0,pts[-1]); ax.plot(P[:,0],P[:,1],color=col,lw=1,ls=':')
    d=np.arccosh(pts[-1][0]); q=disk(pts[-1])
    ly={('+i','+i','+i'):-0.40,('+i','+i','+j'):0.18,('+i','+j','+i'):0.46,('+i','+j','+j'):0.74}[w]
    ax.plot(*q,'o',color=col,ms=6); ax.annotate('%s   d = %.4f,  cosh d = %d/125'%(' '.join(w),d,round(pts[-1][0]*125)),q,
(-0.80,ly),
        color=col,fontsize=8.5,arrowprops=dict(arrowstyle='-',color=col,lw=0.5,alpha=0.6))
ax.plot(0,0,'o',color=INK,ms=5); ax.text(0.02,-0.08,'origin',color=DIM,fontsize=8)
ax.text(0,-0.95,'one turn: cosh d = cosh²(ln 5) = 169/25 (right angle, hyperbolic
Pythagoras)',color=DIM,fontsize=8.8,ha='center')
ax=axs[1]; ax.set_xlim(-1.3,1.3); ax.set_ylim(-1.3,1.3); ax.set_aspect('equal'); ax.axis('off')
ax.set_title('B_i B_j = (pure boost) × (rotation)',color=INK,fontsize=12.5)
th=np.arccos(65/97)
for a0,col,lab in [(0,DIM,'frame before'),(-th,C2,'after the loop')]:
    for v in [(1,0),(0,1)]:
        x=np.cos(a0)*v[0]-np.sin(a0)*v[1]; y=np.sin(a0)*v[0]+np.cos(a0)*v[1]
        ax.annotate('',(x,y),(0,0),arrowprops=dict(arrowstyle='->',color=col,lw=2))
t=np.linspace(-th,0,40); ax.plot(0.45*np.cos(t),0.45*np.sin(t),color=C2,lw=1.2)
ax.text(0.5,-0.32,'θ = %.3f°'%np.degrees(th),color=C2,fontsize=10)
ax.text(0,-0.95,'cos θ = 65/97,  sin θ = 72/97\ntan(θ/2) = 4/9 = Γ²',color=INK,fontsize=11,ha='center',va='top')
ax.text(0,1.2,'two perpendicular steps of ln 5 leave a turn of the frame',color=DIM,fontsize=9,ha='center')
fig.tight_layout(); fig.savefig('figs/fig3_turn.png',dpi=170); plt.close(fig)
# ---------- Figure 4: the knot tree closes
def mob(M,z): return (M[0,0]*z+M[0,1])/(M[1,0]*z+M[1,1])
def cay(z): return (z-1j)/(z+1j)
def hgeo(z1,z2,n=50):  # geodesic in the upper half plane via the disk: map to hyperboloid of H^2
    def toH(z):
        w=cay(z); r2=abs(w)**2; return np.array([(1+r2)/(1-r2),2*w.real/(1-r2),2*w.imag/(1-r2),0.])
    p,q=toH(z1),toH(z2); return geo(p,q,n)
def riley(s,v): return np.array([[s,1],[0,1/s]]),np.array([[s,0],[-v,1/s]])
def axis_pts(M):  # fixed points of a hyperbolic Mobius map on the real line (∞ allowed)
    a,b,c,d=M.ravel()
    if abs(c)<1e-12: return [np.inf,-b/(a-d)]
    disc=np.sqrt((d-a)**2+4*b*c); return [((a-d)+disc)/(2*c),((a-d)-disc)/(2*c)]
def bnd(x): return np.array([0,1.]) if np.isinf(x) else np.array([cay(x).real,cay(x).imag])
fig,axs=plt.subplots(1,3,figsize=(16,5.9))
import sys; sys.path.insert(0,'../atlas_sm/scripts')
cases=[('trefoil, s = 2 (exact)',2.0,13/4,'aba','bab'),]
import sympy as sp
xs=0.5; s=np.exp(xs); R4=lambda v: s*s*v*v+(3*s*s-s**4-1)*v+(3*s*s-s**4-1)
v4=max(np.roots([s*s,3*s*s-s**4-1,3*s*s-s**4-1]).real)
cases.append(('figure-eight, x = 0.5, v = %.5f'%v4,s,v4,None,None))
for ax,(title,s_,v_,_,__) in zip(axs[:2],cases):
    frame(ax,title); a,b=riley(s_,v_); A,Bm=np.linalg.inv(a),np.linalg.inv(b)
    Ms={'a':a,'A':A,'b':b,'B':Bm}
    for M,col in [(a,C1),(b,C4)]:
        f1,f2=axis_pts(M); p,q=bnd(f1),bnd(f2)
        # geodesic between two boundary points: sample interior points via the half plane
        if np.isinf(f1) or np.isinf(f2):
            x0=f2 if np.isinf(f1) else f1; zz=x0+1j*np.logspace(-3,3,200)
        else:
            c0=(f1+f2)/2; R=abs(f1-f2)/2; t=np.linspace(0.002,np.pi-0.002,200); zz=c0+R*np.exp(1j*t)
        ww=cay(zz); ax.plot(ww.real,ww.imag,color=col,lw=1,alpha=0.8)
    # faint depth-3 word tree
    layer=[('',np.eye(2))]
    for dpt in range(3):
        new=[]
        for w,M in layer:
            for g in 'aAbB':
                if w and w[-1]==g.swapcase(): continue
                N=M@Ms[g]; P=hgeo(mob(M,1j),mob(N,1j),20); ax.plot(P[:,0],P[:,1],color=LINE,lw=0.6,zorder=1);
new.append((w+g,N))
        layer=new
    rel='abaBAB' if 'trefoil' in title else 'a'+'bABa'+'B'+'AbaB'   # a W b^-1 W^-1 with W = b a^-1 b^-1 a
    M=np.eye(2); pts=[1j]
    for g in rel: M=M@Ms[g]; pts.append(mob(M,1j))
    for i,(z1,z2) in enumerate(zip(pts[:-1],pts[1:])):
        P=hgeo(z1,z2); ax.plot(P[:,0],P[:,1],color=C2,lw=2.2,zorder=3)
    for z in pts[:-1]:
        q=cay(z); ax.plot(q.real,q.imag,'o',color=C2,ms=4.5,zorder=4)
    ax.plot(0,0,'o',color=INK,ms=7,zorder=5)
    ax.text(0,-1.18,'relator %s walks %d steps and returns (|end − start| = %.1e)'%(rel,len(rel),abs(pts[-1]-
pts[0])),color=DIM,fontsize=9,ha='center')
    ax.text(0,-1.27,'meridian axes: a green, b blue; both translate by 2x = %.4f'%
(2*np.log(s_)),color=DIM,fontsize=9,ha='center')
ax=axs[2]; frame(ax,'Garrett slice, step ln 5: a reduced word never returns')
T=tree(np.log(5),4)
```

```
for w,(M,par) in T.items():
    if par is None: continue
    P=geo(T[par][0]@e0,M@e0); ax.plot(P[:,0],P[:,1],color=LINE,lw=0.6,zorder=1)
w=('+i','+j','-i','-j','+i','+j'); Gs={g:boost(a_,s2,np.log(5)) for g,(a_,s2) in D2.items()}; M=np.eye(4); pts=[e0]
for g in w: M=M@Gs[g]; pts.append(M@e0)
for p,q in zip(pts[:-1],pts[1:]):
    P=geo(p,q); ax.plot(P[:,0],P[:,1],color=C3,lw=2.2,zorder=3)
ax.plot(0,0,'o',color=INK,ms=7,zorder=5)
ax.text(0,-1.18,'the commutator-shaped word i j ī j̄ i j: cosh d = %d/5^6, never 1'%round(pts[-1]
[0]*5**6),color=DIM,fontsize=9,ha='center')
fig.suptitle('The knot\'s tree closes, Garrett\'s tree never does: one hyperbolic disk, two kinds of
tree',color=INK,fontsize=13.5)
fig.tight_layout(rect=(0,0.05,1,1)); fig.savefig('figs/fig4_knots.png',dpi=170); plt.close(fig)
print('figures written; trefoil relation and 4_1 relation drawn; v4 =',v4)
```

POW2_REPORT.md

```
# Two Trees in One Ball: are the boost groups free at z = 2, 4, 8?

Answer: YES for all three (and for every even integer z >= 2). Graded PROVED (elementary proof below; every step written).
Exact-computation cross-checks and the opposite-first relation search agree (no relation found).
Directory: /tmp/claude-0/-home-claude/d7048feb-3787-5f7f-b86a-f71c9a373eed/scratchpad/lanes/open/pow2/
Each script starts with `# SCRIPT: NAME` and states its kill condition in its header before the run.

| z | relation search (exact-hash meet-in-the-middle) | origin-fixing non-backtracking word | planar <B_i,B_j> | commutator
[B_i,B_j] | freeness |
|---|---|---|---|---|---|
| 2 | none, all relations of length <= 22 | none, length <= 14 | none, length <= 28 (origin <= 17) | elliptic, T = 31/16,
infinite order, angle not a rational multiple of pi | PROVED |
| 4 | none, length <= 20 | none, length <= 13 | none, length <= 28 | elliptic, T = 47/64, infinite order, angle irrational |
PROVED |
| 8 | none, length <= 20 | none, length <= 13 | none, length <= 28 | loxodromic, T = -1889/256 (|T|>2), infinite order |
PROVED |

## Step 1. Opposite first: search for a relation
relation_search.c (+ mkgens.py makes the generator files). Words are non-backtracking in the 6 letters B_u^{+-1}.
Method: all non-backtracking words of length n-1 and n are hashed (4x4 matrix times (2z)^{-len} mod 2^31-1, all 16 entries
combined into a 64-bit key; this normalisation makes words of different length comparable). Two distinct words with equal
key are a
relation candidate (any relation of length <= 2n splits as w1 = w2 with lengths n, n or n, n-1). Origin search: all words of
length <= dmax
are applied to e0 (mod p); a hit would be a non-backtracking word fixing the origin.
Kill condition (written in the C header before the run): any collision or origin hit = relation found; none = groups survive
to the stated length.
Positive controls (the search CAN find relations): gens_ctrl.txt (contains B_i(4) = B_i(2)^2) and z = 1 both report
collisions at length 2-3.
Results (files out_search_*.txt): z=2: records 70,312,500, 0 collision groups, 9.2e9 origin nodes, 0 hits (n=11, dmax=14);
z=4 and z=8: n=10 (14,062,500 records), 0 collisions, dmax=13, 0 hits; planar z=2,4,8: n=14 (8,503,056 records), 0
collisions, dmax=17, 0 hits.
No candidate arose, so no exact re-verification was needed. (Mod-p hits would have been verified exactly in Python.)

## Step 2. Commutator type (commutator_type.py, out_commutator_type.txt)
Exact 4x4 trace of [B_a^s, B_b^s'] for all 12 sign/axis pairs equals T^2 with T = 2 - 4 sinh^4(ln z/2) = 2 - (z-1)^4/(4z^2)
(checked exactly; this is the SL2 trace of the commutator, 4x4 trace = T^2 in both the elliptic and hyperbolic case).
* z=2: T = 31/16, |T|<2, elliptic. cos(phi) = T^2/2 - 1 = 449/512, phi/pi = 0.15957...
* z=4: T = 47/64, elliptic. cos(phi) = -5983/8192, phi/pi = 0.76064...
* z=8: T = -1889/256, |T| > 2: loxodromic, no rotation at all.
Finite order test: an SL2(C) element of finite order has eigenvalues that are roots of unity, so its trace is an algebraic
integer.
T = 31/16 and 47/64 are rational non-integers, so the elliptic commutators have INFINITE order; e^{i phi/2} is a root of x^2
- Tx + 1,
not a root of unity; 2cos(phi) = 449/256, -5983/4096 are not algebraic integers, so phi is not a rational multiple of pi.
So no commutator gives a finite-order elliptic relation. (Thomas rotation of B_iB_j: cos = 40/41, 272/353, 2080/4481:
Pythagorean-type, not roots of unity.)

## Step 3. Freeness proof (written in full)
(a) Residues mod other primes (residue_primes.py): det(2zB_u) = 16 z^4, so for every odd prime p the scaled generators have
rank 4 mod p
(no rank-one reduction exists); for p | z^2+1 they become monomial (finite image), for p | z^2-1 scalar-like. Only p = 2
degenerates
(rank 1 mod 2, which is why the Lean argument is silent). The odd-divisor method is dead for powers of 2.
(b) The 2-adic route works, in elementary form: ping-pong on P^1(Q(i)) for the (1+i)-adic valuation. The structure behind it
is the
Bruhat-Tits tree of PGL2(Q_2(i)) (valence 3): the three boosts are hyperbolic with translation length v(z)=2k and pairwise
disjoint axes
at distance 2 from the base vertex; the proof below avoids the tree and uses only valuations.

THEOREM. Let z be an even integer >= 2 (in particular z = 2, 4, 8). Then B_i, B_j, B_k generate a free group of rank 3
(and any two of them a free group of rank 2).

Step 0 (dictionary). Pauli matrices s_x, s_y, s_z. On Hermitian 2x2 matrices X = t + x.s (quadratic form det X = t^2-x^2-
y^2-z^2) put
```

rho(M)X = M X M*/|det M| for M in GL2(C). rho is a homomorphism to the Lorentz group. Its kernel is the scalars: if rho(M)=I, then X=E11, E22 give columns of M parallel to e1, e2, so M=diag(a,b) with |a|^2=|b|^2=|det M|; X=[[1,1],[1,1]] gives a conj(b) = |ab|>0, hence a=b.
Let M_u = a + b s_u with a=z+1, b=z-1 (det = a^2-b^2 = 4z). Split x = x_par u + x_perp. s_u commutes with t + x_par s_u and anticommutes with x_perp.s, so
M X M* = (a+b s_u)^2 (t + x_par s_u) + (a+b s_u)(a-b s_u) x_perp.s, with (a+b s_u)^2 = (a^2+b^2) + 2ab s_u = 2(z^2+1) + 2(z^2-1) s_u.
Dividing by 4z: t' = ((z^2+1)t + (z^2-1)x_par)/(2z), x_par' = ((z^2-1)t+(z^2+1)x_par)/(2z), x_perp' = x_perp. Thus rho(M_u) = B_u (rapidity +ln z; the
other sign is the inverse). (Also verified exactly for z=2,4,8,16, pingpong_check.py CHECK 1.)
Consequence: a reduced word w in B_u^{+-1} equals the identity iff the corresponding product M_w of M_u^{+-1} is a scalar iff M_w acts as the identity on P^1(Q(i)) (Moebius action). So it suffices to show that no nonempty reduced word acts trivially on P^1.

Step 1 (valuation). For x = a+bi in Q(i)^*, v(x) := v_2(a^2+b^2) (v_2 the 2-adic valuation, extended to Q), v(0)=infinity. This is the (1+i)-adic valuation (1+i is the only prime above 2 and has norm 2): v(xy)=v(x)+v(y), v(x+y) >= min(v(x),v(y)) with equality when v(x) != v(y). Values: v(1+i)=v(1-i)=1, v(2)=2, v(unit i)=0, v(odd integer)=0. For z even: v(z) >= 2, v(z-1)=0, v(2z) >= 4.

Step 2 (six sets in P^1(Q(i)) = Q(i) u {inf}).
U_x+ = {w : v(w+1) >= 3}, U_x- = {w : v(w-1) >= 3}, U_y+ = {w : v(w-i) >= 3}, U_y- = {w : v(w+i) >= 3},
U_z+ = {w : v(w) >= 1}, U_z- = {w : v(w) <= -1} u {inf}. (Each nonempty: contains its centre.)
Pairwise disjoint: the centres 1,-1,i,-i have pairwise differences of valuation <= 2 (v(2)=v(2i)=2, v(1-i)=v(1+i)=1), so two radius-3 balls cannot share a point; points of those balls have v(w)=0, so they miss U_z+ (v>=1) and U_z- (v<=-1); and U_z+ cap U_z- is empty.

Step 3 (inclusions). Moebius maps: f_x(w) = ((z+1)w+(z-1))/((z-1)w+(z+1)) (matrix M_x), f_y(w) = ((z+1)w - i(z-1))/(i(z-1)w+(z+1)) (M_y), f_z(w) = z w (M_z ~ diag(2z,2)).
Inverses: f_x^{-1}(w) = -f_x(-w), f_y^{-1}(w) = -f_y(-w), f_z^{-1}(w) = w/z (direct substitution, inverse matrix is M with b -> -b).
Claim: f_s(P^1 \ U_{s^{-1}}) is contained in U_s for each of the six letters s.
* x: f_x(w)+1 = 2z(w+1)/D with D = (z-1)(w-1)+2z. Let w not in U_x-, i.e. v(w-1) <= 2 (or w = inf). If w=inf, f_x(w)+1 = 2z/(z-1), valuation v(2z) >= 4. Else y=w-1 != 0 with v(y)<=2: v((z-1)y)=v(y) <= 2 < 4 <= v(2z), so v(D)=v(y); and w+1 = y+2 has v >= min(v(y),2) = v(y). So v(f_x(w)+1) >= 4 + v(y) - v(y) = 4 >= 3: f_x(w) in U_x+.
 For the inverse: P^1 \ U_x+ = -(P^1 \ U_x-), f_x^{-1}(w) = -f_x(-w) lands in -U_x+ = U_x-.
* y: f_y(w) - i = 2z(w-i)/D_y with D_y = i(z-1)(w+i)+2z. For w not in U_y-, v(Y)<=2 where Y=w+i, the same computation gives v(D_y)=v(Y), v(w-i)=v(Y-2i) >= v(Y), so v(f_y(w)-i) >= 4 (w=inf: f_y(inf)-i = 2z/(i(z-1))). Inverse via -f_y(-w) as for x, with -U_y+ = U_y-.
* z: w not in U_z- means v(w) >= 0, so v(zw) >= 2 >= 1: f_z(w) in U_z+. w not in U_z+ means v(w)<=0 or inf, so v(w/z) <= -2 <= -1 (inf stays inf): f_z^{-1}(w) in U_z-.
(All checked on 100,000 exact test points, plus z = 6,10,12,20,64 pass and the NEGATIVE controls z = 1,3,5,7 fail on 80% of the points, so the test is not blind: pingpong_check.py CHECK 2, pingpong_controls.py.)

Step 4 (ping-pong). Claim: for every nonempty reduced word w = s_1...s_n and every p not in U_{s_n^{-1}}, w(p) lies in U_{s_1}.
Induction on n. n=1 is Step 3. For n>1, s_n(p) is in U_{s_n}, and U_{s_n} is disjoint from U_{s_{n-1}^{-1}} because s_n != s_{n-1}^{-1} (reduced) and distinct letters have disjoint sets; apply the claim to s_1...s_{n-1}.
Now suppose w acts as the identity. Pick a letter t with t not in {s_1, s_n^{-1}} (four or more choices) and a point p in U_t. Then p is outside U_{s_1} and U_{s_n^{-1}} (disjointness), but w(p)=p must lie in U_{s_1}: contradiction. Hence no nonempty reduced word is the identity: by Step 0 B_i,B_j,B_k generate a free group of rank 3; omitting a letter pair gives rank 2. QED.

Where the hypotheses are used: v(z) >= 2 (so v(2z)>=4 > 2 and v(z)>=2 for the z-axis) and z-1 a unit, i.e. z even. For odd z the same computation fails (negative control), and the Lean odd-divisor argument takes over. Together: every integer z >= 2 is covered (Lean: z with an odd divisor m>1; here: every even z, which includes all powers of 2).

Cross-check by an independent algebraic quantity (pingpong_check.py CHECK 3-4): 3000 random reduced words up to length 60 satisfy the ping-pong conclusion; the displacement of the base lattice, d(w) = v(det M_w) - 2 min v(entries of M_w) with entries in Z[i], is > 0 for all 4686 reduced words of length <= 5 and for 900 random reduced words up to length 40 (min 2 / 4 / 6 for z=2/4/8, attained by single z-letters, equal to the translation length 2k).

Open / not claimed
* Not proved: that no non-backtracking word fixes the origin of the ball (as the Lean theorem does for non-powers of 2). Freeness does not imply it (G is not discrete; a word could be a rotation about the origin). Search: none up to length 14 (z=2), 13 (z=4,8), 17 (planar). Status: open, search negative.
* The proof is written and machine-spot-checked in exact arithmetic, but not formalised in Lean (needs the (1+i)-adic valuation on Z[i]/Q(i); the six-set argument is finite and could be formalised with v(x) = v_2(norm)).

Files
mkgens.py, relation_search.c (+ binary), out_search_*.txt, commutator_type.py/out_commutator_type.txt, residue_primes.py/out_residue_primes.txt, pingpong_check.py/out_pingpong_check.txt, pingpong_controls.py/out_pingpong_controls.txt.

mkgens.py

```
# SCRIPT: MKGENS
# Writes generator files for relation_search.c.  Each letter: scale D, then 16 integers (row-major) Q, with B = Q/D.
# Letter order: for axis a in axes: (+boost, -boost); letter 2m and 2m+1 are inverses.
import sys
def boost(z, a, sgn):
    # Q = 2z*B, integer entries; B_u: cosh=(z^2+1)/(2z), sinh=sgn*(z^2-1)/(2z) on (t,axis) block, 1 on other two diagonals
    Q = [[0]*4 for _ in range(4)]
```

```
    for i in range(1,4):
        Q[i][i] = 2*z
    Q[0][0] = z*z+1
    Q[a][a] = z*z+1
    Q[0][a] = sgn*(z*z-1)
    Q[a][0] = sgn*(z*z-1)
    return Q
def write(fname, mats, D):
    with open(fname,'w') as f:
        f.write(str(len(mats))+"\n")
        for Q,d in mats:
            f.write(str(d)+" "+" ".join(str(Q[i][j]) for i in range(4) for j in range(4))+"\n")
if __name__=="__main__":
    for z in (1,2,4,8,16):
        for name,axes in (("full",(1,2,3)),("planar",(1,2))):
            mats=[]
            for a in axes:
                mats.append((boost(z,a,1),2*z)); mats.append((boost(z,a,-1),2*z))
            write("gens_z%d_%s.txt"%(z,name),mats,2*z)
    # positive control: letters B_i(2), B_i(2)^-1, B_i(4), B_i(4)^-1  (relation  b = a a ) plus B_j(2)
    mats=[(boost(2,1,1),4),(boost(2,1,-1),4),(boost(4,1,1),8),(boost(4,1,-1),8),(boost(2,2,1),4),(boost(2,2,-1),4)]
    write("gens_ctrl.txt",mats,0)
```

relation_search.c

```
/* SCRIPT: RELATION_SEARCH
   usage: ./relation_search gensfile n dmax
   KILL CONDITION (written before the run): the search reports FOUND if (a) two distinct non-backtracking words of
   length n or n-1 (n-1 vs n included) give the same matrix mod p (hash of all 16 entries) -> relation candidate,
   later verified exactly in Python; or (b) a non-backtracking word of length <= dmax maps e0 to e0 mod p.
   If nothing is found the groups survive this search up to relation length 2n (collision search) and dmax (origin search).
   Positive control: gens_ctrl.txt contains B_i(4)=B_i(2)^2, so the search MUST report a relation of length 3.
*/
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
typedef uint64_t u64;
static const u64 P = 2147483647ULL; /* 2^31-1 */
static int NL; static u64 G[12][16]; static u64 CK[16];
static u64 pw(u64 a,u64 e){u64 r=1;a%=P;while(e){if(e&1)r=r*a%P;a=a*a%P;e>>=1;}return r;}
typedef struct {u64 key; u64 code;} Rec;  /* code = len<<56 | base-NL digits */
static Rec *R; static size_t nR=0, capR=0; static int N;
static void push(u64 key,u64 code){ if(nR==capR){capR=capR?capR*2:1<<20;R=realloc(R,capR*sizeof(Rec));if(!R)
{fprintf(stderr,"oom\n");exit(1);}} R[nR].key=key;R[nR].code=code;nR++; }
static u64 hashm(const u64*M){u64 k=0;for(int j=0;j<16;j++)k+=M[j]*CK[j];return k;}
static void mulm(const u64*A,const u64*B,u64*C){
  for(int i=0;i<4;i++)for(int j=0;j<4;j++){
    u64 s=A[i*4]*B[j]+A[i*4+1]*B[4+j]+A[i*4+2]*B[8+j]+A[i*4+3]*B[12+j]; /* each <2^62, sum of 4 < 2^64 */
    C[i*4+j]=s%P; } }
static void dfs(int depth,const u64*M,int last,u64 code){
  if(depth>=N-1){ push(hashm(M),((u64)depth<<56)|code); }
  if(depth==N) return;
  for(int l=0;l<NL;l++){ if(depth>0 && l==(last^1)) continue;
    u64 C[16]; mulm(M,G[l],C); dfs(depth+1,C,l,code*NL+l); } }
static int cmp(const void*a,const void*b){u64 x=((Rec*)a)->key,y=((Rec*)b)->key;return x<y?-1:x>y;}
static int dmax; static long long nodes=0, fixed=0;
static u64 E0v[4];
/* origin search: prepend letters; vector v = (word) e0, 4 entries */
static void dfs2(int depth,const u64*v,int first,u64 code){
  nodes++;
  if(depth>0 && v[0]==1 && v[1]==0 && v[2]==0 && v[3]==0){ fixed++; if(fixed<=10) printf("ORIGIN-FIXED len %d code(base %d,
leftmost last digit) %llu\n",depth,NL,(unsigned long long)code); }
  if(depth==dmax) return;
  for(int l=0;l<NL;l++){ if(depth>0 && l==(first^1)) continue;
    u64 w[4]; for(int i=0;i<4;i++){ u64 s=0; for(int j=0;j<4;j++) s+=G[l][i*4+j]*v[j]; w[i]=s%P; }
    dfs2(depth+1,w,l,code+ (u64)l*0 + 0) ; } }
int main(int argc,char**argv){
  FILE*f=fopen(argv[1],"r"); N=atoi(argv[2]); dmax=atoi(argv[3]);
  if(fscanf(f,"%d",&NL)!=1) return 1;
  for(int l=0;l<NL;l++){ long long D; if(fscanf(f,"%lld",&D)!=1) return 1; u64 iD=pw((u64)D%P,P-2);
    for(int j=0;j<16;j++){ long long q; if(fscanf(f,"%lld",&q)!=1) return 1; long long qm=((q%(long long)P)+(long long)P)%
(long long)P; G[l][j]=(u64)qm*iD%P; } }
  u64 s=88172645463325252ULL; for(int j=0;j<16;j++){ s^=s<<13;s^=s>>7;s^=s<<17; CK[j]=s|1; }
  u64 I[16]={0}; for(int i=0;i<4;i++) I[i*5]=1;
  dfs(0,I,-1,0);
  qsort(R,nR,sizeof(Rec),cmp);
  long long ncoll=0; size_t i=0;
  while(i<nR){ size_t j=i; while(j<nR && R[j].key==R[i].key) j++;
    if(j-i>1){ ncoll++; if(ncoll<=10){ printf("COLLISION group:"); for(size_t t=i;t<j&&t<i+4;t++){ printf(" [len %d code
%llu]",(int)(R[t].code>>56),(unsigned long long)(R[t].code&((1ULL<<56)-1))); } printf("\n"); } }
    i=j; }
```

```
  printf("records %zu (lengths %d and %d), collision groups %lld\n",nR,N-1,N,ncoll);
  /* origin search */
  u64 e[4]={1,0,0,0}; dfs2(0,e,-1,0);
  printf("origin search nodes %lld, origin-fixing nonbacktracking words (len<=%d) mod p: %lld\n",nodes,dmax,fixed);
  return 0; }
```

out_search_z2_full.txt

```
records 70312500 (lengths 10 and 11), collision groups 0
origin search nodes 9155273437, origin-fixing nonbacktracking words (len<=14) mod p: 0

real    6m24.077s
user    4m24.102s
sys     0m1.721s
```

out_search_z4_full.txt

```
records 14062500 (lengths 9 and 10), collision groups 0
origin search nodes 1831054687, origin-fixing nonbacktracking words (len<=13) mod p: 0

real    1m46.090s
user    0m52.261s
sys     0m0.315s
```

out_search_z8_full.txt

```
records 14062500 (lengths 9 and 10), collision groups 0
origin search nodes 1831054687, origin-fixing nonbacktracking words (len<=13) mod p: 0

real    1m57.464s
user    0m53.141s
sys     0m0.324s
```

out_search_planar_z2_z4.txt

```
records 8503056 (lengths 13 and 14), collision groups 0
origin search nodes 258280325, origin-fixing nonbacktracking words (len<=17) mod p: 0

real    0m35.723s
user    0m9.677s
sys     0m0.239s
records 8503056 (lengths 13 and 14), collision groups 0
origin search nodes 258280325, origin-fixing nonbacktracking words (len<=17) mod p: 0

real    0m30.132s
user    0m9.679s
sys     0m0.205s
```

out_search_planar_z8.txt

```
records 8503056 (lengths 13 and 14), collision groups 0
origin search nodes 258280325, origin-fixing nonbacktracking words (len<=17) mod p: 0

real    0m13.689s
user    0m9.442s
sys     0m0.165s
```

commutator_type.py

```
# SCRIPT: COMMUTATOR_TYPE
# KILL CONDITION (written before the run): the claim "T = 2 - 4 sinh^4(ln z/2) is the SL2 trace of [B_i,B_j]" is dead if
# the exact 4x4 trace of the commutator differs from T^2 (trace of the Hermitian-matrix representation of an SL2 element
```

```
# with real trace T is T^2 in the hyperbolic case and 2+2cos(2 theta)=T^2 also in the elliptic case, since 2+2cos2th=4cos^2
th).
# Finite order is DECIDED by: SL2 trace T is an algebraic integer if the element has finite order (eigenvalue a root of
unity);
# T rational and not an integer therefore => infinite order.  Rotation angle phi: cos(phi)=T^2/2-1; by Niven, rational cos
of a
# rational multiple of pi lies in {0,+-1/2,+-1}.
from fractions import Fraction as Fr
import itertools
def boost(z,a,sg):
    c=Fr(z*z+1,2*z); s=sg*Fr(z*z-1,2*z)
    B=[[Fr(int(i==j)) for j in range(4)] for i in range(4)]
    B[0][0]=c;B[a][a]=c;B[0][a]=s;B[a][0]=s
    return B
def mm(A,B): return [[sum(A[i][k]*B[k][j] for k in range(4)) for j in range(4)] for i in range(4)]
def tr(A): return sum(A[i][i] for i in range(4))
def inv(z,a,sg): return boost(z,a,-sg)
for z in (2,4,8,16):
    print("=== z =",z)
    T=2-Fr((z-1)**4,4*z*z)      # 2 - 4 sinh^4(ln z /2),  sinh(ln z/2)=(z-1)/(2 sqrt z)
    print(" claimed SL2 trace of commutator T =",T,"=",float(T))
    seen={}
    for (a,sa,b,sb) in itertools.product((1,2,3),(1,-1),(1,2,3),(1,-1)):
        if a>=b: continue
        A=boost(z,a,sa);B=boost(z,b,sb)
        C=mm(mm(A,B),mm(inv(z,a,sa),inv(z,b,sb)))
        t4=tr(C)
        seen.setdefault(t4,[]).append((a,sa,b,sb))
    for t4,l in seen.items():
        print("  4x4 trace of [B_a^sa,B_b^sb] =",t4,"=",float(t4),"  (#pairs with this value:",len(l),") equals
T^2?",t4==T*T)
    t4=list(seen.keys())[0]
    # decide type using the T of the standard commutator [B_i,B_j]
    A=boost(z,1,1);B=boost(z,2,1);C=mm(mm(A,B),mm(inv(z,1,1),inv(z,2,1)))
    t4=tr(C); print(" [B_i,B_j] 4x4 trace =",t4,"T^2 =",T*T,"match:",t4==T*T)
    if abs(T)<2:
        cphi=T*T/2-1
        print(" ELLIPTIC (|T|<2): rotation angle phi with cos(phi) = T^2/2-1 =",cphi,"=",float(cphi))
        import math
        print("  phi (rad) =",math.acos(float(cphi)),"  phi/pi =",math.acos(float(cphi))/math.pi)
        print("  T =",T,"is rational, denominator",T.denominator,"=> not an algebraic integer => eigenvalue e^{i phi/2} is
not a root of unity => INFINITE order")
        print("  2cos(phi) =",2*cphi,"denominator",(2*cphi).denominator,": not an integer, so phi is not a rational multiple
of pi (Niven / algebraic-integer test)")
        print("  e^{i phi/2} is root of x^2 - T x + 1 =",T.numerator,"/",T.denominator,": x^2-(%s)x+1"%T)
        # also: 2cos(phi/2)=T must be algebraic integer for finite order
    else:
        import math
        lam=(float(T)+math.sqrt(float(T)**2-4))/2
        print(" HYPERBOLIC/LOXODROMIC (|T|>2), real trace; |eigenvalue| =",abs(lam),", translation length 2 |ln|lambda||
=",2*abs(math.log(abs(lam))),"  -> infinite order")
    # other commutator traces (sign variants) summarized above
    # Thomas rotation of B_i B_j
    A=boost(z,1,1);B=boost(z,2,1);P=mm(A,B)
    print(" tr(B_i B_j) (4x4) =",tr(P)," Thomas rotation cos(theta)=4z(z^2+1)/(z^4+6z^2+1)=",Fr(4*z*(z*z+1),z**4+6*z*z+1))
```

Output

```
=== z = 2
 claimed SL2 trace of commutator T = 31/16 = 1.9375
  4x4 trace of [B_a^sa,B_b^sb] = 961/256 = 3.75390625   (#pairs with this value: 12 ) equals T^2? True
 [B_i,B_j] 4x4 trace = 961/256 T^2 = 961/256 match: True
 ELLIPTIC (|T|<2): rotation angle phi with cos(phi) = T^2/2-1 = 449/512 = 0.876953125
  phi (rad) = 0.5013113246722616   phi/pi = 0.1595723506990729
  T = 31/16 is rational, denominator 16 => not an algebraic integer => eigenvalue e^{i phi/2} is not a root of unity =>
INFINITE order
  2cos(phi) = 449/256 denominator 256 : not an integer, so phi is not a rational multiple of pi (Niven / algebraic-integer
test)
  e^{i phi/2} is root of x^2 - T x + 1 = 31 / 16 : x^2-(31/16)x+1
 tr(B_i B_j) (4x4) = 81/16  Thomas rotation cos(theta)=4z(z^2+1)/(z^4+6z^2+1)= 40/41
=== z = 4
 claimed SL2 trace of commutator T = 47/64 = 0.734375
  4x4 trace of [B_a^sa,B_b^sb] = 2209/4096 = 0.539306640625   (#pairs with this value: 12 ) equals T^2? True
 [B_i,B_j] 4x4 trace = 2209/4096 T^2 = 2209/4096 match: True
 ELLIPTIC (|T|<2): rotation angle phi with cos(phi) = T^2/2-1 = -5983/8192 = -0.7303466796875
  phi (rad) = 2.389625666581401   phi/pi = 0.7606414739513907
  T = 47/64 is rational, denominator 64 => not an algebraic integer => eigenvalue e^{i phi/2} is not a root of unity =>
INFINITE order
  2cos(phi) = -5983/4096 denominator 4096 : not an integer, so phi is not a rational multiple of pi (Niven / algebraic-
integer test)
  e^{i phi/2} is root of x^2 - T x + 1 = 47 / 64 : x^2-(47/64)x+1
 tr(B_i B_j) (4x4) = 625/64  Thomas rotation cos(theta)=4z(z^2+1)/(z^4+6z^2+1)= 272/353
=== z = 8
 claimed SL2 trace of commutator T = -1889/256 = -7.37890625
```

```
  4x4 trace of [B_a^sa,B_b^sb] = 3568321/65536 = 54.44825744628906   (#pairs with this value: 12 ) equals T^2? True
 [B_i,B_j] 4x4 trace = 3568321/65536 T^2 = 3568321/65536 match: True
 HYPERBOLIC/LOXODROMIC (|T|>2), real trace; |eigenvalue| = 0.13810628730978847 , translation length 2 |ln|lambda|| =
3.9594633847562153  -> infinite order
 tr(B_i B_j) (4x4) = 6561/256  Thomas rotation cos(theta)=4z(z^2+1)/(z^4+6z^2+1)= 2080/4481
=== z = 16
 claimed SL2 trace of commutator T = -48577/1024 = -47.4384765625
  4x4 trace of [B_a^sa,B_b^sb] = 2359724929/1048576 = 2250.409058570862   (#pairs with this value: 12 ) equals T^2? True
 [B_i,B_j] 4x4 trace = 2359724929/1048576 T^2 = 2359724929/1048576 match: True
 HYPERBOLIC/LOXODROMIC (|T|>2), real trace; |eigenvalue| = 0.02108931043973783 , translation length 2 |ln|lambda|| =
7.717977962205797  -> infinite order
 tr(B_i B_j) (4x4) = 83521/1024  Thomas rotation cos(theta)=4z(z^2+1)/(z^4+6z^2+1)= 16448/67073
```

residue_primes.py

```
# SCRIPT: RESIDUE_PRIMES
# Step 3(a): for every prime p<=200 and z in {2,4,8}, scale Q_u=2zB_u (integer) and reduce mod p.  Report rank of Q_u mod p
and,
# for p | z^2+1, z^2-1, the shape of the time/axis block.  KILL CONDITION for the idea "a residue argument like the odd-
divisor one works":
# it needs, for some p, a rank-one (or otherwise ping-pong-able) reduction of the scaled generators.  If every Q_u mod p has
rank 4
# (p odd) or if the reduction is monomial/scalar, the residue idea is dead for that p.  det(Q_u)=16 z^4.
from mkgens import boost
import sympy as sp
primes=[p for p in sp.primerange(2,200)]
for z in (2,4,8):
    print("=== z =",z," det(Q_u) =",16*z**4)
    rows=[]
    for p in primes:
        ranks=set()
        for a in (1,2,3):
            Q=sp.Matrix(boost(z,a,1))
            ranks.add(Q.rank(iszerofunc=lambda x: x%p==0) if False else sp.Matrix(Q).applyfunc(lambda
x:x%p).rank(iszerofunc=lambda x:x%p==0))
        # proper rank mod p via GF(p) elimination
        def rank_mod(Mx,p):
            A=[[int(Mx[i,j])%p for j in range(4)] for i in range(4)]; r=0
            for c in range(4):
                piv=None
                for i in range(r,4):
                    if A[i][c]%p: piv=i;break
                if piv is None: continue
                A[r],A[piv]=A[piv],A[r]; iv=pow(A[r][c],-1,p)
                A[r]=[x*iv%p for x in A[r]]
                for i in range(4):
                    if i!=r and A[i][c]%p:
                        f=A[i][c]; A[i]=[(x-f*y)%p for x,y in zip(A[i],A[r])]
                r+=1
            return r
        rk=rank_mod(sp.Matrix(boost(z,1,1)),p)
        tag=""
        if (z*z+1)%p==0: tag="p | z^2+1: block [[0,-2],[-2,0]] mod p (monomial, finite image)"
        if (z*z-1)%p==0: tag="p | z^2-1: block 2*I mod p (scalar-like on the block)"
        rows.append((p,rk,tag))
    print(" ranks of Q_u mod p, p<=199:",{p:rk for p,rk,t in rows if rk!=4})
    print(" primes with extra structure:",[(p,t) for p,rk,t in rows if t])
    print(" => rank-one reductions for odd p: none (all odd p have rank 4); only p=2 degenerates (det Q has only the prime
2).")
```

Output

```
=== z = 2  det(Q_u) = 256
 ranks of Q_u mod p, p<=199: {2: 1}
 primes with extra structure: [(3, 'p | z^2-1: block 2*I mod p (scalar-like on the block)'), (5, 'p | z^2+1: block [[0,-2],
[-2,0]] mod p (monomial, finite image)')]
 => rank-one reductions for odd p: none (all odd p have rank 4); only p=2 degenerates (det Q has only the prime 2).
=== z = 4  det(Q_u) = 4096
 ranks of Q_u mod p, p<=199: {2: 1}
 primes with extra structure: [(3, 'p | z^2-1: block 2*I mod p (scalar-like on the block)'), (5, 'p | z^2-1: block 2*I mod p
(scalar-like on the block)'), (17, 'p | z^2+1: block [[0,-2],[-2,0]] mod p (monomial, finite image)')]
 => rank-one reductions for odd p: none (all odd p have rank 4); only p=2 degenerates (det Q has only the prime 2).
=== z = 8  det(Q_u) = 65536
 ranks of Q_u mod p, p<=199: {2: 1}
 primes with extra structure: [(3, 'p | z^2-1: block 2*I mod p (scalar-like on the block)'), (5, 'p | z^2+1: block [[0,-2],
[-2,0]] mod p (monomial, finite image)'), (7, 'p | z^2-1: block 2*I mod p (scalar-like on the block)'), (13, 'p | z^2+1:
block [[0,-2],[-2,0]] mod p (monomial, finite image)')]
 => rank-one reductions for odd p: none (all odd p have rank 4); only p=2 degenerates (det Q has only the prime 2).
```

pingpong_check.py

```
# SCRIPT: PINGPONG_CHECK
# Exact verification of the (1+i)-adic ping-pong on P^1(Q(i)) for z = 2^k (k>=1), with exact Gaussian rationals.
# Objects: M_u = (z+1) I + (z-1) sigma_u  (sigma_x,sigma_y,sigma_z Pauli);  rho(M)X = M X M^*/|det M| on Hermitian X.
# Check 1 (kill: rho(M_u) != B_u for some u, z  -> the dictionary boosts <-> M_u is wrong and the proof does not apply):
#   rho(M_u) equals the boost matrix B_u of the task (axis u, rapidity ln z).
# Check 2 (kill: a point w violates  f_s( P^1 \ U_{s^-1} ) in U_s  or two U's intersect -> the ping-pong inclusions are
false):
#   six sets with v = v_pi(.) = v_2(Norm(.)) :  U_x+ = {v(w+1)>=3}, U_x- = {v(w-1)>=3}, U_y+ = {v(w-i)>=3},
#   U_y- = {v(w+i)>=3}, U_z+ = {v(w)>=1}, U_z- = {v(w)<=-1} u {inf}.   M_x,M_y,M_z act by Moebius maps f_x,f_y,f_z.
#   test on exact points (random + all residue-type points near the centres) the inclusions for all 6 letters.
# Check 3 (kill: a random reduced word w=s_1..s_n and a point p outside U_{s_n^-1} with w(p) outside U_{s_1}):
#   the consequence of the lemma, tested for 3000 random words of length up to 60 -> would show the six-set structure fails.
# Check 4 (kill: any nonempty reduced word with displacement d(w)=0 of the base vertex): d(w)=v(det)-2*min v(entries) of the
#   Z[i]-matrix product of N_u^{+-}=(z+1)+-(z-1)sigma_u over all reduced words up to length 6 and random ones up to 40.
import random, itertools, sys
from fractions import Fraction as Fr
class G:  # Gaussian rational
    __slots__=("a","b")
    def __init__(s,a=0,b=0): s.a=Fr(a); s.b=Fr(b)
    def __add__(s,o): o=g(o); return G(s.a+o.a,s.b+o.b)
    __radd__=__add__
    def __sub__(s,o): o=g(o); return G(s.a-o.a,s.b-o.b)
    def __rsub__(s,o): return g(o)-s
    def __mul__(s,o): o=g(o); return G(s.a*o.a-s.b*o.b,s.a*o.b+s.b*o.a)
    __rmul__=__mul__
    def __neg__(s): return G(-s.a,-s.b)
    def norm(s): return s.a*s.a+s.b*s.b
    def inv(s): n=s.norm(); return G(s.a/n,-s.b/n)
    def __truediv__(s,o): return s*g(o).inv()
    def iszero(s): return s.a==0 and s.b==0
    def __eq__(s,o): o=g(o); return s.a==o.a and s.b==o.b
    def __repr__(s): return "(%s+%si)"%(s.a,s.b)
def g(x): return x if isinstance(x,G) else G(x,0)
def v2(n):  # 2-adic valuation of a positive rational
    n=Fr(n); p,q=n.numerator,n.denominator; c=0
    while p%2==0: p//=2; c+=1
    while q%2==0: q//=2; c-=1
    return c
BIG=10**9
def vpi(x):
    x=g(x)
    if x.iszero(): return BIG
    return v2(x.norm())   # v_pi(x) = v_2(Norm x) since N(pi)=2 and odd primes are units
INF=None
def mob(M,w):
    a,b,c,d=M
    if w is INF:
        return INF if c.iszero() else a/c
    den=c*w+d
    if den.iszero(): return INF
    return (a*w+b)/den
I=G(0,1)
def Mmat(z,u,sg=1):
    zp,zm=z+1,sg*(z-1)
    if u=='x': return (G(zp),G(zm),G(zm),G(zp))
    if u=='y': return (G(zp),G(0,-zm),G(0,zm),G(zp))
    if u=='z': return (G(zp+zm),G(0),G(0),G(zp-zm))
def inU(name,w):
    if name=='x+': return w is not INF and vpi(w+1)>=3
    if name=='x-': return w is not INF and vpi(w-1)>=3
    if name=='y+': return w is not INF and vpi(w-I)>=3
    if name=='y-': return w is not INF and vpi(w+I)>=3
    if name=='z+': return w is not INF and vpi(w)>=1
    if name=='z-': return w is INF or vpi(w)<=-1
names=['x+','x-','y+','y-','z+','z-']
invname={'x+':'x-','x-':'x+','y+':'y-','y-':'y+','z+':'z-','z-':'z+'}
def letter_matrix(z,name): return Mmat(z,name[0],1 if name[1]=='+' else -1)
# For letter s, which set is U_s?  g_x+ (M_x) attracts to -1 : U_{x+} = B(-1,1/8) as in the proof; g_y+ attracts to +i; g_z+
attracts to 0.
def rand_pt(rng):
    t=rng.random()
    if t<0.05: return INF
    # random Gaussian rational with various 2-adic sizes
    def rn():
        return Fr(rng.randint(-40,40),rng.choice([1,1,2,3,4,5,8,16,32,64,7]))
    base=G(rn(),rn())
    ctr=rng.choice([G(0),G(1),G(-1),I,-I,None])
    if ctr is not None:
        sh=rng.choice([1,2,4,8,16,32,64]); pw=Fr(1,sh) if rng.random()<.5 else Fr(sh)
        eps=G(rng.randint(-3,3),rng.randint(-3,3))
        # point near centre at pi-adic distance ~ 2^-j : centre + (1+i)^j * small
        j=rng.randint(0,7); pij=G(1)
```

```
        for _ in range(j): pij=pij*G(1,1)
        return ctr+pij*G(rng.randint(-5,5)+rng.choice([0,1]),rng.randint(-5,5))*G(Fr(1,rng.choice([1,3,5,7])))
    return base
def check1():
    # rho(M) X = M X M^*/|det M|, basis X = I, sx, sy, sz ;  coordinates (t,x,y,z): X = t I + x sx + y sy + z sz
    import sympy as sp
    sx=sp.Matrix([[0,1],[1,0]]);sy=sp.Matrix([[0,-sp.I],[sp.I,0]]);sz=sp.Matrix([[1,0],[0,-1]]);I2=sp.eye(2)
    basis=[I2,sx,sy,sz]
    def coords(X):
        return [sp.simplify((X*B).trace()/2) for B in basis]  # tr(X sigma_a)/2 gives coordinate since tr(sa sb)=2 delta
    ok=True
    for z in (2,4,8,16):
        for u,(sig,ax) in {'x':(sx,1),'y':(sy,2),'z':(sz,3)}.items():
            M=(z+1)*I2+(z-1)*sig; det=M.det()
            cols=[]
            for B in basis:
                Y=M*B*M.H/sp.Abs(det)
                cols.append([sp.nsimplify(sp.simplify(c)) for c in coords(sp.simplify(Y))])
            R=sp.Matrix(cols).T   # R[:,j] = image of basis j
            c=sp.Rational(z*z+1,2*z); s=sp.Rational(z*z-1,2*z)
            Bm=sp.eye(4); Bm[0,0]=c;Bm[ax,ax]=c;Bm[0,ax]=s;Bm[ax,0]=s
            same = (R==Bm); ok&=bool(same)
            print("  z=%d axis %s: rho(M_u) == B_u(+ln z): %s"%(z,u,same))
    return ok
if __name__=="__main__":
    print("CHECK 1: rho(M_u)=B_u"); ok1=check1(); print("  ALL OK" if ok1 else "  FAILED (kill)")
    rng=random.Random(12345)
    # Check 2
    print("CHECK 2: disjointness and inclusions")
    bad=0; tested=0
    for z in (2,4,8,16,32):
        for _ in range(4000):
            w=rand_pt(rng)
            mem=[n for n in names if inU(n,w)]
            if len(mem)>1: bad+=1; print("  OVERLAP",w,mem)
            for s in names:
                if not inU(invname[s],w):
                    fw=mob(letter_matrix(z,s),w); tested+=1
                    # which U_s:  letter named 's+' (M_u) maps complement of U_{s-} into U_{s+}, where U_{x+}=B(-1,1/8) etc.
                    if not inU(s,fw): bad+=1; print("  VIOLATION z=%d s=%s w=%s f(w)=%s"%(z,s,w,fw))
    print("  tested inclusions:",tested,"violations/overlaps:",bad)
    # sanity: the points tested are spread over the sets
    cnt={n:0 for n in names}; cnt['none']=0
    for _ in range(4000):
        w=rand_pt(rng); m=[n for n in names if inU(n,w)]
        cnt[m[0] if m else 'none']+=1
    print("  distribution of random test points over the six sets:",cnt)
    # Check 3
    print("CHECK 3: reduced words")
    viol=0
    for z in (2,4,8):
        mats={s:letter_matrix(z,s) for s in names}
        for _ in range(1000):
            n=rng.randint(1,60); w=[rng.choice(names)]
            while len(w)<n:
                s=rng.choice(names)
                if s!=invname[w[-1]]: w.append(s)
            # find a test point outside U_{s_n^-1}
            for _t in range(200):
                p=rand_pt(rng)
                if not inU(invname[w[-1]],p): break
            q=p
            for s in reversed(w): q=mob(mats[s],q)
            if not inU(w[0],q): viol+=1
    print("  words tested 3000, violations:",viol)
    # Check 4
    print("CHECK 4: base-vertex displacement d(w)=v(det)-2 v(content)")
    def mmul(A,B): return (A[0]*B[0]+A[1]*B[2],A[0]*B[1]+A[1]*B[3],A[2]*B[0]+A[3]*B[2],A[2]*B[1]+A[3]*B[3])
    for z in (2,4,8):
        k={2:1,4:2,8:3}[z]; vdet=4+2*k
        mats={s:letter_matrix(z,s) for s in names}
        mind=BIG; cnt=0
        def rec(M,n,last,maxn):
            global mind,cnt
            if n>0:
                d=n*vdet-2*min(vpi(e) for e in M); cnt+=1
                if d<=0: print("   d<=0 for a reduced word!",d); sys.exit(1)
                mind=min(mind,d)
            if n==maxn: return
            for s in names:
                if last is not None and s==invname[last]: continue
                rec(mmul(M,mats[s]) if n>0 else mats[s],n+1,s,maxn)
        rec(None,0,None,5)
        print("  z=%d exhaustive reduced words of length 1..5: %d words, min displacement %d (>0 for all)"%(z,cnt,mind))
        md=BIG
        for _ in range(300):
```

```
            n=rng.randint(6,40); w=[rng.choice(names)]
            while len(w)<n:
                s=rng.choice(names)
                if s!=invname[w[-1]]: w.append(s)
            M=mats[w[0]]
            for s in w[1:]: M=mmul(M,mats[s])
            d=n*vdet-2*min(vpi(e) for e in M); md=min(md,d)
            if d<=0: print("   d<=0 random!"); sys.exit(1)
        print("  z=%d 300 random reduced words length 6..40: min displacement %d (>0)"%(z,md))
```

Output

```
CHECK 1: rho(M_u)=B_u
  z=2 axis x: rho(M_u) == B_u(+ln z): True
  z=2 axis y: rho(M_u) == B_u(+ln z): True
  z=2 axis z: rho(M_u) == B_u(+ln z): True
  z=4 axis x: rho(M_u) == B_u(+ln z): True
  z=4 axis y: rho(M_u) == B_u(+ln z): True
  z=4 axis z: rho(M_u) == B_u(+ln z): True
  z=8 axis x: rho(M_u) == B_u(+ln z): True
  z=8 axis y: rho(M_u) == B_u(+ln z): True
  z=8 axis z: rho(M_u) == B_u(+ln z): True
  z=16 axis x: rho(M_u) == B_u(+ln z): True
  z=16 axis y: rho(M_u) == B_u(+ln z): True
  z=16 axis z: rho(M_u) == B_u(+ln z): True
  ALL OK
CHECK 2: disjointness and inclusions
  tested inclusions: 100000 violations/overlaps: 0
  distribution of random test points over the six sets: {'x+': 608, 'x-': 647, 'y+': 624, 'y-': 621, 'z+': 834, 'z-': 666,
'none': 0}
CHECK 3: reduced words
  words tested 3000, violations: 0
CHECK 4: base-vertex displacement d(w)=v(det)-2 v(content)
  z=2 exhaustive reduced words of length 1..5: 4686 words, min displacement 2 (>0 for all)
  z=2 300 random reduced words length 6..40: min displacement 20 (>0)
  z=4 exhaustive reduced words of length 1..5: 4686 words, min displacement 4 (>0 for all)
  z=4 300 random reduced words length 6..40: min displacement 28 (>0)
  z=8 exhaustive reduced words of length 1..5: 4686 words, min displacement 6 (>0 for all)
  z=8 300 random reduced words length 6..40: min displacement 42 (>0)
```

pingpong_controls.py

```
# SCRIPT: PINGPONG_CONTROLS
# KILL/CONTROL: (a) even z that are not powers of 2 (6,10,12,20) must pass the same inclusion test (the hand proof uses only
z even);
# (b) NEGATIVE control: odd z (3,5,7) and z=1 must produce violations (the proof genuinely needs v_pi(z)>=2 and z-1 a unit).
# If (b) shows zero violations the test is blind and Check 2 proves nothing.
import random, sys
sys.path.insert(0,'.')
from pingpong_check import *
rng=random.Random(777)
for z in (6,10,12,20,2,64,3,5,7,1):
    bad=0;tested=0
    for _ in range(3000):
        w=rand_pt(rng)
        for s in names:
            if not inU(invname[s],w):
                fw=mob(letter_matrix(z,s),w);tested+=1
                if not inU(s,fw): bad+=1
    print("z=%d tested %d violations %d"%(z,tested,bad))
```

Output

```
z=6 tested 15000 violations 0
z=10 tested 15000 violations 0
z=12 tested 15000 violations 0
z=20 tested 15000 violations 0
z=2 tested 15000 violations 0
z=64 tested 15000 violations 0
z=3 tested 15000 violations 12000
z=5 tested 15000 violations 12000
z=7 tested 15000 violations 12000
z=1 tested 15000 violations 12000
```

CONJ_REPORT.md

Are the LPS group and the boost group conjugate in PGL2(Q_5)?

Scripts (same folder): inv.py (exact invariants, sympy), padic.py (5-adic Hensel i_5 mod 5^24, tree check).

Verdict
NOT conjugate. PROVED (exact rational invariant, plus computed checks). Not conjugate in PGL2(Q_5), GL2(Q_5), PGL2(Q(i)), or PGL2(C), under any relabelling of generators, and in fact not even setwise under any isomorphism of the groups.

The separating invariant
For g in GL2 over any field, t(g) = tr(g)^2 / det(g) is unchanged by scaling g and by conjugation.

generator (any u, either sign)	trace	det	tr^2/det
LPS: 1 +/- 2u as 2x2 matrix	2	5	4/5
Boost: 3I +/- 2 sigma_u (sqrt5 x boost)	6	5	36/5

All six generators of each group have the same value, so every relabelling (permutation, sign swap) compares 4/5 with 36/5. These are rational numbers, so the comparison is the same under every embedding Q(i) -> Q_5 (i_5 = +/-2 mod 5) and in C. A conjugating g would force 4/5 = 36/5 in Q_5; instead v_5(36/5 - 4/5) = -1.

Equivalent eigenvalue form: with rho = ratio of eigenvalues, tr^2/det = (1+rho)^2/rho, so conjugacy up to scalar needs rho_A in {rho_B, 1/rho_B}. Boost: rho = 5 (eigenvalues 5, 1 of 3I+2sigma_u). LPS: rho = (1+2i_5)/(1-2i_5) = (-3+4i_5)/5. Computed 5-adically: rho_LPS = 5 and rho_LPS = 1/5 both fail (they agree only mod 5; the differences have valuations 1 and 0). Exact reason: i_5 = 7 would be needed, but 7^2 = 49 is not -1 in Q_5.

Table of further invariants (exact, sympy; u != v, all sign choices give the same value)
word	LPS tr^2/det	Boost tr^2/det
g_u	4/5	36/5
g_u^2	36/25	676/25
g_u g_v and g_u g_v^-1 (u != v)	4/25	324/25
commutator [g_u, g_v]	196/625	196/625

Every single-generator, square and product invariant differs. The commutator value happens to agree (196/625); an agreeing invariant does not imply conjugacy, and the disagreeing ones decide the question. So no linear system g X = Y g needs solving: an intertwiner g that is invertible cannot exist (invertible g X g^-1 = lambda Y forces equal tr^2/det).

Over R / C: where it breaks
LPS elements are images of quaternions q = a+bi+cj+dk, so t = 4a^2/N(q) <= 4 (Cauchy-Schwarz a^2 <= N). For the generator, q/sqrt5 = cos(theta) + sin(theta)u with tan(theta) = 2: an elliptic element (compact, SU(2)), rotation of S^3-action angle arccos(-3/5) = 126.87 degrees about axis u; eigenvalue ratio has modulus 1. Boosts have t = 36/5 > 4: loxodromic, real eigenvalue ratio 5 = e^r (r = ln 5), modulus != 1. Elliptic and hyperbolic classes in PGL2(C) are disjoint, so not conjugate over C (hence not over Q(i) either). Setwise strengthening: every element of the LPS group has t <= 4, so no boost generator (t = 36/5) is conjugate to ANY element of the LPS group, whatever isomorphism is used.

Over Q_5 the distinction is subtler, because both generators are split hyperbolic with translation length 1 on the tree (det valuation 1, primitive). The tree sees them as alike; the multiplier rho (a cross-ratio type quantity) separates them.

The "v2 working report" observation: checked and TRUE
With i_5 = 2 mod 5 (Hensel to 5^24):
- Each 3I +/- 2 sigma_u and each LPS generator is primitive of determinant valuation 1, so moves the base vertex to a neighbour.
- Image lines mod 5 in P^1(F_5): Boost: i+ -> 4, i- -> 1, j+ -> 3, j- -> 2, k+ -> inf, k- -> 0 (six distinct = all of P^1(F_5)). LPS: i+ -> inf, i- -> 0, j+ -> 3, j- -> 2, k+ -> 4, k- -> 1 (also all six distinct).
- Proof of simple transitivity (no freeness assumption needed): let F be the abstract free group on three letters, rho: F -> group the generator map. The orbit map from the 6-regular Cayley tree of F to the Bruhat-Tits tree, f -> rho(f) v0, is equivariant and bijective on neighbours of every vertex (because the six neighbour images of v0 are distinct), hence a covering of trees, hence an isomorphism. So both groups are free of rank 3 and act simply transitively on the vertices. Computed check: all reduced words to length 4 (1+6+30+150+750 = 937 words) give 937 distinct vertices, at distance equal to word length, for both groups.

Consequence (precise statement of what is and is not true)
Both groups are free on matched generators and both act simply transitively on the same tree T. The vertex map gamma_A v0 -> gamma_B v0 (matching words) is therefore an automorphism of T conjugating one group to the other, so they ARE conjugate in Aut(T) (the first-neighbour labelling matches after relabelling i <-> k, j fixed, but that is not needed). That automorphism is NOT in PGL2(Q_5): Aut(T) is far larger than PGL2(Q_5), and the algebraic invariant t separates the groups there. "Same tree action" does not give "same matrices".

Status
- Not conjugate in PGL2(Q_5)/GL2(Q_5)/C, any relabelling, even setwise: PROVED (one exact rational invariant; 4/5 vs 36/5).
- Simple transitivity of the boost group: PROVED (covering argument) and COMPUTED (depth 4).
- Conjugate in Aut(T): PROVED (same argument), not algebraic.
- Open: none for the stated question. Possibly interesting follow-up (not tested): whether a different pair of lattices/other Lorentz-type groups with t <= 4 could match; the boost group with t = 36/5 cannot.

inv.py

```
# SCRIPT: CONJ_INVARIANTS_EXACT
import sympy as sp
from itertools import product
I=sp.I
M=sp.Matrix
def lps(u,s):
    a,b,c,d=1,0,0,0
    if u=='i': b=2*s
    if u=='j': c=2*s
    if u=='k': d=2*s
    return M([[a+b*I,c+d*I],[-c+d*I,a-b*I]])
sig={'i':M([[0,1],[1,0]]),'j':M([[0,-I],[I,0]]),'k':M([[1,0],[0,-1]])}
def boost(u,s): return 3*sp.eye(2)+2*s*sig[u]
def inv(g): return g.inv()
def ratio(g):
    t=sp.simplify(g.trace()); d=sp.simplify(g.det())
    return sp.nsimplify(sp.simplify(sp.expand(t**2/d)))
gens={}
for name,f in (('LPS',lps),('BOOST',boost)):
    gens[name]={(u,s):f(u,s) for u in 'ijk' for s in (1,-1)}
for name in gens:
    print(name)
    for (u,s),g in gens[name].items():
        print(' gen',u,s,'tr',g.trace(),'det',sp.simplify(g.det()),'tr2/det',ratio(g))
# check inverses are generators up to scalar
for name in gens:
    for (u,s),g in gens[name].items():
        h=gens[name][(u,-s)]
        assert sp.simplify(g*h-5*sp.eye(2))==sp.zeros(2,2), (name,u,s)
print('inverse check ok (g*g^-1 = 5I)')
# length-2 and commutator invariants
keys=list(gens['LPS'].keys())
res={}
for name in gens:
    G=gens[name]
    prod2={}; comm={}
    for a,b in product(keys,keys):
        if a[0]==b[0]: continue
        prod2[(a,b)]=ratio(G[a]*G[b])
        comm[(a,b)]=ratio(G[a]*G[b]*G[(a[0],-a[1])]*G[(b[0],-b[1])])
    res[name]=(prod2,comm)
for name in res:
    p,c=res[name]
    print(name,'products g_u g_v (u!=v) tr2/det distinct values:',sorted(set(p.values()),key=lambda z:(sp.re(z),sp.im(z))))
    print(name,'commutators tr2/det distinct values:',sorted(set(c.values()),key=lambda z:(sp.re(z),sp.im(z))))
    # also tr (not squared) of commutator normalised
# square invariants
for name in gens:
    print(name,'g^2 tr2/det', set(ratio(g*g) for g in gens[name].values()))
# compare sets
for k,lab in ((0,'products'),(1,'commutators')):
    A=set(res['LPS'][k].values()); B=set(res['BOOST'][k].values())
    print(lab,'common values LPS vs BOOST:',A&B)
# explicit example row
print('example i,j product LPS', res['LPS'][0][(('i',1),('j',1))], 'BOOST', res['BOOST'][0][(('i',1),('j',1))])
print('example comm [i,j] LPS', res['LPS'][1][(('i',1),('j',1))], 'BOOST', res['BOOST'][1][(('i',1),('j',1))])
# all-words-of-length<=3 : does any boost invariant equal any LPS invariant for ANY pair of words of same shape?
```

Output

```
LPS
 gen i 1 tr 2 det 5 tr2/det 4/5
 gen i -1 tr 2 det 5 tr2/det 4/5
 gen j 1 tr 2 det 5 tr2/det 4/5
 gen j -1 tr 2 det 5 tr2/det 4/5
 gen k 1 tr 2 det 5 tr2/det 4/5
 gen k -1 tr 2 det 5 tr2/det 4/5
BOOST
 gen i 1 tr 6 det 5 tr2/det 36/5
 gen i -1 tr 6 det 5 tr2/det 36/5
 gen j 1 tr 6 det 5 tr2/det 36/5
 gen j -1 tr 6 det 5 tr2/det 36/5
 gen k 1 tr 6 det 5 tr2/det 36/5
 gen k -1 tr 6 det 5 tr2/det 36/5
inverse check ok (g*g^-1 = 5I)
LPS products g_u g_v (u!=v) tr2/det distinct values: [4/25]
LPS commutators tr2/det distinct values: [196/625]
BOOST products g_u g_v (u!=v) tr2/det distinct values: [324/25]
BOOST commutators tr2/det distinct values: [196/625]
LPS g^2 tr2/det {36/25}
```

```
BOOST g^2 tr2/det {676/25}
products common values LPS vs BOOST: set()
commutators common values LPS vs BOOST: {196/625}
example i,j product LPS 4/25 BOOST 324/25
example comm [i,j] LPS 196/625 BOOST 196/625
```

padic.py

```
# SCRIPT: CONJ_PADIC_TREE
from itertools import product
K=24
P=5
MOD=P**K
# Hensel: i5 with i5^2=-1, i5 = 2 mod 5
x=2
for _ in range(10):
    x=(x-(x*x+1)*pow(2*x,-1,MOD))%MOD
assert (x*x+1)%MOD==0 and x%5==2
i5=x
def mm(A,B):
    return [[(A[0][0]*B[0][0]+A[0][1]*B[1][0])%MOD,(A[0][0]*B[0][1]+A[0][1]*B[1][1])%MOD],
            [(A[1][0]*B[0][0]+A[1][1]*B[1][0])%MOD,(A[1][0]*B[0][1]+A[1][1]*B[1][1])%MOD]]
def lps(u,s):
    a,b,c,d=1,0,0,0
    if u=='i': b=2*s
    if u=='j': c=2*s
    if u=='k': d=2*s
    return [[(a+b*i5)%MOD,(c+d*i5)%MOD],[(-c+d*i5)%MOD,(a-b*i5)%MOD]]
def boost(u,s):
    I=1;
    sg={'i':[[0,1],[1,0]],'j':[[0,-i5],[i5,0]],'k':[[1,0],[0,-1]]}[u]
    return [[(3*(r==c)+2*s*sg[r][c])%MOD for c in range(2)] for r in range(2)]
def val(n):
    n%=MOD
    if n==0: return K
    v=0
    while n%5==0: n//=5; v+=1
    return v
def canon(g):
    # lattice spanned by columns, up to scalar. returns (n, canonical data)
    m=min(val(g[r][c]) for r in range(2) for c in range(2))
    g=[[ (g[r][c]//P**m) for c in range(2)] for r in range(2)]
    M2=P**(K-m)
    cols=[(g[0][0]%M2,g[1][0]%M2),(g[0][1]%M2,g[1][1]%M2)]
    def v2(n):
        n%=M2
        if n==0: return 99
        v=0
        while n%5==0: n//=5; v+=1
        return v
    cols.sort(key=lambda c:v2(c[1]))
    w1,w2=cols
    # make w2 second coord zero
    if v2(w2[1])<99:
        q=(w2[1]//P**v2(w1[1]))*pow((w1[1]//P**v2(w1[1])),-1,M2)%M2
        w2=((w2[0]-q*w1[0])%M2,0)
    e=v2(w2[0]); q2=v2(w1[1])
    u=pow(w1[1]//P**q2,-1,M2)
    f=(w1[0]*u)%M2
    f%=P**e
    return (e,q2,f)
# generators as dict
def gens(f): return {(u,s):f(u,s) for u in 'ijk' for s in (1,-1)}
def neigh_line(g):
    # image of g mod 5: column space as point of P1(F5)
    a,b,c,d=[g[r][cc]%5 for r in range(2) for cc in range(2)]
    col=(a,c) if (a,c)!=(0,0) else (b,d)
    # normalise
    if col[0]%5!=0:
        inv=pow(col[0],-1,5); return ('1',(col[1]*inv)%5)
    return ('inf',0)
for name,f in (('LPS',lps),('BOOST',boost)):
    G=gens(f)
    lines={k:neigh_line(g) for k,g in G.items()}
    print(name,'det valuations',{k:val((g[0][0]*g[1][1]-g[0][1]*g[1][0])) for k,g in G.items()})
    print(name,'image lines mod 5 (point of P1(F5))',lines)
    print(name,'six lines distinct:',len(set(lines.values()))==6)
    # reduced words up to depth 4
    keys=list(G.keys())
    level={():[[1,0],[0,1]]}
    seen={}
    total=0
```

```
    cur=[((),[[1,0],[0,1]])]
    verts={canon([[1,0],[0,1]]):()}
    ok=True
    for depth in range(1,5):
        nxt=[]
        for w,g in cur:
            for k in keys:
                if w and w[-1][0]==k[0] and w[-1][1]==-k[1]: continue
                h=mm(g,G[k]); w2=w+(k,)
                c=canon(h)
                n=val(h[0][0]*h[1][1]-h[0][1]*h[1][0])-2*min(val(h[r][cc]) for r in range(2) for cc in range(2))
                if n!=depth: ok=False
                if c in verts: ok=False
                verts[c]=w2
                nxt.append((w2,h))
        cur=nxt
        print(name,'depth',depth,'words',len(cur),'distinct vertices so far',len(verts),'expected',sum(6*5**(d-1) if d else
1 for d in range(depth+1)))
    print(name,'all reduced words up to depth 4: det-valuation=depth and vertices distinct:',ok)
# eigenvalue ratios in Q_5
# LPS: (1+2 i5)/(1-2 i5);  valuation
a=(1+2*i5)%MOD; b=(1-2*i5)%MOD
print('LPS eigenvalues 1+2i5 valuation',val(a),' 1-2i5 valuation',val(b))
# is ratio 5 or 1/5 ? check a == 5*b or b==5*a mod 5^k
print('LPS ratio == 5 ?',(a-5*b)%MOD==0,'  ratio == 1/5 ?',(b-5*a)%MOD==0)
print('5-adic digits of valuation of (a-5b), (b-5a):',val(a-5*b),val(b-5*a))
# also with lambda scaling: eigenvalue ratio invariant. rho_LPS*5 vs: rho=(-3+4i)/5
rho_num=(-3+4*i5)%MOD
print('v(-3+4 i5)=',val(rho_num),' v(-3-4 i5)=',val((-3-4*i5)))
# tr^2/det equality test 4/5 vs 36/5 in Q_5: difference 32/5, valuation -1 (nonzero)
print('v_5(36/5-4/5) = v(32)-1 =',val(32)-1)
```

Output

```
LPS det valuations {('i', 1): 1, ('i', -1): 1, ('j', 1): 1, ('j', -1): 1, ('k', 1): 1, ('k', -1): 1}
LPS image lines mod 5 (point of P1(F5)) {('i', 1): ('inf', 0), ('i', -1): ('1', 0), ('j', 1): ('1', 3), ('j', -1): ('1', 2),
('k', 1): ('1', 4), ('k', -1): ('1', 1)}
LPS six lines distinct: True
LPS depth 1 words 6 distinct vertices so far 7 expected 7
LPS depth 2 words 30 distinct vertices so far 37 expected 37
LPS depth 3 words 150 distinct vertices so far 187 expected 187
LPS depth 4 words 750 distinct vertices so far 937 expected 937
LPS all reduced words up to depth 4: det-valuation=depth and vertices distinct: True
BOOST det valuations {('i', 1): 1, ('i', -1): 1, ('j', 1): 1, ('j', -1): 1, ('k', 1): 1, ('k', -1): 1}
BOOST image lines mod 5 (point of P1(F5)) {('i', 1): ('1', 4), ('i', -1): ('1', 1), ('j', 1): ('1', 3), ('j', -1): ('1', 2),
('k', 1): ('inf', 0), ('k', -1): ('1', 0)}
BOOST six lines distinct: True
BOOST depth 1 words 6 distinct vertices so far 7 expected 7
BOOST depth 2 words 30 distinct vertices so far 37 expected 37
BOOST depth 3 words 150 distinct vertices so far 187 expected 187
BOOST depth 4 words 750 distinct vertices so far 937 expected 937
BOOST all reduced words up to depth 4: det-valuation=depth and vertices distinct: True
LPS eigenvalues 1+2i5 valuation 1  1-2i5 valuation 0
LPS ratio == 5 ? False   ratio == 1/5 ? False
5-adic digits of valuation of (a-5b), (b-5a): 1 0
v(-3+4 i5)= 2  v(-3-4 i5)= 0
v_5(36/5-4/5) = v(32)-1 = -1
```

partA.py

```
# SCRIPT: LN7_PARTA_BOUNDS
# Exact check of l + (L-1) delta <= d(O,wO) <= L l at z = 7 for all non-backtracking words.
import numpy as np, mpmath as mp, itertools, sys
mp.mp.dps = 60
Z = 7
c, s = Z*Z+1, Z*Z-1          # 2z B_u = [[50,48],[48,50]] ; use 7 B_u = [[25,24],[24,25]] (z odd: exact /2)
def Q(axis, sign):
    M = np.diag([7,7,7,7]).astype(object)
    M[0,0] = 25; M[axis,axis] = 25; M[0,axis] = M[axis,0] = 24*sign
    return M
letters = [(1,1),(1,-1),(2,1),(2,-1),(3,1),(3,-1)]
names = ['i','-i','j','-j','k','-k']
Qs = [Q(a,sg) for a,sg in letters]
inv = [1,0,3,2,5,4]
# Lorentz check: Q^T eta Q = 49 eta, Q_u Q_-u = 49 I
eta = np.diag([1,-1,-1,-1]).astype(object)
for u in range(6):
    assert (Qs[u].T.dot(eta).dot(Qs[u]) == 49*eta).all()
    assert (Qs[u].dot(Qs[inv[u]]) == 49*np.diag([1,1,1,1]).astype(object)).all()
ell = mp.log(7)
```

```
delta = mp.acosh(mp.mpf(9)/7)
print('ell =', mp.nstr(ell,12), ' delta =', mp.nstr(delta,12), ' delta/ell =', mp.nstr(delta/ell,12))
assert abs(mp.cosh(ell) - mp.mpf(25)/7) < mp.mpf(10)**-50 and abs(mp.sinh(ell/2)**2 - mp.mpf(9)/7) < mp.mpf(10)**-50
LMAX = int(sys.argv[1]) if len(sys.argv) > 1 else 9
# DFS over words with first letter fixed = i (the symmetry group of the octahedron acts transitively on first letters
# and preserves non-backtracking and d). Row vector r = e0^T Q_{w1} ... Q_{wL}; r[0] = 7^L cosh d.
best = {}   # L -> (min margin in d, word, min cosh d numerator)
worst_ratio = {}
mind = {}
ref = {}
def dfs(r, word, L):
    num = r[0]
    D = mp.acosh(mp.mpf(int(num)) / mp.mpf(7)**L)
    lb = ell + (L-1)*delta
    marg = D - lb
    if L not in best or marg < best[L][0]:
        best[L] = (marg, list(word))
    st = sum(1 for m in range(L-1) if word[m+1]==word[m]); tu = L-1-st
    m2 = D - (ell*(1+st) + tu*delta)
    if tu > 0 and (L not in ref or m2 < ref[L][0]):
        ref[L] = (m2, list(word))
    if L not in mind or D < mind[L][0]:
        mind[L] = (D, list(word))
    assert D <= L*ell + mp.mpf(10)**-40
    if L == LMAX: return
    last = word[-1]
    for u in range(6):
        if u == inv[last]: continue
        word.append(u)
        dfs(r.dot(Qs[u]), word, L+1)
        word.pop()
r0 = np.array([1,0,0,0], dtype=object).dot(Qs[0])
dfs(r0, [0], 1)
print('L  #words(first=i)  min[d-(l+(L-1)delta)]  min d   min d/(L l)   argmin word')
for L in range(1, LMAX+1):
    m, w = best[L]; D, w2 = mind[L]
    print(L, 5**(L-1), mp.nstr(m,10), mp.nstr(D,10), mp.nstr(D/(L*ell),8), ' '.join(names[x] for x in w2))
# best constant c with d >= l + (L-1) c over all checked words (L>=2)
print('refined bound d >= l(1+#straight) + delta(#turns): min margin per L')
print('(straight words i^L give equality d = L l exactly; listed below: words with >= 1 turn)')
for L in range(2, LMAX+1):
    print(L, mp.nstr(ref[L][0],10), ' '.join(names[x] for x in ref[L][1]))
print('overall min margin (L>=2):', mp.nstr(min(best[L][0] for L in range(2,LMAX+1)),12))
print('lower bound fails anywhere (tol 1e-40):', any(best[L][0] < -mp.mpf(10)**-40 for L in best))
print('refined bound fails anywhere (tol 1e-40):', any(ref[L][0] < -mp.mpf(10)**-40 for L in ref))
# exact identity behind the commutator: cosh(tau) = T_4(9/7)
from fractions import Fraction as F
c = F(9,7); print('T4(9/7) =', 8*c**4-8*c**2+1, '  commutator cosh(tau) = 23137/2401')
```

Output

```
ell = 1.94591014906  delta = 0.738997943852  delta/ell = 0.37976981836
L  #words(first=i)  min[d-(l+(L-1)delta)]  min d   min d/(L l)   argmin word
1 1 0.0 1.945910149 1.0 i
2 5 0.5526302452 3.237538338 0.83188279 i j
3 25 0.7894485265 4.213354563 0.72174531 i j -i
4 125 0.8961968264 5.059100807 0.64996588 i j -i -j
5 625 0.9457706911 5.847672616 0.60102185 i j -i -j i
6 3125 0.9691316688 6.610031537 0.56614737 i j -i -j i j
7 15625 0.9802175388 7.360115351 0.54033587 i j -i -j i j -i
8 78125 0.9854959165 8.104391673 0.52060418 i j -i -j i j -i -j
9 390625 0.9880131545 8.845906854 0.50509965 i j -i -j i j -i -j i
refined bound d >= l(1+#straight) + delta(#turns): min margin per L
(straight words i^L give equality d = L l exactly; listed below: words with >= 1 turn)
2 0.5526302452 i j
3 0.5343515109 i i j
4 0.5145971448 i i j j
5 0.5141899167 i i i j j
6 0.513782024 i i i j j j
7 0.5137736979 i i i i j j j
8 0.5137653716 i i i i j j j j
9 0.5137652017 i i i i i j j j j
overall min margin (L>=2): 0.552630245241
lower bound fails anywhere (tol 1e-40): False
refined bound fails anywhere (tol 1e-40): False
T4(9/7) = 23137/2401   commutator cosh(tau) = 23137/2401
```

partA2.py

```
# SCRIPT: LN7_PARTA_COMMUTATOR
# Exact translation length of the commutator [B_i, B_j] = B_i B_j B_-i B_-j and other short periodic words.
import sympy as sp, mpmath as mp, itertools
mp.mp.dps = 50
def Q(axis, sign):
    M = sp.diag(7,7,7,7); M[0,0]=25; M[axis,axis]=25; M[0,axis]=M[axis,0]=24*sign; return M
L = {'i':Q(1,1),'-i':Q(1,-1),'j':Q(2,1),'-j':Q(2,-1),'k':Q(3,1),'-k':Q(3,-1)}
inv = {'i':'-i','-i':'i','j':'-j','-j':'j','k':'-k','-k':'k'}
x = sp.symbols('x')
delta = mp.acosh(mp.mpf(9)/7)
def specrad(M):
    A = mp.matrix([[mp.mpf(int(sp.numer(M[r,c])))/mp.mpf(int(sp.denom(M[r,c]))) for c in range(4)] for r in range(4)])
    E, _ = mp.eig(A)
    return mp.log(max(abs(e) for e in E))
def transl(word):
    M = sp.eye(4)
    for a in word: M = M*L[a]
    M = M/sp.Integer(7)**len(word)
    p = sp.factor(M.charpoly(x).as_expr())
    return specrad(M), p, M.trace()
tau, p, tr = transl(['i','j','-i','-j'])
print('commutator charpoly:', p)
print('trace =', tr, ' tau =', mp.nstr(tau,20), ' tau/4 =', mp.nstr(tau/4,20), ' delta =', mp.nstr(delta,20))
print('cosh(tau) candidates: cosh(4 delta)=', mp.nstr(mp.cosh(4*delta),20))
# minimum per-letter translation over cyclically reduced periodic words up to period 6, first letter i
names=list(L)
res=[]
for P in range(1,7):
    for w in itertools.product(names, repeat=P-1):
        w=('i',)+w
        ok=all(w[(m+1)%P]!=inv[w[m]] for m in range(P))
        if not ok: continue
        M=sp.eye(4)
        for a in w: M=M*L[a]
        t=specrad(M)-P*mp.log(7)
        res.append((t/P,' '.join(w)))
res.sort()
for r in res[:8]: print(mp.nstr(r[0],15), r[1])
```

Output

```
commutator charpoly: (x - 1)**2*(2401*x**2 - 46274*x + 2401)/2401
trace = 51076/2401  tau = 2.9559917754069545138  tau/4 = 0.73899794385173862846  delta = 0.73899794385173862846
cosh(tau) candidates: cosh(4 delta)= 9.6364014993752603082
0.738997943851739 i -j -i j
0.738997943851739 i -k -i k
0.738997943851739 i j -i -j
0.738997943851739 i k -i -k
1.08108461723216 i -j -i -j -i j
1.08108461723216 i -k -i -k -i k
1.08108461723216 i -k -i k i k
1.08108461723216 i -k i -k -i k
```

planes.py

```
# SCRIPT: LN7_PLANE_GAPS
# Checks the geometric inputs of the nested half-space proof at z = 7, exactly where possible.
# Normals (scaled by sqrt 7): N_u = (3, 4 e_u), <N,N> = 9 - 16 = -7, signature (+,-,-,-).
# H_u = { p : <p, N_u> <= 0 } = { x_u / t >= 3/4 = tanh(l/2) }.
import numpy as np, mpmath as mp
from fractions import Fraction as F
mp.mp.dps = 40
eta = np.diag([1,-1,-1,-1])
def ip(a,b): return int(np.array(a).dot(eta).dot(np.array(b)))
E = {'i':(1,1),'-i':(1,-1),'j':(2,1),'-j':(2,-1),'k':(3,1),'-k':(3,-1)}
def N(u):
    a,s = E[u]; v=[3,0,0,0]; v[a]=4*s; return v
def Q(u):
    a,s = E[u]; M=np.diag([7,7,7,7]); M[0,0]=25; M[a,a]=25; M[0,a]=M[a,0]=24*s; return M
inv={'i':'-i','-i':'i','j':'-j','-j':'j','k':'-k','-k':'k'}
print('<N_u,N_u> =', ip(N('i'),N('i')))
for u in E:
    assert ip(N(u),N(u)) == -7
    # Q_u N_{-u} = -7 N_u : B_u carries the plane of H_{-u} onto the plane of H_u, flipping the side
    assert list(Q(u).dot(N(inv[u]))) == [-7*c for c in N(u)]
    for w in E:
```

```
        g = F(abs(ip(N(inv[u]),N(w))),7)   # cosh of the distance between plane(-u) and plane(w)
        if w == inv[u]: continue
        assert g == (F(25,7) if w == u else F(9,7)), (u,w,g)
print('Q_u N_{-u} = -7 N_u for all u: True')
print('cosh d(plane(-u), plane(u)) = 25/7 = cosh l ; cosh d(plane(-u), plane(w perp)) = 9/7 = cosh delta')
# sinh d(point, plane) = |<p, N>| / sqrt7 for unit-time point p
O = [1,0,0,0]; BO = [F(25,7),F(24,7),0,0]
print('sinh d(O, plane i)     =', mp.nstr(abs(ip(O,N('i')))/mp.sqrt(7),15), ' sinh(l/2) =',
mp.nstr(mp.sinh(mp.log(7)/2),15))
v = 3*25-4*24
print('sinh d(B_iO, plane i)  =', mp.nstr(abs(mp.mpf(v)/7)/mp.sqrt(7),15))
# independent numerical check of the plane-plane distance formula by direct minimisation
def pt(r, th, ph):  # point of H^3 with rapidity r in direction (th, ph)
    d = np.array([np.sin(th)*np.cos(ph), np.sin(th)*np.sin(ph), np.cos(th)])
    return np.concatenate([[np.cosh(r)], np.sinh(r)*d])
import itertools
from scipy.optimize import minimize
def on_plane(u, a, b):
    # parametrise plane x_u = (3/4) t by two coordinates in the plane's own hyperboloid: p = cosh-type param
    # plane is a copy of H^2: use orthonormal frame n0 = foot point, e1, e2
    ax,s = E[u]
    foot = np.zeros(4); foot[0]=4/np.sqrt(7); foot[ax]=s*3/np.sqrt(7)   # <foot,foot>=1, <foot,N>=0
    others=[k for k in (1,2,3) if k!=ax]
    e1=np.zeros(4); e1[others[0]]=1; e2=np.zeros(4); e2[others[1]]=1
    r=np.hypot(a,b);
    if r==0: return foot
    return np.cosh(r)*foot + np.sinh(r)*(a*e1+b*e2)/r
def dist(p,q): return float(np.arccosh(max(1.0, p.dot(eta).dot(q))))
f=lambda z: dist(on_plane('-i',z[0],z[1]), on_plane('j',z[2],z[3]))
best=min((minimize(f, x0, method='Nelder-Mead', options={'xatol':1e-12,'fatol':1e-14,'maxiter':20000}) for x0 in
np.random.default_rng(1).normal(size=(20,4))), key=lambda r:r.fun)
print('numerical min distance plane(-i) to plane(j) =', best.fun, '  delta =', float(mp.acosh(mp.mpf(9)/7)))
```

Output

```
<N_u,N_u> = -7
Q_u N_{-u} = -7 N_u for all u: True
cosh d(plane(-u), plane(u)) = 25/7 = cosh l ; cosh d(plane(-u), plane(w perp)) = 9/7 = cosh delta
sinh d(O, plane i)     = 1.13389341902768  sinh(l/2) = 1.13389341902768
sinh d(B_iO, plane i)  = 1.13389341902768
numerical min distance plane(-i) to plane(j) = 0.7389979438517325   delta = 0.7389979438517387
```

AREA_REPORT.md

```
# Why the relator polygon has area 0 at o*: a reflection symmetry (proved)

Grades: **proved** / **computed** / **open**. Scripts: `area_sym.py`, `area_sym2.py` (each starts with `# SCRIPT:` and its
kill conditions); outputs `area_sym_out.txt`, `area_sym2_out.txt`.

## Result

**Theorem (proved).** Let K(p,q), q odd, have a real Riley representation a = [[s,1],[0,1/s]], b = [[s,0],[-v,1/s]] (s real,
v real, v != 0), with W = g_1...g_{p-1}, g_i = b^{e_i} (i odd), a^{e_i} (i even), e_i = (-1)^{floor(iq/p)}, and relator r =
a W b^-1 W^-1. Let P(o) be the geodesic polygon with vertices p_k = h_k.o, h_k the length-k prefixes of r (N = 2m vertices,
m = |W|+1), and A(o) its signed (winding-weighted) area.

1. If v > 0, the matrix K = [[0,1],[v,0]] acts on H as the reflection R: z -> 1/(v conj z) in the geodesic l = {|z| =
1/sqrt(v)}, and A(R o) = -A(o) for every o. In particular **A(o) = 0 for every o on l**, whether or not the axes of a and b
cross.
2. If v < 0, K is a half-turn, A(K o) = +A(o), and nothing forces A = 0.
3. o* (the crossing point of the axes) lies on l, so A(o*) = 0 on every arm with v > 0 and 0 < v < (s - 1/s)^2 (axes cross).
Where the axes do not cross, o* is not on l and A(o*) != 0.

## Proof

**Step 1, an anti-automorphism swapping a and b (proved).** Put D = diag(1,-v) and tau(X) = D X^T D^-1. Then tau(XY) =
tau(Y) tau(X), and direct multiplication gives tau(a) = b (the (2,1) entry of a^T is scaled by -v) and tau(b) = a (the (1,2)
entry -v is scaled by -1/v, giving 1). So tau(g^{+-1}) swaps the type of the letter and keeps the exponent.

**Step 2, W is palindromic (proved).** For 0 < i < p, iq/p is not an integer, so floor((p-i)q/p) = q - 1 - floor(iq/p); with
q odd, e_{p-i} = e_i. Also p is odd, so i and p-i have opposite parity: g_{p-i} has the opposite type to g_i. Hence
tau(g_{p-i}) = g_i and tau(W) = tau(g_{p-1})...tau(g_1) = g_1...g_{p-1} = W. (Checked by machine for all odd q, p <= 59: 364
pairs, no failure.)

**Step 3, tau is an inversion in K (proved).** For X in SL2, X^T = J X^-1 J^-1 with J = [[0,1],[-1,0]]. So tau(X) = K X^-1
K^-1 with K = D J = [[0,1],[v,0]], K^2 = v I. As an isometry, K acts by z -> 1/(v z) if det K = -v > 0 (a half-turn about
i/sqrt(-v)), and by z -> 1/(v conj z) if v > 0, an orientation-reversing involution whose mirror is |z|^2 = 1/v.
Consequences, as exact matrix identities: K a K^-1 = b^-1, K b K^-1 = a^-1, K W K^-1 = W^-1.

**Step 4, the induced map on words (proved).** The substitution phi: a -> b^-1, b -> a^-1 satisfies K h K^-1 = phi(h) for
```

all words h. By Step 2, phi(W) = W^-1 letter for letter (the j-th letter of W^-1 is g_{p-j}^-1, which is the opposite type to g_j with exponent -e_j, which is phi(g_j)). So phi(r) = b^-1 W^-1 a W, as a word.

Step 5, the half-period shift (proved). Write r = r1 r2 with r1 = aW (length m), r2 = b^-1 W^-1. Then phi(r) = r2 r1. Since r = 1 in the group, r2 = r1^-1 as matrices. For o on l (R o = o) and each k (indices mod N):

R(p_k) = K h_k K^-1 . R(o) = phi(h_k).o = g.p_{k+m}, g = r1^-1 = (aW)^-1.

Check: for k <= m, phi(h_k) = q_k is a prefix of r2 and p_{m+k} = r1 q_k o, so g p_{m+k} = q_k o. For k > m, phi(h_k) = r2 h_{k-m} = r1^-1 h_{k-m} and k-m = k+m mod N. So Sigma := g^-1 R is an orientation-reversing isometry with **Sigma p_k = p_{k+m}** for all k: it maps the polygon to itself, keeping the cyclic order of the vertices.

Step 6, the area (proved). Signed area is A(P) = integral over the boundary of a primitive alpha of the area form, so it depends only on the cyclic edge sequence, not on the starting vertex (equivalently, the fan sum is independent of the apex). An orientation-reversing isometry changes its sign. Since Sigma P = P as a cyclic sequence, A(P) = A(Sigma P) = -A(P), so A = 0. For general o, polygon(R o; r) = R polygon(o; phi(r)) and polygon(o; phi(r)) = g polygon(o; r) re-indexed, so A(R o) = -A(o). For v < 0, K is orientation-preserving and the same argument gives A(K o) = +A(o) only. QED.

Step 7, o* (proved + computed). Axis(a) is the line Re z = x_a = s/(1-s^2); axis(b) is the semicircle from 0 to x_b = (1-s^2)/(s v). x_a x_b = 1/v. When the axes cross (x_b < x_a < 0), the crossing point has |z|^2 = x_a^2 + x_a(x_b - x_a) = x_a x_b = 1/v, so it lies on l (R swaps the two axes and so fixes their intersection). When they do not cross, R maps the foot of the common perpendicular on axis(a) to the foot on axis(b), a different point, so o* is not on l.

Why the half-turn J failed: J (orientation-preserving, about o*) inverts a and b but reverses nothing; the true symmetry is the reflection R, composed with the shift g^-1, which is a half-period shift of the cyclic order and not a reversal. The polygon is not a symmetric simple polygon; it is a self-overlapping 2m-gon in which vertices p_k and p_{k+m} are mirror images (Sigma is a reflection in the perpendicular bisector of o and aW.o, checked: Sigma^2 = id to 1e-97), and the signed areas of the two halves cancel.

Numerical confirmation (computed; kill conditions K1-K12 survived)

mpmath, 50 and 100 digits; knots 3_1, 4_1, 5_2, 6_2, 6_1 (9,7), (13,5); x = 0.7 to 3.5, every real root with |v| > 1e-12 (about 40 branch points).

check	worst error
K1 tau(a) = b, tau(b) = a	7e-52
K2 tau(W) = W	1.5e-48
K3 K a^-1 K^-1 = b, K W^-1 K^-1 = W (projective)	2e-49
K4 K p_k = g p_{k+m}, o on l	1.3e-34 at x = 3.5 (loss of digits from s^22), < 1e-36 elsewhere
K5 A = 0 at 6 points of l (v > 0, all 3_1 rows included, where the axes do NOT cross)	8e-33 (x = 3.5, 6_2); 1e-96 at 100 digits
K6 A(R o) = -A(o) at random o	4e-31 (x = 3.5), typically < 1e-40
K7 v < 0: A(K o) = +A(o), and A != 0 at random points (4_1 v- arm: 0.08 to 1.3; 6_1 and 6_2 v < 0 arms: 0.07 to 3.5)	3e-43
K8 o* on l iff axes cross	crossing: ||o*|^2 - 1/v| = 0 to 1e-50, A(o*) < 2e-34; non-crossing: ||o*|^2 - 1/v| = 1e-6 to 0.3, A(o*) = -1.52 (3_1, x = 0.7), -0.52 (x = 2.0), -0.12 (x = 3.5), same on 5_2 and 6_2 v > (s-1/s)^2 arms
K9 Sigma p_k = p_{k+m}; Sigma^2 = id	2e-94
K11 palindrome, 364 (p,q) pairs	0 failures
K12 (13,5), 4 branches: A on l	2e-90

Reading: the zero at o* found earlier is the special case of the reflection fixing l, so it is not an accident of o*. The earlier trefoil value A(o*) -> -pi (x -> 0) is consistent: the trefoil has v > (s-1/s)^2, so o* is off l.

Idea tests (kill condition stated in the script headers)

1. Orientation-reversing symmetry: **survived; this is the mechanism** (reflection R swapping the axes, R a R = b^-1, R W R = W^-1; it needs o on l, composed with the half-period shift g).
2. Decomposition into congruent triangles of opposite orientation: the same fact in pieces: Sigma pairs the fan triangle (p_0, p_k, p_{k+1}) with (p_m, p_{k+m}, p_{k+m+1}), a mirror pair, with the fan apex moved by a cyclic re-indexing (free by Step 6). Not run separately.
3. Any base point on the line of symmetry: **yes, every point of l** (computed at six angles per branch, proved above), and in addition A is odd under R. Grid scan (17 x 17, 5 branches): A changes sign only across l (0 violations in 53 sign-changing edges); this is **computed on a coarse grid, not proved**, that l is the whole zero set.

When it fails

- v < 0 (4_1 v- arm, the v < 0 arms of 6_1 and 6_2, the small negative stretch of 6_2 branch 1): K is a half-turn; no reflection exists; A(o) > 0 at all points tested.
- Off l, for any base point (o = i included): A != 0 in general (4_1 at o = i: 0.84 at the fold).
- Where v > 0 but the axes are ultraparallel (v > (s-1/s)^2: the trefoil, the non-crossing 4_1, 5_2, 6_2 arms at large v): the vanishing holds on l but the named point o* is off l, so A(o*) != 0.
- The proof used: q odd (palindrome), real s and v, v > 0. Not covered: q even normal forms, complex branches, elliptic meridians (|s| = 1).

Open

- Whether l is the entire zero set of A for v > 0 (grid evidence only).
- What A(o) is as a function on H^2 off l (it is odd under R and equals the holonomy angle of the frame around the relator loop).
- Whether the v < 0 half-turn gives a conserved quantity (A(K o) = A(o) is the only constraint found).

area_sym.py

```
# SCRIPT: AREA-SYM
# KILL CONDITIONS (written before the run):
#  K1  tau(X)=D X^T D^-1 (D=diag(1,-v)) swaps a,b: dies if |tau(a)-b| or |tau(b)-a| > 1e-30.
#  K2  tau(W)=W for the Riley word of every tested (p,q), q odd: dies if residual > 1e-30 (projective) for any knot/branch.
#  K3  K=[[0,1],[v,0]] acts as z -> 1/(v conj z): orientation-reversing iff v>0, mirror l = {|z|=1/sqrt(v)}; dies if for
some v>0 branch K a^-1 K^-1 != b or K W^-1 K^-1 != W.
#  K4  Vertex identity: K p_k(o) = g p_{k+m}(o) for o on l (g=(aW)^-1, m=|W|+1, indices mod 2m): dies if error > 1e-30.
#  K5  A(o)=0 for o on l, ANY point, v>0, including branches where the axes do NOT cross (trefoil): dies if |A| > 1e-30 at
any tested point.
#  K6  A(K o) = -A(o) for random o, v>0 (odd under the reflection): dies if error > 1e-30.
#  K7  For v<0, K is a half-turn: A(K o)=+A(o) holds, and A is NOT identically 0 (dies if A=0 at all random points tested on
v<0 branches, i.e. the opposite conclusion).
#  K8  o* lies on l exactly when axes cross (o* is crossing point); where they do not cross, o* is off l and A(o*) != 0.
import sys, random
sys.path.insert(0, '/tmp/claude-0/-home-claude/d7048feb-3787-5f7f-b86a-f71c9a373eed/scratchpad/lanes/compute')
from q2_lib import *
mp.mp.dps = 50
random.seed(7)

def pm(X, Y):  # projective distance in PGL2 (X ~ cY), normalise by determinant sign-free ratio
    d = mp.mpf(0)
    i0 = max(((i, j) for i in range(2) for j in range(2)), key=lambda t: abs(Y[t[0], t[1]]))
    c = X[i0[0], i0[1]] / Y[i0[0], i0[1]]
    return max(abs(X[i, j] - c*Y[i, j]) for i in range(2) for j in range(2)) / max(abs(X[i, j]) for i in range(2) for j in
range(2))

def Kact(v, z):  # isometry z -> 1/(v conj z) (v>0), and half-turn z -> 1/(v z)?? for v<0: matrix [[0,1],[v,0]] acts as
Mobius z->1/(v z)
    if v > 0: return 1/(v*mp.conj(z))
    return 1/(v*z)

def area(M, rel, o):
    pts = orbit(M, rel, o); A, E = poly_area_turn(pts); return A

knots = {'3_1': (3, 1), '4_1': (5, 3), '5_2': (7, 3), '6_2': (11, 3), '6_1': (9, 7), '7_?(13,5)': (13, 5)}
import time
for nm, (p, q) in knots.items():
    t0 = time.time()
    try:
        fs = riley_coeff_funcs(p, q)
    except Exception as ex:
        print(nm, 'riley failed', ex); continue
    rel, W = letters(p, q); m = len(W) + 1
    print("=== %s (p,q)=(%d,%d), |W|=%d, relator length %d  (setup %.1fs)" % (nm, p, q, len(W), len(rel), time.time()-t0))
    for x in ['0.7', '1.2', '2.0', '3.5']:
        s = mp.e**mp.mpf(x)
        for v in real_roots(fs, s):
            if abs(v) < mp.mpf('1e-12'): continue
            M = mats(s, v); a, b = M[('a', 1)], M[('b', 1)]
            D = mp.matrix([[1, 0], [0, -v]]); Kmat = mp.matrix([[0, 1], [v, 0]])
            tau = lambda X: D * X.T * D**-1
            k1 = max(max(abs(tau(a)[i, j] - b[i, j]) for i in range(2) for j in range(2)), max(abs(tau(b)[i, j] - a[i, j])
for i in range(2) for j in range(2)))
            Wm = prod(M, W)
            k2 = max(abs(tau(Wm)[i, j] - Wm[i, j]) for i in range(2) for j in range(2)) / max(abs(Wm[i, j]) for i in
range(2) for j in range(2))
            k3a = pm(Kmat * a**-1 * Kmat**-1, b); k3b = pm(Kmat * Wm**-1 * Kmat**-1, Wm)
            g = (a*Wm)**-1
            xa = s/(1-s*s); xb = (1-s*s)/(s*v); cross = (xb < xa < 0)
            line = 'v>0 reflection' if v > 0 else 'v<0 half-turn'
            # K4: vertices, base points on l (v>0) or arbitrary (v<0 uses half-turn K)
            if v > 0:
                R0 = 1/mp.sqrt(v); th = mp.mpf('0.9')
                o = R0*mp.mpc(mp.cos(th), mp.sin(th))
            else:
                o = mp.mpc(0.3, 1.1)
            pts = orbit(M, rel, o); N = len(pts) - 1
            Ko = Kact(v, o)
            k4 = max(abs(Kact(v, pts[k]) - mobius(g, pts[(k + m) % N])) for k in range(N)) if abs(Ko - o) < mp.mpf(10)**-40
else None
            # K5: A on l at several angles
            Aon = []
            if v > 0:
                for thv in ['0.2', '0.6', '1.0', '1.4', '2.0', '2.8']:
                    oo = R0*mp.mpc(mp.cos(mp.mpf(thv)), mp.sin(mp.mpf(thv)))
                    Aon.append(abs(area(M, rel, oo)))
            # K6/K7: random off-line points
            e6 = mp.mpf(0); Aoff = []
            for _ in range(3):
                oo = mp.mpc(random.uniform(-2, 2), random.uniform(0.3, 2.5))
                A1 = area(M, rel, oo); A2 = area(M, rel, Kact(v, oo)); Aoff.append(A1)
                e6 = max(e6, abs(A2 + A1) if v > 0 else abs(A2 - A1))
```

```
            # K8: o*
            line8 = ''
            if xa*xb > 0 or True:
                ystar2 = xa*(xb - xa)
                if cross:
                    ostar = mp.mpc(xa, mp.sqrt(ystar2))
                    onl = abs(abs(ostar)**2 - 1/v)
                    line8 = "o* crossing pt: ||o*|^2-1/v| = %s, A(o*) = %s" % (mp.nstr(onl, 3), mp.nstr(abs(area(M, rel,
ostar)), 3))
                else:
                    ostar = mp.mpc(xa, mp.sqrt(abs(xa*(xa - xb))))
                    line8 = "axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = %s ; |o*|^2-1/v = %s" %
(mp.nstr(area(M, rel, ostar), 8), mp.nstr(abs(ostar)**2 - 1/v, 4))
            print(" x=%s v=%s [%s%s] K1=%s K2=%s K3=%s,%s K4=%s" % (x, mp.nstr(v, 6), line, ', axes cross' if cross else '',
mp.nstr(k1, 2), mp.nstr(k2, 2), mp.nstr(k3a, 2), mp.nstr(k3b, 2), mp.nstr(k4, 2) if k4 is not None else 'n/a'))
            if v > 0:
                print("      K5 max|A| on l (6 pts) = %s ; K6 max|A(Ko)+A(o)| = %s ; A off l = %s" % (mp.nstr(max(Aon), 3),
mp.nstr(e6, 3), [mp.nstr(t, 5) for t in Aoff]))
            else:
                print("      K7 max|A(Ko)-A(o)| = %s ; A at random pts = %s" % (mp.nstr(e6, 3), [mp.nstr(t, 5) for t in
Aoff]))
            print("      " + line8)
```

Output

```
=== 3_1 (p,q)=(3,1), |W|=2, relator length 6  (setup 0.0s)
 x=0.7 v=3.3018 [v>0 reflection] K1=6.7e-52 K2=0.0 K3=2.0e-52,0.0 K4=1.3e-50
      K5 max|A| on l (6 pts) = 5.28e-50 ; K6 max|A(Ko)+A(o)| = 7.75e-50 ; A off l = ['-1.8315', '-0.30629', '-1.1714']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = -1.5239316 ; |o*|^2-1/v = 0.2632
 x=1.2 v=10.1139 [v>0 reflection] K1=0.0 K2=0.0 K3=0.0,6.4e-52 K4=4.1e-50
      K5 max|A| on l (6 pts) = 2.53e-49 ; K6 max|A(Ko)+A(o)| = 1.03e-48 ; A off l = ['-4.3261', '-4.5206', '-5.3363']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = -1.0538546 ; |o*|^2-1/v = 0.0217
 x=2.0 v=53.6165 [v>0 reflection] K1=0.0 K2=2.2e-52 K3=0.0,2.2e-52 K4=2.1e-49
      K5 max|A| on l (6 pts) = 5.86e-47 ; K6 max|A(Ko)+A(o)| = 9.03e-47 ; A off l = ['-3.1598', '-5.1192', '-2.5184']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = -0.52428353 ; |o*|^2-1/v = 0.0007089
 x=3.5 v=1095.63 [v>0 reflection] K1=0.0 K2=0.0 K3=0.0,0.0 K4=8.5e-47
      K5 max|A| on l (6 pts) = 3.33e-45 ; K6 max|A(Ko)+A(o)| = 7.96e-44 ; A off l = ['-2.5324', '-0.4045', '-1.1214']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = -0.12058826 ; |o*|^2-1/v = 1.668e-6
=== 4_1 (p,q)=(5,3), |W|=4, relator length 10  (setup 0.0s)
 x=0.7 v=-0.662671 [v<0 half-turn] K1=6.7e-52 K2=5.4e-52 K3=3.3e-52,5.4e-52 K4=n/a
      K7 max|A(Ko)-A(o)| = 2.06e-49 ; A at random pts = ['1.0375', '1.345', '1.2697']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 1.2689812 ; |o*|^2-1/v = 3.887
 x=0.7 v=1.96447 [v>0 reflection, axes cross] K1=0.0 K2=2.0e-51 K3=0.0,2.0e-51 K4=1.4e-49
      K5 max|A| on l (6 pts) = 3.49e-49 ; K6 max|A(Ko)+A(o)| = 4.7e-49 ; A off l = ['-0.73445', '0.54678', '-3.8007']
      o* crossing pt: ||o*|^2-1/v| = 1.34e-51, A(o*) = 2.0e-49
 x=1.2 v=-0.90014 [v<0 half-turn] K1=0.0 K2=4.5e-52 K3=0.0,4.5e-52 K4=n/a
      K7 max|A(Ko)-A(o)| = 9.06e-49 ; A at random pts = ['1.0874', '1.0076', '1.0276']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 1.0045377 ; |o*|^2-1/v = 2.441
 x=1.2 v=9.01403 [v>0 reflection, axes cross] K1=0.0 K2=2.3e-51 K3=0.0,5.8e-52 K4=2.1e-47
      K5 max|A| on l (6 pts) = 6.43e-47 ; K6 max|A(Ko)+A(o)| = 9.91e-47 ; A off l = ['-1.6569', '-3.4551', '-2.4665']
      o* crossing pt: ||o*|^2-1/v| = 0.0, A(o*) = 5.43e-47
 x=2.0 v=-0.981343 [v<0 half-turn] K1=0.0 K2=2.7e-53 K3=0.0,1.6e-51 K4=n/a
      K7 max|A(Ko)-A(o)| = 6.27e-47 ; A at random pts = ['0.42437', '0.2471', '0.61674']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 0.5195023 ; |o*|^2-1/v = 2.076
 x=2.0 v=52.5978 [v>0 reflection, axes cross] K1=0.0 K2=2.6e-52 K3=0.0,2.6e-52 K4=1.7e-45
      K5 max|A| on l (6 pts) = 5.54e-44 ; K6 max|A(Ko)+A(o)| = 1.93e-43 ; A off l = ['-4.4756', '-4.5579', '-1.7484']
      o* crossing pt: ||o*|^2-1/v| = 0.0, A(o*) = 1.29e-42
 x=3.5 v=-0.999087 [v<0 half-turn] K1=0.0 K2=6.9e-53 K3=0.0,6.9e-53 K4=n/a
      K7 max|A(Ko)-A(o)| = 1.18e-43 ; A at random pts = ['0.080921', '0.1294', '0.12276']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 0.1205333 ; |o*|^2-1/v = 2.004
 x=3.5 v=1094.63 [v>0 reflection, axes cross] K1=0.0 K2=0.0 K3=0.0,2.2e-54 K4=1.1e-40
      K5 max|A| on l (6 pts) = 7.49e-39 ; K6 max|A(Ko)+A(o)| = 6.81e-38 ; A off l = ['-2.0662', '-3.1904', '-4.6282']
      o* crossing pt: ||o*|^2-1/v| = 0.0, A(o*) = 1.21e-36
=== 5_2 (p,q)=(7,3), |W|=6, relator length 14  (setup 0.1s)
 x=0.7 v=3.07295 [v>0 reflection] K1=0.0 K2=8.4e-51 K3=0.0,5.0e-51 K4=3.0e-48
      K5 max|A| on l (6 pts) = 2.54e-49 ; K6 max|A(Ko)+A(o)| = 1.75e-48 ; A off l = ['-1.9731', '-0.86247', '-2.1353']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = -1.3596515 ; |o*|^2-1/v = 0.218
 x=1.2 v=0.213357 [v>0 reflection, axes cross] K1=0.0 K2=1.3e-51 K3=0.0,1.3e-51 K4=1.3e-49
      K5 max|A| on l (6 pts) = 4.94e-48 ; K6 max|A(Ko)+A(o)| = 3.75e-48 ; A off l = ['-0.031436', '0.15177', '-0.10709']
      o* crossing pt: ||o*|^2-1/v| = 1.07e-50, A(o*) = 6.99e-49
 x=1.2 v=8.99044 [v>0 reflection, axes cross] K1=0.0 K2=2.2e-49 K3=0.0,7.2e-50 K4=1.4e-47
      K5 max|A| on l (6 pts) = 1.67e-47 ; K6 max|A(Ko)+A(o)| = 1.31e-47 ; A off l = ['-4.5597', '-2.8642', '-4.3113']
      o* crossing pt: ||o*|^2-1/v| = 1.67e-52, A(o*) = 4.42e-47
 x=1.2 v=10.024 [v>0 reflection] K1=0.0 K2=7.5e-51 K3=0.0,2.3e-51 K4=8.2e-47
      K5 max|A| on l (6 pts) = 7.49e-47 ; K6 max|A(Ko)+A(o)| = 4.59e-46 ; A off l = ['-2.4401', '-1.8164', '-3.4367']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = -1.0096057 ; |o*|^2-1/v = 0.01992
 x=2.0 v=0.0376832 [v>0 reflection, axes cross] K1=0.0 K2=3.9e-52 K3=0.0,3.9e-52 K4=2.3e-49
      K5 max|A| on l (6 pts) = 7.51e-46 ; K6 max|A(Ko)+A(o)| = 1.52e-45 ; A off l = ['0.090579', '0.038103', '0.096385']
      o* crossing pt: ||o*|^2-1/v| = 0.0, A(o*) = 2.42e-46
 x=2.0 v=52.5971 [v>0 reflection, axes cross] K1=0.0 K2=1.5e-48 K3=0.0,2.0e-49 K4=1.7e-45
      K5 max|A| on l (6 pts) = 8.35e-46 ; K6 max|A(Ko)+A(o)| = 4.16e-45 ; A off l = ['-4.1648', '-1.8834', '-5.0121']
      o* crossing pt: ||o*|^2-1/v| = 0.0, A(o*) = 3.72e-44
 x=2.0 v=53.5982 [v>0 reflection] K1=0.0 K2=9.7e-50 K3=0.0,1.3e-50 K4=3.7e-43
      K5 max|A| on l (6 pts) = 8.32e-44 ; K6 max|A(Ko)+A(o)| = 3.21e-43 ; A off l = ['-3.446', '-1.4632', '-2.7422']
```

```
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = -0.51959195 ; |o*|^2-1/v = 0.0006962
 x=3.5 v=0.00182626 [v>0 reflection, axes cross] K1=0.0 K2=1.3e-52 K3=0.0,1.3e-52 K4=2.1e-47
      K5 max|A| on l (6 pts) = 4.65e-42 ; K6 max|A(Ko)+A(o)| = 1.67e-40 ; A off l = ['0.03649', '0.01805', '0.018836']
      o* crossing pt: ||o*|^2-1/v| = 1.37e-48, A(o*) = 5.08e-43
 x=3.5 v=1094.63 [v>0 reflection, axes cross] K1=0.0 K2=2.8e-45 K3=0.0,8.6e-47 K4=3.3e-40
      K5 max|A| on l (6 pts) = 3.51e-41 ; K6 max|A(Ko)+A(o)| = 4.03e-40 ; A off l = ['-3.75', '-4.6054', '-5.7643']
      o* crossing pt: ||o*|^2-1/v| = 0.0, A(o*) = 7.98e-39
 x=3.5 v=1095.63 [v>0 reflection] K1=0.0 K2=1.0e-48 K3=0.0,3.1e-50 K4=1.5e-37
      K5 max|A| on l (6 pts) = 1.18e-38 ; K6 max|A(Ko)+A(o)| = 3.17e-37 ; A off l = ['-5.2092', '-5.0398', '-5.6514']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = -0.12053335 ; |o*|^2-1/v = 1.666e-6
=== 6_2 (p,q)=(11,3), |W|=10, relator length 22  (setup 0.2s)
 x=0.7 v=-0.0461676 [v<0 half-turn] K1=6.7e-52 K2=5.5e-52 K3=3.3e-52,5.5e-52 K4=n/a
      K7 max|A(Ko)-A(o)| = 1.5e-46 ; A at random pts = ['0.55833', '0.41136', '0.84431']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 0.34240192 ; |o*|^2-1/v = 44.19
 x=0.7 v=2.21533 [v>0 reflection, axes cross] K1=0.0 K2=3.2e-51 K3=0.0,3.2e-51 K4=2.9e-48
      K5 max|A| on l (6 pts) = 4.46e-48 ; K6 max|A(Ko)+A(o)| = 7.6e-48 ; A off l = ['-2.117', '-0.48953', '-1.6131']
      o* crossing pt: ||o*|^2-1/v| = 6.68e-52, A(o*) = 2.63e-49
 x=0.7 v=3.57334 [v>0 reflection] K1=0.0 K2=2.0e-50 K3=0.0,1.1e-50 K4=4.1e-48
      K5 max|A| on l (6 pts) = 1.4e-47 ; K6 max|A(Ko)+A(o)| = 9.03e-48 ; A off l = ['-1.7178', '-4.5347', '-1.9025']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = -1.6833101 ; |o*|^2-1/v = 0.3092
 x=1.2 v=-0.00799675 [v<0 half-turn] K1=0.0 K2=1.1e-51 K3=0.0,1.1e-51 K4=n/a
      K7 max|A(Ko)-A(o)| = 3.1e-43 ; A at random pts = ['0.19033', '0.18065', '0.77572']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 0.098738892 ; |o*|^2-1/v = 250.3
 x=1.2 v=9.08842 [v>0 reflection, axes cross] K1=0.0 K2=1.3e-50 K3=7.4e-53,5.1e-51 K4=5.8e-46
      K5 max|A| on l (6 pts) = 1.89e-45 ; K6 max|A(Ko)+A(o)| = 1.31e-44 ; A off l = ['-0.24627', '-2.5228', '-1.5332']
      o* crossing pt: ||o*|^2-1/v| = 0.0, A(o*) = 2.35e-46
 x=1.2 v=10.2117 [v>0 reflection] K1=0.0 K2=4.1e-50 K3=0.0,1.2e-50 K4=7.2e-46
      K5 max|A| on l (6 pts) = 3.87e-45 ; K6 max|A(Ko)+A(o)| = 1.28e-45 ; A off l = ['-3.6038', '-3.6702', '-2.42']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = -1.0991649 ; |o*|^2-1/v = 0.02359
 x=2.0 v=-0.000335115 [v<0 half-turn] K1=0.0 K2=8.8e-53 K3=0.0,8.8e-53 K4=n/a
      K7 max|A(Ko)-A(o)| = 9.1e-38 ; A at random pts = ['0.21605', '0.08546', '0.087216']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 0.0097315793 ; |o*|^2-1/v = 5968.0
 x=2.0 v=52.6118 [v>0 reflection, axes cross] K1=0.0 K2=8.4e-51 K3=0.0,1.0e-51 K4=4.4e-43
      K5 max|A| on l (6 pts) = 1.07e-40 ; K6 max|A(Ko)+A(o)| = 1.5e-39 ; A off l = ['-1.8126', '-4.1482', '-4.5976']
      o* crossing pt: ||o*|^2-1/v| = 0.0, A(o*) = 4.66e-42
 x=2.0 v=53.6351 [v>0 reflection] K1=0.0 K2=1.7e-48 K3=0.0,2.3e-49 K4=1.6e-41
      K5 max|A| on l (6 pts) = 2.04e-40 ; K6 max|A(Ko)+A(o)| = 2.78e-41 ; A off l = ['-5.2278', '-3.8406', '-1.123']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = -0.52901343 ; |o*|^2-1/v = 0.0007219
 x=3.5 v=-8.31527e-7 [v<0 half-turn] K1=4.2e-53 K2=7.0e-53 K3=1.3e-54,7.0e-53 K4=n/a
      K7 max|A(Ko)-A(o)| = 1.56e-27 ; A at random pts = ['0.016195', '0.014553', '0.026053']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 0.00011004531 ; |o*|^2-1/v = 2.405e+6
 x=3.5 v=1094.63 [v>0 reflection, axes cross] K1=0.0 K2=3.2e-48 K3=0.0,9.7e-50 K4=1.8e-36
      K5 max|A| on l (6 pts) = 5.35e-33 ; K6 max|A(Ko)+A(o)| = 4.09e-31 ; A off l = ['-5.1292', '-2.6634', '-1.5236']
      o* crossing pt: ||o*|^2-1/v| = 0.0, A(o*) = 2.46e-36
 x=3.5 v=1095.63 [v>0 reflection] K1=0.0 K2=1.1e-45 K3=0.0,3.2e-47 K4=1.3e-34
      K5 max|A| on l (6 pts) = 7.97e-33 ; K6 max|A(Ko)+A(o)| = 2.41e-34 ; A off l = ['-2.5112', '-4.6198', '-1.7447']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = -0.12064319 ; |o*|^2-1/v = 1.669e-6
=== 6_1 (p,q)=(9,7), |W|=8, relator length 18  (setup 0.1s)
 x=0.7 v=-0.285329 [v<0 half-turn] K1=0.0 K2=1.2e-51 K3=0.0,1.2e-51 K4=n/a
      K7 max|A(Ko)-A(o)| = 1.94e-49 ; A at random pts = ['0.8637', '0.86083', '0.84844']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 0.8476635 ; |o*|^2-1/v = 7.878
 x=0.7 v=2.28393 [v>0 reflection, axes cross] K1=0.0 K2=0.0 K3=0.0,1.4e-51 K4=7.7e-47
      K5 max|A| on l (6 pts) = 1.56e-46 ; K6 max|A(Ko)+A(o)| = 1.03e-46 ; A off l = ['-0.89062', '-2.1287', '-0.48431']
      o* crossing pt: ||o*|^2-1/v| = 0.0, A(o*) = 8.69e-47
 x=1.2 v=-2.54533 [v<0 half-turn] K1=0.0 K2=1.4e-51 K3=0.0,1.4e-51 K4=n/a
      K7 max|A(Ko)-A(o)| = 9.59e-48 ; A at random pts = ['0.75409', '3.4057', '1.6282']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 1.5673871 ; |o*|^2-1/v = 1.005
 x=1.2 v=-2.10023 [v<0 half-turn] K1=0.0 K2=7.4e-51 K3=0.0,7.4e-51 K4=n/a
      K7 max|A(Ko)-A(o)| = 3.85e-49 ; A at random pts = ['3.4894', '1.568', '0.41066']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 1.4520961 ; |o*|^2-1/v = 1.172
 x=1.2 v=-0.353634 [v<0 half-turn] K1=0.0 K2=8.0e-52 K3=0.0,8.1e-52 K4=n/a
      K7 max|A(Ko)-A(o)| = 7.48e-49 ; A at random pts = ['0.64784', '0.68819', '0.6679']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 0.64603149 ; |o*|^2-1/v = 5.875
 x=1.2 v=9.11309 [v>0 reflection, axes cross] K1=0.0 K2=2.4e-51 K3=0.0,1.2e-51 K4=1.8e-43
      K5 max|A| on l (6 pts) = 2.26e-43 ; K6 max|A(Ko)+A(o)| = 1.61e-42 ; A off l = ['-0.79911', '-3.7229', '-3.4684']
      o* crossing pt: ||o*|^2-1/v| = 1.67e-52, A(o*) = 3.44e-42
 x=2.0 v=-2.60456 [v<0 half-turn] K1=0.0 K2=4.0e-52 K3=0.0,4.0e-52 K4=n/a
      K7 max|A(Ko)-A(o)| = 1.01e-46 ; A at random pts = ['0.77707', '1.0781', '0.90828']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 0.82810462 ; |o*|^2-1/v = 0.8059
 x=2.0 v=-2.01866 [v<0 half-turn] K1=0.0 K2=3.1e-51 K3=0.0,3.1e-51 K4=n/a
      K7 max|A(Ko)-A(o)| = 5.75e-48 ; A at random pts = ['1.2979', '1.158', '0.93272']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 0.73471567 ; |o*|^2-1/v = 1.029
 x=2.0 v=-0.376779 [v<0 half-turn] K1=0.0 K2=2.3e-52 K3=0.0,1.6e-51 K4=n/a
      K7 max|A(Ko)-A(o)| = 2.91e-46 ; A at random pts = ['0.67523', '0.42782', '0.26509']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 0.32459706 ; |o*|^2-1/v = 5.346
 x=2.0 v=52.6165 [v>0 reflection, axes cross] K1=0.0 K2=0.0 K3=0.0,4.5e-53 K4=3.7e-38
      K5 max|A| on l (6 pts) = 3.69e-37 ; K6 max|A(Ko)+A(o)| = 1.59e-36 ; A off l = ['-4.4271', '-1.8456', '-3.6898']
      o* crossing pt: ||o*|^2-1/v| = 0.0, A(o*) = 1.18e-34
 x=3.5 v=-2.61737 [v<0 half-turn] K1=4.2e-53 K2=9.4e-53 K3=1.3e-54,9.4e-53 K4=n/a
      K7 max|A(Ko)-A(o)| = 1.07e-43 ; A at random pts = ['0.35214', '0.23421', '0.16349']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 0.1948533 ; |o*|^2-1/v = 0.766
 x=3.5 v=-2.00091 [v<0 half-turn] K1=0.0 K2=3.3e-50 K3=1.3e-54,3.3e-50 K4=n/a
      K7 max|A(Ko)-A(o)| = 2.41e-46 ; A at random pts = ['0.20938', '0.2706', '0.1885']
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 0.17044759 ; |o*|^2-1/v = 1.001
 x=3.5 v=-0.381714 [v<0 half-turn] K1=0.0 K2=2.5e-53 K3=0.0,2.5e-53 K4=n/a
      K7 max|A(Ko)-A(o)| = 1.86e-43 ; A at random pts = ['0.071261', '0.075069', '0.074242']
```

```
      axes do not cross (v>(s-1/s)^2 or v<0): o* off l, A(o*) = 0.074537854 ; |o*|^2-1/v = 5.241
Traceback (most recent call last):
  File "/tmp/claude-0/-home-claude/d7048feb-3787-5f7f-b86a-f71c9a373eed/scratchpad/lanes/open/area/area_sym.py", line 68, in
<module>
    Aon.append(abs(area(M, rel, oo)))
                   ~~~~^^^^^^^^^^^^^
  File "/tmp/claude-0/-home-claude/d7048feb-3787-5f7f-b86a-f71c9a373eed/scratchpad/lanes/open/area/area_sym.py", line 28, in
area
    pts = orbit(M, rel, o); A, E = poly_area_turn(pts); return A
                                   ~~~~~~~~~~~~~~~~^^^^^
  File "/tmp/claude-0/-home-claude/d7048feb-3787-5f7f-b86a-f71c9a373eed/scratchpad/lanes/compute/q2_lib.py", line 112, in
poly_area_turn
    E += mp.atan2(sn, cs)
         ~~~~~~~~^^^^^^^^
  File "/usr/local/lib/python3.13/dist-packages/mpmath/ctx_mp.py", line 261, in atan2
    return ctx.make_mpf(libmp.mpf_atan2(y._mpf_, x._mpf_, *ctx._prec_rounding))
                                        ^^^^^^^
AttributeError: 'mpc' object has no attribute '_mpf_'. Did you mean: '_mpc_'?
```

area_sym2.py

```
# SCRIPT: AREA-SYM2
# KILL CONDITIONS (written before the run):
#  K9  Sigma = g^-1 K (g=(aW)^-1) satisfies Sigma p_k = p_{k+m} for o on l, and Sigma^2 = identity as an isometry (a
reflection in the perpendicular bisector of o and aW.o): dies if either error > 1e-25.
#  K10 Zero set of A on a grid (v>0, 4_1 and trefoil): sign changes of A occur only across l (A odd under the reflection).
Dies if a sign change is found on a grid edge whose two endpoints lie on the SAME side of l.
#  K11 (3_1 etc.) the general-q statement: for q odd, e_{p-i}=e_i for all (p,q) with p<=60 (palindrome). Dies on any
exception.
#  K12 a knot at larger size (13,5) passes K2-K5 at dps 100.
import sys, random
sys.path.insert(0, '/tmp/claude-0/-home-claude/d7048feb-3787-5f7f-b86a-f71c9a373eed/scratchpad/lanes/compute')
from q2_lib import *
from math import gcd
mp.mp.dps = 100
# K11
bad = 0; cnt = 0
for p in range(3, 61, 2):
    for q in range(1, p, 2):
        if gcd(p, q) != 1: continue
        e = eps(p, q); cnt += 1
        if any(e[i-1] != e[p-i-1] for i in range(1, p)): bad += 1
print("K11 palindrome e_{p-i}=e_i for odd q: %d pairs (p<=59), failures %d" % (cnt, bad))
def area(M, rel, o):
    pts = orbit(M, rel, o); A, E = poly_area_turn(pts); return A
def side(v, z): return abs(z)**2*v - 1   # >0 outside l (|z|>1/sqrt v)
for nm, (p, q), x in [('3_1', (3, 1), '1.0'), ('4_1', (5, 3), '1.2'), ('5_2', (7, 3), '1.2')]:
    fs = riley_coeff_funcs(p, q); rel, W = letters(p, q); m = len(W) + 1; N = 2*m
    s = mp.e**mp.mpf(x)
    for v in real_roots(fs, s):
        if v <= 0: continue
        M = mats(s, v); a = M[('a', 1)]; Wm = prod(M, W); g = (a*Wm)**-1
        o = mp.sqrt(1/v)*mp.mpc(mp.cos(0.8), mp.sin(0.8))
        # Sigma z = g^-1 (K z), K z = 1/(v conj z)
        Sg = lambda z: mobius(g**-1, 1/(v*mp.conj(z)))
        pts = orbit(M, rel, o)
        e1 = max(abs(Sg(pts[k]) - pts[(k+m) % N]) for k in range(N))
        z = mp.mpc(0.37, 0.9); e2 = abs(Sg(Sg(z)) - z)
        print("K9 %s x=%s v=%s: max|Sigma p_k - p_{k+m}| = %s ; |Sigma^2 z - z| = %s" % (nm, x, mp.nstr(v, 6), mp.nstr(e1,
3), mp.nstr(e2, 3)))
        # K10 grid
        mp.mp.dps = 40
        xs = [mp.mpf(-1.5) + mp.mpf(3)*i/16 for i in range(17)]; ys = [mp.mpf('0.15') + mp.mpf('2.2')*j/16 for j in
range(17)]
        G = [[area(M, rel, mp.mpc(xx, yy)) for xx in xs] for yy in ys]
        viol = 0; chg = 0; zero_near = 0
        for j in range(17):
            for i in range(17):
                for (di, dj) in [(1, 0), (0, 1)]:
                    i2, j2 = i+di, j+dj
                    if i2 > 16 or j2 > 16: continue
                    if G[j][i]*G[j2][i2] < 0:
                        chg += 1
                        s1 = side(v, mp.mpc(xs[i], ys[j])); s2 = side(v, mp.mpc(xs[i2], ys[j2]))
                        if s1*s2 > 0: viol += 1
        print("K10 %s v=%s: grid edges with a sign change of A = %d; of these with both ends on the same side of l = %d; A
range [%s, %s]" % (nm, mp.nstr(v, 5), chg, viol, mp.nstr(min(min(r) for r in G), 4), mp.nstr(max(max(r) for r in G), 4)))
        mp.mp.dps = 100
# K12
p, q = 13, 5
fs = riley_coeff_funcs(p, q); rel, W = letters(p, q)
for x in ['1.2', '2.0']:
```

```
    s = mp.e**mp.mpf(x)
    for v in real_roots(fs, s):
        if v <= 0: continue
        M = mats(s, v)
        try:
            ls = []
            for th in ['0.5', '1.1', '2.2']:
                oo = mp.sqrt(1/v)*mp.mpc(mp.cos(mp.mpf(th)), mp.sin(mp.mpf(th)))
                ls.append(abs(area(M, rel, oo)))
            off = area(M, rel, mp.mpc(0.4, 1.3))
            print("K12 (13,5) x=%s v=%s: max|A| on l = %s ; A off l = %s" % (x, mp.nstr(v, 6), mp.nstr(max(ls), 3),
mp.nstr(off, 6)))
        except Exception as ex:
            print("K12 x=%s v=%s exception %r" % (x, mp.nstr(v, 6), ex))
```

Output

```
K11 palindrome e_{p-i}=e_i for odd q: 364 pairs (p<=59), failures 0
K9 3_1 x=1.0 v=6.52439: max|Sigma p_k - p_{k+m}| = 2.4e-99 ; |Sigma^2 z - z| = 6.52e-101
K10 3_1 v=6.5244: grid edges with a sign change of A = 7; of these with both ends on the same side of l = 0; A range
[-5.839, 2.036]
K9 4_1 x=1.2 v=9.01403: max|Sigma p_k - p_{k+m}| = 1.12e-96 ; |Sigma^2 z - z| = 1.79e-97
K10 4_1 v=9.014: grid edges with a sign change of A = 7; of these with both ends on the same side of l = 0; A range [-5.91,
1.518]
K9 5_2 x=1.2 v=0.213357: max|Sigma p_k - p_{k+m}| = 9.25e-98 ; |Sigma^2 z - z| = 2.03e-99
K10 5_2 v=0.21336: grid edges with a sign change of A = 25; of these with both ends on the same side of l = 0; A range
[-0.1355, 2.938]
K9 5_2 x=1.2 v=8.99044: max|Sigma p_k - p_{k+m}| = 7.7e-96 ; |Sigma^2 z - z| = 5.08e-98
K10 5_2 v=8.9904: grid edges with a sign change of A = 7; of these with both ends on the same side of l = 0; A range
[-5.909, 1.523]
K9 5_2 x=1.2 v=10.024: max|Sigma p_k - p_{k+m}| = 1.64e-94 ; |Sigma^2 z - z| = 6.37e-96
K10 5_2 v=10.024: grid edges with a sign change of A = 7; of these with both ends on the same side of l = 0; A range
[-5.918, 1.359]
K12 (13,5) x=1.2 v=9.12776: max|A| on l = 8.44e-97 ; A off l = -1.75555
K12 (13,5) x=1.2 v=10.0014: max|A| on l = 2.22e-96 ; A off l = -1.79572
K12 (13,5) x=2.0 v=52.6168: max|A| on l = 1.46e-90 ; A off l = -2.18801
K12 (13,5) x=2.0 v=53.5974: max|A| on l = 1.68e-90 ; A off l = -2.19145
```

band.py

```
# SCRIPT: KNOT-COMM-BAND
# Where does a two-bridge knot's real Riley curve meet the band where the meridian commutator is a rotation?
# Identity (checked on 5,243 samples and proved in Lean): tr[a,b] = 2 - v((s - 1/s)^2 - v).
# So [a,b] is elliptic  <=>  |tr| < 2  <=>  0 < v(Z - 2) < 4 with Z - 2 = (s - 1/s)^2 - v = tr(ab) - 2,
# i.e. v > 0, tr(ab) > 2 and v (tr(ab) - 2) < 4.
import warnings; warnings.filterwarnings('ignore')
import json, math, numpy as np, sympy as sp
from census_lib import riley_poly, alexander, eps, S_, V_, T
KN = [(n, k['p'], k['q']) for n, k in [(k['name'], k) for k in json.load(open('../../../atlas_sm/knot3d/data_v2.json'))
['knots']]]
xs = np.concatenate([np.linspace(1e-4, 0.05, 60), np.arange(0.05, 6.0, 0.005)])
out = {}
for name, p, q in KN:
    R = sp.Poly(riley_poly(p, q), V_)
    cf = [sp.lambdify(S_, c, 'numpy') for c in R.all_coeffs()]
    ent, pts = [], 0
    vmin_pos = None
    for x in xs:
        s = math.exp(x); co = [complex(f(s)) if callable(f) else complex(f) for f in cf]
        co = [c.real for c in co]
        for v in np.roots(co):
            if abs(v.imag) > 1e-9 * max(1, abs(v)): continue
            v = v.real; pts += 1
            A = (s - 1 / s) ** 2; tr = 2 - v * (A - v)
            if abs(tr) < 2: ent.append((round(x, 4), round(v, 5), round(tr, 5)))
    D = sp.Poly(alexander(p, q), T); r = [complex(z) for z in sp.Poly(D).nroots(n=30)]
    off = sum(1 for z in r if abs(abs(z) - 1) > 1e-8)
    det = abs(int(D.eval(-1)))
    sig = sum(eps(p, q))   # sum of the eps signs (two-bridge: signature is a known function of these; recorded raw)
    out[name] = dict(p=p, q=q, real_points=pts, enters=len(ent) > 0, first=ent[0] if ent else None, last=ent[-1] if ent else
None,
                     det=det, alex_offcircle_roots=off, eps_sum=sig, alexander=str(D.as_expr()))
    print('%-7s p/q=%d/%d  enters=%-5s  first=%-28s det=%-3d off-circle Alexander roots=%d  sum eps=%+d' % (
        name, p, q, len(ent) > 0, str(ent[0]) if ent else '-', det, off, sig), flush=True)
json.dump(out, open('band_out.json', 'w'), indent=1)
```

Output

```
T(2,3)  p/q=3/1  enters=False  first=-                          det=3   off-circle Alexander roots=0  sum eps=+2
T(2,5)  p/q=5/1  enters=False  first=-                          det=5   off-circle Alexander roots=0  sum eps=+4
4_1     p/q=5/3  enters=True   first=(np.float64(0.485), np.float64(0.13928), np.float64(1.87775)) det=5   off-circle
Alexander roots=2  sum eps=+0
T(2,7)  p/q=7/1  enters=False  first=-                          det=7   off-circle Alexander roots=0  sum eps=+6
5_2     p/q=7/3  enters=True   first=(np.float64(0.725), np.float64(1.48801), np.float64(0.49759)) det=7   off-circle
Alexander roots=0  sum eps=+2
6_1     p/q=9/5  enters=True   first=(np.float64(0.35), np.float64(0.02623), np.float64(1.9873)) det=9   off-circle
Alexander roots=2  sum eps=+0
6_2     p/q=11/3  enters=True   first=(np.float64(0.36), np.float64(0.16501), np.float64(1.93793)) det=11  off-circle
Alexander roots=2  sum eps=+2
7_2     p/q=11/5  enters=True   first=(np.float64(0.605), np.float64(0.99262), np.float64(1.3458)) det=11  off-circle
Alexander roots=0  sum eps=+2
7_3     p/q=13/3  enters=True   first=(np.float64(0.475), np.float64(0.50188), np.float64(1.76383)) det=13  off-circle
Alexander roots=0  sum eps=+4
6_3     p/q=13/5  enters=True   first=(np.float64(2.865), np.float64(305.9725), np.float64(1.99486)) det=13  off-circle
Alexander roots=4  sum eps=+0
8_1     p/q=13/7  enters=True   first=(np.float64(0.285), np.float64(0.00095), np.float64(1.99968)) det=13  off-circle
Alexander roots=2  sum eps=+0
7_4     p/q=15/11  enters=True   first=(np.float64(0.485), np.float64(0.72051), np.float64(1.78635)) det=15  off-circle
Alexander roots=0  sum eps=+2
8_2     p/q=17/3  enters=True   first=(np.float64(0.295), np.float64(0.11919), np.float64(1.9715)) det=17  off-circle
Alexander roots=2  sum eps=+4
7_5     p/q=17/5  enters=True   first=(np.float64(2.055), np.float64(58.96299), np.float64(1.99177)) det=17  off-circle
Alexander roots=0  sum eps=+4
8_3     p/q=17/13  enters=True   first=(np.float64(0.25), np.float64(0.04315), np.float64(1.99085)) det=17  off-circle
Alexander roots=2  sum eps=+0
8_4     p/q=19/5  enters=True   first=(np.float64(0.265), np.float64(0.05725), np.float64(1.98682)) det=19  off-circle
Alexander roots=2  sum eps=+2
7_6     p/q=19/7  enters=True   first=(np.float64(0.555), np.float64(0.96769), np.float64(1.61658)) det=19  off-circle
Alexander roots=2  sum eps=+2
7_7     p/q=21/13  enters=True   first=(np.float64(0.725), np.float64(2.4109), np.float64(1.79077)) det=21  off-circle
Alexander roots=4  sum eps=+0
8_7     p/q=23/5  enters=True   first=(np.float64(1.42), np.float64(15.17381), np.float64(1.99422)) det=23  off-circle
Alexander roots=4  sum eps=+2
8_6     p/q=23/7  enters=True   first=(np.float64(0.27), np.float64(0.1387), np.float64(1.9778)) det=23  off-circle
Alexander roots=2  sum eps=+2
8_9     p/q=25/7  enters=True   first=(np.float64(0.285), np.float64(0.05046), np.float64(1.9857)) det=25  off-circle
Alexander roots=6  sum eps=+0
8_8     p/q=25/9  enters=True   first=(np.float64(1.335), np.float64(12.5092), np.float64(1.99978)) det=25  off-circle
Alexander roots=4  sum eps=+0
8_11    p/q=27/17  enters=True   first=(np.float64(0.35), np.float64(0.01562), np.float64(1.99227)) det=27  off-circle
Alexander roots=2  sum eps=-2
8_13    p/q=29/11  enters=True   first=(np.float64(0.57), np.float64(1.02019), np.float64(1.56499)) det=29  off-circle
Alexander roots=4  sum eps=+0
8_12    p/q=29/17  enters=True   first=(np.float64(0.305), np.float64(0.03919), np.float64(1.9865)) det=29  off-circle
Alexander roots=4  sum eps=+0
8_14    p/q=31/13  enters=True   first=(np.float64(0.39), np.float64(0.10615), np.float64(1.94334)) det=31  off-circle
Alexander roots=2  sum eps=+2
```

band_hp.py

```
# SCRIPT: KNOT-COMM-BAND-HP
# High-precision check (mpmath, 60 digits) of the entries into the rotation band for 6_3, 7_5, 8_7, 8_8,
# and of the torus knots' ab trace and v sign.
import warnings; warnings.filterwarnings('ignore')
import mpmath as mp, sympy as sp
from census_lib import riley_poly, S_, V_
mp.mp.dps = 60
def real_roots(p, q, x):
    R = sp.Poly(riley_poly(p, q), V_); s = mp.e ** mp.mpf(x)
    co = [mp.mpf(str(sp.N(c.subs(S_, sp.Float(str(s), 70)), 70))) for c in R.all_coeffs()]
    rts = mp.polyroots(co, maxsteps=400, extraprec=200)
    return s, [mp.re(r) for r in rts if abs(mp.im(r)) < mp.mpf(10) ** -30]
for name, p, q, xs in [('6_3', 13, 5, [2.6, 2.865, 3.2, 4.0, 5.0]), ('7_5', 17, 5, [2.0, 2.055, 2.5, 3.5]),
                       ('8_7', 23, 5, [1.4, 1.42, 1.8, 3.0]), ('8_8', 25, 9, [1.3, 1.335, 1.8, 3.0])]:
    for x in xs:
        s, vs = real_roots(p, q, x); A = (s - 1 / s) ** 2
        trs = [2 - v * (A - v) for v in vs]
        ins = [mp.nstr(t, 12) for t in trs if abs(t) < 2]
        print(name, 'x=%.3f' % x, 'real roots', len(vs), ' tr[a,b] in (-2,2):', ins)
for name, p, q in [('T(2,3)', 3, 1), ('T(2,5)', 5, 1), ('T(2,7)', 7, 1)]:
    for x in [0.3, 1.0, 2.5]:
        s, vs = real_roots(p, q, x)
        print(name, 'x=%.1f' % x, [('v=' + mp.nstr(v, 8), 'tr(ab)=' + mp.nstr(s * s + 1 / (s * s) - v, 10)) for v in vs])
```

Output

```
6_3 x=2.600 real roots 2  tr[a,b] in (-2,2): []
6_3 x=2.865 real roots 2  tr[a,b] in (-2,2): []
6_3 x=3.200 real roots 2  tr[a,b] in (-2,2): []
6_3 x=4.000 real roots 2  tr[a,b] in (-2,2): []
6_3 x=5.000 real roots 2  tr[a,b] in (-2,2): []
7_5 x=2.000 real roots 6  tr[a,b] in (-2,2): []
7_5 x=2.055 real roots 6  tr[a,b] in (-2,2): []
7_5 x=2.500 real roots 6  tr[a,b] in (-2,2): []
7_5 x=3.500 real roots 6  tr[a,b] in (-2,2): []
8_7 x=1.400 real roots 5  tr[a,b] in (-2,2): []
8_7 x=1.420 real roots 5  tr[a,b] in (-2,2): []
8_7 x=1.800 real roots 5  tr[a,b] in (-2,2): []
8_7 x=3.000 real roots 5  tr[a,b] in (-2,2): []
8_8 x=1.300 real roots 4  tr[a,b] in (-2,2): []
8_8 x=1.335 real roots 4  tr[a,b] in (-2,2): []
8_8 x=1.800 real roots 4  tr[a,b] in (-2,2): []
8_8 x=3.000 real roots 4  tr[a,b] in (-2,2): []
T(2,3) x=0.3 [('v=1.3709304', 'tr(ab)=1.0')]
T(2,3) x=1.0 [('v=6.5243914', 'tr(ab)=1.0')]
T(2,3) x=2.5 [('v=147.4199', 'tr(ab)=1.0')]
T(2,5) x=0.3 [('v=0.75289645', 'tr(ab)=1.618033989'), ('v=2.9889644', 'tr(ab)=-0.6180339887')]
T(2,5) x=1.0 [('v=5.9063574', 'tr(ab)=1.618033989'), ('v=8.1424254', 'tr(ab)=-0.6180339887')]
T(2,5) x=2.5 [('v=146.80186', 'tr(ab)=1.618033989'), ('v=149.03793', 'tr(ab)=-0.6180339887')]
T(2,7) x=0.3 [('v=0.5689927', 'tr(ab)=1.801937736'), ('v=1.9258886', 'tr(ab)=0.4450418679'), ('v=3.61791',
'tr(ab)=-1.246979604')]
T(2,7) x=1.0 [('v=5.7224536', 'tr(ab)=1.801937736'), ('v=7.0793495', 'tr(ab)=0.4450418679'), ('v=8.771371',
'tr(ab)=-1.246979604')]
T(2,7) x=2.5 [('v=146.61796', 'tr(ab)=1.801937736'), ('v=147.97486', 'tr(ab)=0.4450418679'), ('v=149.66688',
'tr(ab)=-1.246979604')]
```

band_hp2.py

```
# SCRIPT: KNOT-COMM-BAND-HP2
# 60-digit rescan of 6_3, 7_5, 8_7, 8_8 (x in 0.02..6): for each real Riley root print v, tr(ab)=s^2+s^-2-v,
# and tr[a,b]-2 = -v(tr(ab)-2). Elliptic iff 0 < v(tr(ab)-2) < 4.
import warnings; warnings.filterwarnings('ignore')
import mpmath as mp, sympy as sp, json
from census_lib import riley_poly, S_, V_
mp.mp.dps = 60
out = {}
for name, p, q in [('6_3', 13, 5), ('7_5', 17, 5), ('8_7', 23, 5), ('8_8', 25, 9)]:
    R = sp.Poly(riley_poly(p, q), V_); cs = [sp.lambdify(S_, c, 'mpmath') for c in R.all_coeffs()]
    hits, closest = [], (None, mp.inf)
    x = mp.mpf('0.02')
    while x <= 6:
        s = mp.e ** x
        rts = mp.polyroots([c(s) for c in cs], maxsteps=500, extraprec=300)
        for r in rts:
            if abs(mp.im(r)) < mp.mpf(10) ** -25:
                v = mp.re(r); tab = s * s + 1 / (s * s) - v; k = v * (tab - 2)
                if 0 < k < 4: hits.append((float(x), float(v), float(2 - k)))
                d = min(abs(k), abs(k - 4))
                if k > 0 and d < closest[1]: closest = ((float(x), float(v), float(tab), float(k)), d)
        x += mp.mpf('0.01')
    out[name] = dict(hits=hits[:5], n_hits=len(hits), closest_positive_k=closest[0])
    print(name, 'hits', len(hits), hits[:3], 'closest k>0 (x,v,tr(ab),k):', closest[0], flush=True)
json.dump(out, open('band_hp2_out.json', 'w'), indent=1)
```

Output

```
6_3 hits 0 [] closest k>0 (x,v,tr(ab),k): None
7_5 hits 0 [] closest k>0 (x,v,tr(ab),k): None
8_7 hits 0 [] closest k>0 (x,v,tr(ab),k): None
8_8 hits 15 [(5.7, 89319.72337200104, 1.9999999999999938), (5.73, 94843.07027546126, 1.9999999999999987), (5.74,
96759.06778953563, 1.9999999999999933)] closest k>0 (x,v,tr(ab),k): (5.73, 94843.07027546126, 2.0, 1.4107307997757274e-15)
```

firstx.py

```
# SCRIPT: KNOT-COMM-FIRSTX — 80-digit check of the float scan's first band entry x1 for every knot:
# at x1 some real branch has 0<k<4 (k = v(tr ab - 2), elliptic iff 0<k<4), at x1-0.005 none does.
# Never-enter knots: max k over all real branches at 120 sample x in (0,6] must be <= 0.
# Kill condition: an entering knot with no band point at x1, or a never-enter knot with max k > 0 that survives 80 digits.
import warnings; warnings.filterwarnings('ignore')
import mpmath as mp, sympy as sp, json
from census_lib import riley_poly, S_, V_
```

```
mp.mp.dps = 80
d = json.load(open('band_out.json')); res = {}
NEVER = {'T(2,3)', 'T(2,5)', 'T(2,7)', '6_3', '7_5', '8_7', '8_8'}
def ks(cs, x):
    s = mp.e ** mp.mpf(x); rts = mp.polyroots([c(s) for c in cs], maxsteps=1500, extraprec=800)
    return [mp.re(r) * (s*s + 1/(s*s) - mp.re(r) - 2) for r in rts if abs(mp.im(r)) < mp.mpf(10) ** -40]
for name, e in d.items():
    R = sp.Poly(riley_poly(e['p'], e['q']), V_); cs = [sp.lambdify(S_, c, 'mpmath') for c in R.all_coeffs()]
    if name in NEVER:
        mx = max([max([float(k) for k in ks(cs, 0.05 * i)] or [-9e9]) for i in range(1, 121)])
        res[name] = dict(enters=False, max_k=mx); print(name, 'never: max k over 120 samples =', mx, flush=True)
    else:
        x1 = e['first'][0]
        a = [float(k) for k in ks(cs, x1) if 0 < k < 4]; b = [float(k) for k in ks(cs, round(x1 - 0.005, 3)) if 0 < k < 4]
        res[name] = dict(enters=True, x1=x1, in_band_at_x1=a, in_band_before=b); print(name, 'x1', x1, 'k at x1', a,
'before', b, flush=True)
json.dump(res, open('firstx_out.json', 'w'), indent=1)
```

Output

```
T(2,3) never: max k over 120 samples = -1.0100083361116072
T(2,5) never: max k over 120 samples = -0.14972087797411646
4_1 x1 0.485 k at x1 [0.1222531483628355] before []
T(2,7) never: max k over 120 samples = -0.04121093420920517
5_2 x1 0.725 k at x1 [1.559214023436715, 1.5024079721685908] before []
6_1 x1 0.35 k at x1 [0.012699213363384091] before []
6_2 x1 0.36 k at x1 [0.036723410765915704, 0.06207254979837948] before []
7_2 x1 0.605 k at x1 [0.6794648672740796, 0.6541987160303736] before []
7_3 x1 0.475 k at x1 [0.21869377338853482, 0.23617004144677853] before []
6_3 never: max k over 120 samples = -6.144325609684112e-06
8_1 x1 0.285 k at x1 [0.0003169387156866573] before []
7_4 x1 0.485 k at x1 [0.25808328601329056, 0.21364583288883263] before []
8_2 x1 0.295 k at x1 [0.022072017018148433, 0.028501473339882916] before []
7_5 never: max k over 120 samples = -1.536053088274064e-06
8_3 x1 0.25 k at x1 [0.009151655877896231] before []
8_4 x1 0.265 k at x1 [0.013183670846780515] before []
7_6 x1 0.555 k at x1 [0.451401972491496, 0.3834244084919281] before []
7_7 x1 0.725 k at x1 [0.26397623793835034, 0.20923019223659123] before []
8_7 never: max k over 120 samples = -1.5360719642947195e-06
8_6 x1 0.27 k at x1 [0.012251971674268856, 0.02219966026843743] before []
8_9 x1 0.285 k at x1 [0.014297594591256554] before []
8_8 never: max k over 120 samples = -2.3195655856914496e-16
8_11 x1 0.35 k at x1 [0.007729734305822806] before []
8_13 x1 0.57 k at x1 [0.521279968733845, 0.4350075254657359] before []
8_12 x1 0.305 k at x1 [0.013503092819935822] before []
8_14 x1 0.39 k at x1 [0.021777222566652325, 0.05665520435944509] before []
```

asym.py

```
# SCRIPT: KNOT-COMM-ASYM — limits of k = v(tr ab - 2) on every real branch as x -> infinity (x = 6, 9, 12; 200 digits)
import warnings; warnings.filterwarnings('ignore')
import mpmath as mp, sympy as sp, json
from census_lib import riley_poly, S_, V_
d = json.load(open('band_out.json')); out = {}
mp.mp.dps = 200
for name, e in d.items():
    R = sp.Poly(riley_poly(e['p'], e['q']), V_); cs = [sp.lambdify(S_, c, 'mpmath') for c in R.all_coeffs()]
    rows = {}
    for x in [6, 9, 12]:
        s = mp.e ** mp.mpf(x); rts = mp.polyroots([c(s) for c in cs], maxsteps=2000, extraprec=1500)
        ks = []
        for r in rts:
            if abs(mp.im(r)) < mp.mpf(10) ** -80:
                v = mp.re(r); t = s*s + 1/(s*s) - v; ks.append((float(t), mp.nstr(v*(t-2), 8)))
        rows[x] = sorted(ks, key=lambda z: -z[0])
    out[name] = {str(k): [list(z) for z in v] for k, v in rows.items()}
    small = [(t, k) for t, k in rows[12] if abs(float(k)) < 10]
    print('%-7s x=12 branches with |k|<10 (t, k):' % name, small, flush=True)
json.dump(out, open('asym_out.json', 'w'), indent=1)
```

Output

```
T(2,3)  x=12 branches with |k|<10 (t, k): []
T(2,5)  x=12 branches with |k|<10 (t, k): []
4_1     x=12 branches with |k|<10 (t, k): [(2.000000000037751, '1.0')]
T(2,7)  x=12 branches with |k|<10 (t, k): []
5_2     x=12 branches with |k|<10 (t, k): [(26489122129.84347, '2.0'), (2.000000000037751, '1.0')]
6_1     x=12 branches with |k|<10 (t, k): [(26489122129.84347, '2.0'), (2.0000000000988343, '2.618034'),
```

```
(2.0000000000144196, '0.38196601')]
6_2     x=12 branches with |k|<10 (t, k): [(26489122129.84347, '-3.7751345e-11'), (2.000000000009438, '0.25')]
7_2     x=12 branches with |k|<10 (t, k): [(26489122129.84347, '1.0'), (26489122129.84347, '3.0'), (2.0000000000988343,
'2.618034'), (2.0000000000144196, '0.38196601')]
7_3     x=12 branches with |k|<10 (t, k): [(26489122129.84347, '2.8503282e-21'), (2.000000000009438, '0.25')]
6_3     x=12 branches with |k|<10 (t, k): [(2.0, '-3.7751345e-11')]
8_1     x=12 branches with |k|<10 (t, k): [(26489122129.84347, '1.0'), (26489122129.84347, '3.0'), (2.000000000122578,
'3.2469796'), (2.000000000058702, '1.5549581'), (2.000000000007477, '0.19806226')]
7_4     x=12 branches with |k|<10 (t, k): [(26489122129.84347, '0.5'), (2.0, '1.4251641e-21'), (2.0, '-3.7751345e-11')]
8_2     x=12 branches with |k|<10 (t, k): [(26489122129.84347, '-5.3801862e-32'), (2.0000000000041944, '0.11111111')]
7_5     x=12 branches with |k|<10 (t, k): [(26489122129.84347, '-7.5502691e-11'), (26489122129.84347, '-3.7751345e-11'),
(2.0, '-9.4378364e-12')]
8_3     x=12 branches with |k|<10 (t, k): [(26489122129.84347, '0.5'), (2.0, '3.731128e-21'), (2.0, '5.4436424e-22')]
8_4     x=12 branches with |k|<10 (t, k): [(26489122129.84347, '-3.7751345e-11'), (26489122129.84347, '2.8503282e-21'),
(2.0000000000247087, '0.6545085'), (2.000000000003605, '0.095491503')]
7_6     x=12 branches with |k|<10 (t, k): [(26489122129.84347, '-3.7751345e-11'), (26489122129.84347, '1.0'),
(26489122129.84347, '1.0'), (2.0, '1.4251641e-21')]
7_7     x=12 branches with |k|<10 (t, k): [(26489122129.84347, '-3.7751345e-11'), (2.000000000037751, '1.0'), (2.0,
'1.4251641e-21')]
8_7     x=12 branches with |k|<10 (t, k): [(2.0, '-9.4378364e-12')]
8_6     x=12 branches with |k|<10 (t, k): [(26489122129.84347, '-9.8834305e-11'), (26489122129.84347, '-7.5502691e-11'),
(26489122129.84347, '-1.4419731e-11'), (2.0, '3.5629102e-22')]
8_9     x=12 branches with |k|<10 (t, k): [(26489122129.84347, '-3.7751345e-11'), (2.000000000009438, '0.25'), (2.0,
'-9.4378364e-12')]
8_8     x=12 branches with |k|<10 (t, k): [(26489122129.84347, '-7.5502691e-11'), (26489122129.84347, '-3.7751345e-11'),
(2.0, '-5.3801862e-32')]
8_11    x=12 branches with |k|<10 (t, k): [(26489122129.84347, '2.8503282e-21'), (2.000000000037751, '1.0'),
(2.000000000037751, '1.0'), (2.000000000037751, '1.0'), (2.0, '1.4251641e-21'), (2.0, '-3.7751345e-11')]
8_13    x=12 branches with |k|<10 (t, k): [(26489122129.84347, '2.8503282e-21'), (26489122129.84347, '1.0'),
(26489122129.84347, '1.0'), (2.0, '1.4251641e-21'), (2.0, '-3.7751345e-11')]
8_12    x=12 branches with |k|<10 (t, k): [(26489122129.84347, '-3.7751345e-11'), (26489122129.84347, '1.0'),
(26489122129.84347, '1.0'), (2.0000000000755027, '2.0'), (2.0000000000755027, '2.0'), (2.000000000037751, '1.0'), (2.0,
'3.5629102e-22')]
8_14    x=12 branches with |k|<10 (t, k): [(26489122129.84347, '-9.4378364e-12'), (26489122129.84347, '1.0'),
(26489122129.84347, '1.0'), (26489122129.84347, '2.0'), (2.000000000037751, '1.0'), (2.0, '1.4251641e-21')]
```

k88.py

```
# SCRIPT: KNOT-COMM-8_8 — k = v(t-2) on 8_8's real branches with t near 2, x = 3..12 (200 digits); scaled by s^6
import warnings; warnings.filterwarnings('ignore')
import mpmath as mp, sympy as sp
from census_lib import riley_poly, S_, V_
mp.mp.dps = 200
R = sp.Poly(riley_poly(25, 9), V_); cs = [sp.lambdify(S_, c, 'mpmath') for c in R.all_coeffs()]
for i in range(0, 37):
    x = 3 + 0.25 * i; s = mp.e ** mp.mpf(x)
    rts = mp.polyroots([c(s) for c in cs], maxsteps=3000, extraprec=2000)
    row = []
    for r in rts:
        if abs(mp.im(r)) < mp.mpf(10) ** -90:
            v = mp.re(r); t = s*s + 1/(s*s) - v
            if abs(t - 2) < 0.1: k = v*(t-2); row.append('t-2=%s k=%s k*s^6=%s' % (mp.nstr(t-2, 5), mp.nstr(k, 6),
mp.nstr(k*s**6, 6)))
    print('x=%.2f' % x, ' | '.join(row), flush=True)
```

Output

```
x=3.00 t-2=-3.8223e-11 k=-1.53438e-8 k*s^6=-1.00747
x=3.25 t-2=-5.1477e-12 k=-3.41364e-9 k*s^6=-1.00452
x=3.50 t-2=-6.946e-13 k=-7.60334e-10 k*s^6=-1.00274
x=3.75 t-2=-9.3835e-14 k=-1.69471e-10 k*s^6=-1.00166
x=4.00 t-2=-1.2685e-14 k=-3.77894e-11 k*s^6=-1.00101
x=4.25 t-2=-1.7157e-15 k=-8.42861e-12 k*s^6=-1.00061
x=4.50 t-2=-2.321e-16 k=-1.88022e-12 k*s^6=-1.00037
x=4.75 t-2=-3.1403e-17 k=-4.19474e-13 k*s^6=-1.00022
x=5.00 t-2=-4.2493e-18 k=-9.3589e-14 k*s^6=-1.00014
x=5.25 t-2=-5.7503e-19 k=-2.08814e-14 k*s^6=-1.00008
x=5.50 t-2=-7.7818e-20 k=-4.65912e-15 k*s^6=-1.00005
x=5.75 t-2=-1.0531e-20 k=-1.03957e-15 k*s^6=-1.00003
x=6.00 t-2=-1.4252e-21 k=-2.31957e-16 k*s^6=-1.00002
x=6.25 t-2=-1.9288e-22 k=-5.17561e-17 k*s^6=-1.00001
x=6.50 t-2=-2.6103e-23 k=-1.15483e-17 k*s^6=-1.00001
x=6.75 t-2=-3.5327e-24 k=-2.57677e-18 k*s^6=-1.0
x=7.00 t-2=-4.7809e-25 k=-5.74954e-19 k*s^6=-1.0
x=7.25 t-2=-6.4703e-26 k=-1.28289e-19 k*s^6=-1.0
x=7.50 t-2=-8.7565e-27 k=-2.86252e-20 k*s^6=-1.0
x=7.75 t-2=-1.1851e-27 k=-6.38715e-21 k*s^6=-1.0
x=8.00 t-2=-1.6038e-28 k=-1.42516e-21 k*s^6=-1.0
x=8.25 t-2=-2.1705e-29 k=-3.17997e-22 k*s^6=-1.0
x=8.50 t-2=-2.9375e-30 k=-7.09548e-23 k*s^6=-1.0
```

```
x=8.75 t-2=-3.9755e-31 k=-1.58321e-23 k*s^6=-1.0
x=9.00 t-2=-5.3802e-32 k=-3.53263e-24 k*s^6=-1.0
x=9.25 t-2=-7.2813e-33 k=-7.88236e-25 k*s^6=-1.0
x=9.50 t-2=-9.8542e-34 k=-1.75879e-25 k*s^6=-1.0
x=9.75 t-2=-1.3336e-34 k=-3.9244e-26 k*s^6=-1.0
x=10.00 t-2=-1.8049e-35 k=-8.75651e-27 k*s^6=-1.0
x=10.25 t-2=-2.4426e-36 k=-1.95384e-27 k*s^6=-1.0
x=10.50 t-2=-3.3057e-37 k=-4.35961e-28 k*s^6=-1.0
x=10.75 t-2=-4.4738e-38 k=-9.7276e-29 k*s^6=-1.0
x=11.00 t-2=-6.0546e-39 k=-2.17052e-29 k*s^6=-1.0
x=11.25 t-2=-8.194e-40 k=-4.84309e-30 k*s^6=-1.0
x=11.50 t-2=-1.1089e-40 k=-1.08064e-30 k*s^6=-1.0
x=11.75 t-2=-1.5008e-41 k=-2.41123e-31 k*s^6=-1.0
x=12.00 t-2=-2.0311e-42 k=-5.38019e-32 k*s^6=-1.0
```

signs.py

```
# SCRIPT: KNOT-COMM-SIGNS  — per knot, at sample x: each real Riley root v with t = tr(ab) and k = v(t-2) (60 digits)
import warnings; warnings.filterwarnings('ignore')
import mpmath as mp, sympy as sp, sys
from census_lib import riley_poly, S_, V_
mp.mp.dps = 60
for name, p, q in [('6_3',13,5),('7_5',17,5),('8_7',23,5),('8_8',25,9),('4_1',5,3),('5_2',7,3),('7_3',13,3)]:
    R = sp.Poly(riley_poly(p, q), V_); cs = [sp.lambdify(S_, c, 'mpmath') for c in R.all_coeffs()]
    print(name, 'deg', R.degree())
    for x in [0.05, 0.3, 1.0, 3.0]:
        s = mp.e ** mp.mpf(x); rts = mp.polyroots([c(s) for c in cs], maxsteps=800, extraprec=300)
        row = []
        for r in rts:
            if abs(mp.im(r)) < mp.mpf(10) ** -25:
                v = mp.re(r); t = s*s + 1/(s*s) - v
                row.append('v=%s t=%s k=%s' % (mp.nstr(v, 5), mp.nstr(t, 6), mp.nstr(v*(t-2), 4)))
        print('  x=%.2f' % x, ' | '.join(row))
```

fig7.py

```
# SCRIPT: PEEL-TREES-FIG7
# Figure 7: the knot commutator band. Left: real Riley branches in the plane (x, u), u = v / (4 sinh^2 x)
# (u in (0,1) <=> the meridian axes cross); shaded: the rotation band 0 < A^2 u(1-u) < 4, A = 4 sinh^2 x.
# Right: first entry x for each knot (80-digit check, firstx_out.json); knots that never enter marked.
import json, warnings; warnings.filterwarnings('ignore')
import numpy as np, mpmath as mp, sympy as sp, matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from census_lib import riley_poly, S_, V_
BG='#070c18'; INK='#e3ebff'; DIM='#8fa2c4'; LINE='#2a3a5c'
C1='#4fe3c1'; C2='#ffb547'; C3='#ff6f91'; C4='#7aa8ff'; C5='#c08bff'
plt.rcParams.update({'font.family':'DejaVu Sans','text.color':INK,'axes.edgecolor':LINE,'axes.labelcolor':INK,
                     'xtick.color':DIM,'ytick.color':DIM,'savefig.facecolor':BG,'figure.facecolor':BG,'axes.facecolor':BG})
mp.mp.dps = 50
fig, (a1, a2) = plt.subplots(1, 2, figsize=(12.5, 5.6), gridspec_kw=dict(width_ratios=[1.25, 1]))
X = np.linspace(0.01, 2.6, 400); A = 4*np.sinh(X)**2
# band: 0<u<1 and A^2 u(1-u) < 4
U = np.linspace(-0.6, 1.6, 600); XX, UU = np.meshgrid(X, U); AA = 4*np.sinh(XX)**2; K = AA**2*UU*(1-UU)
a1.contourf(XX, UU, ((K > 0) & (K < 4)).astype(float), levels=[0.5, 1.5], colors=['#1c3b4a'])
a1.axhline(0, color=LINE, lw=1); a1.axhline(1, color=LINE, lw=1)
a1.axvline(np.log(1+np.sqrt(2)), color=DIM, lw=1, ls='--'); a1.text(np.log(1+np.sqrt(2))+0.03, 1.5, 'x = ln(1+√2)\n(Γ² =
1/2)', color=DIM, fontsize=9, va='top')
for name, p, q, col in [('3₁', 3, 1, C3), ('4₁', 5, 3, C1), ('5₂', 7, 3, C2), ('6₃', 13, 5, C5)]:
    R = sp.Poly(riley_poly(p, q), V_); cs = [sp.lambdify(S_, c, 'mpmath') for c in R.all_coeffs()]
    xs, us = [], []
    for x in np.linspace(0.01, 2.6, 260):
        s = mp.e ** mp.mpf(x); Ax = (s - 1/s)**2
        for r in mp.polyroots([c(s) for c in cs], maxsteps=600, extraprec=300):
            if abs(mp.im(r)) < mp.mpf(10)**-20:
                u = float(mp.re(r)/Ax)
                if -0.6 < u < 1.6: xs.append(x); us.append(u)
    a1.scatter(xs, us, s=3, color=col, label=name)
a1.set_xlim(0, 2.6); a1.set_ylim(-0.6, 1.6); a1.set_xlabel('x  (meridian length 2x)'); a1.set_ylabel('u = v / 4 sinh²x')
a1.text(0.05, 0.47, 'axes cross', color=DIM, fontsize=9); a1.text(0.05, 1.08, 'axes disjoint', color=DIM, fontsize=9);
a1.text(0.05, -0.12, 'axes disjoint (v < 0)', color=DIM, fontsize=9)
lg = a1.legend(loc='upper right', markerscale=4, frameon=True, facecolor=BG, edgecolor=LINE, fontsize=9)
a1.set_title('Real branches against the rotation band (shaded)', fontsize=12)
d = json.load(open('../../lanes/open/band/firstx_out.json'))
names = list(d.keys()); ent = [(n, d[n]['x1']) for n in names if d[n]['enters']]; nev = [n for n in names if not d[n]
['enters']]
ent.sort(key=lambda z: z[1]); order = ent + [(n, None) for n in nev]
for i, (n, x1) in enumerate(order):
    if x1 is None: a2.text(0.02, i, 'never (k < 0 on every real branch)', color=C3, fontsize=8.5, va='center')
```

```
    else: a2.barh(i, x1, color=C1, height=0.6)
a2.set_yticks(range(len(order))); SUB=str.maketrans('0123456789','₀₁₂₃₄₅₆₇₈₉'); a2.set_yticklabels([n if n.startswith('T')
else n.split('_')[0]+n.split('_')[1].translate(SUB) for n, _ in order], fontsize=9); a2.set_ylim(len(order)-0.4, -0.6);
a2.axvline(np.log(1+np.sqrt(2)), color=DIM, lw=1, ls='--'); a2.set_xlim(0, 1.0); a2.set_xlabel('first x with an elliptic
commutator')
a2.set_title('First entry into the band, 26 knots', fontsize=12)
plt.tight_layout(); plt.savefig('fig7_band.png', dpi=150); print('ok', len(ent), 'enter,', len(nev), 'never')
```

assemble_v4.py

```
# SCRIPT: PEEL-TREES-ASSEMBLE-V4
# Builds two_trees_v4.html (dark, self-contained) and two_trees_v4_print.html (embedded TrueType fonts for the PDF).
import base64, html, os
D = os.path.dirname(os.path.abspath(__file__))
rd = lambda p: open(os.path.join(D, p), encoding='utf-8').read()
body = rd('body_v4.html')
for f in sorted(os.listdir(os.path.join(D, 'figs'))):
    body = body.replace('FIG:' + f, '' + base64.b64encode(open(os.path.join(D, 'figs', f),
'rb').read()).decode())
assert 'FIG:' not in body
css, pcss = rd('css_all.txt').split('@@PRINT@@')
pre = lambda t: '<pre>%s</pre>' % html.escape(t)
app = ['<h2>Appendix A. Lean</h2>',
 '<p>Five core Lean 4.34.1 files, no Mathlib, 159 theorems. Every finite statement is decided by the kernel
(<code>decide</code>), never by <code>native_decide</code>; algebraic identities use <code>grind</code>, residue steps
<code>omega</code>. A fourth set of files, built on Mathlib as a library only, is drafted for independent checking and has
not been built.</p>',
 '<h3>A.1 PeelTrees.lean (74 theorems)</h3>', '<h4>Run</h4>' + pre(rd('lean_run.txt')), '<h4>Axiom audit</h4>' +
pre(rd('audit_out.txt')),
 '<h4>False controls (each must be rejected)</h4>' + pre(rd('controls.lean') + '\n-- result: %d of 3 rejected\n' %
rd('controls_out.txt').count(': error') + '\n'.join(l[:160] for l in rd('controls_out.txt').splitlines() if ': error' in
l)),
 '<h4>Source</h4>' + pre(rd('PeelTrees.lean')),
 '<h3>A.2 PeelTrees2.lean (49 theorems): every ratio not a power of 2, Thomas for every ratio</h3>', '<h4>Run</h4>' +
pre(rd('lean2_run.txt')),
 '<h4>Axiom audit</h4>' + pre(rd('v3src/audit2_out.txt')), '<h4>Source</h4>' + pre(rd('v3src/PeelTrees2.lean')),
 '<h3>A.3 PeelTrees3.lean (7 theorems): the ln 5 commutator</h3>', '<h4>Run and axiom audit</h4>' +
pre(rd('lean3_run.txt')), '<h4>Source</h4>' + pre(rd('PeelTrees3.lean')),
 '<h3>A.4 Mathlib-based drafts (not built; for independent checking)</h3>',
 '<p>Core.lean in this set is PeelTrees.lean unchanged. Smith.lean ties Γ to Mathlib\'s metric on the upper half-plane
(UpperHalfPlane.tanh_half_dist, dist_of_re_eq); Riley.lean states the meridians as Matrix.IsHyperbolic and the relation
identities over any commutative ring.</p>',
 '<h4>lakefile.toml</h4>' + pre(rd('v3src/lakefile.toml')), '<h4>Smith.lean</h4>' + pre(rd('v3src/Smith.lean')),
'<h4>Riley.lean</h4>' + pre(rd('v3src/Riley.lean')), '<h4>CLAIMS.md</h4>' + pre(rd('v3src/MATHLIB_CLAIMS.md')),
 '<h3>A.5 PeelTrees4.lean (12 theorems): the knot commutator band and the torus knots</h3>', '<h4>Run</h4>' +
pre(rd('v4/lean4_run.txt')), '<h4>Axiom audit</h4>' + pre(rd('v4/audit4_out.txt')),
 '<h4>False controls (each must be rejected)</h4>' + pre('\n'.join(rd('v4/controls4.lean').split('-- FALSE CONTROLS: each
must be rejected.')[1:]) + '\n-- result: %d of 3 rejected\n' % len(set(l.split(': error')[0] for l in
rd('v4/controls4_out.txt').splitlines() if ': error' in l))),
 '<h4>Source</h4>' + pre(rd('v4/PeelTrees4.lean')),
 '<h3>A.6 PingPong7.lean (17 theorems): the ping-pong at ln 7</h3>', '<h4>Run</h4>' + pre(rd('v4/pingpong7_run.txt')),
'<h4>Axiom audit</h4>' + pre(rd('v4/src/pingpong7_audit_out.txt')), '<h4>Source</h4>' + pre(rd('v4/src/PingPong7.lean')),
 '<h2>Appendix B. Scripts and their output</h2>']
B = [('exact.py','exact_out.txt'),('thomas_ints.py','thomas_out.txt'),('knotsym.py','knotsym_out.txt'),
('garrett_hyperbolic.py','garrett_hyperbolic_out.txt'),
     ('knot_boost_tree.py','knot_boost_tree_out.txt'),('census_lib.py',None),('v3src/sanity.py','v3src/sanity_out.txt'),
     ('v3src/KILL_CONDITIONS.txt',None),('v3src/q1_commutator.py','v3src/q1_commutator_out.txt'),('v3src/q1_enum.py',None),
     ('v3src/q1_enum_z4.txt',None),('v3src/q1_enum_z5.txt',None),('v3src/q1_enum_z6.txt',None),
('v3src/q1_enum_z7.txt',None),
     ('v3src/q1_identify.py','v3src/q1_identify_out.txt'),('v3src/q1_orbit_accum.py','v3src/q1_orbit_accum_out.txt'),
     ('v3src/q1_pingpong.py',None),('v3src/q1_pp_sym.txt',None),('v3src/q1_pp_free.txt',None),
('v3src/q1_pp_powers.txt',None),('v3src/q1_pp_sub.txt',None),
     ('v3src/q2_lib.py',None),('v3src/q2_sweep.py',None),('v3src/q2_3_1.txt',None),('v3src/q2_4_1.txt',None),
('v3src/q2_5_2.txt',None),('v3src/q2_6_2.txt',None),
     ('v3src/q2_checks.py','v3src/q2_checks_out.txt'),('v3src/q2_smallx.py','v3src/q2_smallx_out.txt'),
('v3src/make_plots.py',None),
     ('gen2.py',None),('figs.py',None),
     ('v4/src/POW2_REPORT.md',None),('v4/src/mkgens.py',None),('v4/src/relation_search.c',None),
('v4/src/out_search_z2_full.txt',None),('v4/src/out_search_z4_full.txt',None),
     ('v4/src/out_search_z8_full.txt',None),('v4/src/out_search_planar_z2_z4.txt',None),
('v4/src/out_search_planar_z8.txt',None),
     ('v4/src/commutator_type.py','v4/src/out_commutator_type.txt'),
('v4/src/residue_primes.py','v4/src/out_residue_primes.txt'),
     ('v4/src/pingpong_check.py','v4/src/out_pingpong_check.txt'),
('v4/src/pingpong_controls.py','v4/src/out_pingpong_controls.txt'),
     ('v4/src/CONJ_REPORT.md',None),('v4/src/inv.py','v4/src/inv_out.txt'),('v4/src/padic.py','v4/src/padic_out.txt'),
     ('v4/src/partA.py','v4/src/partA_out.txt'),('v4/src/partA2.py','v4/src/partA2_out.txt'),
('v4/src/planes.py','v4/src/planes_out.txt'),
     ('v4/src/AREA_REPORT.md',None),('v4/src/area_sym.py','v4/src/area_sym_out.txt'),
('v4/src/area_sym2.py','v4/src/area_sym2_out.txt'),
     ('v4/src/band.py','v4/src/band_out.txt'),('v4/src/band_hp.py','v4/src/band_hp_out.txt'),
('v4/src/band_hp2.py','v4/src/band_hp2_out.txt'),('v4/src/firstx.py','v4/src/firstx_out.txt'),
```

```
     ('v4/src/asym.py','v4/src/asym_out.txt'),('v4/src/k88.py','v4/src/k88_out.txt'),('v4/src/signs.py',None),
('v4/fig7.py',None),('assemble_v4.py',None)]
for f, out in B:
    app.append('<h3>%s</h3>%s' % (os.path.basename(f), pre(rd(f))))
    if out: app.append('<p class="cap">Output</p>' + pre(rd(out)))
head = ('<!doctype html><html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-
width,initial-scale=1">'
        '<title>Two Trees in One Ball</title><meta name="author" content="Jeromie Beasley">')
page = lambda extra: head + '<style>' + css.strip().removeprefix('<style>').removesuffix('</style>') + '</style>' + extra +
'</head><body>' + body + ''.join(app) + '</body></html>'
open(os.path.join(D, 'two_trees_v4.html'), 'w', encoding='utf-8').write(page(''))
open(os.path.join(D, 'two_trees_v4_print.html'), 'w', encoding='utf-8').write(page(pcss))
print('built', os.path.getsize(os.path.join(D, 'two_trees_v4.html')), 'bytes')
```